

Manual de usuario del panel 3X-UI

العربية · English · Español · فارسی · Bahasa Indonesia · 日本語 · Português · Русский · Türkçe · Українська · Tiếng Việt · 简体中文 · 繁體中文

Versión de 3X-UI: 3.4.0. El manual está elaborado para esta versión y es válido para ella. El resumen de cambios de la 3.4.0 respecto a la 3.3.1 se encuentra en la sección [«Qué hay de nuevo en 3.4.0»](#).

Manual detallado del panel web **3X-UI** (gestión de Xray-core): funciones, configuración y operación, con descripción de cada campo y opción de la interfaz.

Los nombres y etiquetas corresponden a la interfaz del panel. Las palabras *inbound* / *outbound* no se traducen.

Contenido

- [Qué hay de nuevo en 3.4.0](#)
- [1. Introducción, requisitos e instalación](#)
 - [1.1. Qué es 3X-UI](#)
 - [1.2. Sistemas operativos y arquitecturas compatibles](#)
 - [1.3. Métodos de instalación](#)
 - [1.4. Primer inicio y credenciales por defecto](#)
 - [1.5. Ubicación de los archivos](#)
 - [1.6. Comando de gestión `x-ui` \(menú del script\)](#)
 - [1.7. Subcomandos de `x-ui` \(sin menú interactivo\)](#)
 - [1.8. Migración SQLite → PostgreSQL](#)
- [2. Acceso al panel y seguridad](#)
 - [2.1. Formulario de acceso](#)
 - [2.2. Autenticación de dos factores \(2FA / TOTP\)](#)
 - [2.3. Limitación de intentos de inicio de sesión \(login limiter / protección contra fuerza bruta\)](#)
 - [2.4. Cambio de usuario y contraseña del administrador](#)
 - [2.5. Ruta secreta \(URI path / webBasePath\) y puerto del panel](#)
 - [2.6. Duración de la sesión \(tiempo de espera\)](#)
 - [2.7. LDAP \(sincronización y autenticación\)](#)
- [3. Resumen / Dashboard](#)
 - [3.1. Principios generales de recolección de datos](#)
 - [3.2. CPU](#)
 - [3.3. Memoria \(RAM\)](#)
 - [3.4. Swap](#)
 - [3.5. Disco \(Storage\)](#)
 - [3.6. Tiempo de actividad del sistema \(Uptime\)](#)
 - [3.7. Carga del sistema \(Load average\)](#)
 - [3.8. Red: velocidad y volumen total de tráfico](#)
 - [3.9. Direcciones IP del servidor](#)
 - [3.10. Conexiones TCP/UDP](#)
 - [3.11. Estado de Xray y gestión del proceso](#)
 - [3.12. Actualización del panel \(3X-UI\)](#)
 - [3.13. Actualización de archivos geo \(GeoIP / GeoSite\)](#)
 - [3.14. Copia de seguridad y restauración de la base de datos](#)
 - [3.15. Elementos adicionales de la interfaz](#)
- [4. Inbounds: creación y parámetros generales](#)
 - [4.1. Campos generales del formulario](#)
 - [4.2. Sniffing](#)
 - [4.3. Allocate \(estrategia de asignación de puertos\)](#)
 - [4.4. External Proxy \(Proxy externo\)](#)
 - [4.5. Fallbacks](#)
 - [4.6. Reinicio periódico del tráfico](#)
 - [4.7. JSON del inbound \(avanzado\)](#)
 - [4.8. Acciones sobre el inbound: QR / Edit / Reset / Delete y estadísticas](#)

- 5. Protocolos
 - 5.1. Lista de protocolos compatibles
 - 5.2. Qué protocolos admiten TLS / REALITY / transporte
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (reenviador transparente)
 - 5.8. SOCKS / HTTP (protocolo `mixed`)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (v2 por defecto)
 - 5.11. MTPROTO (proxy para Telegram)
 - 5.12. Resumen rápido para elegir protocolo
- 6. Transporte (Stream Settings)
 - 6.1. Selección de la red de transmisión
 - 6.2. RAW / TCP (`tcpSettings`)
 - 6.3. mKCP (`kcpSettings`)
 - 6.4. WebSocket (`wsSettings`)
 - 6.5. gRPC (`grpcSettings`)
 - 6.6. HTTPUpgrade (`httpupgradeSettings`)
 - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
 - 6.8. Transporte Hysteria (`hysteriaSettings`)
 - 6.9. Parámetros adicionales
- 7. Seguridad de la conexión: TLS,XTLS y REALITY
 - 7.1. Diferencias: TLS vs XTLS vs REALITY
 - 7.2. Modo «Ninguno» (`none`)
 - 7.3. Modo TLS
 - 7.4. Modo REALITY
 - 7.5. Recomendaciones prácticas de configuración
- 8. Clientes
 - 8.1. Campos del cliente
 - 8.2. Vinculación al inbound
 - 8.3. Operaciones sobre el cliente
 - 8.4. Operaciones masivas
 - 8.5. Búsqueda, filtros y ordenación
 - 8.6. Iconos y estados
- 9. Grupos de clientes
 - 9.1. Qué es un grupo de clientes y para qué sirve
 - 9.2. Relación del grupo con clientes, inbounds, nodos y protocolos
 - 9.3. Directorio de grupos y grupos «vacíos»
 - 9.4. Campos y columnas del grupo
 - 9.5. Creación de un grupo
 - 9.6. Cambio de nombre del grupo
 - 9.7. Agregar clientes a un grupo
 - 9.8. Eliminar clientes de un grupo (sin eliminar los clientes)
 - 9.9. Reinicio del tráfico del grupo

- 9.10. Eliminación del grupo y de los clientes del grupo
- 9.11. Relación con la página «Clientes»
- 9.12. Resumen de endpoints de la API
- 9.13. Tráfico por grupo
- 10. Suscripciones (Subscription)
 - 10.1. Qué es subld y cómo se forma el enlace
 - 10.2. Configuración del servidor de suscripciones
 - 10.3. Formatos de salida
 - 10.4. Página de información de la suscripción y códigos QR
 - 10.5. Plantillas personalizadas de la página de suscripción
- 11. Xray: enrutamiento, outbounds, DNS y extensiones
 - 11.1. Estructura del editor: pestañas/modos
 - 11.2. Configuración general (General)
 - 11.3. Reglas de enrutamiento (routing)
 - 11.4. Outbounds (conexiones salientes)
 - 11.5. Balanceadores (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Proxy inverso y TUN
 - 11.10. Registros y estadísticas (Stats, metrics)
 - 11.11. Guardar, reiniciar y transformaciones automáticas
 - 11.12. Outbound desde suscripción (con actualización automática)
 - 11.13. Rotación de IP en WARP
- 12. Nodos (multipanel, master/slave)
 - 12.1. Resumen en la parte superior de la lista
 - 12.2. Agregar y editar un nodo
 - 12.3. Verificación TLS (para nodos https)
 - 12.4. Información mostrada por nodo
 - 12.5. Acciones sobre el nodo
 - 12.6. Historial de métricas
 - 12.7. Cómo se sincronizan inbounds y clientes
 - 12.8. Cadenas de nodos (subnodos / nodos transitivos)
 - 12.9. Nodos: novedades en 3.3.0
- 13. Configuración del panel
 - 13.1. Guardar y reiniciar el panel
 - 13.2. Configuración general (pestaña «Panel» / *General*)
 - 13.3. Acceso al panel: IP, puerto, ruta, dominio, certificado
 - 13.4. Sesión, proxy del panel y proxies de confianza (pestaña «Proxy y servidor» / *Proxy and Server*)
 - 13.5. Bot de Telegram (pestaña «Bot de Telegram» / *Telegram Bot*)
 - 13.6. Fecha y hora (pestaña «Fecha y hora» / *Date and Time*)
 - 13.7. Tráfico externo y comportamiento de Xray (pestaña «Tráfico externo» / *External Traffic*)
 - 13.8. Otros: plantilla de configuración de Xray y URL de verificación
 - 13.9. Cuenta de administrador y tokens de API
 - 13.10. Cambios en la API en 3.3.0 (importante para integraciones)

- 14. Bot de Telegram
 - 14.1. Activación y configuración del bot
 - 14.2. Menú principal y botones
 - 14.3. Comandos del bot
 - 14.4. Gestión de clientes (solo administrador)
 - 14.5. Notificaciones e informes
 - 14.6. Copia de seguridad y registros
 - 14.7. Particularidades de funcionamiento
 - 15. Bases geo (geoip / geosite y personalizadas)
 - 15.1. Qué son geoip.dat y geosite.dat
 - 15.2. Archivos geo estándar y su actualización
 - 15.3. Recursos geo personalizados (Custom GeoSite / GeoIP)
 - 15.4. Validación y restricciones
 - 15.5. Comprobación automática al iniciar el panel
 - 15.6. Uso de las bases geo en las reglas de enrutamiento
 - 16. Operación: copias de seguridad, registros, actualización, CLI
 - 16.1. Copia de seguridad y restauración de la base de datos
 - 16.2. Visualización de registros
 - 16.3. Nivel y configuración del registro de Xray
 - 16.4. Gestión de Xray: parada y reinicio
 - 16.5. Reinicio y actualización del panel
 - 16.6. Tareas periódicas (cron)
 - 16.7. Menú de consola y CLI (`x-ui`)
 - 16.8. Desinstalación del panel
 - 16.9. Comando `x-ui migrateDB`
-

Qué hay de nuevo en 3.4.0

Esta sección enumera brevemente los cambios visibles para el usuario de la versión **3.4.0** respecto a la 3.3.1, agrupados por secciones del manual. Los detalles de cada función se encuentran en la sección correspondiente.

3. Resumen / Dashboard

- **Historial de métricas del sistema: nuevos intervalos de agregación 12h/24h/48h** – En la ventana del historial de métricas del sistema se han añadido los valores de promediado 12h, 24h y 48h – ahora los gráficos (CPU, RAM, tráfico, paquetes, conexiones, disco, activos, carga) pueden consultarse para períodos de hasta dos días.

4. Inbounds: creación y parámetros generales

- **Inbound: advertencia de conflicto con el puerto reservado de la API de Xray** – Al crear o modificar un inbound, el panel ahora impide ocupar el puerto reservado de la API interna de Xray (por defecto 62789 en 127.0.0.1): un inbound TCP local en ese puerto sobre loopback se rechaza con un error de conflicto de puerto. En los nodos (nodes) la restricción no aplica – tienen su propio Xray.
- **Tunnel/TPProxy: aceptación de streamSettings sin clave security** – Los inbounds de tipo tunnel/TPProxy cuyo streamSettings no contiene un bloque security ahora se abren y guardan sin error de validación.
- **Inbounds: columna Speed (velocidad en vivo) en la lista** – En la lista de inbounds aparece la columna Speed con la velocidad de tráfico actual (subida/bajada) de cada inbound.

5. Protocolos

- **Shadowsocks-2022: regeneración de PSK de clientes al cambiar de método con diferente tamaño de clave** – Para Shadowsocks-2022: al cambiar el método de cifrado a uno con distinto tamaño de clave (por ejemplo entre aes-256 y aes-128), los PSK de los clientes ahora se regeneran automáticamente con la nueva longitud al guardar el inbound. En consecuencia, los clientes afectados deberán obtener la suscripción de nuevo (los enlaces anteriores dejarán de funcionar).
- **WireGuard: eliminación del campo workers** – De los formularios de WireGuard (inbound y outbound) se ha eliminado el campo workers: xray-core v26.6.22 ya no lo utiliza. Las configuraciones guardadas anteriormente funcionan sin cambios – el campo simplemente se ignora.
- **VLESS+XHTTP: flow xtls-rprx-vision en enlaces y suscripciones** – Para VLESS sobre XHTTP, el flow xtls-rprx-vision ahora se incluye correctamente en los enlaces y suscripciones (incluyendo XHTTP+REALITY y el formato Clash/Mihomo). Antes, el panel guardaba el flow pero los clientes recibían la configuración sin vision.

6. Transporte (Stream Settings)

- **XHTTP: cambio de nombre de campos sessionID + Session ID Table / Length** – En el transporte XHTTP los campos de sesión han sido renombrados: Session ID Placement y Session ID Key (antes – Session Placement / Session Key). Se añaden dos parámetros. Session ID Table – conjunto de caracteres para generar identificadores de sesión: puede elegirse uno predefinido (ALPHABET, Base62, hex, number, etc.) o introducirse una cadena ASCII arbitraria; vacío – valor por defecto de xray-core. Session ID Length –

longitud o rango (por ejemplo 8-16) de los identificadores generados; solo se tiene en cuenta cuando Session ID Table está definido, el mínimo debe ser mayor que 0. Los inbounds guardados anteriormente se migran automáticamente.

- **Inbound: preajuste Real client IP para determinar la IP real detrás de CDN/relay** – En los ajustes de socket (sockopt) del inbound aparece la selección Real client IP – un preajuste para determinar la IP real del visitante cuando el tráfico llega a través de una CDN o relay (de lo contrario se registra la dirección del intermediario). Hay tres opciones disponibles: Off / direct (sin procesamiento), Cloudflare CDN (confiar en X-Forwarded-For) y L4 relay / Spectrum (PROXY) (aceptar cabecera del protocolo PROXY). Los preajustes son mutuamente excluyentes y advierten si el transporte seleccionado no los soporta. Estos campos nunca se envían a los clientes en las suscripciones.
- **gRPC: la cabecera Trusted X-Forwarded-For ahora se tiene en cuenta** – La cabecera Trusted X-Forwarded-For ahora también se tiene en cuenta en el transporte gRPC (antes solo en WebSocket, HTTPUpgrade y XHTTP). Para los inbounds gRPC el panel ya no muestra la advertencia de cabecera no compatible.

7. Seguridad de la conexión: TLS, XTLS y REALITY

- **TLS: nuevos campos Verify Peer Cert By Name, Curve Preferences, Master Key Log, ECH Sockopt** – Verify Peer Cert By Name – nombres (separados por comas) con los que el cliente verifica el certificado del servidor en lugar del SNI; sustituto moderno del eliminado allowInsecure, se transmite en enlaces y suscripciones, no se escribe en el servidor. Curve Preferences – restricción y orden de las curvas de intercambio de claves TLS (por ejemplo X25519MLKEM768, X25519); vacío – valores por defecto. Master Key Log – ruta para escribir las claves TLS (formato SSLKEYLOGFILE) para depuración en Wireshark; dejar vacío en producción. ECH Sockopt – parámetros de socket para obtener la configuración ECH (dialerProxy, Domain Strategy, TCP Fast Open, Multipath TCP).
- **REALITY: limitación de velocidad del fallback (Limit Fallback) y Master Key Log** – Para cada dirección (Upload y Download) se especifican: After Bytes – cuántos bytes pasar a velocidad completa antes de comenzar la limitación (0 – limitar desde el primer byte); Bytes Per Sec – límite de velocidad del tráfico fallback para evitar que los sondeos usen el servidor como canal gratuito (0 – sin límite, desactiva la dirección); Burst Bytes Per Sec – margen para picos momentáneos. También se añade el campo Master Key Log (ruta al archivo SSLKEYLOGFILE para depuración).
- **TLS: botones para completar Pinned Peer Cert SHA-256 desde el certificado del inbound y por SNI** – Junto al campo Pinned Peer Cert SHA-256 ahora hay dos botones de autocompletado en lugar del anterior botón de hash aleatorio. El primero introduce el SHA-256 del certificado del propio inbound. El segundo obtiene el hash del certificado en vivo del servidor realizando una conexión TLS al SNI indicado (serverName debe estar relleno). Los hashes obtenidos se añaden al campo (separados por comas) y se incluyen en los enlaces para anclar el certificado en el cliente.
- **TLS: OCSP stapling desactivado por defecto para nuevos certificados de inbound** – Para los nuevos inbounds, el OCSP stapling está desactivado por defecto (intervalo 0). Esto elimina los errores en los registros de xray para certificados sin respondedor OCSP (por ejemplo Let's Encrypt). El campo se mantiene – puede activarse para certificados que admitan stapling.
- **REALITY: compatibilidad con el campo dest (alias de target)** – Si un inbound REALITY fue creado con el campo dest (por versiones anteriores del panel, mediante API o herramientas externas), ahora se carga correctamente: el valor de dest se transfiere al campo Target. Antes, Target quedaba vacío y volver a guardar rompía REALITY.

8. Clientes

- **Pestaña «Links» en el editor de cliente (enlaces externos y suscripciones)** — Mediante el botón **Add External Link** se añaden enlaces share de terceros (`vless://` , `vmess://` , `trojan://` , `ss://` , `hysteria2://` , `wireguard://`), y con **Add External Subscription** — URLs de suscripciones remotas. Todo ello se incluye en la salida de la suscripción del cliente (formatos raw, JSON y Clash): los enlaces se añaden tal cual, mientras que las suscripciones remotas son descargadas periódicamente por el panel y sus configuraciones se combinan con las propias.
- **El campo «Límite de IP» ahora se desactiva sin Fail2ban** — El campo **Límite de IP** ahora solo funciona si Fail2ban está instalado y activo. Si Fail2ban no está instalado (o el sistema es Windows, o la función está desactivada en el servidor), el campo del editor de cliente se bloquea y al pasar el cursor se muestra una sugerencia para instalar Fail2ban desde el menú bash `x-ui` . Al actualizar el panel, el límite de IP guardado de los clientes en servidores sin Fail2ban se pone a cero, ya que de todas formas no se aplicaba.
- **Eliminación de clientes no vinculados, exportación e importación de clientes** — En el menú **Más** de la página **Clientes** se añaden tres operaciones. **Exportar clientes** muestra una lista JSON de todos los clientes (en formato `{client, inboundIds}`) con botones de copia y descarga (`clients-export.json`). **Importar clientes** acepta el mismo JSON: los clientes con vínculos se recrean y se vinculan a los inbounds, los clientes sin vínculos se restauran como registros independientes, y los emails ya existentes no se sobrescriben (se marcan como omitidos). **Eliminar clientes no vinculados** elimina todos los clientes no vinculados a ningún inbound, junto con su tráfico, registro de IP y enlaces externos; la acción es irreversible.
- **Registro de IP del cliente: hora de conexión y nombre del nodo** — En el registro de IP del cliente (botón de visualización junto al campo «Límite de IP» y en la tarjeta «Información del cliente»), cada entrada ahora contiene, además de la IP, la hora del último acceso y la etiqueta del nodo (`@ nombre_nodo`) por el que se registró la conexión — en una configuración multipanel se puede ver por qué nodo se conectó el cliente.
- **Reinicio de la etiqueta de grupo en el editor individual del cliente** — Ahora si en el editor de un cliente individual se vacía el campo **Grupo** y se guarda, la etiqueta de grupo se elimina correctamente — antes el cliente podía seguir mostrándose bajo el grupo anterior hasta un nuevo guardado.
- **La lista de clientes se actualiza automáticamente (sondeo en segundo plano)** — La lista de clientes ahora se actualiza automáticamente: el panel consulta la página actualizada cada pocos segundos, por lo que los clientes recién conectados y los cambios en el orden de clasificación aparecen sin necesidad de actualizar manualmente.

10. Suscripciones (Subscription)

- **Hosts gestionados: sobreescritura de enlaces de suscripción por hosts** — En la versión 3.4.0 se añade la sección Hosts (elemento del menú lateral). Para cada inbound pueden definirse uno o varios endpoints de Host que sustituyen la dirección, el puerto y los parámetros TLS del propio inbound en los enlaces de suscripción — esto resulta útil para distribuir el tráfico a través de CDN o relay. Para cada host se definen Remark y descripción, vinculación al inbound, Address (vacío — hereda la dirección del inbound) y Port (0 — hereda el puerto del inbound), parámetros Security (same/tls/none/reality), así como Host header, Path, Mux, Sockopt, Final Mask, exclusión de formatos de suscripción (raw/json/clash) y parámetros para Clash/mihomo. Los hosts se ordenan dentro del inbound y admiten operaciones masivas.
- **Remark Template reemplaza al constructor de modelo de remark; variables {{VAR}}** — El anterior constructor del nombre de perfil (selección de Inbound/Email/External Proxy y separador) se reemplaza por el campo «Remark Template». En él se escribe un formato de nombre arbitrario insertando variables

mediante un botón: identificación del cliente ({{EMAIL}}, {{INBOUND}}, {{HOST}}, {{ID}}, {{SUB_ID}}, {{COMMENT}}, {{TELEGRAM_ID}}), tráfico ({{TRAFFIC_USED}}, {{TRAFFIC_LEFT}}, {{TRAFFIC_TOTAL}}, {{UP}}, {{DOWN}}, {{USAGE_PERCENTAGE}}) y plazo/estado ({{DAYS_LEFT}}, {{TIME_LEFT}}, {{EXPIRE_DATE}}, {{JALALI_EXPIRE_DATE}}, {{STATUS}}, {{STATUS_EMOJI}}). Las variables se sustituyen individualmente para cada cliente al generar la suscripción, y se dispone de vista previa. Los segmentos separados por el carácter «|» con valores ilimitados se ocultan automáticamente, y los datos de consumo y plazo solo se muestran en el primer enlace del cliente. Si el campo se deja vacío, se usa el modelo de remark anterior.

- **Enlaces externos y suscripciones remotas por cliente (pestaña Links)** — Aquí se pueden adjuntar al cliente individual enlaces share de terceros (Add External Link) y direcciones de suscripciones externas (Add External Subscription) — se incluirán en su propia suscripción (formatos RAW, JSON y Clash). Las suscripciones externas son descargadas por el panel y combinadas con las configuraciones del cliente. Resulta útil para entregar al cliente servidores adicionales además de sus inbounds.
- **Happ: ocultar la configuración del servidor en el cliente (Hide server settings)** — En la pestaña Happ de la configuración de suscripción se añade el interruptor «Hide server settings». Cuando está activado, en el cliente Happ se oculta la posibilidad de ver y modificar los parámetros del servidor. La opción solo afecta al cliente Happ.
- **El nombre del nodo ya no se añade al nombre del perfil en la suscripción** — El nombre del nodo (Node) ya no se agrega al nombre del perfil en la suscripción. En la aplicación cliente solo se muestra el remark del inbound definido por el administrador, sin el sufijo interno tipo «@nombre-nodo».
- **La etiqueta del modelo de remark se renombra Other → External Proxy (luego reemplazada por la plantilla)** — No es necesario documentarlo por separado: el cambio de nombre del elemento «Other» a «External Proxy» en el modelo de remark queda absorbido por la transición al nuevo campo Remark Template, donde el constructor del modelo se elimina de la interfaz.
- **Corrección de enlaces de suscripción: SS2022, Shadowrocket, SIP002 obfs, XHTTP en Clash** — Se ha mejorado la compatibilidad de los enlaces de suscripción generados: se corrige la codificación de SS2022, el deep-link para Shadowrocket, la salida de Shadowsocks+obfs en formato SIP002 (plugin obfs-local) y el conjunto completo de campos XHTTP en suscripciones Clash/Mihomo. No se requieren ajustes adicionales — los enlaces simplemente son reconocidos de forma más correcta por los clientes.
- **Modelo de remark de suscripciones: el elemento «Other» renombrado a «External Proxy»** — En la configuración de suscripciones, en el modelo de remark, el elemento «Other» pasa a llamarse «External Proxy» (fuente — remark del proxy externo). El comportamiento no cambia, las configuraciones existentes son compatibles.
- **Suscripciones: selección de variables de remark haciendo clic en chips (Remark variable picker)** — Junto al campo Remark Template hay disponible un conjunto agrupado de chips de variables: al hacer clic en una variable {{VAR}} se inserta en la plantilla, y al pasar el cursor se muestra una descripción. En los campos de remark y nombre de host también se admite la notación simplificada entre llaves simples — {DATA_LEFT}, {EXPIRE_DATE}, {PROTOCOL}, {TRANSPORT}, etc.; el panel la convierte automáticamente al formato interno {{...}}.

11. Xray: enrutamiento, outbounds, DNS y extensiones

- **Enrutamiento y Outbounds trasladados a elementos separados del menú lateral** — A partir de esta versión, «Salientes» (Outbounds) y «Enrutamiento» (Routing) están en elementos independientes del menú lateral (justo debajo de «Hosts»), cada uno con su propia dirección — /outbound y /routing. Antes, el enrutamiento se abría dentro del submenú «Configuraciones de Xray», y los outbounds — como pestaña de la página Xray. En el submenú «Configuraciones de Xray» ahora solo quedan: Principal, Balanceador, DNS y

Plantilla avanzada. Los enlaces directos a `/outbound` y `/routing` y la recarga de página funcionan como se esperaba.

- **Las reglas de enrutamiento pueden activarse y desactivarse con un interruptor** – Una regla de enrutamiento individual ahora puede **desactivarse** temporalmente con un interruptor sin necesidad de eliminarla. En la tabla de reglas hay una columna «**Activar**» con un interruptor; en el formulario de la regla el campo «Activar» también es un interruptor. Una regla desactivada no aparece en la configuración final de Xray. La regla de servicio de estadísticas (`api`) no puede desactivarse – su interruptor está bloqueado.
- **La exportación de reglas de enrutamiento y outbounds se abre en una ventana modal de vista previa** – El botón «**Exportar**» de reglas de enrutamiento y salientes ya no descarga el archivo directamente, sino que abre una ventana modal con vista previa del JSON y botones de «**Copiar**» y «**Descargar**». En la pestaña «Enrutamiento», «Importar» y «Exportar» se agrupan en un menú desplegable «**más**» (igual que en la pestaña Outbounds).
- **La prueba de todos los salientes ahora verifica también los outbounds de suscripciones; direct/dns ya no se prueban** – El botón «**Probar todos**» en la página «Salientes» ahora también verifica los outbounds obtenidos de suscripciones (tabla «de suscripciones») – sus filas también se resaltan con el resultado. Los outbounds `freedom` («direct») y `dns` ya no se prueban en ningún modo (no son proxies): el botón de prueba no está disponible para ellos, y «Probar todos» los omite.
- **FinalMask: arrays por fragmento Lengths/Delays en lugar de campos únicos Length/Delay** – En la máscara de fragmento (FinalMask), los campos únicos Length y Delay se reemplazan por listas Lengths y Delays: para cada segmento se puede definir un rango de longitud independiente (por ejemplo 100-200) y un retardo en ms (por ejemplo 10-20 o 0). Las filas pueden añadirse y eliminarse; los valores guardados anteriormente se migran automáticamente.
- **Loopback outbound: bloque Sniffing añadido** – El outbound de tipo loopback ahora tiene un bloque Sniffing con los mismos parámetros que el inbound: activación, destOverride, Metadata Only, Route Only y lista de dominios excluidos.
- **Hysteria2 / Salamander: modo Gecko (packetSize) y TLS para la máscara Realm** – En la máscara UDP (FinalMask) para Hysteria2 se amplían las posibilidades. En la máscara Salamander se añade el selector Mode: el modo Gecko añade relleno aleatorio de paquetes con campos Min/Max de tamaño (de 1 a 2048, por defecto 512-1200) para proteger contra el análisis de longitud de paquetes. En la máscara Realm aparece un bloque TLS Config opcional: Server Name (SNI), ALPN (h3/h2/http/1.1), Fingerprint y Allow Insecure.
- **La importación de share-link en outbound guarda la configuración de xmux** – Al importar un outbound desde un share-link, ahora se conservan los ajustes del multiplexor **xmux** (XHTTP): antes se perdían silenciosamente. Tras la importación, los valores se cargan en el subformulario XMUX.
- **Las etiquetas de outbounds procedentes de suscripciones conservan caracteres no ASCII (cirílico)** – Las etiquetas de los outbounds obtenidos de suscripciones ahora conservan los caracteres no ASCII (por ejemplo, cirílico) y permanecen legibles, en lugar de reducirse solo a dígitos.

12. Nodos (multipanel, master/slave)

- **Nodos: nuevo modo de verificación TLS – Mutual TLS (certificado de cliente)** – En el formulario del nodo, el modo de verificación TLS ahora tiene cuatro opciones: Verify (CA del sistema), Pin (fijación del certificado por SHA-256), Skip (sin verificación) y el nuevo Mutual TLS (certificado de cliente). En el modo Mutual TLS, el panel también se autentica ante el nodo con un certificado de cliente emitido por su propia CA; el token de API para ese nodo se vuelve opcional. Para activar mTLS: en el nodo establezca el modo

Mutual TLS, copie la CA del panel de gestión desde la sección Node mTLS, indíquela en el nodo como CA raíz de confianza y reinicie el nodo.

- **Nodos: sección «Node mTLS» – copia de la CA del panel y CA raíz de confianza** – En la página Nodos se añade la sección Node mTLS para configurar la autenticación TLS mutua entre paneles. El botón «Copiar CA de este panel» copia el certificado raíz del panel al portapapeles – debe transferirse a los nodos gestionados que verificarán el certificado de cliente del panel. El campo «CA raíz de confianza» se usa cuando el panel actúa él mismo como nodo: pegue aquí la CA del panel de gestión para exigir su certificado de cliente y reinicie el panel. El TLS mutuo es opcional; si los campos están vacíos, los nodos funcionan según el esquema anterior – solo con token de API.
- **Enrutamiento de la conexión saliente del panel al nodo (Connection outbound)** – En el formulario del nodo aparece el campo «Connection outbound» (conexión saliente). Si se selecciona una etiqueta de Xray-outbound, el tráfico del panel hacia la API de ese nodo pasará por el outbound indicado (el panel añade automáticamente un bridge-inbound sobre loopback a la configuración activa y lo aplica sin reinicio). Valor vacío = conexión directa. En la lista las etiquetas se agrupan en «Salientes» y «Balanceadores»; los outbounds blackhole están ocultos.
- **Nodos: enrutamiento del tráfico panel→nodo a través del outbound seleccionado («Connection outbound»)** – En el formulario del nodo aparece el campo «Connection outbound»: permite dirigir el tráfico de las consultas del panel al nodo a través del Xray outbound seleccionado (disponibles outbounds normales y balanceadores). El panel añade automáticamente un loopback-bridge inbound a la configuración activa y aplica los cambios sin reinicio. Deje el campo vacío para conexión directa.
- **Nodos: la eliminación de un nodo queda bloqueada mientras tenga inbounds vinculados** – Un nodo solo puede eliminarse después de que se le hayan desvinculado todos los inbounds. Si todavía hay al menos un inbound vinculado al nodo, el panel rechazará la eliminación con un error – primero desvincule o elimine esos inbounds, luego elimine el nodo.
- **Nodos: en la página de nodos se muestra la velocidad en vivo de los inbounds alojados en el nodo** – En la página Nodos, para los inbounds alojados en el nodo ahora se muestran los clientes activos, contadores y la velocidad de transferencia actual. El chip «terminados» (ended) cuenta solo los clientes caducados y sin tráfico (los desactivados ya no se incluyen en él).

14. Bot de Telegram

- **Notificaciones: nuevo bus de eventos con canales Telegram y Email (SMTP)** – Se añade un sistema de notificaciones de eventos con dos canales de entrega – Telegram y Email. En la pestaña de notificaciones los eventos se agrupan por tarjetas: Outbound (caída/recuperación), Xray Core (cierres inesperados), Nodes (nodo no disponible/recuperado), System (alta carga de CPU y memoria con umbral configurable en %), Security (intento de inicio de sesión). Cada grupo tiene un interruptor maestro y un contador de eventos seleccionados. El conjunto de eventos habilitados se configura de forma independiente para Telegram y Email; por defecto están activados «intento de inicio de sesión» y «alta carga de CPU».
- **Notificaciones: nuevo canal Email/SMTP y configuración del servidor SMTP** – Se añade un nuevo canal de notificaciones por correo electrónico mediante SMTP. En la pestaña de configuración SMTP se definen: activación de notificaciones por email, host y puerto SMTP (por defecto 587), nombre de usuario, contraseña (almacenada de forma oculta), lista de destinatarios (separados por comas) y tipo de cifrado – none, STARTTLS (por defecto) o TLS. El botón «Enviar email de prueba» verifica la conexión e indica en qué etapa (conexión, autenticación, envío) se produjo el error. En la segunda pestaña se seleccionan los eventos sobre los que se recibirán mensajes.

- **Notificaciones: alerta de alta carga de memoria (umbral de RAM)** – A las alertas de alta carga de CPU se añade una alerta de alta carga de memoria RAM. En el grupo de eventos «System» aparece «Memory high (%)» con su propio campo de umbral (por defecto 80%); el panel comprueba la carga de RAM cada minuto y al superarse el umbral envía una notificación a los canales seleccionados.

15. Bases geo (geoip / geosite y personalizadas)

- **Actualización de bases geo: estado por archivo y omisión del reinicio si no hay cambios** – La actualización de las bases geo (geoip/geosite, incluidos los conjuntos IR y RU) ahora muestra el estado de cada archivo: actualizado, ya estaba al día o error de descarga. El reinicio de Xray (y por tanto la interrupción de las conexiones activas) solo ocurre si al menos un archivo fue efectivamente actualizado; si no hay cambios, el panel no se reinicia. El mismo comportamiento aplica al comando x-ui update-all-geofiles.

16. Operación: copias de seguridad, registros, actualización, CLI

- **El límite de IP del cliente solo funciona con fail2ban instalado; el campo se bloquea en caso contrario** – La restricción por número de IP para un cliente ahora solo funciona si fail2ban está instalado en el servidor. Si no lo está, el campo «IP Limit» en el formulario del cliente y en la adición masiva queda desactivado con una nota explicativa (en Windows – con un mensaje separado), y los límites previamente definidos en esos servidores se ponen automáticamente a cero, ya que de todos modos no se aplicaban. El bloqueo por fail2ban ahora abarca tanto TCP como UDP.
- **Instalación automática de fail2ban al instalar y actualizar el panel** – Al instalar y actualizar el panel en un servidor ordinario, fail2ban ahora se instala y configura automáticamente (antes esto solo ocurría en Docker), para que la función de límite de IP funcione desde el primer momento. El comportamiento lo controla la variable de entorno XUI_ENABLE_FAIL2BAN: la configuración se realiza si la variable no está definida o es igual a true. La ejecución manual está disponible mediante el comando x-ui setup-fail2ban; un error de fail2ban no interrumpe la instalación ni la actualización.
- **Sobreescritura del puerto del panel mediante la variable XUI_PORT** – Se añade la variable de entorno XUI_PORT, que establece el puerto del panel web solo durante la ejecución del proceso actual, sin modificar el valor guardado webPort en la base de datos. Se admiten valores de 1 a 65535; un valor vacío, incorrecto o fuera del rango se ignora (se usa webPort) con una advertencia en el registro. Al usar Docker con red bridge, el puerto publicado del contenedor debe coincidir con XUI_PORT, por ejemplo XUI_PORT=8080 y ports: «8080:8080».
- **CLI: los flags -webCert/-webCertKey ahora se aplican en el subcomando setting** – En la CLI, los flags -webCert y -webCertKey ahora también funcionan en el subcomando x-ui setting (antes se ignoraban silenciosamente y el panel quedaba sin HTTPS). Al especificarlos, se puede definir directamente la ruta al certificado y clave del panel web sin necesidad de llamar a un subcomando separado cert.
- **El nombre del archivo de copia de seguridad de la BD se forma a partir de la dirección del servidor** – Los archivos de copia de seguridad de la base de datos ahora se nombran según la dirección del servidor, y no con el nombre fijo x-ui.db / x-ui.dump. Al descargarse desde el navegador, el nombre se toma de la dirección del panel en la barra de direcciones; en su defecto, del dominio web configurado, y si no hay ninguno, de la IP pública (primero IPv4, luego IPv6). Así los backups de distintos servidores se distinguen fácilmente. La extensión sigue siendo .db para SQLite y .dump para PostgreSQL.
- **Soporte para instalación y actualización en hosts solo con IPv6** – Los scripts de instalación y actualización ahora funcionan correctamente en servidores solo con IPv6: la descarga del release y los archivos

auxiliares ya no fuerza el uso de IPv4, por lo que el panel puede instalarse y actualizarse en un host sin dirección IPv4.

1. Introducción, requisitos e instalación

1.1. Qué es 3X-UI

3X-UI es un panel de administración web de código abierto para servidores [Xray-core](#). El panel ofrece una interfaz web multilingüe unificada para desplegar, configurar y monitorizar un amplio conjunto de protocolos proxy y VPN: desde un único VPS hasta configuraciones distribuidas con varios nodos.

3X-UI es un fork ampliado del proyecto original X-UI. En comparación con él, se ha añadido soporte para más protocolos, mayor estabilidad, contabilidad de tráfico por cliente y numerosas funciones de comodidad.

Funciones principales:

- **Inbound de distintos protocolos** – VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN y **MTProto** (proxy de Telegram, añadido en 3.3.0).
- **Transportes modernos y cifrado** – TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade y XHTTP, protegidos con TLS, XTLS y REALITY.
- **Fallback** – atención de varios protocolos en un mismo puerto (por ejemplo, VLESS y Trojan en el 443) mediante fallback en Xray.
- **Gestión por cliente** – cuotas de tráfico, fechas de expiración, límites de IP, indicador de estado «en línea», enlaces de invitación con un clic, códigos QR y suscripciones.
- **Estadísticas de tráfico** – por cada inbound, cliente y outbound, con posibilidad de restablecimiento.
- **Soporte de varios nodos** – administración y escalado a múltiples servidores desde un único panel.
- **Outbound y enrutamiento** – WARP, NordVPN, reglas de enrutamiento personalizadas, balanceadores de carga, cadenas de proxies.
- **Servidor de suscripciones integrado** con varios formatos de salida.
- **Bot de Telegram** para monitorización y administración remota.
- **REST API** con documentación Swagger integrada.
- **Almacenamiento flexible** – SQLite (por defecto) o PostgreSQL.
- **13 idiomas de interfaz**, temas oscuro y claro.
- **Integración con Fail2ban** para aplicar límites de IP por cliente.

Importante: el proyecto está destinado únicamente al uso personal. No se recomienda utilizarlo con fines ilegales ni en entornos de producción.

1.2. Sistemas operativos y arquitecturas compatibles

Sistemas operativos

El script de instalación detecta la distribución a partir del campo `ID` de `/etc/os-release` (o `/usr/lib/os-release`). Se admiten oficialmente:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine y Windows.

En sistemas de la familia Alpine se utiliza el servicio OpenRC (`rc-service` / `rc-update`); en el resto, systemd. Para CentOS 7 los paquetes se instalan con `yum` ; en versiones más recientes, con `dnf` . Si la distribución no se reconoce, el script intenta utilizar por defecto el gestor de paquetes `apt-get` .

Arquitecturas de procesador

La arquitectura se detecta a partir de la salida de `uname -m` y se normaliza a uno de los valores admitidos:

Valor de <code>uname -m</code>	Arquitectura en 3X-UI
<code>x86_64</code> , <code>x64</code> , <code>amd64</code>	<code>amd64</code>
<code>i*86</code> , <code>x86</code>	<code>386</code>
<code>armv8*</code> , <code>arm64</code> , <code>aarch64</code>	<code>arm64</code>
<code>armv7*</code> , <code>arm</code>	<code>armv7</code>
<code>armv6*</code>	<code>armv6</code>
<code>armv5*</code>	<code>armv5</code>
<code>s390x</code>	<code>s390x</code>

Si la arquitectura no figura en esta lista, el script muestra «Unsupported CPU architecture!» y cancela la instalación.

Dependencias básicas

Antes de instalar el panel, el script instala automáticamente un conjunto básico de paquetes (los nombres varían según la distribución): `cron` / `cronie` / `dcron` , `curl` , `tar` , `tzdata` / `timezone` , `socat` , `ca-certificates` , `openssl` .

1.3. Métodos de instalación

Método 1. Script de instalación (recomendado)

La instalación se realiza con un único comando como root:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

El script requiere obligatoriamente privilegios de root: si se ejecuta sin ellos, muestra «Please run this script with root privilege» y termina con error.

Lo que hace el instalador paso a paso:

1. Detecta el sistema operativo y la arquitectura.
2. Instala las dependencias básicas.
3. Descarga el archivo del release `x-ui-linux-<arch>.tar.gz` y lo descomprime en el directorio `/usr/local/x-ui` .
4. Descarga el script de administración `x-ui.sh` y lo instala como el comando `/usr/bin/x-ui` .

5. Crea el directorio de logs `/var/log/x-ui`.
6. Ejecuta la configuración inicial: selección de base de datos, generación de credenciales, elección de puerto y configuración opcional de SSL.
7. Instala e inicia el servicio de arranque automático (unidad `systemd` `x-ui.service` o script `init` OpenRC para Alpine).

Selección de base de datos durante la instalación. El instalador ofrece:

- 1) SQLite (por defecto, recomendado con menos de 500 clientes) – un único archivo `/etc/x-ui/x-ui.db`, sin configuración adicional.
- 2) PostgreSQL (recomendado con gran número de clientes o varios nodos). PostgreSQL puede instalarse localmente (se crea un usuario y base de datos dedicados con el nombre `xui`) o se puede indicar el DSN de un servidor ya existente. Los parámetros de conexión se escriben en el archivo de entorno del servicio (`/etc/default/x-ui`, `/etc/conf.d/x-ui` o `/etc/sysconfig/x-ui` según la distribución) como las variables `XUI_DB_TYPE` y `XUI_DB_DSN`.

Ejemplo: escritura de parámetros PostgreSQL en el archivo de entorno del servicio. Tras seleccionar PostgreSQL e indicar el DSN, el instalador añadirá al archivo de entorno líneas similares a estas:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Aquí `xui` es el nombre del usuario y la base de datos, `127.0.0.1:5432` es la dirección y el puerto del servidor, y `sslmode=disable` es adecuado para una conexión local (para un servidor remoto se suele usar `require`).

Instalación de una versión específica (antigua). Se puede indicar explícitamente una etiqueta de versión: el instalador descargará el release correspondiente:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

La versión mínima admitida para este tipo de instalación es `v2.3.5`; si se indica una más antigua, se muestra «Please use a newer version (at least v2.3.5)».

Método 2. Docker

Ejecución con la base de datos SQLite por defecto:

```
docker compose up -d
```

Para ejecutar con el servicio PostgreSQL integrado, hay que descomentar las líneas `XUI_DB_*` en `docker-compose.yml` e iniciar con el perfil:

```
docker compose --profile postgres up -d
```


La imagen incluye Fail2ban (activo por defecto) para aplicar límites de IP por cliente. Fail2ban bloquea a los infractores mediante `iptables`, lo que requiere la capacidad `NET_ADMIN`. En `docker-compose.yml` ya está concedida mediante `cap_add`. Al iniciar manualmente con `docker run`, las capacidades deben añadirse manualmente; de lo contrario, los bloqueos solo se registrarán en el log pero no se aplicarán:

Ejemplo: comando completo `docker run`. Variante mínima con exposición del puerto del panel, capacidades de red y volumen persistente para la base de datos:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

El volumen `/etc/x-ui` conserva el archivo `x-ui.db` entre reinicios del contenedor; sin él, la configuración y las cuentas se perderían.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

En Docker, el panel es el proceso principal del contenedor: el arranque automático se controla mediante la política de reinicio del contenedor (por ejemplo, `restart: unless-stopped`), no a través de un servicio interno al contenedor.

1.4. Primer inicio y credenciales por defecto

En la primera instalación (cuando aún se utilizan las credenciales por defecto), el instalador **genera valores aleatorios** para el nombre de usuario, la contraseña, la ruta web y el puerto:

Parámetro	Cómo se genera durante la instalación	Nota
Nombre de usuario (Username)	cadena aleatoria de 10 caracteres	se genera automáticamente
Contraseña (Password)	cadena aleatoria de 10 caracteres	se genera automáticamente
Ruta web del panel (WebBasePath)	cadena aleatoria de 18 caracteres	protege el panel de ser detectado por la URL raíz
Puerto del panel (Port)	por defecto, puerto aleatorio en el rango 1024–62000; puede establecerse manualmente si se desea	el valor «de fábrica» de <code>webPort</code> es <code>2053</code> , pero el instalador lo sobrescribe

Al final de la instalación, el script muestra un resumen: nombre de usuario, contraseña, puerto, ruta web, token de API y el enlace de acceso (Access URL) con el formato:

```
https://<dominio-o-IP>:<puerto>/<ruta-web>
```

Si no hay certificado SSL configurado, el enlace será por `http://` y el script mostrará un aviso sobre la necesidad de configurar SSL (opción de menú 19).

Cambio obligatorio de credenciales. Como el nombre de usuario y la contraseña se generan aleatoriamente, deben **guardarse inmediatamente tras la instalación**. Pueden cambiarse en cualquier momento mediante la opción de menú «Reset Username & Password» (véase más adelante) o desde la interfaz web en la configuración del panel. Tras el restablecimiento, el script recuerda: «Please use the new login username and password to access the X-UI panel. Also remember them!».

Tras la instalación, el comando `x-ui` abre el menú de administración (véase la sección 1.6).

1.5. Ubicación de archivos

Ruta	Descripción
<code>/usr/local/x-ui/</code>	directorio de instalación del panel (binario <code>x-ui</code> , script <code>x-ui.sh</code>)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	binario de Xray-core (en armv5/armv6/armv7 se renombra a <code>xray-linux-arm</code>)
<code>/usr/bin/x-ui</code>	script de administración (comando <code>x-ui</code>)
<code>/etc/x-ui/x-ui.db</code>	archivo de base de datos SQLite (por defecto)
<code>/var/log/x-ui/</code>	directorio de logs del panel
<code>/etc/systemd/system/x-ui.service</code>	unidad systemd del servicio (no en Alpine)
<code>/etc/init.d/x-ui</code>	script init de OpenRC (solo Alpine)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	archivo de variables de entorno del servicio (la ruta depende de la distribución); aquí se escriben <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code>

El directorio de la base de datos puede redefinirse con la variable de entorno `XUI_DB_FOLDER` (por defecto `/etc/x-ui`), y el directorio de los binarios de Xray con `XUI_BIN_FOLDER` (por defecto `bin` relativo al directorio del panel). El nombre del archivo de base de datos es `x-ui.db`.

Ejemplo: mover la base de datos a un disco separado. Para almacenar `x-ui.db` no en `/etc/x-ui` sino, por ejemplo, en un disco montado en `/data`, defina la variable en el archivo de entorno del servicio y reinicie el panel:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

La ruta completa a la base de datos será `/data/x-ui/x-ui.db`.

Variables de entorno principales

Variable	Descripción	Por defecto
XUI_DB_TYPE	backend de BD: <code>sqlite</code> o <code>postgres</code>	<code>sqlite</code>
XUI_DB_DSN	cadena de conexión PostgreSQL (cuando <code>XUI_DB_TYPE=postgres</code>)	–
XUI_DB_FOLDER	directorio del archivo de BD SQLite	<code>/etc/x-ui</code>
XUI_INIT_WEB_BASE_PATH	URI inicial del panel web (solo en la primera inicialización)	<code>/</code>
XUI_DB_MAX_OPEN_CONNS	máximo de conexiones abiertas (pool de PostgreSQL)	–
XUI_DB_MAX_IDLE_CONNS	máximo de conexiones inactivas (pool de PostgreSQL)	–
XUI_ENABLE_FAIL2BAN	activar la aplicación de límites de IP mediante Fail2ban	<code>true</code>
XUI_LOG_LEVEL	nivel de registro (<code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code>)	<code>info</code>
XUI_DEBUG	modo de depuración	<code>false</code>

Ejemplo: activar el registro detallado temporalmente. Para diagnosticar un problema, eleve el nivel de logs a `debug` y reinicie el servicio:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log      # ver el log de depuración
```

Tras el diagnóstico, restaure el valor `info` para que el log no crezca indefinidamente.

Ruta inicial del panel web mediante variable de entorno. La variable `XUI_INIT_WEB_BASE_PATH` establece el URI del panel web (`webBasePath`) durante la inicialización inicial de la configuración. Es útil al desplegar en Docker o mediante systemd para fijar desde el principio la ruta de acceso al panel. El valor se normaliza automáticamente: se añaden las barras inicial y final si es necesario, y un valor vacío o compuesto solo de espacios se ignora (en ese caso se aplica la ruta por defecto `/`). La variable afecta **únicamente a la inicialización inicial**: si la configuración ya existe, la ruta se cambia desde la interfaz web o mediante la opción de menú «Reset Web Base Path».

1.6. Comando de administración `x-ui` (menú del script)

Tras la instalación, el comando `x-ui` (ejecutado como root) abre el menú interactivo «3X-UI Panel Management Script». La selección de una opción se realiza introduciendo su número (rango 0–27). Muchas opciones también están disponibles como subcomandos para scripts (véase la sección 1.7).

El menú se divide en bloques temáticos.

Instalación y actualización

- **1. Install** – instalación del panel (ejecuta `install.sh`). Antes de instalar se comprueba que el panel no esté ya instalado.

- **2. Update** – actualización de todos los componentes de x-ui a la última versión. Los datos no se pierden; tras la actualización el panel se reinicia automáticamente. Requiere confirmación.
- **3. Update Menu** – actualización únicamente del script de administración (`x-ui.sh` / comando `x-ui`) a la versión actual, sin reinstalar el panel.
- **4. Legacy Version** – instalación de una versión anterior concreta del panel. El script solicita el número de versión (por ejemplo, `2.4.0`) y descarga el release correspondiente.
- **5. Uninstall** – eliminación completa del panel **junto con Xray**. Se detiene y deshabilita el servicio, se eliminan los directorios `/etc/x-ui/` y `/usr/local/x-ui/`, el archivo de entorno del servicio y el propio script de administración. Requiere confirmación (por defecto «no»).

Credenciales y configuración

- **6. Reset Username & Password** – restablecimiento del nombre de usuario y la contraseña del panel. Se pueden introducir valores propios o dejarlos vacíos para generación aleatoria (nombre aleatorio de 10 caracteres, contraseña aleatoria de 18 caracteres). Adicionalmente, se ofrece la posibilidad de desactivar la autenticación de dos factores (2FA) si está configurada. Tras el restablecimiento, el panel se reinicia.
- **7. Reset Web Base Path** – restablecimiento de la ruta web del panel: se genera una nueva ruta aleatoria (18 caracteres) y el panel se reinicia. Se utiliza cuando la ruta anterior ha sido comprometida u olvidada.
- **8. Reset Settings** – restablecimiento de todos los ajustes del panel a los valores por defecto. **Las credenciales (nombre de usuario y contraseña) y los datos de las cuentas no se pierden.** Requiere confirmación; tras el restablecimiento el panel se reinicia.
- **9. Change Port** – cambio del puerto del panel web. Se solicita el número de puerto (1–65535); tras establecerlo es necesario reiniciar para que el puerto entre en vigor.
- **10. View Current Settings** – visualización de la configuración actual (`x-ui setting -show`). Muestra, entre otras cosas, el backend de BD en uso (SQLite o PostgreSQL con la contraseña enmascarada en el DSN) y el enlace de acceso (Access URL). Si SSL no está configurado, ofrece la posibilidad de emitir un certificado Let's Encrypt para la dirección IP.

Gestión del servicio

- **11. Start** – inicio del servicio del panel. Si el panel ya está en ejecución, se muestra un mensaje indicando que no es necesario volver a iniciarlo.
- **12. Stop** – detención del servicio del panel.
- **13. Restart** – reinicio del servicio del panel.
- **14. Restart Xray** – reinicio únicamente del núcleo Xray-core sin reiniciar el propio panel (mediante `systemctl reload x-ui` ; en Docker, con la señal `USR1` al proceso del panel).
- **15. Check Status** – comprobación del estado del servicio (`systemctl status x-ui` o `rc-service x-ui status`).
- **16. Logs Management** – gestión de logs: visualización del log de depuración (Debug Log, mediante `journalctl`) y, excepto en Alpine, borrado de todos los logs (Clear All logs).

Arranque automático

- **17. Enable Autostart** – activar el arranque automático del panel al iniciar el sistema operativo (`systemctl enable x-ui` o `rc-update add`).
- **18. Disable Autostart** – desactivar el arranque automático al iniciar el sistema operativo.

En Docker, el arranque automático se controla mediante la política de reinicio del contenedor, por lo que estas opciones solo muestran el aviso correspondiente.

Seguridad y red

- **19. SSL Certificate Management** – gestión de certificados SSL mediante acme.sh: emisión de certificado para un dominio, revocación, renovación forzada, visualización de los dominios existentes, indicación de rutas al certificado para el panel, y emisión de un certificado de corta duración (~6 días, con renovación automática) para una dirección IP.
- **20. Cloudflare SSL Certificate** – emisión de certificado SSL mediante validación DNS de Cloudflare.
- **21. IP Limit Management** – gestión de los límites de número de IP por cliente (basada en Fail2ban): visualización y eliminación de bloqueos, etc.
- **22. Firewall Management** – gestión del cortafuegos (apertura/cierre de puertos y visualización de reglas).
- **23. SSH Port Forwarding Management** – configuración del reenvío de puertos SSH para acceder al panel desde la máquina local a través de un túnel SSH.

Rendimiento y mantenimiento

- **24. Enable BBR** – activación/desactivación del algoritmo de control de congestión TCP BBR (submenú con las opciones Enable BBR / Disable BBR).
- **25. Update Geo Files** – actualización de las bases de datos geo (archivos `.dat`) con selección de fuente: Loyalsoldier (`geoip.dat`, `geosite.dat`), chocolate4u (`geoip_IR.dat`, `geosite_IR.dat`), runetfreedom (`geoip_RU.dat`, `geosite_RU.dat`) o All (todas a la vez). Tras la actualización, el panel se reinicia.
- **26. Speedtest by Ookla** – ejecución de un test de velocidad de red con Speedtest by Ookla.
- **27. PostgreSQL Management** – gestión de la instancia PostgreSQL integrada/asociada (activación y operaciones relacionadas).
- **0. Exit Script** – salir del menú.

1.7. Subcomandos de `x-ui` (sin menú interactivo)

Para su uso en scripts, el comando `x-ui` admite subcomandos directos (ejecutar `x-ui` sin argumentos abre el menú):

Comando	Acción
<code>x-ui</code>	abrir el menú de administración
<code>x-ui start</code>	iniciar el panel
<code>x-ui stop</code>	detener el panel
<code>x-ui restart</code>	reiniciar el panel
<code>x-ui restart-xray</code>	reiniciar Xray
<code>x-ui status</code>	estado actual del servicio
<code>x-ui settings</code>	configuración actual
<code>x-ui enable</code>	activar el arranque automático al iniciar el SO

Comando	Acción
<code>x-ui disable</code>	desactivar el arranque automático
<code>x-ui log</code>	ver logs
<code>x-ui banlog</code>	ver logs de bloqueos de Fail2ban
<code>x-ui update</code>	actualizar el panel
<code>x-ui update-all-geofiles</code>	actualizar todos los archivos geo
<code>x-ui migrateDB [file]</code>	conversión <code>.db</code> <code>[?]</code> <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	instalar una versión antigua
<code>x-ui install</code>	instalar el panel
<code>x-ui uninstall</code>	desinstalar el panel

1.8. Migración de SQLite a PostgreSQL

Una instalación existente con SQLite puede migrarse a PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# luego establecer XUI_DB_TYPE y XUI_DB_DSN en /etc/default/x-ui y reiniciar:
systemctl restart x-ui
```

El archivo SQLite original permanece intacto – elimínalo manualmente solo después de verificar el correcto funcionamiento del nuevo backend.

Ejemplo: verificación del cambio a PostgreSQL. Tras la migración, compruebe que el panel realmente funciona con el nuevo backend mediante el comando de visualización de configuración – en la salida debe aparecer PostgreSQL (la contraseña en el DSN se enmascara):

```
x-ui settings | grep -i -E 'db|dsn'
```

Si el panel se abre y las cuentas están en su lugar, el archivo `x-ui.db` original puede eliminarse.

2. Acceso al panel y seguridad

Este apartado describe todo lo relativo a la autenticación del administrador del panel 3X-UI: el formulario de inicio de sesión, la autenticación de doble factor (TOTP), la protección contra fuerza bruta, el cambio de credenciales, la modificación de la ruta secreta y el puerto del panel, el tiempo de vida de la sesión, así como la sincronización y autenticación mediante LDAP.

2.1. Formulario de inicio de sesión

La página de inicio de sesión se sirve en la raíz de la ruta secreta del panel (`webBasePath`). Si el usuario ya está autenticado, es redirigido automáticamente a `.../panel/` . La página incluye un selector de tema, un selector de idioma de interfaz y el propio formulario.

Campos del formulario:

Campo	Etiqueta/ encabezado	Obligatorio	Descripción
Nombre de usuario	«Nombre de usuario»	Sí	Nombre de usuario del administrador. Un valor vacío se rechaza en el cliente y, en el servidor, con el mensaje «Introduzca el nombre de usuario».
Contraseña	«Contraseña»	Sí	Contraseña del administrador. Un valor vacío se rechaza con el mensaje «Introduzca la contraseña».
Código 2FA	«Código 2FA»	Solo si 2FA está activado	El campo aparece únicamente si la autenticación de doble factor está habilitada en el panel. Código de 6 dígitos generado por la aplicación autenticadora.

El botón «**Iniciar sesión**» envía el formulario a `POST /login` .

Comportamiento y mensajes:

- Tras un inicio de sesión exitoso se muestra «Sesión iniciada correctamente» y se produce la redirección a `.../panel/` .
- Ante cualquier error de credenciales o código 2FA incorrecto, el servidor devuelve un mensaje **unificado**: «Datos de cuenta incorrectos.» (en inglés: *Invalid username or password or two-factor code*). Esto es intencional: el panel no indica qué dato es incorrecto (usuario, contraseña o código) para no facilitar los ataques de fuerza bruta.
- El campo «Código 2FA» se muestra u oculta en función de la respuesta a `POST /getTwoFactorEnable` , que devuelve el estado actual de 2FA incluso antes de que el usuario se autentique.
- Si la sesión del servidor ha expirado, el siguiente request muestra «La sesión ha expirado. Vuelva a iniciar sesión» y el usuario es redirigido a la página de inicio de sesión.

Nota sobre CSRF: antes de enviar el formulario, el cliente obtiene un token CSRF (`GET /csrf-token`); las rutas `/login` y `/logout` están protegidas con verificación CSRF.

Ejemplo: inicio de sesión a través de la API. Cuando 2FA está desactivado basta con el nombre de usuario y la contraseña; si 2FA está activado se añade el campo `twoFactorCode` :

```
# Sin 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# Con 2FA activado – se añade el código de 6 dígitos
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

Si la operación tiene éxito, el servidor devolverá `Set-Cookie` con la cookie de sesión, que deberá enviarse en las peticiones posteriores a `/panel/api/...`.

2.2. Autenticación de doble factor (2FA / TOTP)

La 2FA en 3X-UI está implementada según el estándar **TOTP** y es compatible con cualquier aplicación autenticadora (Google Authenticator, Aegis, FreeOTP, etc.). Los parámetros están fijados en el código: algoritmo **SHA1**, 6 dígitos, período de **30** segundos, emisor (issuer) `3x-ui`, etiqueta `Administrator`.

Ejemplo: URI otpauth codificada en el código QR. Si la aplicación autenticadora no puede leer la cámara, el token puede añadirse manualmente con el siguiente enlace (sustituya su secreto en Base32 por `JBSWY3DPEHPK3PXP`):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

Los parámetros `algorithm=SHA1`, `digits=6`, `period=30` corresponden a los valores fijos del panel y no deben modificarse.

La configuración se encuentra en **Configuración** → **Cuenta de usuario**, pestaña «**Autenticación de doble factor**».

Elemento	Texto	Descripción
Interruptor	«Activar 2FA»	Activa o desactiva la autenticación de doble factor.
Descripción	«Añade un nivel adicional de autenticación para mayor seguridad.»	Texto de ayuda bajo el interruptor.

Cómo activar 2FA

Al activar el interruptor, el panel **genera localmente un nuevo secreto**: una cadena aleatoria en codificación Base32 (alfabeto A–Z y 2–7). Se abre la ventana «Activar autenticación de doble factor» con instrucciones paso a paso:

1. **«Escanee este código QR en la aplicación autenticadora o copie el token que aparece junto al código QR e introdúzcalo en la aplicación».** Bajo el código QR se muestra el secreto en texto plano; al hacer clic en el QR se copia al portapapeles (aparece «Copiado»).
2. **«Introduzca el código de la aplicación»:** hay que introducir el código de 6 dígitos generado por la aplicación. El código se verifica **en el navegador**: el panel calcula el TOTP actual con el secreto recién generado y lo compara con el introducido. Si es incorrecto, muestra «Código incorrecto»; el campo solo acepta exactamente 6 dígitos.

Solo tras la confirmación exitosa se guardan el secreto y el indicador de activación. Al guardar se muestra «La autenticación de doble factor se ha configurado correctamente».

Importante: los cambios en la sección de configuración se aplican con el botón general **«Guardar»**, tras lo cual normalmente es necesario reiniciar el panel («Guarde los cambios y reinicie el panel para que surtan efecto»). Al activar 2FA por primera vez, el servidor adicionalmente **invalida todas las sesiones activas** (incrementa el «login epoch»), por lo que después de aplicar la configuración será necesario volver a iniciar sesión, esta vez con el código 2FA.

Cómo desactivar 2FA

Al volver a pulsar el interruptor se abre la ventana «Desactivar autenticación de doble factor» con el texto «Introduzca el código de la aplicación para desactivar la autenticación de doble factor.». Tras introducir el código correcto, el indicador y el secreto se borran, y se muestra «La autenticación de doble factor se ha eliminado correctamente».

Verificación del código al iniciar sesión

Al iniciar sesión, el servidor toma el secreto almacenado y compara el TOTP actual con el código 2FA enviado. Si no coinciden, el intento se considera fallido, pero al usuario se le muestra el mismo mensaje unificado «Datos de cuenta incorrectos.».

Recuperación de acceso (recovery)

3X-UI **no** dispone de un mecanismo de «códigos de recuperación». Si se pierde el acceso a la aplicación autenticadora, no es posible recuperar el inicio de sesión desde la interfaz del panel. La única vía es desactivar 2FA directamente en la base de datos del servidor: restablecer la clave `twoFactorEnable` a `false` (y si es necesario borrar `twoFactorToken`) en la tabla de configuración y reiniciar el panel. Por eso se recomienda guardar el secreto (token Base32) en un lugar seguro al activar 2FA.

Ejemplo: desactivación de emergencia de 2FA en el servidor. Con acceso SSH al servidor, detenga el panel, restablezca las claves en la tabla de configuración e inícielo de nuevo:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

Después de esto, el acceso se realiza solo con nombre de usuario y contraseña, y si se desea puede configurarse 2FA de nuevo.

Relación con el cambio de credenciales: al cambiar el nombre de usuario o la contraseña (véase 2.4), 2FA **se desactiva automáticamente** en el servidor para que el antiguo secreto no bloquee el acceso con la nueva cuenta.

2.3. Limitación de intentos de inicio de sesión (login limiter / protección contra fuerza bruta)

El panel incluye un limitador de intentos de inicio de sesión fallidos integrado (equivalente a fail2ban a nivel de aplicación). Los parámetros están fijados en el código y **no son configurables** desde la interfaz:

Parámetro	Valor	Función
Máximo de fallos	5	Número de intentos fallidos permitidos dentro de la ventana.
Ventana de conteo	5 minutos	Ventana deslizante en la que se acumulan los fallos (los más antiguos se descartan).
Bloqueo (cooldown)	15 minutos	Tiempo durante el que la clave queda bloqueada tras superar el umbral.

Funcionamiento:

- La clave de bloqueo se construye a partir de la **combinación «IP + nombre de usuario»** (el nombre de usuario se convierte a minúsculas y se eliminan los espacios). Es decir, el bloqueo se aplica al par concreto «dirección + nombre de usuario», no al panel en su conjunto.
- Con cada intento fallido (usuario/contraseña incorrectos o código 2FA incorrecto) el contador aumenta. Al alcanzar **5 fallos en 5 minutos**, la clave queda bloqueada durante **15 minutos**. Durante el bloqueo, cualquier intento de ese par se rechaza inmediatamente con el mismo mensaje «Datos de cuenta incorrectos.», aunque las credenciales sean correctas.
- **Un inicio de sesión exitoso restablece de inmediato** el contador y elimina el bloqueo para ese par.
- La dirección IP del cliente se determina teniendo en cuenta los proxies de confianza (véase `trustedProxyCIDRs`): las cabeceras `X-Real-IP` y `X-Forwarded-For` solo se aceptan si la petición proviene de una dirección de confianza. De lo contrario se usa la dirección real de la conexión y, si no puede obtenerse, la cadena `unknown`.

Todos los intentos se registran en el log. Los intentos fallidos generan una advertencia en el log del servidor con el nombre de usuario, la IP, el motivo y, en caso de bloqueo, el tiempo `blocked_until`. Si las notificaciones de inicio de sesión están habilitadas a través del bot de Telegram (`tgNotifyLogin` – «Notificación de inicio de sesión»), el administrador recibe adicionalmente el nombre de usuario, la IP y la hora de cada intento: exitoso, fallido o bloqueado.

Ejemplo: notificación de inicio de sesión en Telegram. Con `tgNotifyLogin` activado, tras cada intento el administrador recibe un mensaje similar al siguiente:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Para el par «IP + nombre de usuario» bloqueado, el estado indicará que el intento fue rechazado por el limitador.

2.4. Cambio de nombre de usuario y contraseña del administrador

Sección **Configuración** → **Cuenta de usuario**, pestaña «**Credenciales de administrador**». Campos:

Campo	Texto	Descripción
Usuario actual	«Usuario actual»	Nombre de usuario en uso. Debe coincidir con el nombre de usuario actual; de lo contrario, el cambio se rechaza.
Contraseña actual	«Contraseña actual»	Contraseña en uso para verificar la identidad.
Nuevo usuario	«Nuevo usuario»	Nuevo nombre de usuario. No puede estar vacío.
Nueva contraseña	«Nueva contraseña»	Nueva contraseña. No puede estar vacía.

El cambio se aplica con el botón «**Confirmar**» y se envía a `POST /panel/setting/updateUser`.

Lógica y mensajes del servidor:

- Si «Usuario actual» no coincide con el real o «Contraseña actual» es incorrecta: «Se produjo un error al cambiar las credenciales del administrador.» con la aclaración «Nombre de usuario o contraseña incorrectos».
- Si el nuevo nombre de usuario o la nueva contraseña están vacíos: «El nuevo nombre de usuario y la nueva contraseña deben estar rellenos».
- Si la operación es exitosa: «Ha cambiado correctamente las credenciales del administrador.». La contraseña se almacena como hash bcrypt.

Ejemplo: cambio de credenciales a través de la API. La petición requiere una cookie de sesión válida (obtenida al iniciar sesión) y la confirmación del nombre de usuario y contraseña actuales:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=ВАША_СЕССИОННАЯ_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

Tras el éxito, la sesión actual se invalida y será necesario volver a iniciar sesión con las nuevas credenciales.

Efectos importantes del cambio de credenciales:

- **Todas las sesiones existentes se invalidan** (se incrementa el contador `login_epoch` del usuario), por lo que tras el cambio el panel cierra la sesión automáticamente y redirige a la página de inicio de sesión: habrá que volver a autenticarse.
- Si en el momento del cambio **2FA estaba activado, se desactiva automáticamente** (el indicador y el secreto se borran). Será necesario configurar la autenticación de doble factor de nuevo tras el cambio de nombre de usuario y contraseña.

Si 2FA está activado, antes de enviar el formulario se abre la ventana «Cambiar credenciales» con el texto «Introduzca el código de la aplicación para cambiar las credenciales del administrador.»: solo es posible cambiar las credenciales tras confirmar el código 2FA actual.

2.5. Ruta secreta (ruta URI / webBasePath) y puerto del panel

Estos parámetros se encuentran en **Configuración** → **Panel** y afectan directamente a la «visibilidad» y accesibilidad del panel. Se aplican tras guardar y **reiniciar el panel**.

Campo	Texto	Valor por defecto	Descripción
Puerto del panel	«Puerto del panel» (<code>panelPort</code>), ayuda «Puerto en el que opera el panel»	2053	Puerto TCP de la interfaz web.
URI-путь	«URI-путь» (<code>panelUrlPath</code>), ayuda «Debe comenzar con '/' y terminar con '/'»	/	Ruta base secreta (<code>webBasePath</code>). El panel solo es accesible a través de ella (por ejemplo, <code>/mi-secreto/</code>).
Dirección IP para administrar el panel	«Dirección IP para administrar el panel» (<code>panelListeningIP</code>), ayuda «Déjelo vacío para permitir conexiones desde cualquier IP»	vacío	Dirección en la que escucha el panel. Vacío = todas las interfaces.
Dominio del panel	«Dominio del panel» (<code>panelListeningDomain</code>), ayuda «Déjelo vacío para permitir conexiones desde cualquier dominio e IP.»	vacío	Restricción de acceso por dominio (Host).
Ruta al certificado público del panel	<code>publicKeyPath</code> , ayuda «Introduzca la ruta completa comenzando con '/'»	vacío	Certificado TLS para el acceso HTTPS al panel.
Ruta a la clave privada del certificado del panel	<code>privateKeyPath</code> , misma ayuda	vacío	Clave privada TLS.

Comportamiento de la ruta base (`webBasePath`):

- El valor se normaliza automáticamente: si no comienza con `/` , el carácter se añade al inicio; si no termina con `/` , se añade al final. Es decir, la ruta siempre tiene la forma `/.../` .
- La ruta base se aplica al propio panel, a los assets y a la cookie de sesión (la cookie solo se emite para esta ruta).

Recomendaciones de seguridad (sección «Avisos de seguridad»): el panel muestra avisos si la configuración es «demasiado pública»:

- «El panel opera sobre HTTP sin cifrar – configure TLS para producción.»
- «El puerto estándar 2053 es ampliamente conocido – cámbielo por uno aleatorio.»
- «La ruta base por defecto "/" es ampliamente conocida – cámbiela por una aleatoria.»

En otras palabras, para un servidor en producción conviene definir un **puerto no estándar**, una **ruta URI no trivial** y un **certificado TLS**.

Ejemplo: configuración «discreta» del panel para producción. En **Configuración** → **Panel** introduzca valores similares a los siguientes:

Campo	Valor
Puerto del panel	34571 (aleatorio, en lugar de 2053)
URI-путь	/aXf9Qm2/ (no trivial, comienza y termina con /)
Ruta al certificado público del panel	/etc/letsencrypt/live/panel.example.com/fullchain.pem
Ruta a la clave privada del certificado del panel	/etc/letsencrypt/live/panel.example.com/privkey.pem

Tras guardar y reiniciar, el panel solo será accesible en `https://panel.example.com:34571/aXf9Qm2/` y los avisos de seguridad desaparecerán.

2.6. Tiempo de vida de la sesión (timeout)

El campo «**Duración de la sesión**» (`sessionMaxAge`) se encuentra entre los ajustes del panel e intervalos.

Campo	Texto	Valor por defecto	Unidad	Descripción
Duración de la sesión	«Duración de la sesión», ayuda «Duración de la sesión en el sistema (valor: minuto)»	360	minutos	Tiempo de vida de la cookie de sesión del administrador.

Comportamiento:

- El valor se especifica en **minutos** (por defecto 360 minutos = 6 horas) y se convierte a segundos al configurar la cookie.
- Si el valor es **mayor que 0**, la cookie de sesión recibe el `MaxAge` correspondiente. Una vez transcurrido ese tiempo, la cookie deja de ser válida y en la siguiente petición el usuario recibe «La sesión ha expirado. Vuelva a iniciar sesión».
- La sesión también se invalida antes de tiempo al cambiar las credenciales o al activar 2FA por primera vez (mediante el mecanismo `login_epoch`, véase 2.4 y 2.2) y al cerrar sesión explícitamente (`POST /logout`).
- La cookie de sesión se marca como `HttpOnly`, con política `SameSite=Lax`; el indicador `Secure` se activa cuando el acceso al panel es directamente por HTTPS.

Además del propio timeout existe una notificación relacionada: **«Retraso de notificación de expiración de sesión»** (`expireTimeDiff` , ayuda «Recibir una notificación sobre la expiración de la sesión antes de alcanzar el valor umbral (valor: día)», por defecto `0`) – permite recibir un aviso con antelación.

2.7. LDAP (sincronización y autenticación)

La sección LDAP ofrece dos posibilidades: (1) autenticar el inicio de sesión del administrador mediante LDAP si la contraseña local no coincide, y (2) sincronizar periódicamente el estado de los clientes (indicador VLESS activado/desactivado) desde el directorio.

Uso en el inicio de sesión: el servidor comprueba primero el hash bcrypt local de la contraseña. Si **no coincide** y LDAP está activado, el panel intenta autenticar al usuario en el directorio: si se ha definido un `Bind DN` , se realiza un bind de servicio y se busca la entrada del usuario con el filtro y el atributo indicados; a continuación se intenta un bind con el DN encontrado y la contraseña introducida. Si tiene éxito, se concede el acceso. (Tras una autenticación LDAP exitosa, si 2FA está activado se comprueba igualmente el código TOTP.)

Campos de la sección:

Campo	Texto	Valor por defecto	Descripción
Activar sincronización LDAP	«Activar sincronización LDAP» (<code>enable</code>)	false	Interruptor principal de la integración LDAP.
Host LDAP	«Host LDAP» (<code>host</code>)	vacío	Dirección del servidor LDAP.
Puerto LDAP	«Puerto LDAP» (<code>port</code>)	389	Puerto. Para LDAPS normalmente 636.
Usar TLS (LDAPS)	«Usar TLS (LDAPS)» (<code>useTls</code>)	false	Al activarlo se usa el esquema <code>ldaps://</code> con verificación del certificado del servidor (sin omitir la verificación).
Bind DN	«Bind DN» (<code>bindDn</code>)	vacío	DN de la cuenta de servicio para el bind/búsqueda inicial. Si está vacío, no se realiza bind (búsqueda anónima).
Contraseña de bind	ayudas: «Configurado; déjelo vacío para conservar la contraseña actual.» / «No configurado.» / «Configurado – introduzca un nuevo valor para sustituirlo»	vacío	Contraseña para <code>Bind DN</code> . Se almacena por separado; para conservar la anterior, se deja el campo vacío.
Base DN	«Base DN» (<code>baseDn</code>)	vacío	Raíz del subárbol en el que se realiza la búsqueda (búsqueda recursiva en todo el subárbol).
Filtro de usuario	«Filtro de usuario» (<code>userFilter</code>)	(<code>objectClass=person</code>)	Filtro LDAP para seleccionar cuentas. Durante la autenticación, el nombre de usuario se sustituye en el filtro con escape.

Campo	Texto	Valor por defecto	Descripción
Atributo de usuario (username/email)	«Atributo de usuario (username/email)» (<code>userAttr</code>)	<code>mail</code>	Atributo que se compara con el nombre de usuario/ identificador del cliente (por ejemplo, <code>mail</code> o <code>uid</code>).
Atributo del indicador VLESS	«Atributo del indicador VLESS» (<code>vlessField</code>)	<code>vless_enabled</code>	Atributo que determina si el acceso VLESS del cliente debe estar activado.
Atributo de indicador general (opc.)	«Atributo de indicador general (opc.)» (<code>flagField</code>), ayuda «Si se define, reemplaza al indicador VLESS – p. ej. <code>shadowInactive</code> .»	vacío	Si se define, se usa en lugar de <code>vless_enabled</code> .
Valores verdaderos	«Valores verdaderos» (<code>truthyValues</code>), ayuda «Separados por coma; por defecto: <code>true,1,yes,on</code> »	<code>true,1,yes,on</code>	Lista de valores del atributo de indicador que se interpretan como «activado».
Invertir indicador	«Invertir indicador» (<code>invertFlag</code>), ayuda «Actívalo cuando el atributo signifique «desactivado» (p. ej. <code>shadowInactive</code>).»	<code>false</code>	Invierte el significado del indicador.
Programación de sincronización	«Programación de sincronización» (<code>syncSchedule</code>), ayuda «Cadena tipo cron, p. ej. <code>@every 1m</code> »	<code>@every 1m</code>	Frecuencia de sincronización en formato similar a cron.
Etiquetas de inbounds	«Etiquetas de inbounds» (<code>inboundTags</code>), ayuda «Inbounds en los que la sincronización LDAP puede crear o eliminar clientes automáticamente.»	vacío	Limita en qué inbounds están permitidas las operaciones automáticas. Si no hay inbounds: «No se encontraron inbounds. Cree primero un inbound.»
Creación automática de clientes	«Creación automática de clientes» (<code>autoCreate</code>)	<code>false</code>	Crear un cliente en los inbounds indicados si aparece en el directorio.
Eliminación automática de clientes	«Eliminación automática de clientes» (<code>autoDelete</code>)	<code>false</code>	Eliminar un cliente si desaparece del directorio.
Volumen por defecto (GB)	«Volumen por defecto (GB)» (<code>defaultTotalGb</code>)	<code>0</code>	Límite de tráfico para los clientes creados automáticamente (0 = sin límite).
Plazo por defecto (días)	«Plazo por defecto (días)» (<code>defaultExpiryDays</code>)	<code>0</code>	Período de validez para los clientes creados automáticamente (0 = sin expiración).
Límite de IP por defecto	«Límite de IP por defecto» (<code>defaultIpLimit</code>)	<code>0</code>	Límite de IPs simultáneas (0 = sin límite).

Detalles de la lógica del indicador de sincronización: al leer el atributo de indicador (`flagField` , por defecto `vless_enabled`), el valor se considera «activado» si pertenece a la lista de valores verdaderos; si la inversión está habilitada, el resultado se invierte. El atributo de usuario (`userAttr`) se usa como clave de correspondencia (email/nombre); las entradas sin valor en ese atributo se omiten.

Seguridad: se recomienda activar **TLS (LDAPS)** para que las contraseñas de bind y las contraseñas verificadas no se transmitan en texto plano, y usar para Bind DN una cuenta con los permisos mínimos necesarios de lectura.

Ejemplo: configuración típica de sincronización LDAP (Active Directory). Valores de los campos para un directorio donde el estado de acceso se almacena en un atributo similar a `userAccountControl` y la correspondencia se realiza por correo electrónico:

Campo	Valor
Host LDAP	<code>ldap.example.com</code>
Puerto LDAP	<code>636</code>
Usar TLS (LDAPS)	activado
Bind DN	<code>CN=svc-3xui,OU=Service,DC=example,DC=com</code>
Base DN	<code>OU=Users,DC=example,DC=com</code>
Filtro de usuario	<code>(objectClass=person)</code>
Atributo de usuario (username/email)	<code>mail</code>
Atributo del indicador VLESS	<code>vless_enabled</code>
Valores verdaderos	<code>true,1,yes,on</code>
Programación de sincronización	<code>@every 5m</code>

Con esta configuración, cada 5 minutos el panel recorrerá el subárbol `OU=Users` , emparejará los clientes por `mail` y activará o desactivará el acceso VLESS según el valor de `vless_enabled` .

3. Resumen / Panel de control

El panel de control («Panel de control», en la interfaz en inglés — *Overview*) es la página de inicio del panel. Muestra en tiempo real el estado del servidor y del proceso Xray. Todos los indicadores provienen del lado del servidor. El planificador en segundo plano reconstruye el snapshot **cada 2 segundos** y lo distribuye a todas las pestañas abiertas mediante WebSocket; una vez por minuto, las filas de métricas acumuladas se guardan en disco. El endpoint HTTP `GET /status` devuelve el último snapshot almacenado en caché.

A continuación se describe cada indicador y cada elemento de control de la página.

3.1. Principios generales de recopilación de datos

- El snapshot se recopila con la biblioteca `gopsutil`. Si una medición concreta falla, el campo queda en cero y se escribe una advertencia en el registro (`get cpu percent failed`, `get uptime failed`, etc.) — esto no tumba todo el panel, simplemente el mosaico correspondiente mostrará 0/N-A.
- Las velocidades «instantáneas» (CPU %, red, I/O de disco) se calculan como la diferencia entre el snapshot actual y el anterior dividida por el intervalo en segundos. Por eso, en la primera carga de la página los valores de velocidad pueden ser cero hasta que se acumule una segunda medición.
- El historial puede consultarse en la sección «Historia del sistema» (*System History*) — los gráficos se construyen a partir de las mismas filas de datos descritas a continuación (véase el punto 3.12).

3.2. CPU

El mosaico «CPU» (*CPU*) muestra la carga actual del procesador en porcentaje, así como los parámetros del propio procesador.

Indicador	Descripción
Carga de CPU, %	Proporción del tiempo de procesador utilizado en el último intervalo. Se suaviza mediante una media exponencial (EMA, coeficiente <code>alpha = 0.3</code>) para que los picos no sacudan el indicador. El valor siempre se limita al rango 0–100 %. En la primera medición se devuelve 0 (inicialización del punto base).
Procesadores lógicos	Número de núcleos lógicos, es decir, con Hyper-Threading incluido.
Núcleos físicos	Número de núcleos físicos.
Frecuencia	Frecuencia base del procesador en MHz. Se solicita de forma diferida y se almacena en caché: la primera medición exitosa se guarda, el reintento se realiza como máximo una vez cada 5 minutos, y la propia solicitud tiene un tiempo límite de 1,5 s (en algunos sistemas la consulta de frecuencia responde lentamente).

El cálculo de la carga de CPU funciona así: si existe una implementación nativa de la plataforma, se utiliza esa; de lo contrario, se calcula a partir de las deltas de los contadores de tiempo de procesador (*busy / total*). Los tiempos *Guest* y *GuestNice* se excluyen para no contarlos dos veces.

3.3. Memoria (RAM)

El mosaico «Memoria» (*RAM*) muestra utilizada y total. Se muestra como «utilizada / total» y/o porcentaje de uso. En el historial se registra el porcentaje.

3.4. Swap

El mosaico «Swap» (*Swap*) muestra utilizado y total. Si no hay archivo/partición de swap configurado (total = 0), el indicador es cero; en la serie histórica, cuando no hay swap, se registra 0.

3.5. Disco (Storage)

El mosaico «Disco» (*Storage*) muestra utilizado y total, teniendo en cuenta **solo la partición raíz** /. En el historial «Uso del disco» (*Disk Usage*) se registra el porcentaje de ocupación. Por separado se recopila el I/O de disco (lectura / escritura, bytes/s) como delta de los contadores por intervalo — se muestra en la pestaña «Disco I/O» del historial.

3.6. Tiempo de actividad del sistema (Uptime)

El indicador «Tiempo de actividad del sistema» (*Uptime*). Es el tiempo desde el arranque **del servidor completo** (en segundos), no el tiempo de funcionamiento del panel o de Xray. Aparte se almacena el uptime del proceso Xray (véase el punto 3.9), así como el número de hilos del panel («Hilos» / *Threads*).

3.7. Carga del sistema (Load average)

El bloque «Carga del sistema» (*System Load*) — un array de tres números [Load1, Load5, Load15] . Descripción emergente: «Carga media del sistema en los últimos 1, 5 y 15 minutos» (*System load average for the past 1, 5, and 15 minutes*). El gráfico del historial se llama «Carga media del sistema (1 / 5 / 15 min)». En las series históricas los valores se escriben por separado: load1 , load5 , load15 .

Es el indicador Unix estándar: el número medio de procesos en cola de ejecución. Como referencia, hay que compararlo con el número de núcleos: una carga que supera de forma sostenida el número de núcleos físicos indica sobrecarga.

3.8. Red: velocidad y volumen total de tráfico

Solo se tienen en cuenta **las interfaces físicas**. Las interfaces virtuales y de túnel se excluyen: lo / lo0 , y todo lo que comienza por loopback , docker , br- , veth , virbr , tun , tap , wg , tailscale , zt . Los valores se suman en todas las interfaces restantes.

Velocidad global (*Overall Speed*) — velocidad instantánea, delta de contadores por intervalo:

Indicador	Descripción
Subida (etiqueta «Subida» / <i>Upload</i>)	Velocidad de salida, bytes/s.
Bajada (etiqueta «Bajada» / <i>Download</i>)	Velocidad de entrada, bytes/s.

Volumen total de tráfico (*Total Data*) – contadores acumulados desde el arranque del sistema:

Indicador	Descripción
Enviado (etiqueta «Enviado» / <i>Sent</i>)	Total de bytes enviados.
Recibido (etiqueta «Recibido» / <i>Received</i>)	Total de bytes recibidos.

Adicionalmente se recopilan velocidades de paquetes (paquetes/s) y contadores totales de paquetes – se muestran en la pestaña «Paquetes de red» (*Network Packets*) del historial. Series históricas de red: `netUp`, `netDown`, `pktUp`, `pktDown`.

3.9. Direcciones IP del servidor

El bloque «Direcciones IP del servidor» (*IP Addresses*) muestra `IPv4` e `IPv6`. Las direcciones externas se determinan mediante servicios de terceros (`api4.ipify.org`, `ipv4.icanhazip.com`, `v4.api.ipinfo.io/ip`, `ipv4.myexternalip.com/raw`, `4.ident.me`, `check-host.net/ip` para `IPv4` y equivalentes para `IPv6`). La lista se recorre en orden hasta la primera respuesta exitosa; el tiempo límite por solicitud es de 3 s.

Particularidades:

- El resultado se **almacena en caché** durante la vida útil del proceso: una dirección determinada con éxito no se vuelve a solicitar.
- Si ningún servicio responde, el campo queda como `N/A`. Para `IPv6`, al primer `N/A` las solicitudes `IPv6` se desactivan completamente para no perder tiempo en redes sin `IPv6`.
- Junto a las direcciones hay un botón «ojo» para ocultar/mostrar las direcciones – descripción emergente: «Ocultar o mostrar las direcciones IP del servidor» (*Toggle visibility of the IP*). Esto es solo ocultado visual en la interfaz (por ejemplo, para capturas de pantalla) y no afecta a las propias direcciones.

3.10. Conexiones TCP/UDP

El bloque «Estadísticas de conexiones» (*Connection Stats*) muestra el número total de conexiones TCP y UDP activas en el servidor (en todo el sistema, no solo Xray). El gráfico del historial es «Conexiones activas (TCP / UDP)» (*Active Connections*), series `tcpCount`, `udpCount`.

3.11. Estado de Xray y control del proceso

La tarjeta «Xray» muestra el estado del proceso Xray-core y permite controlarlo.

Estados

Valor	Etiqueta	Traducción	Cuándo se establece
<code>running</code>	«Ejecutándose»	<i>Running</i>	El proceso Xray está en ejecución.
<code>stop</code>	«Detenido»	<i>Stopped</i>	El proceso no está en ejecución y no hay ningún error de inicio registrado.

Valor	Etiqueta	Traducción	Cuándo se establece
error	«Error»	<i>Error</i>	El proceso no está en ejecución, pero se registró un error de inicio. El texto del error se muestra en una ventana emergente con el título «Error al ejecutar Xray» (<i>An error occurred while running Xray</i>).
–	«Desconocido»	<i>Unknown</i>	Se muestra mientras el estado aún no se ha recibido.

Junto al estado se muestra la **versión de Xray**.

Botones de control

- **Detener** (*Stop*). Llama a `POST /stopXrayService`. Si tiene éxito, el panel envía por WebSocket el nuevo estado `stop` y la notificación «Xray detenido correctamente» (*Xray service has been stopped*); si hay error, el estado `error` con el texto. Importante: si el panel es accesible *a través del propio Xray*, detener Xray puede interrumpir la conexión con el panel – si la conexión es directa al panel, no hay problema.
- **Reiniciar** (*Restart*). Llama a `POST /restartXrayService`. Antes de la acción se muestra una confirmación «¿Reiniciar xray?» con la explicación «Recarga el servicio xray con la configuración guardada». Si tiene éxito – estado `running` y notificación «Xray reiniciado correctamente» (*Xray service has been restarted successfully*). El reinicio aplica la configuración guardada actual – úselo después de cambiar los ajustes.

Nota. En este fork se ha añadido al panel de control un control completo Start / Stop / Restart para todos los tipos de autorización; en la interfaz original de 3x-ui no hay un botón «iniciar» independiente – el inicio se realiza mediante reinicio.

Botón de visualización de registros de Xray

En la tarjeta Xray hay un botón para ver los registros de Xray (*Logs*). Solo aparece cuando el access-log está configurado en la configuración de Xray: el visor integrado lee precisamente ese archivo, por lo que sin access-log el botón se oculta. La visibilidad del botón está vinculada a una propiedad separada `accessLogEnable` y ya no depende del límite de IP – la lista en línea y el límite de direcciones IP siguen funcionando incluso sin access-log (véase el punto 8).

Selección de versión de Xray

La sección «Selección de versión» (*Version*) permite cambiar Xray-core a otra versión. La lista de versiones se carga mediante `GET /getXrayVersion`:

- La fuente es la API de GitHub del repositorio `XTLS/Xray-core (/releases)`. Las solicitudes se almacenan en caché durante **15 minutos**; si GitHub falla, se devuelve la última lista obtenida con éxito para que el selector no quede vacío.
- Solo se incluyen en la lista las versiones con formato `X.Y.Z` y **no anteriores al 26.4.25**.

Descripciones emergentes: «Seleccione la versión deseada» (*Choose the version you want to switch to.*) y la advertencia «Importante: las versiones antiguas pueden no ser compatibles con la configuración actual» (*Choose carefully, as older versions may not be compatible with current configurations.*).

Cambio de versión: `POST /installXray/:version` . Escenario:

Ejemplo. Cambiar a una versión específica de Xray-core (la cookie de sesión ya debe haberse obtenido mediante autenticación):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Aquí `v25.6.8` es la etiqueta de la lista que devuelve `GET /getXrayVersion` . La versión debe estar presente en esa lista, de lo contrario el panel rechazará la solicitud.

1. La versión seleccionada se verifica en la lista actual de versiones (de lo contrario, se rechaza).
2. Xray se detiene.
3. Se descarga de GitHub el archivo `Xray-<os>-<arch>.zip` para el sistema operativo y arquitectura actuales (se soportan amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x; para Windows – `xray.exe`). El tamaño del archivo y del binario está limitado a 200 MB.
4. El binario se reemplaza de forma atómica (mediante archivo temporal + renombrado) y se marca como ejecutable.
5. Xray se reinicia.

Antes del cambio se muestra el diálogo «Cambiar versión de Xray» (*Do you really want to change the Xray version?*) con la descripción «Esto cambiará la versión de Xray a #version#». Si tiene éxito – notificación «Xray actualizado correctamente» (*Xray updated successfully*).

3.12. Actualización del panel (3X-UI)

Bloque de comprobación de actualizaciones del panel. Los datos llegan mediante `GET /getPanelUpdateInfo` :

Campo	Descripción
Versión actual del panel	Versión del panel instalado.
Última versión del panel	Último release de 3x-ui obtenido desde GitHub.
Actualización disponible	Indicador de que la última versión es más reciente que la actual. Si no se necesita actualización, se muestra «Panel actualizado» / «Actualizado».

El botón **«Actualizar panel»** (*Update Panel*) ejecuta `POST /updatePanel` . Descripción emergente: «Esto actualizará 3X-UI a la última versión y reiniciará el servicio del panel». Antes de ejecutar – confirmación «¿Realmente desea actualizar el panel?» con el texto «Esto actualizará 3X-UI a la versión #version# y reiniciará el servicio del panel».

Particularidades y limitaciones:

- La autoactualización solo es compatible **con Linux** (en otros sistemas operativos se devuelve un error).

- El script de actualización se descarga del repositorio oficial (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`, límite de 2 MB) y se ejecuta mediante `bash`, si es posible de forma aislada mediante `systemd-run`.
- Si el inicio es exitoso, se muestra «Actualización del panel iniciada» (*Panel update started*); si la comprobación de actualización falló – «La comprobación de actualización del panel falló». Durante la instalación se muestra la advertencia «Instalación en curso. No actualice la página».

3.13. Actualización de archivos geográficos (GeoIP / GeoSite)

El botón/diálogo de actualización de bases de datos geográficas llama a `POST /updateGeofile` (todos los archivos) o `POST /updateGeofile/:fileName` (un archivo). La actualización funciona con una lista blanca estricta de nombres y fuentes:

Archivo	Fuente
<code>geoip.dat</code> , <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> (latest)
<code>geoip_IR.dat</code> , <code>geosite_IR.dat</code>	<code>chocolate4u/iran-v2ray-rules</code> (latest)
<code>geoip_RU.dat</code> , <code>geosite_RU.dat</code>	<code>runetfreedom/russia-v2ray-rules-dat</code> (latest)

Comportamiento:

- El nombre del archivo se valida: están prohibidos `..`, barras y rutas absolutas; solo se permiten `[a-zA-Z0-9._-]+.dat`. Los archivos fuera de la lista blanca no se descargan.
- Se utiliza una solicitud condicional `If-Modified-Since`: si el archivo en el servidor fuente no ha cambiado (HTTP 304), no se vuelve a descargar, solo se actualiza la marca de tiempo.
- Después de la descarga, Xray **se reinicia** (para que cargue las nuevas bases de datos).

Ejemplo. Actualizar solo las bases de datos geográficas rusas sin tocar los demás archivos:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b cookie.txt
```

Para actualizar todos los archivos de la lista blanca a la vez, llame a `POST /updateGeofile` sin nombre de archivo.

- Diálogos: «¿Realmente desea actualizar el archivo geográfico?» con «Esto actualizará el archivo `#filename#`» para un archivo individual y «Esto actualizará todos los archivos geográficos» para el botón «Actualizar todos». Éxito – «Archivos geográficos actualizados correctamente».

3.14. Copia de seguridad y restauración de la base de datos

Bloque «Copia de seguridad y restauración» (*Backup & Restore*). El comportamiento depende del sistema de gestión de base de datos utilizado (SQLite por defecto o PostgreSQL).

Exportar base de datos (Copia de seguridad)

El botón «Exportar base de datos» / «Copia de seguridad» (*Back Up*) llama a `GET /getDb`. El archivo se entrega como adjunto:

- **SQLite:** primero se realiza un checkpoint (vaciado del WAL), luego se descarga el archivo `x-ui.db`. Descripción emergente: «Haga clic para descargar el archivo .db que contiene la copia de seguridad de su base de datos actual...».
- **PostgreSQL:** se descarga un volcado `x-ui.dump` en formato personalizado (`pg_dump --format=custom --no-owner --no-privileges`). Las herramientas de cliente PostgreSQL deben estar instaladas en el servidor; de lo contrario, se produce un error indicando la ausencia de `pg_dump`.

Importar base de datos (Restauración)

El botón «Importar base de datos» / «Restauración» (*Restore*) sube el archivo mediante `POST /importDB` (campo de formulario `db`). Descripción emergente: «Haga clic para seleccionar y cargar el archivo .db... para restaurar la base de datos desde la copia de seguridad».

El escenario para **SQLite** es seguro, con reversión:

1. Se verifica el formato SQLite del archivo y se guarda en un archivo temporal, luego se comprueba su integridad.
2. Xray se detiene, la base de datos actual se cierra y se renombra a `*.backup` (fallback).
3. El nuevo archivo ocupa el lugar de la base de datos activa, se realiza la inicialización y la migración. Si algo falla, se restaura el fallback.
4. Xray se reinicia.

Para **PostgreSQL** se carga el `.dump` (se verifica la firma `PGDMP`) y se aplica mediante `pg_restore --clean --if-exists --single-transaction ...`. La descripción emergente advierte explícitamente: «Esto reemplazará todos los datos actuales».

Mensajes: «Base de datos importada correctamente», «Se produjo un error al importar la base de datos», «...al leer la base de datos», «...al obtener la base de datos».

Archivo de migración (entre SQLite y PostgreSQL)

El botón «Descargar archivo de migración» (*Download Migration*) llama a `GET /getMigration` y genera una exportación portátil para ejecutar el panel en otro sistema de base de datos:

- En **SQLite** se descarga `x-ui.dump` (volcado SQL en texto).
- En **PostgreSQL** se descarga `x-ui.db` — una base SQLite preparada, construida a partir de los datos de PostgreSQL.

3.15. Elementos adicionales de la interfaz

- **Indicador de clientes en línea.** El panel de control mantiene una serie `onLine` (*Online Clients* / «Clientes en línea») — el número de clientes con conexión activa. Se calcula cuando Xray está en ejecución (de lo contrario, 0) y se registra en el historial en el mismo ciclo de 2 segundos. Gráfico — pestaña «En línea».

- **Historial del sistema (gráficos).** Botón/sección «Gráficos» → «Historial del sistema» con pestañas: «Ancho de banda», «Paquetes», «Disco I/O», «En línea», «Carga», «Conexiones», «Uso del disco». Los datos se obtienen mediante `GET /history/:metric/:bucket`; los intervalos de agregación permitidos (bucket, seg): **2, 30, 60, 120, 180, 300, 720, 1440, 2880** (los tres últimos corresponden a los preajustes **12h, 24h y 48h** en el selector de intervalo), cada pestaña recibe hasta 60 puntos. El búfer circular de métricas almacena datos durante un período de hasta **48 horas**, por lo que los gráficos (CPU, RAM, tráfico, paquetes, conexiones, disco, en línea, carga) pueden consultarse para un período de hasta dos días. Métricas permitidas: `cpu`, `mem`, `swap`, `netUp`, `netDown`, `pktUp`, `pktDown`, `diskRead`, `diskWrite`, `diskUsage`, `tcpCount`, `udpCount`, `online`, `load1`, `load5`, `load15`. La etiqueta «Últimos 2 minutos» corresponde a `bucket = 2` (modo en tiempo real).

Ejemplo. Obtener la serie de carga de CPU de los últimos ~2 minutos (`bucket = 2` s, hasta 60 puntos) y la misma serie agregada por 5 minutos (`bucket = 300` s):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

La métrica puede reemplazarse por cualquier métrica permitida (`mem`, `netUp`, `tcpCount`, `load1`, etc.). Un bucket fuera de la lista `2, 30, 60, 120, 180, 300, 720, 1440, 2880` será rechazado.

- **Métricas de Xray** – bloque separado con el consumo de memoria y la recolección de basura de Xray (series `xrAlloc`, `xrSys`, `xrHeapObjects`, `xrNumGC`, `xrPauseNs`) y el «Observatorio» (estado de las conexiones salientes). Solo funcionan si en la configuración de Xray se define el bloque `metrics` (`listen 127.0.0.1:11111`, `tag metrics_out`); de lo contrario, se muestra «El endpoint de métricas de Xray no está configurado».

Ejemplo del bloque que activa el mosaico de métricas de Xray. En la sección de configuración de Xray deben estar presentes simultáneamente `metrics` (con `tag`) y un `inbound` que escuche ese `tag`:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

La dirección `127.0.0.1:11111` no se expone al exterior intencionalmente – el panel la consulta de forma local.

- **Selector de tema oscuro.** Se encuentra en el menú general/encabezado, no en el propio panel de control. Opciones: «Tema» (*Theme*) con variantes «Oscuro» y «Muy oscuro» (*Ultra Dark*). Es un ajuste puramente visual del aspecto, no afecta al funcionamiento del panel.

- **Otros enlaces** en el entorno del panel de control (desde el menú/barra inferior): «Registros», «Configuración» – visualización del JSON final de Xray (`GET /getConfigJson`), «Documentación».
-

4. Inbounds: creación y parámetros generales

La sección **«Entrantes»** (inbounds) es la lista de todos los puntos de entrada de Xray a través de los cuales se conectan los clientes. Cada inbound almacena tanto campos «de panel» (nota, límite de tráfico, calendario de reinicio) como bloques JSON sin procesar de configuración de Xray (`settings` , `streamSettings` , `sniffing`).

La creación se realiza con el botón **«Crear conexión»** (*Add Inbound*), y la edición con **«Modificar conexión»** (*Modify Inbound*). Ambas operaciones se envían a los endpoints de API `POST /add` y `POST /update/:id`.

A continuación se describen todos los campos del formulario que **no** pertenecen a la configuración de un protocolo específico (clientes, cifrado, REALITY/TLS) y que **no** pertenecen al transporte/flujo (pestañas **«Flujo»**, **«Seguridad»**) – esos son temas de secciones separadas.

4.1. Campos generales del formulario

Remark (Nota)

Parámetro	Valor
Campo	<code>remark</code>
Tipo	cadena
Por defecto	vacío

Nombre legible por humanos del inbound, que se muestra en la lista y en los encabezados de los diálogos («¿Eliminar conexión "{remark}"?» etc.). La etiqueta del campo es **«Nota»**. No afecta al funcionamiento de Xray; sirve únicamente para facilitar la administración. Se recomienda asignar nombres únicos y descriptivos, ya que se usan en los nombres de los archivos exportados y en las confirmaciones de operaciones masivas.

Protocol (Protocolo)

Parámetro	Valor
Campo	<code>protocol</code>
Etiqueta	«Protocolo»
Validación	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

Lista desplegable del protocolo del inbound. Valores permitidos:

Valor	Nota
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

Valor	Nota
shadowsocks	
wireguard	
hysteria	Hysteria v2 es hysteria con streamSettings.version = 2 ; no existe un protocolo separado
http	
mixed	socks/http en un mismo puerto
tunnel	
tun	aceptado por el validador; no existe una constante de protocolo separada

El campo es obligatorio (required). La elección del protocolo determina qué campos de configuración de clientes y qué transportes estarán disponibles (véanse las secciones específicas de cada protocolo).

Importante: al guardar, el servicio normaliza streamSettings . Las configuraciones de transporte se conservan solo para los protocolos vmess , vless , trojan , shadowsocks , hysteria ; para el resto (http , mixed , tunnel , wireguard , tun) el campo streamSettings se **borra forzosamente**.

Para inbounds de tipo tunnel /TPProxy cuyo bloque streamSettings no contiene la clave security (variante sin transporte), el formulario se abre y guarda sin el error de validación streamSettings.security Invalid input .

Listen IP (IP de escucha)

Parámetro	Valor
Campo	listen
Tipo	cadena
Por defecto	vacío → Xray escucha en 0.0.0.0 (todas las IPs)

Dirección IP en la que el inbound acepta conexiones. Sugerencia del campo:

«Dejar vacío para escuchar en todas las direcciones IP».

Al generar la configuración de Xray, el valor vacío se reemplaza por 0.0.0.0 . Además de una IP, el campo acepta una **ruta de Unix socket** — sugerencia:

«También puede especificar una ruta de Unix socket (por ejemplo, /run/xray/in.sock) o un nombre de socket abstracto con el prefijo @ (por ejemplo, @xray/in.sock) para escuchar en un socket en lugar de un puerto TCP — en ese caso establezca el puerto en 0».

Así, el campo acepta dos formas de Unix socket: una ruta en el sistema de archivos (`/run/xray/in.sock`) y un nombre de socket abstracto con el prefijo `@` (`@xray/in.sock`). En ambos casos establezca **Port** en `0` .

Este campo se modifica cuando se necesita restringir el inbound a una sola interfaz (por ejemplo, `127.0.0.1` para un inbound que funcione únicamente como destino fallback detrás de Nginx) o cuando el inbound escucha en un Unix socket.

Ejemplo. Inbound que escucha solo en la interfaz local (destino fallback típico detrás de Nginx) y en un Unix socket:

```
listen = 127.0.0.1    puerto = 8443
listen = /run/xray/in.sock    puerto = 0
```

Port (Puerto)

Parámetro	Valor
Campo	<code>port</code>
Etiqueta	«Puerto»
Validación	<code>gte=0,lte=65535</code>
Por defecto	— (lo establece el usuario)

Puerto TCP/UDP de escucha. Se permiten valores de `0` a `65535` . El valor `0` se usa únicamente en combinación con la escucha en un Unix socket (véase arriba).

Al guardar, el servicio comprueba conflictos de puerto: dos inbounds no pueden ocupar simultáneamente el mismo `listen:port` para el mismo transporte (TCP/UDP). El transporte se determina a partir del protocolo y de `streamSettings / settings` : por ejemplo, `hysteria` y `wireguard` siempre ocupan UDP, `kcp / quic` — UDP, y la mayoría de los demás — TCP. En caso de conflicto, el guardado se rechaza con un error.

Adicionalmente, el panel no permite ocupar el **puerto reservado de la API interna de Xray** (etiqueta `api` , por defecto `62789` en `127.0.0.1`): un inbound TCP local cuya dirección de escucha coincide con ese puerto en loopback se rechaza con el mismo error de conflicto de puerto. El puerto real de la API se lee desde la plantilla de configuración de Xray (con valor de reserva `62789`). En los nodos (nodes) esta restricción no aplica — tienen su propio Xray.

La etiqueta de Xray (Tag , única) se genera automáticamente a partir del puerto y el transporte con el formato `in-<puerto>-<tcp|udp|tcpudp|any>` ; para un inbound desplegado en un nodo se añade el prefijo `n<nodeId>-` . En caso de colisión se añaden sufijos `-2` , `-3` , etc. Normalmente el usuario no edita la etiqueta.

Total traffic (Tráfico total, GB)

Parámetro	Valor
Campo	<code>total</code> (en bytes)
Etiqueta	«Uso total»
Por defecto	<code>0</code>

Límite de tráfico total del inbound. En el formulario el valor se introduce en gigabytes; en la base de datos se almacena en bytes. Sugerencia del campo:

«= Sin límite. (unidad: GB)».

Es decir, `0` **significa sin límite**. Es un límite a nivel del inbound completo (no de clientes individuales); el tráfico efectivamente consumido se almacena en los campos `up` (enviado) y `down` (recibido) y se compara con `total`.

Expiry date / Duration (Fecha de vencimiento / duración)

Parámetro	Valor
Campo	<code>expiryTime</code> (marca de tiempo Unix)
Etiqueta	«Fecha de vencimiento» (<i>Duration</i>)
Por defecto	vacío / <code>0</code>

Período de validez del inbound. Sugerencia:

«Dejar vacío para que sea indefinido».

El valor vacío (`0`) significa un inbound sin fecha de vencimiento. El valor se almacena como marca Unix; el formulario permite establecer tanto una fecha concreta como un período en días (cuenta relativa desde el momento actual – etiqueta en inglés del campo *Duration*).

Enabled (Habilitar)

Parámetro	Valor
Campo	<code>enable</code>
Etiqueta	«Habilitar» (<i>Enabled</i>)
Por defecto	se establece al crear

Indicador de actividad del inbound. Cambiar este indicador en la lista se procesa mediante un endpoint «ligero» separado `POST /setEnabled/:id`, en lugar de una actualización completa – esto se hace a propósito

para no reserializar todo el bloque `settings` (de todos los clientes) con cada clic en el interruptor de un inbound con miles de clientes. Al deshabilitar, el inbound se elimina del Xray en ejecución; al habilitar, se añade de nuevo.

Node / Deploy to (Nodo / Desplegar en)

Parámetro	Valor
Campo	<code>nodeId</code>
Etiqueta	«Desplegar en», «Panel local»
Por defecto	vacío (panel local)

Selección de dónde funciona físicamente el inbound: en el panel local o en uno de los nodos registrados. Detalle de implementación: `nodeId = 0` se normaliza a `nil`, ya que `0` no es un id de nodo válido sino un artefacto del enlace del formulario; `nil / 0` indica el panel local. Al guardar un inbound en un nodo desconectado, puede aparecer el mensaje «el cambio se sincronizará cuando el nodo se vuelva a conectar».

Estrategia de dirección para enlaces (Share address strategy)

Parámetro	Valor
Campo	estrategia + (opcionalmente) dirección personalizada
Etiqueta	«Estrategia de dirección para enlaces» (<i>Share address strategy</i>)
Por defecto	«Dirección de escucha del inbound» (<i>Inbound listen</i>)

La lista desplegable determina qué dirección se inserta en los **enlaces de compartición y códigos QR exportados** de este inbound. Valores:

Valor	Etiqueta	Qué se inserta
<code>node</code>	«Dirección del nodo» (<i>Node address</i>)	dirección del nodo en el que funciona el inbound
<code>listen</code>	«Dirección de escucha del inbound» (<i>Inbound listen</i>)	dirección de escucha del propio inbound
<code>custom</code>	«Personalizada» (<i>Custom</i>)	dirección propia del campo «Dirección de compartición personalizada» (<i>Custom share address</i>)

Al seleccionar «Personalizada» aparece el campo «Dirección de compartición personalizada»; en él se introduce el host o IP **sin esquema ni puerto** (el valor se valida). La opción «Dirección del nodo» solo aparece en la lista si existe un nodo habilitado en el que pueda funcionar este inbound; en caso contrario se oculta y el valor se ajusta a «Dirección de escucha del inbound».

Esta estrategia afecta **únicamente** a los enlaces de compartición directos y los códigos QR. **No** afecta a la entrega de suscripciones – allí la dirección sigue determinándose por la lógica habitual del panel.

4.2. Sniffing (Análisis de tráfico)

La pestaña «**Sniffing**» edita el bloque `sniffing` de la configuración de Xray, que se almacena como JSON sin procesar. Sniffing permite a Xray «inspeccionar» el nombre de dominio real o el protocolo dentro de la conexión con fines de enrutamiento.

Subcampo	Etiqueta	Propósito
<code>enabled</code>	(interruptor de pestaña)	Habilita/deshabilita el sniffing para el inbound
<code>destOverride</code>	—	Lista de protocolos para los que se intercepta la dirección de destino: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
<code>metadataOnly</code>	«Solo metadatos»	Usar únicamente los metadatos de la conexión, sin leer la carga útil
<code>routeOnly</code>	«Solo enrutamiento»	Aplicar el resultado del sniffing solo para el enrutamiento, sin reescribir la dirección de destino
<code>domainsExcluded</code>	«Dominios excluidos»	Dominios excluidos del sniffing
(IPs excluidas)	«IPs excluidas»	Direcciones IP excluidas del sniffing

- **`destOverride`** — conjunto de analizadores: `http` (detecta el dominio a partir del encabezado HTTP Host), `tls` (a partir del SNI), `quic` (a partir del ClientHello de QUIC), `fakedns` (coincidencia con el pool de FakeDNS). Normalmente se habilitan `http` y `tls` para detectar dominios.

Ejemplo del bloque `sniffing` (detectar dominio por HTTP y TLS, usar el resultado solo para enrutamiento sin tocar la red local):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** — cuando está habilitado, Xray no lee el contenido del primer paquete y se basa únicamente en los metadatos; es útil para no interferir con protocolos cuyos datos no pueden «inspeccionarse».
- **`routeOnly`** — el resultado del sniffing se usa únicamente por las reglas de enrutamiento; la dirección de la conexión en el outbound no se reescribe por el dominio detectado.

Nota: el panel almacena `sniffing` como un bloque JSON opaco y no añade nada al guardarlo — todos los valores por defecto de estas casillas se forman en el lado de la aplicación cliente. El bloque puede editarse en formato sin procesar a través de la sección «JSON del entrante» (véase más abajo).

4.3. Allocate (estrategia de distribución de puertos)

El bloque `allocate` en `streamSettings` controla cómo Xray distribuye los puertos de escucha. Forma parte de la configuración de Xray; el panel lo almacena y transmite como parte de `streamSettings` /JSON del inbound. Parámetros (según la terminología de Xray-core):

Subcampo	Propósito	Valores / por defecto
<code>strategy</code>	Estrategia de asignación de puertos	<code>always</code> – escuchar siempre en el puerto indicado (por defecto); <code>random</code> – cambiar periódicamente los puertos escuchados dentro del rango
<code>refresh</code>	Intervalo de cambio de puertos (minutos) con <code>random</code>	número entero de minutos (se recomienda 5; mínimo 2)
<code>concurrency</code>	Cuántos puertos mantener abiertos simultáneamente con <code>random</code>	entero (por defecto 3; no más de un tercio del ancho del rango de puertos)

`strategy: always` mantiene el inbound en un solo puerto (modo estándar). `strategy: random` se usa en escenarios anti-bloqueo, cuando el inbound «salta» periódicamente por un rango de puertos; en ese caso `refresh` y `concurrency` cobran sentido. Estos valores solo deben modificarse cuando se usa deliberadamente el modo de puertos aleatorios.

Ejemplo del bloque `allocate` en `streamSettings` (modo de puertos aleatorios: mantener 3 puertos abiertos, rotar cada 5 minutos):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Para que esto funcione, el `port` del inbound debe especificarse como un rango (por ejemplo, `20000-20100`).

4.4. External Proxy (Proxy externo)

El campo **«External Proxy»** pertenece a la configuración de generación de enlaces de invitación y se almacena en `streamSettings` del inbound. Define una lista de direcciones externas alternativas (host/puerto, opcionalmente con TLS forzado – **«TLS forzado»**) que se insertan en los enlaces de cliente en lugar del `listen:port` real del inbound.

Se utiliza cuando los clientes deben conectarse no directamente al servidor sino a través de un proxy externo/reverse/CDN: en ese caso los enlaces compartidos contienen la dirección pública de ese frontend. No afecta al proceso de aceptación de conexiones de Xray – es una función «cosmética» de los enlaces generados. Campos del formulario relacionados: **«TLS forzado»**, **«Fingerprint»**, etiquetas de cada entrada.

4.5. Fallbacks (Fallbacks)

La sección **«Fallbacks»** define las reglas de redirección para las conexiones que no coinciden con ningún cliente del inbound. Está disponible para inbounds maestros con transporte TLS (VLESS/Trojan TCP-TLS). Se gestiona a través de los endpoints `GET /:id/fallbacks` / `POST /:id/fallbacks`.

Sugerencia de la sección:

«Cuando una conexión en este inbound no coincide con ningún cliente, se redirige a otro lugar. Seleccione un inbound hijo a continuación para que los campos de enrutamiento (SNI / ALPN / Path / xver) se rellenen automáticamente a partir de su transporte, o deje la selección vacía y establezca Dest directamente (por ejemplo, 8080 o 127.0.0.1:8080) para redirigir a un servidor externo como Nginx. Cada inbound hijo debe escuchar en 127.0.0.1 con security=none».

La sección de fallbacks se muestra solo para inbounds VLESS/Trojan sobre RAW (TCP) con seguridad TLS o REALITY. Un nuevo inbound comienza con `security=none`, por lo que la sección puede parecer ausente inicialmente. En ese estado (VLESS/Trojan, RAW/TCP, seguridad aún no configurada), en lugar de la sección se muestra una sugerencia integrada: los fallbacks estarán disponibles después de seleccionar TLS o Reality en la pestaña **«Seguridad»**.

Campos de una fila de fallback

Campo	Por defecto	Descripción
(inbound hijo)	—	Selección del inbound hijo (etiqueta «Seleccionar inbound»). Si se selecciona, los campos Name/Alpn/Path/Dest pueden rellenarse automáticamente a partir de su transporte
Name	vacío (= cualquiera)	Condición de coincidencia por nombre (SNI/nombre). Etiqueta para «cualquiera» — «cualquiera»
Alpn	vacío	Condición de coincidencia por ALPN
Path	vacío	Condición de coincidencia por ruta (para transportes WS/HTTP del inbound hijo)
Dest	auto	Hacia dónde redirigir. Marcador de posición «auto (listen:puerto del hijo)» . Se puede indicar un puerto (<code>8080</code>) o <code>host:puerto</code> (<code>127.0.0.1:8080</code>)
Xver	<code>0</code>	Versión del protocolo PROXY («Xver»): <code>0</code> — deshabilitado, <code>1</code> o <code>2</code> — la versión correspondiente de PROXY protocol
(orden)	por posición	Orden de aplicación de las reglas; se establece con los botones «Subir»/«Bajar»

Lógica de guardado: toda la lista de fallbacks del maestro se reemplaza de forma atómica. Una fila que no tiene ni inbound hijo seleccionado (`childId <= 0`) ni `Dest` definido se **omite**. Si el inbound hijo seleccionado coincide con el id del maestro, se anula. Al generar el JSON final: si `Dest` está vacío, se calcula a partir del inbound hijo como `listen:port`, donde `0.0.0.0 / :: / ::0` se sustituyen por `127.0.0.1`; los campos vacíos `name / alpn / path` no se incluyen en el JSON de salida; `xver` se añade solo si es mayor que 0.

Ejemplo del `settings.fallbacks` resultante (el tráfico con `alpn=h2` va al destino WS en la ruta `/ws` ; todo lo demás va al Nginx local en el puerto 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

La última fila sin `name` / `alpn` / `path` es la regla «por defecto» que captura todo lo demás.

Botones y sugerencias de la sección de fallbacks

- **«Añadir fallback»** – añadir una fila; **«Aún no hay fallbacks»** – estado vacío.
- **«Añadir rápidamente todos los adecuados»** / **«Añadir todos»** – añade una fila de fallback para cada inbound adecuado que aún no esté conectado. Resultado: «Se añadieron {n} fallback(s)» o «No hay nuevos inbounds adecuados».
- **«Rellenar desde hijo»** – volver a obtener los campos de enrutamiento (SNI/ALPN/Path/xver) del transporte del inbound hijo seleccionado; tras ejecutar – «Rellenado desde hijo».
- **«Modificar campos de enrutamiento»** / **«Ocultar avanzado»** – mostrar/ocultar los campos detallados de la fila.
- Las etiquetas **«Enruta cuando»** y **«Por defecto – captura todo lo demás»** explican la condición de activación de cada fila.

Después de guardar los fallbacks, el servidor reinicia Xray para que los nuevos `settings.fallbacks` entren en vigor.

4.6. Reinicio periódico del tráfico

El bloque **«Reinicio de tráfico»** configura el reinicio automático de los contadores de tráfico del inbound según una programación. Descripción:

«Reinicio automático del contador de tráfico a los intervalos indicados».

Parámetro	Valor
Campo	<code>trafficReset</code>
Validación	<code>omitempty,oneof=never hourly daily weekly monthly</code>
Por defecto	<code>never</code>
Campo asociado	<code>lastTrafficResetTime</code> – marca del último reinicio (etiqueta «Último reinicio»)

Lista desplegable:

Valor	Etiqueta
never	«Nunca»
hourly	«Cada hora»
daily	«Diariamente»
weekly	«Semanalmente»
monthly	«Mensualmente»

Para cada período hay un cron registrado que se ejecuta según el calendario correspondiente (@hourly , @daily , @weekly , @monthly). El cron selecciona todos los inbounds con el trafficReset indicado y para cada uno reinicia los contadores del propio inbound (up=0 , down=0) y el tráfico de todos sus clientes. Es decir, el reinicio periódico afecta tanto al inbound como a sus clientes.

Ejemplo del valor del campo. Para que los contadores se restablezcan el primer día de cada mes, en el formulario se selecciona «Mensualmente», lo que se guarda como:

```
{ "trafficReset": "monthly" }
```

El valor never (por defecto) desactiva completamente el reinicio automático.

4.7. JSON del entrante (avanzado)

La sección «**Secciones JSON del entrante**» proporciona acceso directo a los bloques JSON sin procesar del inbound. Descripción:

«JSON completo del entrante y editores individuales para settings, sniffing y streamSettings».

Editores disponibles:

Pestaña	Etiqueta	Qué edita
Todo	«Objeto completo del entrante con todos los campos en un solo editor»	todo el objeto Inbound
Configuración	«Contenedor del bloque settings de Xray»	campo settings
Sniffing	«Contenedor del bloque sniffing de Xray»	campo sniffing
Stream	«Contenedor del bloque stream de Xray»	campo streamSettings

Estos campos se serializan como objetos JSON anidados: los bloques vacíos se devuelven como null , y el texto que no es JSON válido se envuelve en una cadena para que los datos no se pierdan. Los errores de análisis al guardar se muestran con el prefijo «**JSON avanzado**».

La ventana de visualización «JSON del entrante», al igual que la ventana de importación de inbound, utiliza un editor de código completo con resaltado de sintaxis JSON (en lugar de un campo de texto ordinario): la visualización de la configuración está en modo resaltado de solo lectura, y la importación en modo editable, lo que facilita la lectura y la edición.

4.8. Acciones sobre el inbound: QR / Edit / Reset / Delete y estadísticas

En la lista y en la tarjeta del inbound están disponibles las siguientes acciones (menú «Menú»):

Estadísticas de tráfico

Se muestra el tráfico agregado del inbound: «**Enviado/recibido**» (campos `up` / `down`), «**Tráfico total**», «**Conexiones totales**». En la tarjeta también aparecen «**Creado**», «**Actualizado**», «**Fecha de vencimiento**».

En la lista de inbounds hay una columna **Speed** con la velocidad de tráfico actual de cada inbound (subida/bajada), calculada a partir de los incrementos de los contadores entre sondeos; la misma velocidad en tiempo real se muestra en la ventana de estadísticas del inbound. Cuando un sondeo no produce incremento, el valor de velocidad se restablece.

En el resumen de clientes de la página de inbounds, el estado se determina según la prioridad «agotado/finalizado»: los clientes cuyo período ha vencido o cuyo tráfico se ha agotado (y a los que la tarea automática ha quitado el `enable`) se clasifican como «**Agotado/Finalizado**» (*Depleted/Ended*) y no como el gris «**Deshabilitado**» (*Disabled*), sin contarlos dos veces. La clasificación coincide con la mostrada en la tarjeta del propio cliente y tiene en cuenta correctamente a los clientes vinculados a varios inbounds.

Código QR y copia de enlaces

- «**Más detalles**» – expande los enlaces de conexión y suscripción.
- Código QR del cliente: sugerencia «**Haga clic en el código QR para copiar**».
- «**Copiar enlace**» (*Copy URL*), «**Exportar enlaces**».

Edit (Modificar)

«**Modificar conexión**» – abre el formulario de edición (`POST /update/:id`). Al actualizar, el servicio vuelve a leer la fila existente, transfiere los campos modificados, regenera la etiqueta si es necesario (si la etiqueta anterior fue autogenerada) y sincroniza el runtime de Xray. Éxito – notificación «**Conexión actualizada correctamente**».

Reset Traffic (Reiniciar tráfico)

«**Reiniciar tráfico**» – pone a cero los contadores `up` / `down` de este inbound (`POST /:id/resetTraffic` , establece `up=0` , `down=0`). Confirmación:

«¿Reiniciar el tráfico de "{remark}"?» / «Restablece los contadores de envío/recepción de esta conexión a 0».

El reinicio del tráfico del inbound **no** afecta a los contadores de sus clientes (para ellos existen acciones separadas «Reiniciar tráfico de clientes»). Tras el reinicio se inicia un reinicio de Xray. Éxito – notificación «**Tráfico entrante reiniciado**». También existe una variante masiva – «**Reiniciar tráfico de todas las conexiones**» (`POST /resetAllTraffics`).

Delete (Eliminar)

«**Eliminar conexión**» (`POST /del/:id`). Confirmación:

«¿Eliminar la conexión "{remark}"?» / «La conexión y todos sus clientes serán eliminados. Esta acción no se puede deshacer».

La eliminación retira el inbound del Xray en ejecución (con reinicio si es necesario). Éxito – notificación **«Conexión eliminada correctamente»**. Eliminación masiva – `POST /bulkDel` , con informe por elemento y no más de un reinicio de Xray.

Otras acciones sobre los clientes del inbound

En el menú también están disponibles: **«Clonar»** (copia del inbound con nuevo puerto y lista de clientes vacía), **«Eliminar todos los clientes»** (`POST /:id/deleteAllClients` – elimina todos los clientes; el inbound en sí se conserva), **«Eliminar clientes deshabilitados»**, **«Vincular/Desvincular clientes»**, **«Importar»/«Exportar conexiones»** (`POST /import`). Los detalles de las operaciones con clientes corresponden a la sección sobre clientes.

5. Protocolos

Al crear una conexión entrante (inbound), lo primero que se selecciona es el **Protocolo** («Protocol»). El protocolo determina qué método de autenticación y cifrado de tráfico aplicará Xray-core a ese inbound, qué conjunto de campos en `settings` deberá completarse y qué transportes (`network`) y tipos de seguridad (TLS / REALITY) están disponibles para él.

El campo de protocolo se establece una sola vez al crear el inbound y **no puede modificarse al editar** (en el formulario de edición, la lista desplegable está bloqueada). Para cambiar el protocolo es necesario crear un nuevo inbound.

5.1. Lista de protocolos admitidos

El servidor acepta el siguiente conjunto de valores para el campo `Protocol` :

```
oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

A partir de la versión **3.3.0** se añadió el valor `mtproto` (proxy de Telegram) a la lista.

Valor en la configuración	Propósito	Modelo de cliente
<code>vless</code>	Protocolo proxy principal (predeterminado al crear un inbound)	Clientes con UUID, compatibilidad con flow y cifrado poscuántico
<code>vmess</code>	Protocolo proxy clásico de Xray	Clientes con UUID y parámetro <code>security</code>
<code>trojan</code>	Proxy que imita HTTPS convencional	Clientes con contraseña
<code>shadowsocks</code>	Proxy Shadowsocks (incluido SIP022 / 2022-blake3)	Un usuario o varios (2022)
<code>wireguard</code>	Inbound WireGuard	Peers (no clientes)
<code>hysteria</code>	Inbound Hysteria (versión 2 por defecto)	Clientes con token <code>auth</code>
<code>http</code>	Proxy HTTP clásico (forward proxy)	Cuentas user/pass, sin contabilidad de tráfico
<code>mixed</code>	Proxy SOCKS + HTTP combinado	Cuentas user/pass
<code>tunnel</code>	Forwarder transparente (xray <code>dokodemo-door</code>)	Sin clientes
<code>tun</code>	Interfaz TUN (solo renderización de los existentes)	Sin clientes
<code>mtproto</code>	Proxy de Telegram (MTPROTO), añadido en 3.3.0; gestionado por un proceso separado <code>mtg</code> , no por Xray	Sin clientes (acceso por secreto)

Nota sobre `tun` : el valor se mantiene en la lista por compatibilidad y para **mostrar** inbounds guardados anteriormente, pero en la versión actual el backend no recomienda crearlo — su soporte está considerado obsoleto. No tiene sentido crear nuevos inbounds de este tipo.

Nota sobre Hysteria 2: no existe un protocolo «hysteria2» separado. Es el protocolo `hysteria` con el campo `streamSettings.version = 2`. El esquema de enlace `hysteria2://` se selecciona automáticamente al generar enlaces de compartición cuando la versión del stream es 2.

No todos los protocolos admiten distribución por nodos (nodes). Solo pueden desplegarse en nodos: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. Los protocolos `http`, `mixed`, `tunnel`, `tun`, `mtproto` funcionan únicamente en el panel local.

5.2. Qué protocolos admiten TLS / REALITY / transporte

La posibilidad de activar una u otra capa de seguridad y transporte depende del protocolo y la red seleccionada (`streamSettings.network`):

Capacidad	Disponible para protocolos	Redes permitidas (<code>network</code>)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (y siempre para <code>hysteria</code>)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
flow (<code>xtls-rprx-vision</code>)	solo <code>vless</code>	solo <code>tcp</code> , con <code>security = tls</code> o <code>reality</code>
Stream / transporte (pestaña «Flujo»)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	—

Para los protocolos `http`, `mixed`, `tunnel`, `tun`, `wireguard`, la pestaña de transporte no está disponible — no tienen configuraciones de stream de Xray.

5.3. VLESS

Propósito: protocolo proxy moderno principal. Admite XTLS-Vision (`flow`), REALITY y cifrado poscuántico a nivel del propio VLESS (campos `decryption` / `encryption`). Se usa por defecto para nuevos inbounds.

Campos del bloque `settings` :

Campo	Valor predeterminado	Descripción
<code>clients</code>	<code>[]</code>	Lista de clientes. Cada uno tiene: <code>id</code> (UUID), <code>email</code> (obligatorio), <code>flow</code> , límites (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>
<code>decryption</code>	<code>none</code>	Parámetro de descifrado en el lado del servidor. Etiqueta en la UI: «Descifrado» (en inglés «Decryption»)

Campo	Valor predeterminado	Descripción
encryption	none	Parámetro de cifrado complementario (incluido en el enlace del cliente). Etiqueta: «Cifrado» (en inglés «Encryption»)
fallbacks	[]	Lista de fallbacks (ver sección sobre fallbacks); disponible cuando network = tcp y security = TLS o REALITY
testseed	(4 números: 900, 500, 900, 256)	«Vision testseed» – 4 enteros positivos para el relleno XTLS-Vision. Se aplica solo a clientes con flow xtls-rprx-vision ; en caso contrario se ignora

flow (xtls-rprx-vision)

flow se establece en el cliente, no en el inbound, y acepta uno de tres valores:

Valor	Significado
`` (vacío)	Sin XTLS-flow (predeterminado)
xtls-rprx-vision	XTLS-Vision – modo recomendado para VLESS sobre TCP+TLS/REALITY
xtls-rprx-vision-udp443	El mismo Vision, pero con manejo de UDP/443 (QUIC)

El campo flow solo está disponible para selección cuando se cumplen todas las condiciones: protocolo vless , network = tcp y security = tls o reality . El campo **Vision testseed** en el formulario se muestra solo bajo las mismas condiciones.

Excepción para XHTTP: con VLESS sobre network = xhttp con autenticación poscuántica VLESS habilitada (encryption / decryption , vlessenc), el flow xtls-rprx-vision también es válido – independientemente de la capa de seguridad, incluida REALITY. En este caso el panel transmite correctamente xtls-rprx-vision en los enlaces de compartición y en las suscripciones (incluido el formato Clash/Mihomo), por lo que el cliente recibe la configuración con Vision.

Descifrado / Cifrado (autenticación poscuántica VLESS)

Los campos decryption y encryption corresponden a la autenticación a nivel del propio VLESS (separada del TLS/REALITY de transporte). Por defecto ambos son none . En el formulario hay tres botones adyacentes:

- **Autenticación X25519** (en inglés «X25519 auth») – genera el par decryption / encryption basado en X25519.
- **Autenticación ML-KEM-768** (en inglés «ML-KEM-768 auth») – variante poscuántica (Post-Quantum).
- **Limpiar** – restablece ambos campos a none .

Debajo de los botones se muestra la cadena de estado «Seleccionado: {auth}», donde el valor se determina según el último segmento de la cadena encryption : vacío/ none → «None», clave muy larga (>300 caracteres) → ML-KEM-768, corta → X25519, en caso contrario «Personalizado».

Técnicamente los botones llaman a GET /panel/api/server/getNewVlessEnc (generación de claves mediante xray vlessenc) y rellenan **ambos** campos con valores emparejados del tipo

`mlkem768x25519plus.native.<rtt>.<role>` (por ejemplo, `decryption = mlkem768x25519plus.native.600s.server-x25519`, `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). El parámetro `decryption` permanece en el servidor; `encryption` se incluye en el enlace del cliente.

Importante: al generar la configuración del inbound para Xray, el panel elimina lo innecesario: si en `settings` queda `encryption` (que corresponde al lado del cliente), este **se elimina** de la configuración del servidor. En el servidor solo permanece `decryption`.

Cuándo elegir VLESS: es la opción recomendada por defecto para un nuevo inbound, especialmente combinado con REALITY (sin certificado propio) o con TLS + XTLS-Vision.

Ejemplo: bloque `settings` de un inbound VLESS con un cliente y XTLS-Vision. El campo `flow` está en el cliente; `decryption` permanece en el servidor:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

Para la combinación con REALITY, el bloque `streamSettings` correspondiente (pestaña «Transport» → Security: REALITY) tiene este aspecto:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<clave privada X25519>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Propósito: protocolo proxy clásico de Xray. Autenticación mediante UUID; en el cliente se configura adicionalmente el método de cifrado de la carga útil (`security`).

Campos del bloque `settings` :

Campo	Valor predeterminado	Descripción
<code>clients</code>	<code>[]</code>	Lista de clientes

Cada cliente VMess (además de los campos comunes `email`, límites, `enable`, `tgId`, `subId`, `comment`, `reset`):

Campo del cliente	Valor predeterminado	Descripción
<code>id</code>	—	UUID del cliente
<code>security</code>	<code>auto</code>	Método de cifrado de la carga útil VMess. Valores válidos: <code>aes-128-gcm</code> , <code>chacha20-poly1305</code> , <code>auto</code> , <code>none</code> , <code>zero</code>

Valores de `security` :

- `auto` — Xray elige el cifrado según la plataforma (recomendado);
- `aes-128-gcm`, `chacha20-poly1305` — cifrados AEAD fijos;
- `none` — sin cifrado de carga útil (solo tiene sentido sobre TLS);
- `zero` — sin cifrado ni autenticación de carga útil.

Compatibilidad histórica: los registros antiguos podían almacenar `security: ""` — al leer, la cadena vacía se normaliza a `auto`. Al generar la configuración del servidor, el campo `security` de los clientes VMess **se elimina** de `settings`, ya que no es necesario para el inbound.

Cuándo elegir VMess: para compatibilidad con clientes antiguos o configuraciones existentes. Para nuevos despliegues, generalmente se prefiere VLESS.

5.5. Trojan

Propósito: proxy que imita tráfico HTTPS convencional. Autenticación mediante contraseña. Al igual que VLESS, admite fallbacks y (con `network = tcp`) REALITY/TLS.

Campos del bloque `settings` :

Campo	Valor predeterminado	Descripción
<code>clients</code>	<code>[]</code>	Lista de clientes
<code>fallbacks</code>	<code>[]</code>	Lista de fallbacks (disponible con <code>network = tcp</code> y TLS/REALITY)

El campo clave de cada cliente Trojan:

Campo del cliente	Valor predeterminado	Descripción
<code>password</code>	—	Contraseña del cliente (obligatoria, mínimo 1 carácter)

Campo del cliente	Valor predeterminado	Descripción
email	–	Identificador único del cliente

Los demás campos del cliente son comunes (`limitIp` , `totalGB` , `expiryTime` , `enable` , `tgId` , `subId` , `comment` , `reset`).

Cuándo elegir Trojan: cuando se necesita enmascaramiento como HTTPS en el puerto 443, incluyendo fallbacks a un servidor web (Nginx) para conexiones no solicitadas.

Ejemplo: bloque `settings` de Trojan con fallback a un servidor web local. Las conexiones no solicitadas (sin contraseña válida) se redirigen a Nginx, que escucha en `127.0.0.1:8080` :

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Para el fallback se requieren `network = tcp` y `Security = TLS` o `REALITY`; de lo contrario, el campo `fallbacks` no está disponible.

5.6. Shadowsocks

Propósito: proxy Shadowsocks ligero. Admite tanto cifrados AEAD obsoletos como los nuevos métodos SIP022 (`2022-blake3-*`). Puede funcionar en modo mono-usuario o multiusuario.

Campos del bloque `settings` :

Campo	Valor predeterminado	Descripción
method	<code>2022-blake3-aes-256-gcm</code>	Método de cifrado del inbound. Etiqueta en la UI: «Método de cifrado» (en inglés «Encryption method»)
password	``	Contraseña del inbound (para los métodos 2022 se genera automáticamente ajustada al método seleccionado)
network	<code>tcp,udp</code>	Transporte. Etiqueta: «Red» (en inglés «Network»). Opciones: <code>tcp,udp</code> (TCP, UDP), <code>tcp</code> , <code>udp</code>
clients	<code>[]</code>	Lista de clientes
ivCheck	<code>false</code> (desactivado)	Interruptor «ivCheck» – protección contra reutilización de IV

Métodos de cifrado (`method`)

Conjunto válido:

Método	Categoría
aes-256-gcm	AEAD obsoleto
chacha20-poly1305	AEAD obsoleto
chacha20-ietf-poly1305	AEAD obsoleto
xchacha20-ietf-poly1305	AEAD obsoleto
2022-blake3-aes-128-gcm	SS 2022 (recomendado)
2022-blake3-aes-256-gcm	SS 2022 (predeterminado)
2022-blake3-chacha20-poly1305	SS 2022, mono-usuario

Lógica del panel según los métodos:

- **Métodos 2022** (2022-blake3-*) se consideran «SS 2022». El método 2022-blake3-chacha20-poly1305 es **mono-usuario** (no admite multi-usuario); los demás métodos 2022 permiten varios clientes. El campo de contraseña (con botón de generación que ajusta la longitud de la clave al método) se muestra en el formulario precisamente para los métodos 2022.
- **Cifrados obsoletos** (aes-* , chacha20-*) funcionan según el esquema clásico «un método + una contraseña».

Normalización antes de iniciar Xray: para los cifrados obsoletos, cada cliente debe tener `method` coincidente con el método del inbound (de lo contrario Xray falla con «unsupported cipher method:»). Para los métodos 2022 es al contrario – el campo `method` del cliente debe estar **vacío** (de lo contrario Xray rechaza el inbound con «users must have empty method»). El panel ajusta los datos automáticamente al cambiar el método.

Regeneración de claves de cliente al cambiar el tamaño de clave: para Shadowsocks-2022, al cambiar el método de cifrado a uno con un tamaño de clave diferente (por ejemplo, entre 2022-blake3-aes-256-gcm y 2022-blake3-aes-128-gcm), el panel regenera automáticamente las PSK de los clientes para la nueva longitud al guardar el inbound. De lo contrario, las claves antiguas mantendrían la longitud anterior y Xray las rechazaría. Consecuencia: los clientes afectados deben obtener la suscripción de nuevo – los enlaces anteriores dejarán de funcionar.

Cuándo elegir Shadowsocks: para despliegues sencillos sin enmascaramiento TLS; la elección moderna son los métodos 2022-blake3.

Ejemplo: bloque `settings` de Shadowsocks para el método 2022-blake3 (modo multiusuario). El inbound tiene su propia contraseña (clave base64 de la longitud requerida); cada cliente tiene la suya; el campo `method` del cliente está **vacío**:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWlcmU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMilieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Para cifrados legacy (aes-256-gcm y similares) es al contrario: hay una sola contraseña para el inbound, y el `method` del cliente debe coincidir con el método del inbound.

5.7. Dokodemo-door / Tunnel (forwarder transparente)

Propósito: forwarder transparente (en el panel – protocolo `tunnel`, que implementa el comportamiento de `dokodemo-door`). Acepta tráfico y lo reenvía a la dirección/puerto especificados, sin autenticación ni clientes.

Campos del bloque `settings`:

Campo	Valor predeterminado	Descripción
<code>rewriteAddress</code>	(ninguno)	«Reescribir dirección» (en inglés «Rewrite address») – dirección de destino a la que se redirige el tráfico
<code>rewritePort</code>	(ninguno)	«Reescribir puerto» (en inglés «Rewrite port») – puerto de destino (0–65535)
<code>allowedNetwork</code>	<code>tcp,udp</code>	«Red permitida» (en inglés «Allowed network»). Opciones: <code>tcp,udp</code> , <code>tcp</code> , <code>udp</code>
<code>portMap</code>	<code>{}</code>	«Asignación de puertos» – mapa puerto→puerto (Record<string,string>)
<code>followRedirect</code>	<code>false</code> (desactivado)	«Seguir redirect» (en inglés «Follow redirect») – usar la dirección de destino original de la conexión interceptada

Pestaña «Transport» para Tunnel: los inbounds de este tipo tienen disponible la pestaña «**Transport**», limitada a la configuración de `sockopt` – suficiente para el modo **TProxy** (proxy transparente/redirect a través de `sockopt.tproxy`). La lista desplegable de selección de transporte (`network`) y la pestaña «Security» para Tunnel están ocultas, ya que este tipo no admite TLS/REALITY.

Cuándo elegir: para proxy transparente/redirección de puertos a servicios internos.

5.8. SOCKS / HTTP (protocolo `mixed`)

En esta compilación no existe un protocolo `socks` separado – SOCKS y el proxy HTTP están unificados en el protocolo `mixed` (SOCKS + HTTP combinados). Además, existe un proxy `http` puro por separado.

5.8.1. Mixed (SOCKS + HTTP)

Campos del bloque `settings` :

Campo	Valor predeterminado	Descripción
<code>auth</code>	<code>password</code>	«Auth» – modo de autenticación. Opciones: <code>password</code> (por usuario/contraseña) o <code>noauth</code> (sin autorización)
<code>accounts</code>	(opcional)	«Cuentas» – lista de pares user/pass. Con <code>auth = noauth</code> el campo no se escribe en la configuración
<code>udp</code>	<code>false</code> (desactivado)	Interruptor «UDP» – soporte UDP a través de SOCKS
<code>ip</code>	<code>127.0.0.1</code>	«UDP IP» – dirección local para asociaciones UDP. El campo solo se muestra si <code>udp</code> está activado

Las cuentas se añaden con el botón «Agregar»; al añadir se generan un nombre de usuario aleatorio (8 caracteres) y una contraseña (12 caracteres), que pueden editarse.

5.8.2. HTTP (proxy puro)

Propósito: proxy HTTP de reenvío clásico. A nivel de Xray no rastrea clientes como «facturables» (sin email/límites) – solo existe una lista de cuentas.

Campos del bloque `settings` :

Campo	Valor predeterminado	Descripción
<code>accounts</code>	<code>[]</code>	«Cuentas» – lista de pares user/pass (ambos campos son obligatorios)
<code>allowTransparent</code>	<code>false</code> (desactivado)	«Permitir transparente» (en inglés «Allow transparent») – reenviar solicitudes con el encabezado Host original

Cuándo elegir SOCKS/HTTP: para acceso proxy local o de servicio sin enmascaramiento complejo. `mixed` es conveniente porque un solo puerto atiende tanto clientes SOCKS como HTTP.

5.9. WireGuard (inbound)

Propósito: inbound WireGuard. A diferencia de los protocolos proxy, no opera con «clientes» – en su lugar se configuran **peers** (dispositivos que el servidor acepta). El transporte y TLS/REALITY no son aplicables.

Campos del bloque `settings` :

Campo	Valor predeterminado	Descripción
<code>secretKey</code>	—	Clave privada del servidor (obligatoria). Hay un botón de generación adyacente; la clave pública se muestra automáticamente (campo de solo lectura)
<code>mtu</code>	(opcional)	MTU de la interfaz
<code>noKernelTun</code>	<code>false</code> (desactivado)	«TUN sin kernel» (en inglés «No-kernel TUN») – usar TUN en espacio de usuario en lugar del del kernel
<code>domainStrategy</code>	(opcional)	«Domain Strategy» – estrategia de resolución de dominios: <code>ForceIP</code> , <code>ForceIPv4</code> , <code>ForceIPv4v6</code> , <code>ForceIPv6</code> , <code>ForceIPv6v4</code>
<code>peers</code>	<code>[]</code>	Lista de peers

Campos de cada peer:

Campo del peer	Valor predeterminado	Descripción
<code>privateKey</code>	(opcional)	Clave privada del cliente – se almacena para que el panel pueda generar la configuración para el usuario (solo en los peers del inbound)
<code>publicKey</code>	—	Clave pública del peer (obligatoria)
<code>preSharedKey</code> (PSK)	(opcional)	Clave compartida adicional
<code>allowedIPs</code>	<code>[]</code>	IPs permitidas. Al añadir un nuevo peer, el panel propone automáticamente la siguiente dirección libre (por defecto <code>10.0.0.2/32</code>)
<code>keepAlive</code>	(opcional)	«Keep-alive» – intervalo de mantenimiento de conexión
<code>comment</code>	(opcional)	«Comment» – etiqueta arbitraria del peer; se muestra junto al encabezado «Peer N» y se incluye en el enlace de compartición y en el <code>remark</code> del archivo <code>.conf</code>

El botón «Agregar peer» genera un nuevo par de claves y asigna el siguiente `allowedIPs` . Cada peer puede eliminarse (para el último que quede, la eliminación no está disponible).

El campo «Comment» del peer ayuda a distinguir los dispositivos: su texto se muestra en el formulario junto al encabezado «Peer N», y también aparece en el enlace de compartición y en el `remark` del archivo `.conf` generado, por lo que el dispositivo es fácilmente identificable en la aplicación cliente. Este campo es propio del panel – xray-core ignora los campos desconocidos del peer.

Domain Strategy y pestaña Transport

Además de los peers, el inbound WireGuard tiene el campo **Domain Strategy** (estrategia de resolución de dominios: `ForceIP` , `ForceIPv4` , `ForceIPv4v6` , `ForceIPv6` , `ForceIPv6v4`). El campo es opcional y se escribe en la configuración solo si está definido.

El campo **Workers** (`workers` , número de hilos de trabajo) se eliminó de los formularios de WireGuard (tanto inbound como outbound): a partir de xray-core v26.6.22 el motor ya no lo utiliza y se basa en el

mecanismo interno de wireguard-go. Las configuraciones guardadas anteriormente funcionan sin cambios – al leerlas el campo simplemente se descarta; no se requiere migración.

Para WireGuard también está disponible la pestaña «**Transport**» – pero en versión reducida: en ella solo se configuran `sockopt` y la ofuscación **Finalmask**. La lista desplegable de selección de transporte (`network`) está oculta, ya que WireGuard siempre escucha por UDP. En los registros de ruido (noise) de Finalmask, **Rand Range** se define como un campo separado (rango de bytes 0–255, con validación), y como método de ofuscación para WireGuard e Hysteria está disponible **Salamander**.

Cuándo elegir WireGuard: cuando se necesita un túnel VPN WireGuard propiamente dicho, y no un proxy enmascarado.

5.10. Hysteria (v2 por defecto)

Propósito: inbound Hysteria sobre QUIC. El panel trabaja por defecto con la versión 2. Cada cliente se autentica con el token `auth` en lugar de UUID/contraseña. TLS para Hysteria está siempre disponible (ver tabla de capacidades en 5.2).

Campos del bloque `settings` :

Campo	Valor predeterminado	Descripción
<code>version</code>	2	Versión del protocolo (mínimo 1; el panel usa 2 por defecto)
<code>clients</code>	[]	Lista de clientes

El campo clave de cada cliente es `auth` (token, obligatorio) más los campos comunes (`email` , límites, `enable` , `tgId` , `subId` , `comment` , `reset`).

Los parámetros adicionales se establecen en `streamSettings.hysteriaSettings` :

Campo	Valor / opciones	Descripción
<code>version</code>	fijado en 2 (campo bloqueado)	«Versión» (en inglés «Version»)
<code>udpIdleTimeout</code>	(entero ≥ 1, seg.)	«UDP idle timeout (s)» – tiempo de espera de inactividad UDP
<code>masquerade</code>	desactivado por defecto	«Masquerade» – enmascaramiento como servidor web normal ante solicitudes «no solicitadas»

Al activar `masquerade` se puede elegir el tipo (`type`):

- ```` – predeterminado (página 404);
- `proxy` – proxy inverso (campos «Upstream URL», «Reescribir Host», «Omitir TLS verify»);
- `file` – servir directorio (campo «Directorio», por ejemplo `/var/www/html`);
- `string` – respuesta fija (campos «Código de estado», «Body», «Encabezados»).

Cuándo elegir Hysteria: cuando se necesita transporte QUIC y resistencia en canales inestables/móviles; el enmascaramiento aumenta el sigilo del punto de entrada.

5.11. MTProto (proxy para Telegram)

Añadido en la versión **3.3.0**. Valor del protocolo — `mtproto`.

MTProto es el protocolo proxy nativo de Telegram. En 3X-UI, este inbound **no es gestionado por Xray sino por un proceso separado `mtg`**, controlado por el propio panel. El panel verifica periódicamente los inbounds MTProto habilitados contra los procesos `mtg` en ejecución: levanta los que faltan, detiene los sobrantes y recoge contadores de tráfico de las métricas de `mtg`. Por ello, la **contabilidad de tráfico** de este inbound funciona como en los protocolos habituales.

Aviso oficial en el formulario:

«MTProto es gestionado por un proceso separado `mtg`, no por Xray. Las configuraciones de transporte y los clientes no aplican aquí — comparte el enlace a continuación en Telegram.»

Consecuencias:

- Las pestañas «**Transport**» (**Stream Settings**) y «**Clientes**» **no se aplican a este inbound** — el acceso se define por un único secreto, no por una lista de clientes.
- El inbound MTProto se inicia **solo en el panel principal**; no se despliega en nodos secundarios (los inbounds con `NodeID` definido se omiten).
- La pestaña «**Sniffing**» está oculta para MTProto — este protocolo es gestionado por el proceso `mtg`, no por Xray, por lo que el sniffing no le aplica.

Campos. Se almacenan en `settings` del inbound:

Campo en la UI	Clave	Descripción
Remark	<code>remark</code>	Etiqueta del inbound.
Listen IP	<code>listen</code>	IP de escucha (vacío = todas las interfaces).
Port	<code>port</code>	Puerto del proxy.
Secreto	<code>settings.secret</code>	Secreto de acceso en formato FakeTLS .
Dominio de cobertura (FakeTLS)	<code>settings.fakeTlsDomain</code>	Dominio bajo cuyo tráfico HTTPS se enmascara el proxy.

Formato del secreto (FakeTLS). El panel ajusta automáticamente el secreto al formato correcto: resultado = `ee` + 32 caracteres hex + código hex del dominio de cobertura, es decir, `ee<hex32><hex(fakeTlsDomain)>`. El prefijo `ee` activa el modo FakeTLS, y el dominio (por ejemplo, un sitio conocido) sirve para enmascarar el tráfico como HTTPS convencional. Basta con indicar el dominio — el panel construirá el resto automáticamente.

Domain-fronting y opciones avanzadas de mtg

El inbound MTProto tiene parámetros adicionales del proceso `mtg`. Los campos **Domain fronting IP**, **Domain fronting port** y **Domain fronting PROXY protocol** especifican a dónde `mtg` envía el tráfico que no es de Telegram (por ejemplo, a un sitio NGINX falso): si se deja el IP vacío, se usa el dominio FakeTLS mediante DNS; el puerto por defecto es `443`. Adicionalmente están disponibles **Accept PROXY protocol** (para el listener), **IP preference** (`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`) y **Debug logging**. Cada valor se escribe en el archivo `mtg-<id>.toml` solo si está definido.

Enrutamiento del tráfico de Telegram a través de Xray

El interruptor «**Route through Xray**» (desactivado por defecto) y el campo opcional **Outbound** permiten someter el egress de Telegram al enrutador de Xray. Al activarlo, el panel inserta en la configuración de Xray un puente SOCKS local con la etiqueta del propio inbound, y `mtg` envía el tráfico de Telegram a través de él. A partir de ese momento el tráfico puede emparejarse con reglas en la pestaña «Routing» o dirigirse forzosamente al outbound o balanceador seleccionado mediante el campo **Outbound** (si el campo está vacío, deciden las reglas de enrutamiento).

Cómo distribuirlo al usuario. Para el inbound MTProto el panel genera un enlace de invitación:

Ejemplo: secreto FakeTLS y enlace listo. Si el dominio de cobertura es `www.cloudflare.com`, el secreto se construye como `ee` + 32 caracteres hex + código hex del dominio, por ejemplo:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Enlace de invitación listo (se envía al usuario junto con el código QR en Telegram):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<dirección>&port=<puerto>&secret=<secreto>
```

(equivalente – `https://t.me/proxy?server=...&port=...&secret=...`). Este enlace y el código QR deben enviarse al usuario de Telegram – al abrirlo, el proxy se añade directamente a la aplicación. El enlace también se sirve a través del servidor de suscripciones.

Cuándo usar. Método estándar para eludir el bloqueo de Telegram; el enmascaramiento FakeTLS (dominio de cobertura) hace que el tráfico se asemeje a una visita normal al sitio indicado.

5.12. Guía rápida para elegir protocolo

- **VLESS** – elección por defecto; la mejor opción con REALITY o TLS + XTLS-Vision, admite autenticación poscuántica.
- **Trojan** – enmascaramiento como HTTPS con fallbacks a un servidor web.
- **VMess** – compatibilidad con clientes antiguos.
- **Shadowsocks** – proxy sencillo sin TLS; la elección moderna son los métodos `2022-blake3-*`.

- **Hysteria** – QUIC, resistencia en canales deficientes.
 - **mixed / http** – proxies SOCKS/HTTP de servicio.
 - **WireGuard** – túnel VPN completo.
 - **tunnel** – redirección de puertos transparente.
 - **MTProto** – proxy para eludir el bloqueo de Telegram (FakeTLS); proceso separado `mtg`.
-

6. Transporte (Stream Settings)

El transporte (en la interfaz del panel – campo «**Transporte**», en inglés *Transmission*) determina la forma en que Xray-core transfiere datos dentro de un inbound: qué protocolo de red se utiliza sobre TLS/Reality y cómo se encuadra exactamente el tráfico. Estos parámetros se guardan en el objeto `streamSettings` de la configuración de Xray y se configuran en la pestaña de transporte del editor de inbound. El cifrado (TLS / Reality) se trata en una sección aparte – aquí solo se describe la selección de red y sus parámetros.

6.1. Selección de la red de transmisión

La red se selecciona en el desplegable «**Transporte**» (`streamSettings.network`). El valor predeterminado es `tcp` (que aparece en la lista como **RAW**). Las opciones disponibles son:

Valor en la lista	Campo <code>network</code>	Transporte
RAW	<code>tcp</code>	TCP estándar (renombrado a RAW en versiones recientes de Xray), opcionalmente con ofuscación HTTP
mKCP	<code>kcp</code>	Transporte UDP fiable mKCP
WebSocket	<code>ws</code>	WebSocket sobre HTTP(S)
gRPC	<code>grpc</code>	Túnel gRPC (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP – transporte multiplexado moderno

Al cambiar el valor, el panel borra el bloque de configuración de la red anterior y rellena el bloque de la nueva red con los valores predeterminados de su esquema, de modo que cada campo del subformulario siempre tiene un valor inicial con sentido.

Importante. En esta versión del panel **el transporte HTTP/2 (`h2`) no está disponible en la lista** – fue excluido del conjunto de redes; para un túnel bidireccional similar a HTTP/2 se usa gRPC, y para el transporte moderno enmascarado en HTTP – XHTTP. El transporte **Hysteria** (`hysteria`) no se selecciona a través de esta lista: está vinculado de forma fija al protocolo Hysteria y aparece automáticamente cuando el propio inbound se crea con el protocolo Hysteria (ver apartado 6.8).

A continuación se describe cada red y cada uno de sus campos por separado.

6.2. RAW / TCP (`tcpSettings`)

Transporte TCP básico. De forma predeterminada el tráfico se transmite «tal cual»; opcionalmente se puede enmascarar como un intercambio HTTP/1.1 ordinario.

Campo	Valor predeterminado	Descripción
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (desact.)	Aceptar la cabecera PROXY protocol de un balanceador/proxy superior
Ofuscación HTTP (<code>header.type</code>)	<code>none</code> (desact.)	Activa el enmascaramiento del tráfico como HTTP/1.1

Proxy Protocol

El interruptor «**Proxy Protocol**» (`acceptProxyProtocol`). Cuando está activado, Xray espera en la conexión entrante la cabecera PROXY protocol y extrae de ella la IP real del cliente. Se activa únicamente si delante del panel hay un proxy inverso o balanceador (por ejemplo, HAProxy o nginx con `send-proxy`) que añada dicha cabecera. Desactivado de forma predeterminada.

Ofuscación HTTP (camouflage)

El interruptor «**HTTP Obfuscation**». Controla el campo `header` :

- **Desactivado** → `header.type = "none"` (el campo `header` simplemente no aparece en el cable). TCP limpio.
- **Activado** → `header.type = "http"`. El tráfico se encuadra simulando una petición y respuesta HTTP/1.1. Al activarlo, el panel rellena inmediatamente los subobjetos `request` y `response` con valores predeterminados.

Cuando la ofuscación HTTP está activada, aparecen los campos de configuración de la petición y respuesta simuladas.

Cabecera de petición (`header.request`):

Campo	Clave	Valor predeterminado	Descripción
Versión de petición	<code>request.version</code>	<code>1.1</code>	Versión HTTP en la línea de inicio de la petición
Método de petición	<code>request.method</code>	<code>GET</code>	Método HTTP de la petición simulada
Ruta de petición	<code>request.path</code>	<code>/</code>	Ruta(s). Se introduce como lista de valores separados por coma; en el cable es un array de cadenas. Si se deja vacío, se sustituye por <code>/</code>
Cabeceras de petición	<code>request.headers</code>	<code>{}</code> (vacío)	Tabla «Nombre/Valor» de cabeceras HTTP. Se almacena como mapa <code>nombre → [valores]</code> (a un mismo nombre pueden corresponder varios valores)

Cabecera de respuesta (`header.response`):

Campo	Clave	Valor predeterminado	Descripción
Versión de respuesta	<code>response.version</code>	<code>1.1</code>	Versión HTTP en la línea de inicio de la respuesta
Estado de respuesta	<code>response.status</code>	<code>200</code>	Código de estado HTTP de la respuesta simulada
Motivo de respuesta	<code>response.reason</code>	<code>OK</code>	Descripción textual del estado (reason-phrase)
Cabeceras de respuesta	<code>response.headers</code>	<code>{}</code> (vacío)	Tabla «Nombre/Valor» de cabeceras de respuesta (mapa <code>nombre → [valores]</code>)

Los campos de cabeceras se editan línea a línea – cada línea define el nombre de la cabecera (`Nombre`) y su valor (`Valor`). Estos parámetros solo se usan para enmascarar el aspecto externo del tráfico; no afectan a la criptografía. Los valores predeterminados (`GET / HTTP/1.1` , respuesta `200 OK`) son adecuados para la mayoría de los escenarios – conviene modificarlos solo si se necesita imitar un sitio o servicio concreto.

Ejemplo de `streamSettings` para RAW con ofuscación HTTP:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
    },
    "response": {
      "version": "1.1",
      "status": "200",
      "reason": "OK"
    }
  }
}
```

Nótese que `path` en el cable es un array de cadenas, y cada cabecera también es un array de valores (`Host → ["www.example.com"]`).

6.3. mKCP (`kcpSettings`)

mKCP es un transporte fiable sobre UDP. Resulta útil en canales con pérdida de paquetes y alta latencia, pero genera mayor tráfico de control. Todos los valores predeterminados coinciden con los recomendados en xray-core.

Campo	Clave	Predeterminado	Permitido	Descripción
MTU	<code>mtu</code>	1350	576–1460	Tamaño máximo de paquete (bytes). Se reduce ante problemas de fragmentación
TTI (ms)	<code>tti</code>	20	10–100	Intervalo de transmisión (ms). Menor = menor latencia, pero mayor sobrecarga
Uplink (MB/s)	<code>uplinkCapacity</code>	5	≥ 0	Ancho de banda estimado de subida (MB/s)
Downlink (MB/s)	<code>downlinkCapacity</code>	20	≥ 0	Ancho de banda estimado de bajada (MB/s)
Multiplicador CWND	<code>cwndMultiplier</code>	1	≥ 1	Multiplicador de la ventana de congestión (congestion window)
Ventana máx. de envío	<code>maxSendingWindow</code>	2097152	≥ 0	Tamaño máximo de la ventana de envío

Notas sobre los campos:

- **Uplink / Downlink capacity** determinan la agresividad con la que mKCP ocupa el canal. Se ajustan según el ancho de banda real: valores demasiado altos generan tráfico innecesario; demasiado bajos suponen una infrautilización del canal.
- **TTI** afecta directamente al compromiso «latencia [?] sobrecarga»: valores menores reducen la latencia pero aumentan el volumen de paquetes de control.
- **MTU** limita el tamaño de un paquete mKCP; reducirlo ayuda en canales donde los paquetes UDP grandes se fragmentan o se pierden.

En esta versión del panel el campo «seed» (contraseña de ofuscación de mKCP) y el desplegable de **tipo de cabecera/ofuscación** (`none` , `srtp` , `utp` , `wechat-video` , `dtls` , `wireguard`) **no están presentes como campos independientes** en el subformulario de mKCP — la ofuscación a nivel de transporte se ha trasladado al mecanismo general «FinalMask» (incluido el modo `mkcp-legacy`), descrito en la sección correspondiente. El parámetro «congestion» como casilla independiente tampoco está expuesto; el control de congestión se configura mediante `cwndMultiplier` y `maxSendingWindow` .

6.4. WebSocket (`wsSettings`)

Transporte WebSocket sobre HTTP(S). Se propaga bien a través de CDN y proxies inversos, y se camufla como tráfico web ordinario.

Campo	Clave	Predeterminado	Descripción
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Aceptar la cabecera PROXY protocol del proxy superior (ver apartado 6.2)
Host	<code>host</code>	<code>""</code> (vacío)	Valor de la cabecera HTTP <code>Host</code> . Se especifica al trabajar tras CDN/domain-fronting
Ruta	<code>path</code>	<code>/</code>	Ruta en la línea de petición del handshake WebSocket

Campo	Clave	Predeterminado	Descripción
Período de heartbeat	heartbeatPeriod	0	Intervalo de envío de tramas heartbeat (en segundos). 0 desactiva el heartbeat
Cabeceras	headers	{ } (vacío)	Cabeceras HTTP adicionales del handshake. Mapa «Nombre → Valor» (solo valores de cadena, sin arrays)

Notas:

- **Ruta** debe coincidir en el servidor (inbound) y en el cliente. A menudo, el servidor web enmascara el punto de entrada tras esta ruta.
- **Host** tiene sentido especificarlo si el inbound está detrás de un CDN o se usa domain-fronting; de lo contrario puede dejarse vacío.
- **Período de heartbeat** mantiene la conexión «viva» al pasar por proxies/CDN que terminan agresivamente las sesiones inactivas. Por defecto (0) el heartbeat está desactivado.
- A diferencia de RAW, la tabla de cabeceras de WebSocket usa el formato «plano» `nombre → valor` (un único valor por cabecera).

Ejemplo de `streamSettings` para WebSocket detrás de CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

Los valores de `host` y `path` deben coincidir en el cliente; a diferencia de RAW, el valor de la cabecera aquí es una cadena ordinaria, no un array.

6.5. gRPC (`grpcSettings`)

El transporte con menor número de parámetros. Tuneliza el tráfico dentro de llamadas gRPC (sobre HTTP/2); es muy compatible con CDN que admiten gRPC. No hay ofuscación de cabeceras.

Campo	Clave	Predeterminado	Descripción
Nombre de servicio (Service Name)	serviceName	"" (vacío)	Nombre del servicio gRPC (en la práctica, la «ruta» del túnel). Debe coincidir en servidor y cliente
Authority	authority	"" (vacío)	Valor de la pseudo-cabecera <code>:authority</code> (equivalente a <code>Host</code> en HTTP/2). Se especifica al trabajar con CDN/ dominio

Campo	Clave	Predeterminado	Descripción
Multi Mode	<code>multiMode</code>	<code>false</code> (desact.)	Activa la multiplexación de varios flujos gRPC paralelos dentro de una misma conexión

Notas:

- **Service Name** es el identificador principal del canal gRPC; debe ser idéntico en ambos lados. Un valor vacío es válido, pero normalmente se usa una cadena no obvia para el enmascaramiento.
- **Authority** afecta al `:authority` que se envía en los fotogramas HTTP/2; es necesario principalmente al proxyar a través de CDN.
- **Multi Mode** permite que varios flujos lógicos circulen por una misma conexión física; se activa para mejorar el rendimiento cuando tanto el servidor como el cliente lo soportan.

Ejemplo de `streamSettings` para gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (aquí `GunService`) actúa como «ruta» del túnel y debe coincidir en servidor y cliente.

6.6. HTTPUpgrade (`httpupgradeSettings`)

Transporte basado en el mecanismo HTTP Upgrade (similar a WebSocket, pero sin el protocolo WebSocket en sí). También se propaga bien a través de proxies y CDN. El conjunto de campos repite el de WebSocket, pero **sin** período de heartbeat.

Campo	Clave	Predeterminado	Descripción
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Aceptar la cabecera PROXY protocol del proxy superior
Host	<code>host</code>	<code>""</code> (vacío)	Valor de la cabecera HTTP <code>Host</code>
Ruta	<code>path</code>	<code>/</code>	Ruta de la petición HTTP con cabecera <code>Upgrade</code>
Cabeceras	<code>headers</code>	<code>{}</code> (vacío)	Cabeceras HTTP adicionales. Mapa «plano» <code>nombre</code> → <code>valor</code> (igual que en WebSocket)

El propósito de los campos **Host**, **Ruta** y **Cabeceras** coincide con el de WebSocket (apartado 6.4). El heartbeat no está previsto para HTTPUpgrade — es una particularidad de WebSocket.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (también conocido como SplitHTTP) es un transporte HTTP multiplexado moderno de xray-core. Divide los flujos ascendente y descendente en peticiones HTTP separadas, lo que resulta muy adecuado para CDN y entornos con restricciones en la duración de las conexiones. No todos los campos son visibles en el editor a la vez: algunos aparecen en función del modo seleccionado (`mode`) y de los interruptores activados.

Campos básicos (siempre visibles)

Campo	Clave	Predeterminado	Descripción
Host	<code>host</code>	<code>""</code> (vacío)	Valor de la cabecera HTTP <code>Host</code>
Ruta	<code>path</code>	<code>/</code>	Ruta base de las peticiones HTTP
Modo (<code>Mode</code>)	<code>mode</code>	<code>auto</code>	Modo de transmisión (ver más abajo)
Server Max Header Bytes	<code>serverMaxHeaderBytes</code>	<code>0</code>	Límite del tamaño de cabeceras de petición en el servidor (bytes). <code>0</code> – valor predeterminado de xray-core
Padding Bytes	<code>xPaddingBytes</code>	<code>100-1000</code>	Rango de relleno (padding) aleatorio (en bytes, formato <code>mín-máx</code>) para dificultar el análisis de tamaños
Cabeceras	<code>headers</code>	<code>{}</code> (vacío)	Cabeceras HTTP adicionales. Mapa «plano» <code>nombre → valor</code>
Método HTTP Uplink	<code>uplinkHTTPMethod</code>	<code>""</code> (Default = POST)	Método HTTP de las peticiones ascendentes. Opciones: vacío (predeterminado POST), <code>POST</code> , <code>PUT</code> , <code>GET</code> (el último solo disponible en modo <code>packet-up</code>)
Padding Obfs Mode	<code>xPaddingObfsMode</code>	<code>false</code> (desact.)	Activa la ofuscación avanzada de padding y muestra campos adicionales (ver más abajo)
No SSE Header	<code>noSSEHeader</code>	<code>false</code> (desact.)	No enviar la cabecera <code>Content-Type: text/event-stream</code> (SSE). Se activa si interfiere con el paso por nodos intermedios

Campo «Modo» (`mode`)

Lista desplegable con los valores:

Valor	Descripción
<code>auto</code>	Selección automática de modo (predeterminado)
<code>packet-up</code>	El flujo ascendente se divide en peticiones HTTP individuales (una petición por paquete)
<code>stream-up</code>	El flujo ascendente se transmite en una única petición de streaming prolongada
<code>stream-one</code>	Una única petición de streaming bidireccional compartida

La elección del modo determina qué campos adicionales se hacen visibles.

Campos visibles solo con mode = packet-up :

Campo	Clave	Predeterminado	Descripción
Máx. peticiones POST en búfer	scMaxBufferedPosts	30	Máximo de peticiones POST ascendentes en búfer simultáneamente
Tamaño máx. de subida (bytes)	scMaxEachPostBytes	1000000	Tamaño máximo de una petición POST ascendente (bytes)
Uplink Data Placement	uplinkDataPlacement	"" (Default = body)	Dónde colocar los datos del flujo ascendente: body , header , cookie , query
Uplink Data Key	uplinkDataKey	""	Nombre de clave/cabecera para los datos uplink. Aparece solo si uplinkDataPlacement está definido y no es body

Campo visible solo con mode = stream-up :

Campo	Clave	Predeterminado	Descripción
Stream-Up Server	scStreamUpServerSecs	20-80	Rango de tiempo de mantenimiento de la conexión de streaming del servidor (en segundos, formato mín-máx)

Campos de ofuscación de padding (visibles con xPaddingObfsMode = act.)

Campo	Clave	Predeterminado	Descripción
Padding Key	xPaddingKey	"" (placeholder x_padding)	Nombre de clave para el padding
Padding Header	xPaddingHeader	"" (placeholder X-Padding)	Nombre de la cabecera HTTP en la que se transmite el padding
Padding Placement	xPaddingPlacement	"" (Default = queryInHeader)	Dónde colocar el padding: queryInHeader , header , cookie , query
Padding Method	xPaddingMethod	"" (Default = repeat-x)	Método de generación del padding: repeat-x o tokenish

Ubicación de sesión y secuencia (siempre visibles)

Campo	Clave	Predeterminado	Descripción
Session ID Placement	sessionIDPlacement	"" (Default = path)	Dónde transmitir el identificador de sesión: path , header , cookie , query
Session ID Key	sessionIDKey	"" (placeholder x_session)	Nombre de clave de sesión. Aparece solo si sessionIDPlacement está definido y no es path

Campo	Clave	Predeterminado	Descripción
Session ID Table	<code>sessionIDTable</code>	<code>""</code> (placeholder <code>Base62</code>)	Conjunto de caracteres para la generación de identificadores de sesión. Se puede elegir uno predefinido del desplegable con autocompletado (<code>ALPHABET</code> , <code>Alphabet</code> , <code>BASE36</code> , <code>Base62</code> , <code>HEX</code> , <code>alphabet</code> , <code>base36</code> , <code>hex</code> , <code>number</code>) o introducir una cadena ASCII arbitraria. Vacío – valor predeterminado de xray-core
Session ID Length	<code>sessionIDLength</code>	<code>""</code> (vacío)	Longitud o rango (por ejemplo <code>8-16</code>) de los identificadores generados. Se muestra solo cuando <code>Session ID Table</code> está definido; el mínimo debe ser mayor que 0
Sequence Placement	<code>seqPlacement</code>	<code>""</code> (Default = <code>path</code>)	Dónde transmitir el número de secuencia del paquete: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Sequence Key	<code>seqKey</code>	<code>""</code> (placeholder <code>x_seq</code>)	Nombre de clave de secuencia. Aparece solo si <code>seqPlacement</code> está definido y no es <code>path</code>

Los campos de sesión fueron renombrados en xray-core v26.6.22: antes se llamaban **Session Placement** / **Session Key** (`sessionPlacement` / `sessionKey`) – ahora son **Session ID Placement** / **Session ID Key** (`sessionIDPlacement` / `sessionIDKey`); el nombre anterior ya no es reconocido por el núcleo. Los inbounds guardados antes de la actualización se migran a las nuevas claves automáticamente – no es necesario volver a guardarlos.

Recomendaciones:

- Para la mayoría de las instalaciones es suficiente dejar **Modo = auto** , definir **Ruta/Host** y (al trabajar con CDN) sincronizarlos con el cliente.
- Los campos de ubicación (`*Placement` / `*Key`) y de ofuscación de padding solo son necesarios para un ajuste fino en escenarios específicos anti-DPI/CDN; cuando están vacíos se utilizan los valores predeterminados de xray-core indicados entre paréntesis.
- Los parámetros relativos al lado cliente/outbound (por ejemplo, intervalos de reintento de POST, tamaños de chunk) no se muestran en el formulario de inbound – el listener del servidor los ignora. El multiplexor XMUX, en cambio, sí está disponible en el formulario de inbound (ver más abajo).
- **Los valores predeterminados de servicio no se escriben.** El panel ya no escribe en las configuraciones XHTTP los valores predeterminados de servicio `scMaxEachPostBytes` y `scMinPostsIntervalMs` – se aplican los valores internos de xray-core. Esto elimina la firma DPI constante (el literal `scMinPostsIntervalMs=30`) por la que anteriormente podía bloquearse el tráfico. Para los inbounds ya guardados, los valores que coinciden con los predeterminados de xray-core no se incluyen en los enlaces ni en la suscripción (no es necesario volver a guardar el inbound); los valores definidos manualmente se conservan.

Ejemplo de `streamSettings` para XHTTP (modo `auto`):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Para la mayoría de las instalaciones son suficientes estos cuatro campos; los campos de ubicación de sesión/ secuencia y de ofuscación de padding se dejan vacíos – en ese caso se usan los valores predeterminados de xray-core.

XMUX (multiplexación de conexiones)

El interruptor **XMUX** (`enableXmux`) activa una capa de multiplexación que distribuye las peticiones paralelas entre un pequeño conjunto de conexiones físicas. Al activarlo se despliegan seis campos configurables (almacenados en `xhttpSettings.xmux`):

Campo	Clave	Predeterminado	Descripción
Max Concurrency	<code>maxConcurrency</code>	16-32	Máximo de peticiones simultáneas por conexión (rango mín-máx)
Max Connections	<code>maxConnections</code>	0	Máximo de conexiones físicas (0 – sin límite)
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (vacío)	Cuántas veces reutilizar una conexión
Max Request Times	<code>hMaxRequestTimes</code>	600-900	Máximo de peticiones por conexión (rango)
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800-3000	Tiempo durante el cual una conexión puede reutilizarse (segundos, rango)
Keep Alive Period	<code>hKeepAlivePeriod</code>	"" (vacío)	Período keep-alive para mantener la conexión activa

Importante. No se puede definir **Max Connections** y **Max Concurrency** al mismo tiempo – xray-core rechazará dicha configuración. De forma predeterminada, al activar XMUX el panel asigna `Max Concurrency = 16-32` ; si se define **Max Connections** (valor mayor que 0), el panel eliminará el valor predeterminado de `Max Concurrency` para evitar el conflicto.

6.8. Transporte Hysteria (`hysteriaSettings`)

El transporte **Hysteria** no se selecciona en la lista «Transporte»: se activa automáticamente cuando el inbound se crea con el protocolo Hysteria y está oculto para otros protocolos (al abandonar el protocolo Hysteria, la red vuelve forzosamente a `tcp`). Parámetros:

Campo	Clave	Predeterminado	Descripción
Versión	<code>version</code>	2 (fijo, campo bloqueado)	Versión de Hysteria. Solo se admite Hysteria 2
UDP idle timeout (s)	<code>udpIdleTimeout</code>	60	Tiempo de espera de inactividad de la sesión UDP (segundos). Rango permitido — 2–600; xray-core rechaza valores fuera de este intervalo al arrancar
Masquerade	<code>masquerade</code>	desact. (ausente)	Activa el enmascaramiento del listener como servidor HTTP/3 durante el sondeo

Con **Masquerade** activado aparece la selección de tipo (`type`) y los campos dependientes de él:

- **"" — default (404 page)**: se devuelve una página 404 estándar (no se requieren campos adicionales).
- **proxy (reverse proxy)**: proxy inverso hacia un sitio externo.
 - `url` (**Upstream URL**, placeholder `https://www.example.com`) — dirección de destino;
 - `rewriteHost` (**Reescribir Host**, predeterminado `false`) — sustituir la cabecera `Host`;
 - `insecure` (**Omitir verificación TLS**, predeterminado `false`) — no verificar el certificado TLS del upstream.
- **file (serve directory)**: servir archivos desde un directorio.
 - `dir` (**Directorio**, placeholder `/var/www/html`).
- **string (fixed body)**: respuesta HTTP fija.
 - `statusCode` (**Código de estado**, predeterminado `0`, rango 0–599);
 - `content` (**Body**) — cuerpo de la respuesta;
 - `headers` (**Cabeceras**) — mapa `nombre → valor`.

Masquerade permite que el inbound basado en Hysteria parezca un servidor HTTP/3 ordinario ante sondeos activos, lo que aumenta el sigilo. De forma predeterminada el enmascaramiento está desactivado.

Ejemplo de `hysteriaSettings` con proxy inverso (`masquerade` → `proxy`):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

En este caso, ante un sondeo, el listener devuelve la respuesta de `https://www.example.com`, enmascarándose como un sitio HTTP/3 ordinario.

6.9. Parámetros complementarios

Además de la selección de red, en la misma pestaña hay disponibles dos bloques generales independientes del transporte concreto (descritos en detalle en las secciones correspondientes):

- **External Proxy** (`externalProxy`) – lista de direcciones/puertos externos que se sustituyen en los enlaces de suscripción en lugar de la dirección del propio panel.
- **Sockopt** (`sockopt`) – opciones de socket de bajo nivel (TCP Fast Open, mark, estrategia de dominio, proxy transparente, etc.).

Real client IP (determinación de la IP real detrás de CDN/relay)

Cuando el inbound está detrás de un intermediario (CDN como Cloudflare, túnel L4/relay u otro panel), Xray ve la dirección del intermediario, no la del visitante real. Esa dirección aparece en la lista de clientes en línea y se usa para el límite de IP por cliente, lo que hace que ambos sean inútiles detrás de un proxy. Para recuperar la IP real, en la sección **Sockopt** del formulario de inbound hay un selector de preset **Real client IP** que combina las configuraciones de `acceptProxyProtocol` y `trustedXForwardedFor`:

Preset	Qué hace	Cuándo aplicar
Off / direct	Borra ambos campos.	El inbound es accesible directamente por los clientes
Cloudflare CDN	Establece <code>sockopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> .	WebSocket / HTTPUpgrade / XHTTP / gRPC detrás de CDN Cloudflare (nube naranja)
L4 relay / Spectrum (PROXY)	Activa <code>acceptProxyProtocol = true</code> .	Túnel L4/relay delante del inbound o Cloudflare Spectrum

Los presets son mutuamente excluyentes: elegir uno borra el campo del otro, por lo que un `trustedXForwardedFor` obsoleto no sobrescribe la IP recuperada por PROXY protocol. Debajo del preset siguen visibles el interruptor «raw» **Proxy Protocol** y la lista **Trusted X-Forwarded-For** – el preset simplemente los rellena por usted, y pueden modificarse manualmente si es necesario. Si el preset seleccionado no es compatible con el transporte actual (por ejemplo, PROXY protocol en mKCP), el formulario muestra una advertencia. Estos campos pertenecen únicamente al lado del servidor y **nunca se envían a los clientes en las suscripciones**.

Use solo uno. `acceptProxyProtocol` lee la IP real de la cabecera L4 del PROXY protocol, mientras que `trustedXForwardedFor` la lee de la cabecera HTTP de la petición; mezclarlos manualmente solo tiene sentido si su cadena de intermediarios lo requiere.

- **FinalMask** (`finalmask`) – mecanismo general de ofuscación a nivel de transporte (incluida la ofuscación legacy de mKCP), que reemplaza a los campos individuales «seed»/«header type» dentro de los subformularios de red.

7. Seguridad de la conexión: TLS, XTLS y REALITY

Cada inbound que admite transmisión a través de un flujo de transporte (VMess, VLESS, Trojan, Shadowsocks, Hysteria) dispone de la pestaña «Seguridad» en el editor. En ella se configura cómo se cifra y enmascara el canal de transporte. Hay tres modos disponibles, que se seleccionan mediante botones de radio:

Modo	Etiqueta en UI	Disponibilidad
none	Ninguno	Siempre (excepto Hysteria, donde TLS es obligatorio)
tls	TLS	Para VMess/VLESS/Trojan/Shadowsocks en redes <code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code> ; para Hysteria – siempre
reality	Reality	Solo para VLESS/Trojan en redes <code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>

El botón **Ninguno** no se muestra si el protocolo es Hysteria (para él, TLS es obligatorio). El botón **Reality** aparece únicamente con una combinación válida de protocolo y red (véase la tabla anterior).

Al cambiar de modo, el panel reconstruye completamente el bloque `streamSettings`: elimina `tlsSettings` y `realitySettings` del modo anterior e inserta los valores predeterminados para el modo seleccionado. En particular, al elegir **Reality**, el panel automáticamente: inserta un par aleatorio de `target` + `serverNames` (SNI) de su lista integrada de dominios populares, genera `shortIds` aleatorios y realiza una solicitud al servidor para obtener un par de claves X25519 actualizadas (`privateKey/publicKey`).

7.1. Las diferencias: TLS vs XTLS vs REALITY

- **TLS** – cifrado clásico del transporte mediante el protocolo TLS 1.2/1.3. El servidor debe tener un certificado válido (dominio propio + cadena de confianza). El tráfico tiene el aspecto de HTTPS ordinario, pero para un censor activo el handshake TLS hacia su dominio es reconocible; si se bloquea por SNI o el certificado no es de confianza, la conexión se corta o genera un error.
- **XTLS (Vision)** – no es un modo independiente en la lista «Seguridad», sino un mecanismo de *flow* sobre TLS o Reality. Se activa en el inbound del lado del cliente mediante el campo **Flow** = `xtls-rprx-vision` (o `xtls-rprx-vision-udp443`). Vision está disponible para VLESS en red `tcp` con `security = tls` o `security = reality`, así como para VLESS sobre el transporte `xhttp` con el cifrado VLESS habilitado (`vlessenc` / ML-KEM) – en ese caso el campo **Flow** también puede establecerse en `xtls-rprx-vision`, y el valor se incluye correctamente en el enlace `vless://` (`flow=xtls-rprx-vision`). Vision reduce el «doble cifrado» (TLS-in-TLS) entregando la carga útil directamente tras el handshake, lo que acelera la transmisión y mejora el camuflaje. Por eso la combinación **VLESS + Reality + Flow xtls-rprx-vision** se considera la configuración moderna recomendada.
- **REALITY** – mecanismo de camuflaje sin certificado propio. El servidor «toma prestado» el handshake TLS de un sitio externo real (`target` / `serverNames`), por lo que para un observador la conexión es indistinguible de una visita a ese sitio, y no se necesita ningún certificado. La autenticación se basa en un par de claves X25519 y un conjunto de `shortIds`. REALITY es resistente al sondeo activo (`active probing`) y al bloqueo por SNI, ya que el SNI apunta a un dominio externo real. El costo es una configuración más estricta (un `target` correcto con puerto, sincronización de claves con el cliente).

Regla breve para elegir:

- tiene su propio dominio y certificado válido y necesita apariencia de HTTPS simple → **TLS** (preferiblemente con Vision);
- no tiene dominio/certificado o necesita máxima invisibilidad ante DPI → **REALITY** (con Vision para VLESS/TCP).

7.2. Modo «Ninguno» (`none`)

El transporte se transmite sin envoltura TLS: los bloques `tlsSettings` y `realitySettings` se excluyen de `streamSettings`. El modo no tiene campos adicionales. Es adecuado cuando:

- el inbound escucha únicamente en `127.0.0.1` y sirve como destino de fallback (según la regla del panel, el inbound hijo de fallback debe escuchar en `127.0.0.1` con `security=none`);
- el cifrado/camuflaje lo proporciona una capa externa (por ejemplo, un proxy inverso Nginx delante del panel);
- se utiliza un protocolo con cifrado propio (Shadowsocks) en una red interna.

Para inbounds accesibles desde el exterior, el modo «Ninguno» no se recomienda.

Ejemplo: bloque `streamSettings` para TLS en red `tcp` (VLESS/Trojan/VMess). Así queda el resultado tras seleccionar el modo **TLS** y completar el SNI y las rutas al certificado:

```
"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}
```

7.3. Modo TLS

Campos del bloque `tlsSettings`. Los valores predeterminados se toman del esquema del panel.

Parámetros principales

Campo (etiqueta)	Valor predeterminado	Descripción
SNI (<code>serverName</code>)	<code>""</code> (vacío)	Server Name Indication – nombre de dominio presentado en el handshake TLS. Debe coincidir con el dominio del certificado. Marcador de posición en inglés: «Server Name Indication».
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → Auto	Lista de conjuntos de cifrado permitidos. Por defecto vacío – la elección queda en manos de Xray/Go (opción Auto). Modifíquelo solo si necesita restringir explícitamente los cifrados.
Versión mín/máx (<code>minMaxVersion</code>)	mín = <code>1.2</code> , máx = <code>1.3</code>	Versiones mínima y máxima de TLS. Valores disponibles: <code>1.0</code> , <code>1.1</code> , <code>1.2</code> , <code>1.3</code> . Se recomienda dejar <code>1.2</code> – <code>1.3</code> ; reducir el mínimo a 1.0/1.1 no es deseable (versiones obsoletas e inseguras).
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (en el formulario – el elemento None = <code>""</code> está disponible)	Huella TLS del client hello que se imita (uTLS fingerprint), para que el handshake parezca el de un navegador popular. Véase la lista más abajo. En TLS, el primer elemento de la lista es None (<code>""</code>), que desactiva la imitación.
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	Lista de protocolos de capa de aplicación negociados en TLS (selección múltiple). Valores admitidos: <code>h3</code> , <code>h2</code> , <code>http/1.1</code> . Por defecto se ofrecen <code>h2</code> y <code>http/1.1</code> .

Valores posibles de **uTLS fingerprint** (idénticos para TLS y REALITY): `chrome` , `firefox` , `safari` , `ios` , `android` , `edge` , `360` , `qq` , `random` , `randomized` , `randomizednoalpn` , `unsafe` . En el formulario TLS también está disponible la opción vacía **None** (no se aplica imitación de huella).

Valores disponibles de **Cipher Suites** (TLS 1.3 y conjuntos ECDHE): `TLS_AES_128_GCM_SHA256` , `TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA` , `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA` , `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256` , `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384` , `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256` , `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256` .

Interruptores TLS

Interrupción	Predeterminado	Descripción
Rechazar SNI desconocido (<code>rejectUnknownSni</code>)	desact. (<code>false</code>)	Si está activado, el servidor cierra la conexión cuando el SNI presentado por el cliente no coincide con el nombre del certificado. Aumenta el camuflaje (el servidor no responde a solicitudes «ajenas»), pero requiere que el SNI del cliente coincida exactamente.
Deshabilitar System Root (<code>disableSystemRoot</code>)	desact. (<code>false</code>)	Desactiva el uso del almacén de certificados raíz de confianza del sistema.

Interruptor	Predeterminado	Descripción
Reanudación de sesión (<code>enableSessionResumption</code>)	desact. (<code>false</code>)	Activa la reanudación de sesión TLS (session resumption / session tickets).

Parámetros adicionales de TLS (vcn, curvas, registro de claves, ECH Sockopt)

Debajo de los ajustes principales de TLS hay campos adicionales.

Campo (etiqueta)	Predeterminado	Descripción
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	Nombres (separados por comas) con los que el cliente verifica el certificado del servidor en lugar del SNI. Es el reemplazo moderno del campo <code>allowInsecure</code> eliminado de Xray tras el 2026-06-01. Solo para el panel: no se escribe en el config de xray del servidor, pero se transmite en los enlaces de invitación y suscripciones (<code>vcn=...</code>) para que el cliente lo aplique. Marcador de posición: <code>example.com</code> .
Curve Preferences (<code>curvePreferences</code>)	""	Restricción y orden de las curvas de intercambio de claves TLS, en orden de preferencia (por ejemplo <code>X25519MLKEM768</code> , <code>X25519</code>). Vacío – se usan los valores predeterminados de Xray-core.
Master Key Log (<code>masterKeyLog</code>)	""	Ruta para registrar las TLS master keys en formato <code>SSLKEYLOGFILE</code> (para descifrar tráfico en Wireshark durante la depuración). Marcador de posición: <code>/path/to/sslkeylog.txt</code> . En producción déjelo vacío – el archivo permite descifrar todo el tráfico.
ECH Sockopt (<code>echSockopt</code>)	desact.	Interruptor con parámetros de socket para la conexión a través de la cual Xray solicita la lista de configuración ECH. Al activarlo se muestran: Dialer Proxy (<code>dialerProxy</code> – enrutar la solicitud a través del outbound indicado por etiqueta), Domain Strategy (<code>domainStrategy</code>), TCP Fast Open (<code>tcpFastOpen</code>), Multipath TCP (<code>tcpMptcp</code>). Déjelo desactivado si no es necesario.

Los campos `verifyPeerCertByName` , `curvePreferences` , `masterKeyLog` y `echSockopt` se encuentran en el nivel superior de `tlsSettings` y sobreviven al «recorte» de campos del panel al guardar la configuración.

Certificados

La sección **Certificado SSL** (en la UI el encabezado es «Certificado SSL») se define como una lista: el botón **+** añade una nueva entrada de certificado, y el botón **- Eliminar** la quita (el botón de eliminar solo está disponible si hay más de una entrada). Por defecto, al activar TLS se crea una entrada vacía.

Para cada entrada, el selector de modo de entrada (`useFile`):

- **Ruta al certificado** (valor `useFile = true` ,predeterminado) – se especifican las rutas a los archivos en el servidor:
 - **Clave pública** (`certificateFile`) – ruta al archivo de certificado (`.crt` / `.pem`);
 - **Clave privada** (`keyFile`) – ruta al archivo de clave privada (`.key`).
- **Contenido del certificado** (valor `useFile = false`) – el contenido se pega directamente en los campos (áreas de texto multilínea):
 - **Clave pública** (`certificate`) – contenido PEM del certificado;
 - **Clave privada** (`key`) – contenido PEM de la clave.

Bajo los campos del modo «Ruta al certificado» hay dos botones:

- **Usar certificado del panel** – rellena los campos con las rutas al certificado SSL del propio panel. Para un inbound en el panel central se toma su certificado (`POST /panel/setting/all → webCertFile / webKeyFile`); para un inbound asignado a un nodo, se toma el certificado del propio nodo (`GET /panel/api/nodes/webCert/{nodeId}`), porque las rutas del panel central no existen en el nodo. Si el certificado no está configurado, se muestra la advertencia: «*No se ha configurado un certificado para el panel. Primero configúrelo en Ajustes.*» (El certificado del panel se configura en la sección «Ajustes → Seguridad»).
- **Limpiar** – borra ambas rutas.

Campos adicionales de cada entrada de certificado:

Campo	Predeterminado	Descripción
OCSF Stapling (<code>ocspStapling</code>)	<code>0</code> (desact.)	Intervalo de actualización del OCSF stapling en segundos (mínimo <code>0</code>). Para los nuevos inbounds está desactivado por defecto (<code>0</code>): esto elimina errores en los logs de xray para certificados sin respondedor OCSF (por ejemplo, Let's Encrypt, que abandonó OCSF). Actívalo solo para certificados que admitan stapling.
Carga única (<code>oneTimeLoading</code>)	desact. (<code>false</code>)	Si está activado, el certificado se lee del disco una sola vez al arrancar y no se vuelve a leer si el archivo cambia.
Opción de uso (<code>usage</code>)	<code>encipherment</code>	Finalidad del certificado. Valores admitidos: <code>encipherment</code> (cifrado – certificado de servidor habitual), <code>verify</code> (verificación), <code>issue</code> (emisión – el servidor firma/emite certificados propios).
Build Chain (<code>buildChain</code>)	desact. (<code>false</code>)	Se muestra solo con <code>usage = issue</code> . Construir la cadena de certificados.

No existe un botón separado de certificado autofirmado en el editor de inbound: el panel no genera certificados autofirmados al vuelo para los inbounds. El certificado se especifica mediante una ruta o su contenido, o bien se obtiene de los ajustes del panel con el botón «Usar certificado del panel». La emisión/obtención del certificado SSL del propio panel (incluida la carga de archivos y la vinculación a un dominio) se realiza en la sección **Ajustes → Seguridad**; no hay endpoints ACME/Let's Encrypt para inbounds aquí.

ECH y anclaje de certificado (campos TLS avanzados)

Campo	Predeterminado	Descripción
ECH key (<code>echServerKeys</code>)	<code>""</code>	Claves del servidor para Encrypted Client Hello.
ECH config (<code>settings.echConfigList</code>)	<code>""</code>	Lista de configuración ECH (parte del cliente, se incluye en el enlace).
SHA-256 del certificado del par (<code>settings.pinnedPeerCertSha256</code>)	<code>[]</code>	Hashes SHA-256 del certificado del par (cadenas hexadecimales, separadas por comas). Texto de ayuda literal: «Hashes SHA-256 del certificado del par como cadena hexadecimal (p. ej. e8e2d3...), separados por comas. Solo para el panel – no se escribe en el config de xray del servidor, pero se incluye en los enlaces de invitación para que los clientes puedan anclar el certificado.»

Botones: Junto al campo **SHA-256 del certificado del par** hay dos botones de autocompletado:

- **Fill from this inbound's certificate** (icono de escudo) – rellena el hash SHA-256 del certificado de este propio inbound (se obtiene localmente a través del endpoint `getCerHash`).
- **Fetch the hash by pinging the SNI (xray tls ping)** (icono de descarga) – obtiene el hash del certificado en vivo del servidor realizando una conexión TLS al SNI indicado (en el servidor se llama a `getRemoteCerHash`). El campo **SNI** (`serverName`) debe estar relleno – de lo contrario se muestra el aviso «Set the SNI (serverName) first to ping the remote certificate.»

Los hashes obtenidos se añaden al campo (separados por comas) y se incluyen en los enlaces de invitación para que el cliente pueda anclar el certificado.

- **Obtener nuevo certificado ECH** – solicita al servidor un nuevo par ECH para el SNI actual (`POST /panel/api/server/getNewEchCert` ; en el servidor se ejecuta `xray tls ech --serverName <SNI>`); rellena los campos **ECH key** y **ECH config**.
- **Limpiar** – vacía ambos campos ECH.

7.4. Modo REALITY

Campos del bloque `realitySettings` . REALITY no utiliza certificado SSL: en su lugar usa el handshake TLS tomado prestado de un dominio externo y un par de claves X25519.

Parámetros de camuflaje

Campo (etiqueta)	Valor predeterminado	Descripción
Mostrar (<code>show</code>)	desact. (<code>false</code>)	Salida de depuración de REALITY en los logs de Xray. Normalmente se deja desactivado.
Xver (<code>xver</code>)	<code>0</code>	Versión del protocolo PROXY transmitida al backend (<code>0</code> – desactivado). Mínimo <code>0</code> .

Campo (etiqueta)	Valor predeterminado	Descripción
uTLS (settings.fingerprint)	chrome	Huella TLS que se imita (la misma lista que en TLS, pero sin la opción vacía None).
Destino (target)	"" (el panel inserta uno aleatorio al activarlo)	Campo obligatorio. Dominio real cuyo handshake TLS toma prestado REALITY. Texto de ayuda literal: «Obligatorio. Debe incluir el puerto (p. ej., example.com:443). Sin puerto, Xray-core no arranca.» La validación del panel comprueba la presencia y corrección del puerto; de lo contrario se muestran los errores «El destino de REALITY es obligatorio» / «El destino de REALITY debe contener un puerto...» / «El destino de REALITY tiene un puerto no válido». El botón de actualización adyacente inserta un par aleatorio de la lista integrada.
SNI (serverNames)	[] (se inserta junto con el destino)	Lista de SNI admitidos (entrada múltiple con etiquetas). Debe corresponder al dominio de Destino . El botón de actualización inserta el SNI junto con el destino aleatorio.
Diferencia máxima de tiempo (ms) (maxTimediff)	0	Máxima diferencia de relojes admitida entre cliente y servidor en milisegundos (0 – sin límite). Mínimo 0 .
Versión mínima del cliente (minClientVer)	""	Versión mínima del cliente Xray (marcador de posición 25.9.11). Vacío – sin restricción.
Versión máxima del cliente (maxClientVer)	""	Versión máxima del cliente Xray. Vacío – sin restricción.
Short IDs (shortIds)	[] (se generan al activarlo)	Lista de identificadores cortos (hex) que distinguen a los clientes. Entrada múltiple con etiquetas; el botón de actualización genera un conjunto aleatorio.
SpiderX (settings.spiderX)	/	Ruta del «araña» (parte del cliente de REALITY), utilizada al imitar el acceso al sitio externo. Se incluye en el enlace de invitación.

Destino (target) y **SNI** (serverNames) al activar REALITY y al pulsar el botón de actualización se rellenan con un par aleatorio de la lista integrada del panel: `www.amazon.com` , `aws.amazon.com` , `www.oracle.com` , `www.nvidia.com` , `www.amd.com` , `www.intel.com` , `www.sony.com` (cada uno con el puerto `:443`). Elija un sitio HTTPS externo grande y estable que no esté detrás de su propio servidor.

Ejemplo: bloque streamSettings para REALITY en red tcp (VLESS). No se necesita certificado – en su lugar se usan el dominio prestado y el par de claves X25519:

```

"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": [ "", "6ba85179e30d4fc2" ],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
}

```

Aquí el campo **Destino** (`target`) del panel corresponde a `dest` en el config final de Xray. Si un inbound REALITY fue creado con el destino en la clave `dest` (por versiones antiguas del panel, a través de la API o herramientas externas), el panel al cargarlo normaliza `dest` → `target` cuando `target` está vacío – por lo que dicho inbound se carga correctamente, el campo **Destino** no aparece vacío y volver a guardarlo no rompe el REALITY en funcionamiento.

Claves REALITY (X25519)

Campo	Predeterminado	Descripción
Clave pública (<code>settings.publicKey</code>)	<code>""</code>	Clave pública X25519 (el cliente la coloca en su configuración/enlace).
Clave privada (<code>privateKey</code>)	<code>""</code>	Clave privada X25519 (se almacena solo en el servidor).

Botones bajo las claves:

- **Obtener nuevo certificado** – solicita al servidor un nuevo par de claves (`GET /panel/api/server/getNewX25519Cert` ; en el servidor se ejecuta `xray x25519`), rellena **Clave privada** y **Clave pública**. Este mismo par se genera automáticamente al activar el modo REALITY por primera vez.

Ejemplo: obtener un par de claves X25519 mediante la API (fuera del formulario, por ejemplo para un script). La solicitud devuelve la clave privada y la pública:

```

curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Ответ:
# {"success":true,"obj":{"privateKey":"...","publicKey":"..."}}

```

`cookie.txt` – archivo de cookie de sesión obtenido tras iniciar sesión mediante `POST /login` .

- **Limpiar** – vacía ambas claves.

Firma poscuántica ML-DSA-65 (mldsa65)

Capa adicional (opcional) de autenticación poscuántica de REALITY:

Campo	Predeterminado	Descripción
mldsa65 Seed (mldsa65Seed)	""	Seed de la clave ML-DSA-65 del servidor.
mldsa65 Verify (settings.mldsa65Verify)	""	Valor de verificación (parte del cliente, se incluye en el enlace).

Botones:

- **Obtener nuevo Seed** – solicita un nuevo par (GET /panel/api/server/getNewmldsa65 ; en el servidor se ejecuta xray mldsa65), rellena **mldsa65 Seed** y **mldsa65 Verify**.
- **Limpiar** – vacía ambos campos.

Limitación de velocidad de fallback y registro de claves de REALITY

En los ajustes de REALITY hay disponible una limitación de velocidad del tráfico de fallback – evita que las sondas activas usen el servidor como canal gratuito hacia el dominio prestado. La configuración se establece por separado para dos direcciones – **Limit Fallback Upload** y **Limit Fallback Download** (limitFallbackUpload / limitFallbackDownload), cada una con el mismo conjunto de campos:

Campo (etiqueta)	Predeterminado	Descripción
After Bytes (afterBytes)	0	Cuántos bytes dejar pasar a velocidad plena antes de aplicar la limitación. 0 – limitar desde el primer byte.
Bytes Per Sec (bytesPerSec)	0	Techo de velocidad del tráfico de fallback en bytes por segundo tras el umbral. 0 – sin límite (desactiva esta dirección).
Burst Bytes Per Sec (burstBytesPerSec)	0	Margen para picos breves por encima de la velocidad constante (tamaño del token-bucket). Si es menor que Bytes Per Sec , se eleva a ese valor.

También se añade el campo **Master Key Log** (masterKeyLog) – ruta para registrar las TLS master keys en formato SSLKEYLOGFILE para depuración en Wireshark; déjelo vacío en producción.

7.5. Recomendaciones prácticas de configuración

1. **VLESS + Reality (recomendado)**: cree un inbound VLESS en red tcp , en la pestaña «Seguridad» seleccione **Reality** – el panel insertará automáticamente target /SNI aleatorios, shortIds y generará las claves X25519. Si es necesario, pulse «Obtener nuevo certificado» para generar su propio par de claves. Para los clientes VLESS active **Flow** = xtls-rprx-vision (XTLS Vision) – esto proporcionará el máximo rendimiento y camuflaje.

Ejemplo: enlace de cliente final VLESS + Reality + Vision. Así es el enlace de invitación que el panel genera para este tipo de inbound (los valores de claves/ID son ilustrativos):


```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Aquí **pbk** es la clave pública X25519, **sni** es el dominio prestado de **Destino**, **sid** es uno de los **Short IDs**, **flow=xtls-rprx-vision** es XTLS Vision activado. 2. **TLS con dominio propio:** seleccione **TLS**, rellene **SNI** con el nombre del dominio, añada el certificado (mediante ruta a los archivos o su contenido), o pulse «Usar certificado del panel» si el dominio y el certificado ya están configurados en «Ajustes → Seguridad». Deje **Versión mín/máx** = 1.2 – 1.3 y **uTLS** = **chrome** para camuflarse como un navegador habitual. 3. No deje el modo **Ninguno** para inbounds accesibles desde el exterior – úselo solo para destinos de fallback locales (127.0.0.1) o cuando TLS lo proporcione un proxy externo. 4. Consejo de la interfaz: para la mayoría de los campos avanzados se muestra el aviso «*Se recomienda dejar la configuración predeterminada*» – modifíquelos solo si comprende las consecuencias.

8. Clientes

Un cliente es una cuenta de usuario VPN: un conjunto de credenciales (UUID o contraseña) vinculado a uno o varios inbound, con su propia cuota de tráfico, fecha de vencimiento y límite de conexiones simultáneas. En este fork el cliente es una entidad independiente (tabla `clients`): el mismo cliente puede estar vinculado a varios inbound a la vez, conservando un UUID/contraseña compartidos y un contador de tráfico común. La sección **Clientes** muestra todas las cuentas del panel independientemente del inbound, con búsqueda, filtros, ordenación y operaciones masivas.

8.1. Campos del cliente

A continuación se describe cada campo del editor **Añadir cliente** / **Editar cliente**.

El formulario del cliente se divide en dos pestañas: **Principal** (email, vinculación a inbound, límites, vencimiento, grupo, comentario, etiqueta inversa) y **Credenciales** (UUID/contraseña/auth, Flow, VMess Security). En las etiquetas de los campos la cuota aparece como **Límite de tráfico (GB)**, y los plazos como **Duración (días)** y **Renovación automática (días)**; los campos **Límite de tráfico (GB)** y **Límite de IP** tienen sugerencias que explican que `0` significa «sin restricciones». Al editar un cliente ya existente, el botón de generación de email aleatorio está oculto y el botón del registro de IP se muestra directamente junto al campo **Límite de IP** con el número de direcciones registradas.

Campo	Clave JSON	Por defecto	Descripción
Email	<code>email</code>	– (obligatorio)	Identificador único del cliente
UUID	<code>id</code>	generado	Identificador para VMess/VLESS
Contraseña	<code>password</code>	generada	Contraseña para Trojan/Shadowsocks
Autorización	<code>auth</code>	generada	Contraseña para Hysteria
Flow	<code>flow</code>	vacío	Control de flujo (XTLS), solo VLESS
VMess Security	<code>security</code>	<code>auto</code>	Método de cifrado VMess
Límite de IP	<code>limitIp</code>	<code>0</code> (sin límite)	Máximo de IP simultáneas
Total enviado/recibido (GB)	<code>totalGB</code>	<code>0</code> (sin límite)	Cuota de tráfico
Fecha de vencimiento	<code>expiryTime</code>	<code>0</code> (sin vencimiento)	Fecha de expiración
Renovación automática	<code>reset</code>	<code>0</code> (desactivado)	Período de reinicio de tráfico, días
ID de usuario de Telegram	<code>tgId</code>	<code>0</code> (ninguno)	ID numérico de Telegram
ID de suscripción	<code>subId</code>	generado	Identificador de suscripción
Grupo	<code>group</code>	vacío	Etiqueta lógica de agrupación
Comentario	<code>comment</code>	vacío	Nota libre
Habilitado	<code>enable</code>	<code>true</code>	Si la cuenta está activa

Email (identificador)

El campo **Email** es el identificador principal y obligatorio del cliente. A pesar del nombre, no tiene por qué ser una dirección de correo: sirve cualquier etiqueta de texto (nombre de usuario, número). El valor debe ser **único** dentro del panel; intentar crear un segundo cliente con un email ya ocupado se rechaza (`email already in use`), salvo cuando el `subId` también coincide (lo que se interpreta como vinculación del mismo cliente).

El Email **no puede estar vacío** (`client email is required`) y **no puede contener espacios, los caracteres `/`, `\` ni caracteres de control** («El email no puede contener espacios, `/`, `\` ni caracteres de control»). El email interviene en la contabilidad de tráfico, en el registro de IP, en la lista de usuarios en línea y en los nombres de las operaciones; no se recomienda cambiarlo a posteriori.

UUID / Contraseña / Autorización (credenciales)

El campo de credenciales concreto depende del protocolo del inbound al que se vincula el cliente. Los valores se generan automáticamente si se deja el campo vacío:

- **UUID** (campo `id`) – para los protocolos **VMess** y **VLESS**. Si no se especifica, se genera un UUID v4 aleatorio.
- **Contraseña** (campo `password`) – para **Trojan** y **Shadowsocks**. Para Trojan se genera por defecto un UUID sin guiones. Para Shadowsocks se genera una clave de la longitud adecuada en Base64 según el método de cifrado del inbound: 16 bytes para `2022-blake3-aes-128-gcm`, 32 bytes para `2022-blake3-aes-256-gcm` y `2022-blake3-chacha20-poly1305`; para los demás métodos, un UUID sin guiones. Si la clave introducida manualmente no es compatible con el método `2022-blake3`, será reemplazada por una generada automáticamente.
- **Autorización** (campo `auth`) – contraseña para **Hysteria**. Por defecto, UUID sin guiones.

Dado que un mismo cliente puede estar vinculado a inbound de distintos protocolos, el registro del cliente puede tener simultáneamente UUID, contraseña y auth; cada protocolo utiliza su propio campo.

Ejemplo: cómo se ven las credenciales del cliente en settings de distintos inbound. El mismo cliente se identifica por `id` en un inbound VLESS, por `password` en uno Trojan y por `password` (clave Base64) en uno Shadowsocks:

```
// фрагмент settings.clients y VLESS-inbound
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// тот же клиент в Trojan-inbound
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// тот же клиент в Shadowsocks-inbound (метод 2022-blake3-aes-256-gcm)
{ "password": "GPy0aA3f7C00az53eaQ8eqMfRDjmBl0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (campo `flow`) – control de flujo XTLS. Aplicable **únicamente a VLESS** y solo cuando el inbound está configurado para XTLS Vision: VLESS sobre transporte **TCP** con security **tls** o **reality**. El valor válido es

`xtls-rprx-vision` (así como el histórico `xtls-rprx-vision-udp443`); un valor vacío indica ausencia de flow.

Si el inbound no es compatible con XTLS-flow, el flow especificado **se borra silenciosamente** al guardar el cliente: para un mismo cliente vinculado a varios inbound, el flow solo se aplica donde sea admisible. Cámbielo únicamente si utiliza deliberadamente VLESS-Vision.

VMess Security

VMess Security (campo `security`) – método de cifrado de la carga útil para VMess. El valor por defecto es `auto` (Xray elige el cifrado automáticamente). Los valores válidos son los estándar de VMess: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. Para los demás protocolos este campo no se usa.

Límite de IP (conexiones simultáneas)

Límite de IP (campo `limitIp`) – número máximo de **direcciones IP distintas** desde las que el cliente puede estar conectado simultáneamente. El valor por defecto es `0`, que significa **sin restricción**. Con un valor positivo, el panel rastrea las IP activas del cliente y, al superar el límite, desactiva la cuenta mediante una tarea en segundo plano. (A partir de la versión **3.3.1** el recuento de IP se realiza a través de la API online-stats del núcleo Xray y **no requiere** el registro de acceso; en versiones anteriores del núcleo, el panel vuelve a leer el registro de acceso, que en ese caso debe estar habilitado.) Úselo para impedir compartir una suscripción entre muchos dispositivos: por ejemplo, `2` permite dos dispositivos.

El Límite de IP se aplica mediante **Fail2ban**, por lo que el campo **Límite de IP** solo está activo cuando Fail2ban está instalado y en funcionamiento (el panel comprueba su estado a través de `GET /panel/api/server/fail2banStatus`). Si Fail2ban no está instalado, el campo del editor del cliente (y del formulario de adición masiva) queda bloqueado y al pasar el cursor aparece una sugerencia con la propuesta de instalar Fail2ban desde el menú bash `x-ui` («Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option.»); en Windows la sugerencia indica que Fail2ban no está disponible («Fail2ban is not available on Windows, so the IP limit cannot be enforced.»), y si la función está deshabilitada en el servidor: «The IP limit feature is disabled on this server.». Al actualizar el panel, en clientes de servidores sin Fail2ban el límite de IP guardado se pone a cero mediante una migración puntual, ya que de todos modos no se aplicaba allí.

Ejemplo de valores. `limitIp: 0` – sin restricción; `limitIp: 1` – estrictamente un dispositivo simultáneo; `limitIp: 3` – hasta tres IP distintas. Con una cuarta IP activa, la tarea en segundo plano desactivará al cliente (`enable = false`) hasta que ejecute **Restablecer límite de IP**.

Operaciones relacionadas: **Registro de IP** muestra la lista de IP registradas del cliente; cada entrada contiene, además de la propia IP, la hora del último acceso y la etiqueta del nodo (`@ nombre_nodo`) a través del cual se registró la conexión – en una configuración multipanel se puede ver por qué nodo se conectó el cliente.

Restablecer límite de IP vacía el registro de IP acumulado para que el cliente pueda volver a conectarse sin esperar a que caduquen los registros.

Total enviado/recibido (GB) – cuota de tráfico

Total enviado/recibido (GB) (campo `totalGB`) – cuota de tráfico total (envío + recepción). El valor por defecto – 0 – significa **sin límite**. Al alcanzar la cuota (`up + down >= total`) el cliente se considera **agotado** (depleted) y se desactiva. En la interfaz normalmente se introduce en gigabytes; en la base de datos se almacena en bytes.

En la lista de clientes la columna **Tráfico** muestra una barra de uso con colores: el volumen de tráfico consumido, la etiqueta del límite (o el símbolo ∞ si no hay límite) y, al pasar el cursor, un desglose en enviado/recibido y saldo restante. El mismo indicador compacto se muestra en las tarjetas de clientes en móvil.

Fecha de vencimiento (Expiry)

Fecha de vencimiento (campo `expiryTime`) define el momento de expiración de la cuenta. El campo tiene tres modos:

- **Sin vencimiento** – 0. El cliente nunca expira por tiempo.
- **Fecha concreta** – timestamp Unix positivo (en milisegundos). Al llegar ese momento (`expiryTime <= ahora`) el cliente se considera expirado (expired) y se desactiva. En la interfaz normalmente se especifica eligiendo una fecha o una duración en días (**Duración**, unidad – **Días**).
- **Inicio tras el primer uso** – valor **negativo** que codifica una duración. Mientras el cliente no haya transmitido ningún byte, el plazo permanece negativo («inicio diferido»). En el primer ciclo de contabilidad de tráfico el panel lo convierte a una fecha absoluta: `ahora + |duración|`. Esto permite vender, por ejemplo, «30 días desde la primera conexión» sin saber de antemano cuándo se activará el cliente. La conversión se realiza una sola vez por email, de modo que todos los inbound vinculados reciben la misma fecha de vencimiento.

Ejemplo de codificación del plazo. Fecha fija del 1 de marzo de 2026, 00:00 UTC → `expiryTime: 1772323200000` (timestamp positivo en milisegundos). «30 días desde la primera conexión» → `expiryTime: -2592000000` (valor negativo, $30 \times 24 \times 60 \times 60 \times 1000$); al primer byte de tráfico el panel lo reemplazará por `ahora + 2592000000`. Sin vencimiento → `expiryTime: 0`.

Renovación automática (período de reinicio de tráfico del cliente)

El campo **Renovación automática** (campo `reset`) es el período de renovación/reinicio automático en días. Sugerencia: «Renovación automática tras el vencimiento. (0 = desactivado) (unidad: día)».

- 0 – renovación automática **desactivada** (valor por defecto). Al expirar, el cliente simplemente queda agotado.
- > 0 – la tarea en segundo plano, al expirar el plazo, **reinicia los contadores de tráfico a cero** (`up = down = 0`), **adelanta la fecha de vencimiento** en `reset` días (si es necesario, varios períodos hasta que la nueva fecha quede en el futuro) y, si procede, **vuelve a activar** al cliente. Esto implementa una suscripción periódica (por ejemplo, mensual). La renovación automática **no se aplica a los inbound en nodos remotos** (`node_id IS NOT NULL`).

Consecuencia importante: los clientes con `reset > 0` **quedan excluidos** del concepto «agotado» en las operaciones de eliminación masiva – su tráfico/plazo se reinicia por la renovación automática y no convierte la cuenta en candidata a eliminación.

ID de usuario de Telegram

ID de usuario de Telegram (campo `tgId`) – identificador numérico de Telegram del usuario para vincularlo al bot de Telegram integrado en el panel (notificaciones, consulta autónoma de estadísticas). Sugerencia: «ID numérico del usuario de Telegram (0 = ninguno)». El valor por defecto `0` indica que no hay vinculación. Este campo admite filtrado (**Sí / No**).

ID de suscripción (subId)

ID de suscripción (campo `subId`) – identificador bajo el cual el cliente se incluye en una **suscripción** (subscription). Todos los clientes con el mismo `subId` se entregan a través de un mismo enlace de suscripción. Si el campo se deja vacío al crear el cliente, el panel **genera automáticamente** un `subId` aleatorio (UUID). El valor debe ser **único** entre clientes con distinto email (`subId already in use`) y está sujeto a las mismas restricciones de caracteres que el email («El ID de suscripción no puede contener espacios, '/', ' ' ni caracteres de control»).

Si `subId` el enlace de suscripción para el cliente no está disponible («Este cliente no tiene subId; el enlace de acceso compartido no está disponible.»).

Pestaña Links (enlaces externos y suscripciones)

Además de las pestañas **Principal** y **Credenciales**, el editor del cliente tiene una tercera pestaña **Links** (sugerencia: «Add third-party share links and remote subscription URLs to include in this client's subscription.»). En ella se añaden enlaces de terceros (`vless://` , `vmess://` , `trojan://` , `ss://` , `hysteria2://` , `wireguard://`) mediante el botón **Add External Link**, y URL de suscripciones remotas (por ejemplo, `https://provider.example/sub/...`) mediante **Add External Subscription**.

Todo lo anterior se mezcla en la salida de la suscripción de este cliente (formatos raw, JSON y Clash): los enlaces se añaden tal cual, y las suscripciones remotas el panel las descarga periódicamente (con caché y un tiempo de espera corto) y combina sus configuraciones con las propias. Así, en un único enlace de suscripción del cliente se pueden entregar conjuntamente los servidores propios y configuraciones externas.

Grupo

Grupo (campo `group`) – etiqueta lógica para agrupar clientes relacionados. Sugerencia: «Etiqueta lógica para agrupar clientes relacionados (por ejemplo, equipo, cliente, región). Se puede filtrar desde la barra de herramientas.», marcador de posición – «por ejemplo, customer-a». El campo es opcional (vacío por defecto). Se puede filtrar la lista por grupo y realizar operaciones masivas; para eliminar la etiqueta de un cliente se usa la acción **Desagrupar**.

También se puede eliminar el grupo directamente en el editor de un cliente: si se borra el campo **Grupo** y se guarda, la etiqueta se elimina correctamente y el cliente deja de aparecer bajo el grupo anterior.

Comentario

Comentario (campo `comment`) – nota de texto libre para el administrador (vacío por defecto). El contenido se incluye en la búsqueda y admite filtrado (**Sí / No** comentario).

Habilitado

Habilitado (campo `enable`) – indicador de actividad de la cuenta. Por defecto **habilitado** (`true`); al crear, aunque no se transmita el indicador, el panel fuerza `true`. Un cliente deshabilitado (`enable = false`) no puede conectarse y en el resumen pertenece a la categoría de **inactivos** (deactive). El panel desactiva automáticamente a los clientes que han agotado la cuota, han expirado o han superado el límite de IP.

Campos de solo lectura

En la tarjeta del cliente también se muestran campos de servicio: **Creado** (`created_at`) y **Actualizado** (`updated_at`) – marcas de tiempo de creación y última modificación, que se rellenan automáticamente y no son editables. El campo **Etiqueta inversa** (`reverse`) – etiqueta Reverse opcional para el proxy inverso simple VLESS («Etiqueta Reverse opcional»).

8.2. Vinculación a inbound

Cada cliente debe estar vinculado a al menos un inbound; al crear se requiere un mínimo de uno (`at least one inbound is required`). En el editor esto es el campo **Inbound vinculados** con la sugerencia **Seleccione uno o varios inbound**.

- **Vincular** – añadir el cliente a los inbound seleccionados (mismo UUID/contraseña y tráfico compartido). Las vinculaciones existentes se conservan.
- **Desvincular** – eliminar el cliente de los inbound seleccionados. El registro del cliente se conserva (para eliminarlo completamente use **Eliminar**). Los pares en los que el cliente no estaba vinculado se omiten silenciosamente.

Al guardar un cliente vinculado a varios inbound, los campos incompatibles con el protocolo/transporte concreto (por ejemplo, Flow fuera de VLESS-Vision) se ajustan automáticamente a los valores admisibles para cada inbound.

Sobre la lista de selección de inbound (en el formulario del cliente, al añadir clientes en masa y en las ventanas de vinculación/desvinculación masiva) hay botones **Seleccionar todos** y **Limpiar**. En estas listas cada inbound aparece con su observación (remark) si está definida; en caso contrario, con la etiqueta del inbound.

8.3. Operaciones sobre el cliente

Para un cliente individual (a través de la tarjeta **Información del cliente** o el menú contextual **Acciones**) están disponibles:

Consulta de información, código QR y enlace

- **Información del cliente** – tarjeta con todos los campos, tráfico usado/restante (**Saldo**), fecha de vencimiento e inbound vinculados.

La consulta del cliente a través de la API (`GET /panel/api/clients/get/:email`) devuelve, junto a los campos `client` e `inboundIds` , el campo adicional `usedTraffic` – tráfico realmente consumido (enviado + recibido, incluyendo datos de los nodos), lo que facilita comparar el consumo con la cuota `totalGB` .

- **Código QR y Enlace** – enlace de configuración del cliente para importar en la aplicación cliente. Se genera a partir de todos los inbound vinculados con protocolo compatible (`GET /links/:email`). Si no hay enlaces válidos: «No hay enlaces de acceso compartido – vincule primero el cliente a un inbound con un protocolo compatible.».
- **Enlace de suscripción** – URL de suscripción por `subId` (`GET /subLinks/:subId`). Disponible solo si el cliente tiene `subId` y el servicio de suscripción está habilitado en **Configuración del panel** → **Suscripción** (si no, «El servicio de suscripción está desactivado.»). También se entrega la **URL de suscripción JSON**.

Restablecer tráfico

Restablecer tráfico (`POST /resetTraffic/:email`) pone a cero los contadores de envío/recepción (`up` , `down`) del cliente concreto. La cuota (`totalGB`) y la fecha de vencimiento **no se ven afectadas** – solo se reinicia el volumen usado. Notificación emergente: «Tráfico restablecido». Si el cliente no está vinculado a ningún inbound: «Vincule primero este cliente a un inbound.».

El botón **Restablecer tráfico** también está disponible desde el formulario de edición del cliente – en el panel inferior, junto a **Cancelar** / **Guardar** (antes del reinicio se solicita confirmación). Si el cliente estaba desactivado por agotamiento de tráfico, el reinicio (tanto individual como masivo) lo **vuelve a activar** automáticamente (`enable = true`) y propaga inmediatamente el cambio a los nodos remotos – ya no es necesario volver a activar el cliente manualmente en el maestro y los nodos.

Restablecer límite de IP

Vacía el registro de IP acumulado del cliente (`POST /clearIps/:email`) para levantar el bloqueo temporal por superación del límite de conexiones simultáneas. Notificación emergente: «El registro ha sido vaciado».

Eliminar

Eliminar (`POST /del/:email`) – eliminación completa del cliente. Diálogo de confirmación: título «¿Eliminar el cliente {email}?», texto «El cliente será eliminado de todos los inbound vinculados y su registro de tráfico será destruido. Esta acción no se puede deshacer.». La eliminación desvincula al cliente de **todos** los inbound y destruye su registro de tráfico. Notificación emergente: «Cliente eliminado».

8.4. Operaciones masivas

En la lista de clientes se pueden marcar varios registros (**Seleccionar todos**, **Deseleccionar todos**); el contador muestra «{count} seleccionados». Sobre los seleccionados están disponibles:

- **Eliminar ({count})** (`POST /bulkDel`) – eliminación en grupo. Confirmación: «¿Eliminar {count} clientes?», «Cada cliente seleccionado se elimina de todos los inbound vinculados y su registro de tráfico es destruido. Esta acción no se puede deshacer.». Notificación emergente: «Clientes eliminados: {count}»; en caso de fallo parcial: «Eliminados: {ok}, fallidos: {failed}».

- **Editar ({count}) / Ajuste** (POST /bulkAdjust) – modificación masiva del plazo y/o la cuota. Diálogo «Editar {count} clientes» con la sugerencia «Los valores positivos añaden, los negativos reducen. Los clientes con plazo o tráfico ilimitados se omiten para el campo correspondiente.». Campos: **Añadir días** y **Añadir tráfico (GB)**. Lógica:
 - **Plazo:** los clientes con plazo indefinido (expiryTime == 0) se omiten («unlimited expiry»); para los clientes con fecha, el plazo se desplaza el número de días indicado; para los clientes en modo «tras el primer uso» (plazo negativo) se ajusta la duración de espera. Una reducción que exceda el saldo restante se omite («reduction exceeds remaining time/delay window»).
 - **Tráfico:** los clientes sin límite (totalGB == 0) se omiten («unlimited traffic»); en caso contrario la cuota cambia en el volumen indicado sin bajar de cero.
 - Si no se especifican ni días ni tráfico: «Especifique días o tráfico antes de aplicar.». Notificación emergente: «Editados: {count}» / «Editados: {ok}, omitidos: {skipped}».

Ejemplo: ampliar los clientes seleccionados 30 días y añadir 50 GB. En el diálogo **Editar** indique **Añadir días** = 30 , **Añadir tráfico (GB)** = 50 . Para, por el contrario, restar una semana y reducir la cuota en 10 GB, introduzca valores negativos: **Añadir días** = -7 , **Añadir tráfico (GB)** = -10 (los clientes con plazo indefinido o sin límite de tráfico en el campo correspondiente serán omitidos).

- **Vincular ({count}) / Desvincular ({count})** (POST /bulkAttach / bulkDetach) – vinculación/desvinculación masiva de los clientes seleccionados a los inbound seleccionados. Los destinos son solo inbound multiusuario. Resultado de la desvinculación: «Desvinculados {detached}, omitidos {skipped}.».
- **Sub-enlaces ({count})** – tabla resumen de las URL de suscripción y suscripción JSON de los clientes seleccionados con botón **Copiar todos**. Si ninguno tiene subId: «Ninguno de los clientes seleccionados tiene ID de suscripción.».
- **Añadir al grupo y Desagrupar** – asignación y eliminación de la etiqueta de grupo.

Reinicio de tráfico y eliminación por estado

- **Restablecer tráfico de todos los clientes** (POST /resetAllTraffics) – pone a cero los contadores up / down de **todos** los clientes del panel. Confirmación: «¿Restablecer el tráfico de todos los clientes?» y «Los contadores de envío/recepción de todos los clientes se ponen a cero. Las cuotas y las fechas de vencimiento no se ven afectadas. Esta acción no se puede deshacer.». Notificación emergente: «Tráfico de todos los clientes restablecido».
- **Eliminar agotados** (POST /delDepleted) – elimina todos los clientes cuya **cuota esté agotada** (total > 0 and up + down >= total) o **cuyo plazo haya vencido** (expiry_time > 0 and expiry_time <= ahora), siempre que reset = 0 (los clientes con renovación automática no se eliminan). Confirmación: «¿Eliminar los clientes agotados?», «Se eliminan todos los clientes cuya cuota de tráfico esté agotada o cuyo plazo haya vencido. Esta acción no se puede deshacer.». Notificación emergente: «Clientes agotados eliminados: {count}».

Exportación, importación y eliminación de clientes sin vincular

Cuando no hay nada seleccionado, en el menú **Más** de la página **Clientes** hay disponibles tres operaciones.

Exportar clientes (GET /clients/export) abre un visor con la lista JSON de todos los clientes en formato {client, inboundIds} con botones de copia y descarga (archivo clients-export.json). **Importar clientes** (POST /clients/import) abre un editor en el que se pega ese mismo JSON y se pulsa **Import**: los clientes

con `inboundIds` se crean y vinculan a los inbound; los clientes sin vinculaciones se restauran como registros independientes «desnudos», y los email ya existentes **nunca se sobrescriben** – quedan en la lista de omitidos. Notificaciones emergentes: «{count} clients imported», «{ok} imported, {failed} skipped».

Eliminar clientes sin vincular (`POST /clients/deleteOrphans`) – operación peligrosa: elimina todos los clientes que no estén vinculados a ningún inbound, junto con su registro de tráfico, registro de IP y enlaces externos. Confirmación: «Delete clients without an inbound?», «Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.». Notificación emergente: «{count} unattached clients deleted». La acción es irreversible.

8.5. Búsqueda, filtros y ordenación

Encima de la lista hay una barra de búsqueda («Buscar email, comentario, sub ID, UUID, contraseña, auth...») – busca por email, comentario, subId, UUID, contraseña y auth. Contador de resultados: «Mostrando {shown} de {total}».

La lista de clientes se actualiza automáticamente: el panel consulta la página actual cada pocos segundos, por lo que los clientes recién conectados y los cambios en el orden de clasificación aparecen sin necesidad de actualizar manualmente (el indicador de carga no parpadea durante el sondeo en segundo plano).

El panel **Filtro de clientes** permite filtrar por estado (categoría), protocolo, inbound vinculado, rango de fecha de vencimiento, rango de tráfico usado, presencia de renovación automática (**Sí/No**), presencia de ID de Telegram y comentario, y también por grupo. En paneles con nodos aparece un selector múltiple **Nodos**: se puede limitar la lista a los clientes de los nodos seleccionados; la opción **Panel local** filtra los clientes de inbound sin vinculación a nodo (el filtro solo es visible cuando hay nodos). Ordenación: **Primero los más antiguos/recientes, Actualizados recientemente, Conectados recientemente, Email A→Z / Z→A, Mayor tráfico, Mayor saldo, Próximos a vencer**.

8.6. Iconos y estados

Prioridad de estados: agotado/expirado → inactivo → próximo a vencer → activo.

- **En línea / Fuera de línea** – cliente con conexión activa (presente en la lista en línea actual) y **habilitado**. La lista en línea se actualiza con solicitudes independientes (`/onlines` , `/onlinesByGuid`).
- **Agotado** (depleted) – la cuota está consumida (`up + down >= totalGB`) o el plazo ha vencido (`expiryTime <= ahora`). Este cliente se desactiva automáticamente y queda bajo la acción de **Eliminar agotados**.
- **Próximo a vencer / agotarse** (expiring) – cliente habilitado al que le queda menos del intervalo umbral hasta el vencimiento o menos del volumen umbral hasta agotar la cuota (los umbrales se definen en la configuración del panel). No se aplica si el cliente ya está agotado/desactivado.
- **Inactivo** (deactive) – cliente con `enable = false` (desactivado manualmente o por una tarea en segundo plano).
- **Activo** (active) – habilitado, no agotado, plazo no vencido y aún lejos de los umbrales.

9. Grupos de clientes

Esta es una función específica de este fork de 3X-UI. En el proyecto original 3x-ui (MHSanaei) no existe el concepto de «grupo de clientes» — aquí se han añadido una tabla separada de grupos, la página **Grupos** en el menú del panel y los métodos de API correspondientes. Si migra la configuración al 3x-ui original, la etiqueta de grupo simplemente no se tendrá en cuenta en ningún lugar.

9.1. Qué es un grupo de clientes y para qué sirve

Un **grupo** es una etiqueta lógica con nombre (label) que puede asignarse a uno o varios clientes. No crea un nuevo método de conexión y no es ni un inbound ni un nodo — es puramente una etiqueta organizativa que facilita filtrar y procesar clientes de forma masiva.

La idea clave del modelo de clientes en este fork: **el cliente es una entidad de primer nivel, identificada por email** (el campo `email` en la tabla `clients` tiene un índice único). El mismo cliente (un email con las mismas credenciales) puede pertenecer simultáneamente a varios inbound e incluso a varios nodos, incluso con protocolos distintos. La etiqueta de grupo se almacena **una sola vez por cliente**, por lo que se aplica automáticamente a todas sus asociaciones con inbound a la vez.

La etiqueta de grupo es una etiqueta lógica de agrupación:

Capa	Dónde se almacena	Campo
Registro del cliente (BD)	tabla <code>clients</code>	<code>group_name</code> (por defecto cadena vacía <code>''</code>)
Catálogo de grupos (BD)	tabla <code>client_groups</code>	<code>name</code> (índice único, no vacío)
Configuración del inbound (Xray)	JSON <code>settings.clients[].group</code>	se replica en cada objeto de cliente de cada inbound al que pertenece el cliente

Para qué sirve en la práctica:

- **Un cliente en varios inbound/nodos.** Si un cliente «se vende» como acceso a varios inbound a la vez (por ejemplo, distintos protocolos o distintos nodos), el grupo permite gestionarlo como una unidad: reiniciar el tráfico, eliminar, renombrar la etiqueta — con una sola operación sobre todos sus inbound.
- **Operaciones masivas y filtrado.** En la página **Clientes** la lista puede filtrarse por grupo; en la página **Grupos** están disponibles acciones masivas sobre todos los miembros del grupo.
- **Organización de un gran número de clientes.** Etiquetas como `vip`, `trial`, `team-A` permiten clasificar miles de clientes en categorías lógicas sin necesidad de crear inbound adicionales.

9.2. Relación del grupo con clientes, inbound, nodos y protocolos

Este es el apartado más importante para comprender el funcionamiento, ya que la sincronización de la etiqueta no es trivial.

El grupo describe al cliente, no al inbound. La etiqueta vive en el registro del cliente (`clients.group_name`). Cuando un cliente está asociado a varios inbound, ante cualquier cambio de grupo el panel recorre **todos** los

inbound en los que está ese cliente y actualiza/elimina el campo `group` dentro de su configuración de Xray (`settings.clients[]`). Técnicamente esto funciona así: a partir del email del cliente se localizan todos los inbound en los que está, y luego en el JSON de configuración de cada uno de esos inbound se modifica el objeto de cliente con ese email. Por tanto:

- El grupo **no depende del protocolo**. Un mismo email puede ser cliente VLESS en un inbound y cliente Hysteria en otro — la etiqueta de grupo es la misma y se aplicará a ambos (las credenciales de cada protocolo son propias y se almacenan por separado).
- El grupo **abarca nodos**. Los inbound que pertenecen a nodos se distinguen de los inbound del panel principal por el campo `nodeId` (en los inbound del panel principal es `null / 0`). La etiqueta de grupo se propaga a los objetos de cliente en los inbound independientemente de si es un inbound principal o de nodo — siempre que el cliente con ese email esté presente.

La etiqueta de grupo es resistente a la sincronización con nodos y a la reconstrucción de configuraciones. Este comportamiento está diseñado expresamente:

- Cuando un nodo envía un snapshot de tráfico, sus datos **no sobrescriben** los campos `group_name` y `comment` del cliente en la BD del panel. El grupo y el comentario se consideran campos «locales del panel» — el nodo no los gestiona.
- Al reconstruir la configuración de un inbound, un valor vacío de `group` en los datos entrantes **no borra** la etiqueta ya guardada. El grupo se gestiona exclusivamente a través de la página **Grupos** (no mediante la edición de la configuración de Xray del inbound), por lo que un «grupo vacío» en una reconstrucción normal se interpreta como «no tocar», no como «borrar».

Conclusión práctica: las únicas operaciones que **intencionalmente borran** la etiqueta son eliminar el grupo y eliminar explícitamente un cliente del grupo (véase más abajo). La edición habitual del inbound o la sincronización en segundo plano con el nodo no harán desaparecer el grupo.

9.3. Catálogo de grupos y grupos «vacíos»

La lista de grupos que se muestra en la página se forma combinando dos fuentes:

1. **Grupos derivados (derived)** — todos los valores no vacíos de `group_name` que realmente aparecen en los clientes, con el recuento de clientes.
2. **Grupos almacenados (stored)** — registros de la tabla `client_groups` .

Esta unión produce un efecto importante: un grupo puede existir **sin ningún cliente**. Este tipo de grupo se crea con el botón «Añadir grupo» (registro en `client_groups`) y aparece en la lista con el contador `0` . Estos registros son los llamados **grupos vacíos**. La lista siempre está ordenada por nombre sin distinción de mayúsculas y minúsculas.

Contadores de resumen en la página:

Campo	Qué muestra
Total de grupos	Número total de grupos (almacenados y derivados juntos).
Cientes con grupo	Cuántos clientes tienen una etiqueta de grupo no vacía.
Grupos vacíos	Cuántos grupos existen sin clientes (contador <code>0</code>).

Campo	Qué muestra
Clientes en el grupo	Número de clientes en un grupo concreto (columna de la tabla).

9.4. Campos y columnas del grupo

El registro de grupo en la tabla `client_groups` contiene:

Campo	Tipo	Por defecto	Descripción
<code>Id</code>	int	autoincremento	Clave primaria del registro de grupo.
<code>Name</code>	string	– (obligatorio)	Nombre del grupo. Índice único, no puede estar vacío. En la UI – columna Nombre .
<code>CreatedAt</code>	int64 (ms)	momento de creación	Momento de creación del registro de grupo.
<code>UpdatedAt</code>	int64 (ms)	momento de modificación	Momento de la última modificación.

En la tabla de la página se muestran al menos las columnas **Nombre** y **Clientes en el grupo**, además de los botones de acción (véase más abajo).

9.5. Creación de un grupo

Botón **Añadir grupo**.

Pasos:

1. Haga clic en **Añadir grupo**.
2. Introduzca el nombre del grupo.
3. Confirme.

Comportamiento del backend (`POST /panel/api/clients/groups/create` , cuerpo `{"name": "..."}`):

- El nombre se recorta de espacios en los extremos. Un nombre vacío se rechaza con el error «group name is required».
- Si ya existe un grupo con ese nombre – error «group already exists».
- Si tiene éxito, se crea un registro en `client_groups` (inicialmente sin clientes – es un grupo vacío).

Mensaje de éxito: **«Grupo «{name}» creado.»**

Ejemplo: crear un grupo vacío a través de la API. Prepare un conjunto de etiquetas de antemano, antes de añadir clientes:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}'
```

Respuesta en caso de éxito:

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

Una llamada repetida con el mismo nombre devolverá `"success": false` y el mensaje `group already exists`.

Crear un grupo vacío de antemano es conveniente cuando desea preparar un conjunto de etiquetas y luego añadirles clientes a través de «Añadir clientes...».

9.6. Cambio de nombre de un grupo

Botón **Renombrar**, título del diálogo – «**Renombrar {name}**».

Comportamiento (`POST /panel/api/clients/groups/rename` , cuerpo `{"oldName": "...", "newName": "..."}:`

- Ambos nombres se recortan de espacios. Nombre antiguo vacío – error «old group name is required», nombre nuevo vacío – «new group name is required».
- Si el nombre nuevo coincide con el antiguo – no se realiza ninguna acción (0 clientes afectados).
- De lo contrario, el cambio de nombre se ejecuta de forma atómica:
 - el registro en `client_groups` se renombra;
 - en todos los clientes con `group_name = oldName` el campo se actualiza a `newName` ;
 - en **todos los inbound** en los que están los clientes afectados (incluidos los de nodos), en la configuración de Xray el valor de `group` se cambia del antiguo al nuevo.
- Tras el cambio de nombre, el panel marca Xray como pendiente de reinicio y envía una notificación de cambio de clientes.

Mensajes:

- Éxito: «**Grupo renombrado para {count} cliente(s).**»
- Conflicto de nombres en la UI: «**Ya existe un grupo con el nombre «{name}».**»

9.7. Añadir clientes a un grupo

Botón **Añadir clientes...**, título – «**Añadir clientes al grupo «{name}»**».

Texto literal de la ayuda en el diálogo:

«Seleccione los clientes que desea añadir a este grupo. Las asociaciones existentes con inbound se conservan; solo cambia la etiqueta de grupo. Los clientes que ya pertenecen a este grupo no se muestran.»

Si no hay nadie a quien añadir, se muestra «**No hay otros clientes para añadir.**»

Comportamiento (`POST /panel/api/clients/groups/bulkAdd` , cuerpo `{"emails": [...], "group": "..."}:`

- El nombre del grupo es obligatorio (de lo contrario, error «group name is required»); lista de emails vacía – la operación no hace nada.
- Si dicho grupo todavía no existe ni en `client_groups` ni entre los derivados – se creará automáticamente.
- Para los emails seleccionados se establece `group_name = group` en los clientes; **las asociaciones de los clientes con inbound no cambian** – solo se modifica la etiqueta. Luego en todos los inbound de esos clientes se establece el campo `group`.
- Se devuelve el número de registros de clientes afectados; Xray se marca para reinicio.

Mensaje de éxito: «**{count} cliente(s) añadido(s) a {name}.**»

Ejemplo: etiquetar varios clientes con un grupo en una sola solicitud. Los clientes se especifican por email; las asociaciones con inbound no cambian:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

Si el grupo `vip` aún no existe, se creará automáticamente. Tras la solicitud, en el registro de estos clientes se establecerá `group_name = "vip"`, y en la configuración de Xray de cada uno de sus inbound el objeto de cliente recibirá el campo `"group": "vip"`:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Eliminación de clientes de un grupo (sin eliminar los propios clientes)

Botón **Eliminar clientes...**, título – «**Eliminar clientes del grupo «{name}»**».

Texto literal de la ayuda:

«Seleccione los miembros que desea eliminar de este grupo. Los propios clientes se conservan (utilice «Eliminar clientes del grupo» para la eliminación completa).»

Comportamiento (`POST /panel/api/clients/groups/bulkRemove` , cuerpo `{"emails": [...]}`): técnicamente es lo mismo que «Añadir al grupo» con un grupo vacío. En los clientes seleccionados se borra `group_name`, y en sus inbound se elimina el campo `group` de la configuración de Xray. Los propios clientes y sus asociaciones con inbound se conservan.

Mensaje de éxito: «**{count} cliente(s) eliminado(s) de {name}.**»

9.9. Reinicio del tráfico del grupo

Botón **Reiniciar tráfico**.

Diálogo de confirmación:

- Título: «¿Reiniciar el tráfico del grupo {name}?»
- Texto: «Esto pondrá a cero up/down de los {count} cliente(s) de este grupo.»

Comportamiento: para todos los emails de los miembros del grupo se ponen a cero `up` y `down` en la tabla de tráfico y el campo `enable` se establece en `true` (el cliente se activa). La operación se ejecuta en lotes dentro de una transacción.

Mensaje de éxito: «Tráfico reiniciado para {count} cliente(s).»

9.10. Eliminación del grupo y eliminación de clientes del grupo

En la página existen **dos operaciones de eliminación fundamentalmente distintas** — es fácil confundirlas, por lo que la diferencia es crítica.

9.10.1. Eliminar el grupo (conservar los clientes)

Botón «Eliminar el grupo (conservar los clientes)».

Diálogo:

- Título: «¿Eliminar el grupo {name}?»
- Texto: «Esto elimina el grupo y borra su etiqueta en {count} cliente(s). Los propios clientes no se eliminan.»

Comportamiento (`POST /panel/api/clients/groups/delete` , cuerpo `{"name": "..."}`): el registro del grupo se elimina de `client_groups` , en todos sus clientes se borra `group_name` , y de sus inbound se elimina el campo `group` . **Los clientes, sus conexiones y su tráfico se conservan.** Xray se marca para reinicio.

Mensaje de éxito: «Grupo borrado para {count} cliente(s).»

9.10.2. Eliminar los clientes del grupo (eliminación completa)

Botón «Eliminar clientes del grupo».

Diálogo:

- Título: «¿Eliminar todos los clientes de {name}?»
- Texto: «Esto elimina de forma irreversible {count} cliente(s) junto con sus registros de tráfico. La etiqueta de grupo también se borra. Esta acción no se puede deshacer.»

Esta es una operación destructiva: elimina los propios clientes (mediante eliminación masiva por email, endpoint `POST /panel/api/clients/bulkDel`), incluyendo sus registros de tráfico, y por tanto los elimina de todos los inbound.

Mensajes:

- Éxito: «{count} cliente(s) eliminado(s).»
- Resultado parcial: «{ok} eliminado(s), {failed} omitido(s)»

Si el grupo está vacío, las acciones sobre sus miembros no están disponibles – se muestra **«Este grupo aún no tiene clientes.»**

9.11. Relación con la página «Clientes»

La etiqueta de grupo es visible y se utiliza también fuera de la página **Grupos**:

- En el registro compacto del cliente existe el campo `group`, por lo que en la lista de clientes se muestra la pertenencia al grupo.
- La lista de clientes (`/panel/api/clients/list/paged`) acepta el parámetro de filtro `group`: puede pasarse un nombre o varios separados por comas. La comparación se realiza con lógica «O» dentro del campo, sin distinción de mayúsculas y minúsculas. Caso especial: un elemento vacío en la lista de grupos del filtro significa «clientes sin grupo» (aquellos cuyo `group` está vacío).
- En la respuesta de la página de clientes se devuelve el array `groups` – lista completa de nombres de los grupos existentes, para que la UI pueda construir el menú desplegable de filtro.

Ejemplo: filtrar clientes por grupos. La solicitud devuelve solo los clientes con las etiquetas `vip` o `trial` (varios nombres separados por comas, lógica «O»):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Para obtener los clientes **sin** grupo, pase un elemento vacío en la lista – por ejemplo, el valor de filtro `group=` (cadena vacía) o `group=vip,` (etiqueta `vip` más clientes sin grupo).

9.12. Resumen de endpoints de API

Todas las rutas de grupos están montadas bajo `/panel/api/clients`:

Método y ruta	Propósito	Cuerpo de la solicitud
GET <code>/panel/api/clients/groups</code>	Lista de grupos con contadores de clientes	–
GET <code>/panel/api/clients/groups/:name/emails</code>	Emails de todos los miembros del grupo (ordenados por email)	–
POST <code>/panel/api/clients/groups/create</code>	Crear un grupo vacío	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Renombrar un grupo	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Eliminar el grupo conservando los clientes (borrar la etiqueta)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Añadir clientes a un grupo (por email)	<code>{"emails": [...], "group"}</code>
POST <code>/panel/api/clients/groups/bulkRemove</code>	Quitar clientes de un grupo (borrar la etiqueta)	<code>{"emails": [...]}</code>

Método y ruta	Propósito	Cuerpo de la solicitud
POST /panel/api/clients/ bulkDel	Eliminación completa de clientes (utilizado por «Eliminar clientes del grupo»)	<code>{"emails": [...], "keepTraffic"}</code>

Ejemplo: escenario típico del ciclo de vida de un grupo a través de la API.

```
# 1. Crear la etiqueta trial
curl -s ../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. Asignarla a dos clientes
curl -s ../panel/api/clients/groups/bulkAdd -d '{"emails": ["u1@example.com", "u2@example.com"], "group":"trial"}'

# 3. Reiniciar el tráfico de todos los miembros (por email de /groups/trial/emails)
curl -s ../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Eliminar el grupo pero conservar los clientes (solo borrar la etiqueta)
curl -s ../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

El paso 4 elimina el registro del grupo y borra `group_name` en sus clientes, pero los propios clientes, sus conexiones y su tráfico permanecen. Para la eliminación irreversible de los propios clientes se utiliza `bulkDel` en su lugar.

Las operaciones que modifican la etiqueta en los clientes (`rename` , `delete` , `bulkAdd` , `bulkRemove`) marcan Xray como pendiente de reinicio y envían una notificación de cambio de clientes.

9.13. Tráfico por grupo

Novedad de la versión 3.3.0: en la sección **Grupos** (página «Clientes», pestaña de gestión de grupos) la tabla de grupos ahora muestra no solo el número de clientes en cada grupo, sino también el tráfico total consumido por el grupo. La columna lleva el título «**Tráfico utilizado**».

Qué muestra la columna

Para cada fila de grupo se muestra la suma del tráfico de todos los clientes pertenecientes a ese grupo — es decir, la suma de `up` + `down` (tráfico enviado + recibido) de todos sus miembros. Esto da una respuesta rápida a la pregunta «cuánto ha descargado/subido en total el grupo», sin necesidad de abrir los clientes uno a uno y sumar manualmente.

Junto a la tabla de grupos también se muestran:

Columna	Qué significa
Nombre	Nombre del grupo
Clientes	Cuántos clientes están etiquetados con este grupo (antes la columna se llamaba «Clientes en el grupo»)
Enviado	<code>up</code> total (tráfico enviado) de todos los clientes del grupo
Recibido	<code>down</code> total (tráfico recibido) de todos los clientes del grupo

Columna	Qué significa
Tráfico utilizado	up + down total de todos los clientes del grupo

El tráfico enviado y recibido se muestran en columnas separadas **Enviado** y **Recibido**, y la columna **Tráfico utilizado** muestra su suma. La columna del número de clientes se llama simplemente **Clientes**.

El resumen sobre la tabla muestra adicionalmente agregados de todos los grupos – «**Total de grupos**» y «**Clientes con grupo**», y el tráfico total se divide en dos tarjetas: «**Total enviado / recibido**» (con flechas arriba/abajo – tráfico enviado y recibido por separado de todos los grupos) y «**Tráfico total**» (con icono de diagrama – su suma total).

Cómo se calcula

El cálculo se realiza con una sola consulta SQL a la tabla de clientes con una unión (`LEFT JOIN`) a la tabla de contabilidad de tráfico:

- por el campo de etiqueta de grupo (`group_name`) los clientes se agrupan y se cuenta su número – esto es «Clientes en el grupo»;
- el tráfico se toma como la suma de `up + down` de la tabla unida `client_traffics` . Es decir, se suman tanto los bytes enviados (`up`) como los recibidos (`down`) de cada cliente;
- dado que el email es único tanto en la tabla de clientes como en la tabla de tráfico, la unión no duplica el tráfico de un cliente.

Particularidades de los valores:

- **Los clientes sin registro de tráfico** se contabilizan en el contador de miembros, pero aportan 0 a la suma, por lo que un grupo recién creado muestra tráfico `0` .
- **Los grupos vacíos** (creados pero sin clientes) también están presentes en la lista con contador y tráfico cero: además de los grupos «derivados» de las etiquetas de los clientes, en el resultado se mezclan los grupos explícitamente almacenados, y luego la lista se ordena por nombre sin distinción de mayúsculas y minúsculas.
- Los clientes sin etiqueta de grupo (`group_name` vacío) no se incluyen en el cálculo.

Acciones relacionadas

Desde la tabla de grupos siguen estando disponibles las acciones sobre el grupo completo, entre ellas «**Reiniciar tráfico**» – pone a cero `up / down` de todos los clientes del grupo seleccionado. Tras dicho reinicio, la columna «Tráfico utilizado» para ese grupo muestra `0` .

10. Suscripciones (Subscription)

Una suscripción (subscription) es un mecanismo que permite entregar al cliente un único enlace permanente (URL) mediante el cual la aplicación VPN descarga y actualiza periódicamente el conjunto completo de configuraciones. En lugar de reenviar manualmente al usuario un enlace separado para cada inbound, se le proporciona una dirección única del tipo `https://dominio:puerto/sub/<subId>`. A través de esta dirección, el panel ensambla al vuelo todas las configuraciones vinculadas a dicho cliente y las entrega en el formato que el cliente necesita. Cuando cambia algún parámetro en el servidor (nueva dirección, rotación de claves Reality, incorporación de un inbound), el cliente recibe la configuración actualizada en la próxima actualización automática, sin necesidad de ninguna acción por parte del usuario.

La suscripción es atendida por un servidor HTTP/HTTPS independiente dentro del panel, que se inicia de forma autónoma respecto a la interfaz web y escucha en su propio puerto. Esto se hace por motivos de seguridad: el puerto de suscripción puede abrirse al exterior sin necesidad de abrir el puerto del propio panel.

10.1. Qué es subId y cómo se forma el enlace

Cada cliente en un inbound tiene el campo `subId` (en la interfaz — «ID de suscripción»). Este valor es la clave de la suscripción: el panel busca en todos los inbounds los clientes cuyo `subId` coincida con el solicitado y combina sus configuraciones en una sola respuesta.

- Si varios clientes (en uno o distintos inbounds) tienen el mismo `subId`, sus configuraciones se incluirán en una única suscripción. Esta es la forma habitual de entregar a un usuario varios servidores/protocolos a través de un solo enlace.

Ejemplo: un usuario — dos servidores con un solo enlace. Supongamos que hay dos inbounds (VLESS en el servidor A y Trojan en el servidor B). Para entregar al usuario ambas configuraciones con un único enlace, asigne a sus dos clientes el mismo `subId`:

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

Entonces, en la dirección `https://sub.example.com:2096/sub/ivan2025` el panel devolverá ambas configuraciones a la vez. Si más adelante añade un tercer inbound con el mismo `subId`, aparecerá para el usuario en la próxima actualización automática de la suscripción, sin necesidad de enviar un nuevo enlace.

- Si el campo `subId` de un cliente está vacío, no es posible compartir el enlace de acceso general. En la interfaz, esto se indica con el aviso: «Este cliente no tiene subId, el enlace de acceso compartido no está disponible.»

Enlaces externos y suscripciones del cliente (pestaña «Links»)

En el formulario del cliente existe la pestaña **«Links»**, donde para un cliente concreto se pueden adjuntar fuentes adicionales de configuraciones que se mezclan específicamente en su suscripción (formatos RAW, JSON y Clash):

- **Add External Link** – enlace de compartición externo (`vless://`, `trojan://`, `ss://`, etc.). Se añade a la respuesta tal cual, y para JSON/Clash además se descompone en una configuración.
- **Add External Subscription** – dirección de una suscripción externa. El panel la descarga por sí mismo (con caché y un tiempo de espera reducido) e incorpora las configuraciones obtenidas a la lista general del cliente.

Esto resulta útil para ofrecer al cliente servidores adicionales sobre sus inbounds a través del mismo enlace único. Si la respuesta de la suscripción remota es demasiado grande, ya no se trunca silenciosamente: el panel devuelve un error y continúa usando el último valor almacenado en caché con éxito.

- El valor de `subId` no puede establecerse de forma arbitraria: al guardar se verifica que no contenga espacios, los caracteres `/`, `\` ni caracteres de control. El mensaje de validación correspondiente es: «El ID de suscripción no puede contener espacios, '/', '' ni caracteres de control».

El enlace resultante se construye como `<base>/<subPath>/<subId>` (véase la sección sobre la configuración del servidor de suscripciones y el campo «URI de proxy inverso»). Si no se encuentra ningún cliente para el `subId` (el cliente fue eliminado o el `subId` no existe), el servidor devuelve HTTP 404 sin cuerpo. En caso de error interno – HTTP 500. Las aplicaciones VPN se guían únicamente por el código de respuesta, por lo que el cuerpo del error está intencionalmente vacío.

Orden de los enlaces inbound en la suscripción

Cada inbound tiene el campo **«Orden en la suscripción»** (`subSortIndex`) – un número desde 1 que indica la posición de los enlaces de ese inbound en la respuesta de la suscripción. Los valores menores van primero; en caso de igualdad, se mantiene el orden de creación original (por id). El orden se aplica a todos los formatos de respuesta: texto plano, página de suscripción, JSON y Clash. Este campo no afecta al orden de los inbounds en el panel en sí.

El campo se edita en el formulario de inbound junto a la configuración de la dirección del enlace (share address) y se sincroniza a los nodos según las reglas habituales. Si al menos un inbound tiene un orden distinto de 1, aparece en la lista de Inbounds una columna compacta **«Orden»**.

10.2. Configuración del servidor de suscripciones

Todos los parámetros de suscripción se encuentran en la sección de configuración del panel, en la pestaña **«Suscripción»**. A continuación se describe cada parámetro; entre paréntesis se indica la clave interna de configuración y el valor predeterminado.

La propia sección está dividida en pestañas: **«Configuración del panel»**, **«Información»**, **«Perfil»**, **«Certificados»**, **«Happ»** y **«Clash / Mihomo»**. Los campos de título de suscripción, URL de soporte, página de perfil, anuncio y directorio de tema se encuentran en la pestaña «Perfil»; las reglas de enrutamiento de Happ y Clash/Mihomo – en las pestañas homónimas; el intervalo de actualización de la suscripción – en la pestaña «Información».

Parámetros principales

Campo (UI)	Clave	Valor predeterminado	Descripción
Habilitar suscripción	<code>subEnable</code>	<code>true</code> (habilitado)	Inicia un servidor de suscripciones independiente. Descripción: «Función de suscripción con configuración independiente». Si está deshabilitado, el servidor de suscripciones no se inicia y ninguno de los enlaces funciona.
IP de escucha	<code>subListen</code>	vacío	Dirección IP en la que el servidor de suscripciones acepta conexiones. Descripción: «Déjelo vacío de forma predeterminada para rastrear todas las direcciones IP».
Puerto de suscripción	<code>subPort</code>	<code>2096</code>	Puerto TCP del servidor de suscripciones. Descripción: «El número de puerto para el servicio de suscripción no debe estar en uso en el servidor» – el puerto debe estar libre y no entrar en conflicto con el panel o Xray.
Ruta URI	<code>subPath</code>	<code>/sub/</code>	Ruta en la que se sirven las suscripciones normales. Descripción: «Debe comenzar con '/' y terminar con '/'».
Dominio de escucha	<code>subDomain</code>	vacío	Dominio desde el que se permite el acceso a la suscripción (validación Host). Descripción: «Déjelo vacío de forma predeterminada para escuchar en todos los dominios y direcciones IP». Si se define, las solicitudes con un Host diferente son rechazadas.

Importante sobre seguridad: la ruta predeterminada `/sub/` (y `/json/` para JSON) es ampliamente conocida y fácil de adivinar. El panel muestra una advertencia: «La ruta de suscripción predeterminada `/sub/` es ampliamente conocida – cámbiela.» y una similar para JSON. Se recomienda establecer una ruta propia no obvia.

TLS / certificado

Campo (UI)	Clave	Predeterminado	Descripción
Ruta al archivo de clave pública del certificado de suscripción	<code>subCertFile</code>	vacío	Ruta completa al archivo de certificado (<code>.crt</code> / <code>fullchain</code>). Descripción: «Introduzca la ruta completa comenzando con '/'».
Ruta al archivo de clave privada del certificado de suscripción	<code>subKeyFile</code>	vacío	Ruta completa al archivo de clave privada. Descripción: «Introduzca la ruta completa comenzando con '/'».

Si ambas rutas están definidas y el certificado se carga correctamente, el servidor de suscripciones funciona por **HTTPS**. Si los campos están vacíos o el certificado no pudo leerse, el servidor vuelve a **HTTP** (el error se escribe en el log). La presencia de TLS válido también influye en la formación de la URL base: con el puerto 443 con TLS y el puerto 80 sin TLS, el número de puerto se omite en el enlace.

Intervalo de actualización

Campo (UI)	Clave	Predeterminado	Descripción
Intervalos de actualización de suscripción	subUpdates	12	Con qué frecuencia (en horas) la aplicación cliente debe volver a solicitar la suscripción. Descripción: «Intervalo entre actualizaciones en la aplicación cliente (en horas)».

El valor se transmite al cliente en el encabezado HTTP `Profile-Update-Interval`; los clientes modernos lo usan como período de actualización automática de la configuración.

Formato e información en la respuesta

Campo (UI)	Clave	Predeterminado	Descripción
Codificar	subEncrypt	true	Descripción: «Cifrar las configuraciones devueltas en la suscripción». Técnicamente no es cifrado, sino codificación Base64 de todo el cuerpo de la suscripción normal (formato que espera la mayoría de los clientes). Si está deshabilitado, los enlaces se entregan como texto plano, uno por línea.
Mostrar información de uso	subShowInfo	true	Descripción: «Mostrar el tráfico restante y la fecha de vencimiento después del nombre de la configuración». Cuando está habilitado, se añaden al nombre (remark) de cada configuración marcadores de tráfico restante () y fecha de vencimiento (por ejemplo, 5D, 3H); para clientes expirados/no disponibles se muestra N/A .
Incluir Email en el nombre	subEmailInRemark	true	Descripción: «Incluir el email del cliente en el nombre del perfil de suscripción.». Añade el email del cliente al remark del perfil.

Plantilla de remark (Remark Template)

El nombre visible (remark) de cada configuración en la suscripción se forma mediante la **plantilla de remark** – el campo «**Plantilla de nota**» (`remarkTemplate`) en la pestaña «**Información**» de la configuración de suscripción. El antiguo constructor del modelo de nota (selección individual de partes inbound/email/proxy externo y símbolo separador) ha sido eliminado de la interfaz; ahora se escribe un formato de nombre libre y se insertan variables. El valor predeterminado es `{{INBOUND}} | {{TRAFFIC_LEFT}} | {{DAYS_LEFT}}D` . Si el campo se deja vacío, se aplica el modelo de remark anterior (no configurable a través de la interfaz).

Las variables están agrupadas en secciones **Client**, **Traffic** y **Time & status** y se muestran junto al campo como chips clicables `{{VAR}}` con descripción emergente al pasar el cursor; al hacer clic se inserta el token en la plantilla; hay una vista previa en tiempo real. Cada variable se sustituye individualmente para el cliente específico en el momento de la generación de la suscripción. También se admite la notación simplificada con llaves simples (`{DATA_LEFT}` , `{EXPIRE_DATE}` , `{PROTOCOL}` , `{TRANSPORT}` , etc.) – el panel la convierte automáticamente al formato interno `{{...}}` .

Variables disponibles:

- **Identificación del cliente:** `{{EMAIL}}`, `{{INBOUND}}` (remark del propio inbound), `{{HOST}}` (remark del host), `{{ID}}` (UUID), `{{SHORT_ID}}` (primeros 8 caracteres del UUID), `{{SUB_ID}}`, `{{COMMENT}}`, `{{TELEGRAM_ID}}`, `{{PROTOCOL}}`, `{{TRANSPORT}}`.
- **Tráfico:** `{{TRAFFIC_USED}}`, `{{TRAFFIC_LEFT}}`, `{{TRAFFIC_TOTAL}}` (y sus variantes `*_BYTES` en bytes exactos), `{{UP}}`, `{{DOWN}}`, `{{USAGE_PERCENTAGE}}`.
- **Plazo y estado:** `{{DAYS_LEFT}}`, `{{TIME_LEFT}}`, `{{EXPIRE_DATE}}` (AAAA-MM-DD), `{{JALALI_EXPIRE_DATE}}` (fecha en calendario jalali), `{{EXPIRE_UNIX}}`, `{{CREATED_UNIX}}`, `{{RESET_DAYS}}`, `{{STATUS}}` (active / expired / disabled / depleted), `{{STATUS_EMOJI}}`.

La plantilla puede dividirse en segmentos mediante la barra vertical `|`. Un segmento en el que una variable produce un valor «ilimitado» (∞) – por ejemplo `{{TRAFFIC_LEFT}}` o `{{DAYS_LEFT}}` para un cliente sin restricciones – se oculta automáticamente. Además, el bloque con el consumo de tráfico y el plazo se muestra una sola vez, en el primer enlace del cliente, para no repetirse en cada configuración.

Ejemplo. La plantilla `{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D` para un cliente con 42 GB restantes y 7 días producirá el nombre `ivan@vpn 42.00GB 7D`, y para un cliente sin límites – simplemente `ivan@vpn` (los segmentos con ∞ se omiten). | Plantilla de remark | `remarkTemplate | {{INBOUND}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D` | Plantilla libre del nombre visible (remark) de cada configuración con sustitución de variables `{{VAR}}`. Se sustituye individualmente para cada cliente al generar la suscripción. El antiguo constructor «modelo de nota» (selección de inbound/email/proxy externo y separador) ha sido eliminado de la interfaz y se usa solo como alternativa de reserva si el campo se deja vacío. Más detalles – véase «Plantilla de remark (Remark Template)» más arriba. |

Metadatos del perfil (encabezados de respuesta)

Estas cadenas se transmiten al cliente en los encabezados HTTP de la respuesta y se muestran en la aplicación VPN como metadatos del perfil. Todas están vacías de forma predeterminada.

Campo (UI)	Clave	Encabezado	Descripción
Título de suscripción	<code>subTitle</code>	<code>Profile-Title</code> (en Base64)	«Nombre de la suscripción que ve el cliente en la aplicación VPN». Para Clash también se usa como nombre del perfil importado mediante <code>Content-Disposition</code> .
URL de soporte	<code>subSupportUrl</code>	<code>Support-Url</code>	«Enlace al soporte técnico que se muestra en la aplicación VPN».
URL del perfil	<code>subProfileUrl</code>	<code>Profile-Web-Page-Url</code>	«Enlace a su sitio web que se muestra en la aplicación VPN». Si no se define, se sustituye la URL real de la solicitud de suscripción.
Anuncio	<code>subAnnounce</code>	<code>Announce</code> (en Base64)	«Texto del anuncio que se muestra en la aplicación VPN».

Además, en cada respuesta se transmite el encabezado `Subscription-Userinfo` con los datos de tráfico agregados del cliente: `upload`, `download`, `total` y `expire` (momento de vencimiento en segundos). A partir de él, el cliente muestra el tráfico restante y la fecha de vencimiento.

Enrutamiento (solo para el cliente Happ)

Campo (UI)	Clave	Predeterminado	Descripción
Habilitar enrutamiento	<code>subEnableRouting</code>	<code>false</code>	«Configuración global para habilitar el enrutamiento en la aplicación VPN. (Solo para Happ)». Se transmite en el encabezado <code>Routing-Enable</code> .
Reglas de enrutamiento	<code>subRoutingRules</code>	vacío	«Reglas de enrutamiento globales para la aplicación VPN. (Solo para Happ)». Se transmiten en el encabezado <code>Routing</code> .

| Ocultar configuración del servidor | `subHideSettings` | `false` | «Ocultar la configuración del servidor en la suscripción (solo para Happ)». Cuando está habilitado, en el cliente Happ se oculta la posibilidad de ver y modificar los parámetros del servidor. La opción solo tiene efecto para el cliente Happ. |

URI de proxy inverso

Campo (UI)	Clave	Predeterminado	Descripción
URI de proxy inverso	<code>subURI</code>	vacío	«Modificar el URI base de la URL de suscripción para su uso detrás de servidores proxy».

Si el campo está vacío, el panel construye la dirección base del enlace a partir del dominio y el puerto de suscripción (teniendo en cuenta TLS). Si la suscripción se distribuye a través de un proxy inverso/CDN externo en otro dominio o ruta, en este campo se establece el URI base resultante y todos los enlaces se construirán a partir de él. Existen campos independientes equivalentes para JSON (`subJsonURI`) y Clash (`subClashURI`).

Si solo se define el `subURI` general y los campos individuales para JSON y Clash se dejan vacíos, los enlaces de esos formatos en la página de suscripción heredan el esquema y el host de `subURI` (y no el puerto del servidor sub y `http`) – de modo que coinciden con la dirección del proxy inverso.

Ejemplo: suscripción detrás de un proxy inverso. La propia suscripción escucha en `2096`, pero desde fuera está disponible a través de nginx/CDN en `https://cfg.example.com/u/`. Para que los enlaces en la respuesta se construyan desde la dirección externa y no desde el `dominio:2096` interno, en el campo «Reverse proxy URI» se establece el URI base resultante:

```
Reverse proxy URI: https://cfg.example.com/u
```

Entonces el enlace final tomará la forma `https://cfg.example.com/u/ivan2025`. Para los formatos JSON y Clash, si es necesario, se rellenan los campos separados `subJsonURI` y `subClashURI` del mismo modo.

10.3. Formatos de salida

La suscripción puede entregarse en tres formatos independientes, cada uno con su propio endpoint que puede habilitarse/deshabilitarse por separado.

Dirección del servidor y nodos en la respuesta

La dirección del servidor en los enlaces de suscripción se sustituye según la misma estrategia de dirección del enlace que los enlaces normales y los códigos QR del panel: «listen» – dirección de enlace enrutable, «custom» – dirección personalizada definida por el usuario (`shareAddr`), «node» (predeterminado) – dirección del nodo. Para los inbounds sin una estrategia explícitamente definida, la respuesta de la suscripción no cambia. Esto permite que un inbound de nodo, vinculado a una IP pública concreta, entregue a los clientes una dirección accesible. La estrategia se aplica a los formatos RAW, JSON y Clash.

El nombre del nodo (Node) no se añade al nombre (remark) del perfil en la suscripción: en la aplicación cliente solo se muestra el remark del inbound establecido por el administrador, sin el sufijo interno del tipo `@nombre-nodo`. Para distinguir entradas con el mismo nombre en una suscripción multinodo, asigneles remarks diferentes de forma manual o use hosts gestionados (Hosts) con su propio Remark.

Si por desincronización entre nodos el mismo cliente acaba en un inbound JSON de servicio dos veces, la respuesta de la suscripción elimina automáticamente esos duplicados por email en los tres formatos, de modo que no aparecen perfiles repetidos en la respuesta.

Hosts gestionados (Hosts)

La sección **Hosts** (elemento del menú lateral; página de resumen con el recuento Total/Enabled/Disabled y la lista) define sustituciones de dirección para los enlaces de suscripción. Para cada inbound se puede añadir uno o varios **hosts** – endpoints que se sustituyen en los enlaces de suscripción entregados al cliente **en lugar de la dirección, el puerto y los parámetros TLS del propio inbound**. Esto resulta útil para distribuir el tráfico a través de CDN o un relay sin modificar el inbound en sí.

Para cada host se definen:

- **Remark** y descripción (Description), vinculación a un **Inbound** concreto, interruptor **Enable** y asignación a nodos (**Nodes**).
- **Address** (vacío – se hereda la dirección del inbound) y **Port** (`0` – se hereda el puerto del inbound); **Tags** (se tienen en cuenta solo en la suscripción RAW).
- Pestaña **Security** – `same` / `tls` / `none` / `reality` con SNI, huella digital (fingerprint), ALPN, certificado anclado (pinned-cert), `allowInsecure` y ECH.
- Pestaña **Advanced** – encabezado Host, Path, ruta VLESS, Mux, Sockopt, Final Mask y exclusión del host de formatos de suscripción individuales (raw / json / clash).
- Pestaña **Clash (mihomo)** – versión IP, Mihomo X25519, mezcla de hosts (Shuffle host).

Los hosts se ordenan dentro de su inbound y admiten habilitación, deshabilitación y eliminación masiva. Los hosts gestionados reemplazan al anterior array External Proxy.

Enlaces normales (SUB) – Base64 / texto plano

Formato base, endpoint `subPath` (predeterminado `/sub/`). Siempre habilitado (cuando la suscripción en general está habilitada). Devuelve una lista de enlaces Xray (`vless://`, `vmess://`, `trojan://`, `ss://`, etc.) – uno por línea. Con la opción «Codificar» (`subEncrypt`) habilitada, toda la lista se codifica en Base64; cuando está deshabilitada, se entrega como texto plano. Este formato es compatible con prácticamente todos los clientes (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ y otros).

Ejemplo: cuerpo de respuesta con «Codificar» deshabilitado. Con `subEncrypt = false` el endpoint `/sub/` entrega texto plano – un enlace por línea:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

Con `subEncrypt = true` (predeterminado) la misma lista se codifica íntegramente en Base64 y se entrega en una sola línea – este es el formato que espera la mayoría de los clientes.

Suscripción JSON (sing-box y compatibles)

Endpoint `subJsonPath` (predeterminado `/json/`), se habilita con una casilla independiente.

Campo (UI)	Clave	Predeterminado	Descripción
Suscripción JSON	<code>subJsonEnable</code>	<code>false</code>	«Habilitar/deshabilitar el endpoint JSON de suscripción de forma independiente.».

Devuelve la configuración JSON completa (formato compatible con sing-box y clientes derivados – Podkop, OpenWRT sing-box, Karing, NekoBox). Para este formato hay parámetros adicionales disponibles (pestaña `subFormats`):

- **Mux** (`subJsonMux` , predeterminado vacío) – configuración JSON de multiplexación (Mux) que se inyecta en el outbound de cada flujo de suscripción JSON. «Transmisión de múltiples flujos de datos independientes en una sola conexión.».
- **Final Mask** (`subJsonFinalMask` , predeterminado vacío) – «Máscaras finalmask de xray (TCP/UDP) y configuración QUIC añadidas a cada flujo de suscripción JSON. Requiere una versión reciente de xray en el cliente.» Se configura mediante subcampos: «Paquetes» (`packets`), «Longitud» (`length`), «Intervalo» (`interval`), «División máx.» (`maxSplit`), «Ruidos» (`noises` : «Tipo»/ `type` , «Paquete»/ `packet` , «Retardo (ms)»/ `delayMs` , «Aplicar a»/ `applyTo` , botón «+ Ruido»), así como «Concurrencia» (`concurrency`), «Concurrencia xudp» (`xudpConcurrency`) y «xudp UDP 443» (`xudpUdp443`).
- **Reglas de enrutamiento** (`subJsonRules` , predeterminado vacío) – reglas globales añadidas a la configuración JSON.

Suscripción Clash / Mihomo (YAML)

Endpoint `subClashPath` (predeterminado `/clash/`), se habilita con una casilla independiente.

Campo (UI)	Clave	Predeterminado	Descripción
Suscripción Clash / Mihomo	<code>subClashEnable</code>	<code>false</code>	Habilita la generación de configuración YAML para clientes Clash y Mihomo.
Habilitar enrutamiento	<code>subClashEnableRouting</code>	<code>false</code>	«Añadir reglas de enrutamiento globales de Clash/Mihomo a las suscripciones YAML generadas.».
Reglas de enrutamiento globales	<code>subClashRules</code>	vacío	«Reglas de Clash/Mihomo añadidas al inicio de cada suscripción YAML antes de MATCH,PROXY.».

La respuesta se entrega con el tipo `application/yaml; charset=utf-8`. Si se ha definido el «Título de suscripción» (`subTitle`), también se transmite en el encabezado `Content-Disposition ( attachment; filename*=UTF-8' '<title> )`, para que el cliente Clash nombre el perfil importado con ese nombre.

El formato de los enlaces generados y el YAML se mantiene actualizado para los clientes modernos: Shadowsocks-2022 (SS2022) ya no codifica la información de usuario en Base64; los enlaces Shadowsocks con ofuscación http se entregan en formato SIP002 con el plugin `obfs-local`; para las suscripciones Clash/Mihomo se implementa el conjunto completo de campos XHTTP. Esto no requiere ninguna configuración adicional – los enlaces simplemente son reconocidos con mayor precisión por los clientes.

Nota: en esta compilación se admiten exactamente tres formatos – enlaces normales (Base64/texto), JSON (compatible con sing-box) y Clash/Mihomo (YAML). No existe un formato Outline independiente en el servidor de suscripciones.

10.4. Página de información de suscripción y códigos QR

Si se abre el enlace de suscripción en un navegador (o se añade explícitamente el parámetro `?html=1` o `?view=html` a la URL, o se envía el encabezado `Accept: text/html`), el servidor en lugar de la respuesta «en bruto» entrega una **página visual de información de suscripción** («Información de suscripción»). Las aplicaciones VPN siguen recibiendo la respuesta en formato máquina, ya que no solicitan HTML.

La página (aplicación de una sola página, construida con Vite) muestra:

- **Información de la suscripción** (bloque Descriptions):
 - «ID de suscripción» – valor de `subId`;
 - «Estado» – «Activa», «Inactiva» o «Sin límite». El estado «inactiva» se establece si el cliente está deshabilitado, ha agotado el límite de tráfico o ha caducado;
 - «Descargado» y «Enviado» – volúmenes de tráfico;
 - «Límite total» – límite de tráfico o ∞ si no hay restricción;
 - «Fecha de vencimiento» – fecha de finalización o «Sin vencimiento»;
 - tráfico restante y hora de la última conexión.
 - Las fechas se muestran en calendario gregoriano o jalali según la configuración «Calendar Type» del panel (`datepicker` , predeterminado `gregorian`).
- **Enlaces de suscripción**: para cada formato habilitado – una fila separada con etiqueta de color (verde **SUB**, violeta **JSON**, dorado **CLASH**), botón de copia y botón de **código QR** (ventana emergente, tamaño 240 px). La fila con JSON y CLASH aparece solo si el formato correspondiente está habilitado en la configuración.
- **Enlaces individuales** («Copiar enlace»): lista completa de configuraciones individuales incluidas en la suscripción, cada una con su etiqueta de protocolo, botón de copia y código QR (para enlaces post-quantum no se genera QR).
- **Botón «Copiar todas las configuraciones»** (sobre la lista de enlaces individuales): con un solo clic copia al portapapeles todos los enlaces de configuraciones (cada uno en una línea nueva) sin necesidad de copiarlos uno a uno; al finalizar se muestra la notificación «Todas las configuraciones copiadas».

- **Botones de importación rápida en aplicaciones** (menús desplegables por plataforma): para Android – v2box, v2rayNG (deep-link `v2rayng://install-config?url=...`), Sing-box, V2RayTun, NPV Tunnel, Happ (`happ://add/...`); para iOS – Shadowrocket (mediante el parámetro `flag=shadowrocket`), v2box (`v2box://install-sub?url=...&name=...`), Streisand (`streisand://import/...`), V2RayTun, NPV Tunnel, Happ. Estos botones abren el deep-link de la aplicación correspondiente con la dirección de suscripción ya insertada, o copian el enlace al portapapeles.

La página de información se entrega con encabezados de prohibición de caché (`Cache-Control: no-cache`) para que el cliente vea siempre los datos actualizados sobre tráfico y fecha de vencimiento.

10.5. Plantillas personalizadas de la página de suscripción

A partir de 3.3.0 es posible reemplazar la página de aterrizaje estándar de la suscripción con su propia plantilla HTML. De forma predeterminada, en la dirección de suscripción se entrega la página integrada, pero si se especifica un directorio con una plantilla propia, el panel la renderizará e insertará en ella los datos actuales del cliente (tráfico, fecha de vencimiento, enlaces, etc.).

Importante: el panel **no incluye** plantillas listas para usar. En el repositorio solo hay el directorio `sub_templates/` con el archivo de instrucciones `sub_templates/README.md` ; el tema propio debe crearse de forma independiente.

Dónde se activa

El directorio del tema se establece en la configuración del panel:

Configuración → **Suscripción** → **sección «Información de suscripción»**, campo **«Directorio del tema de suscripción»** (`subThemeDir`).

Descripción del campo en la interfaz: «Ruta absoluta a la carpeta con la plantilla personalizada (index.html/sub.html) para la página de suscripción (por ejemplo, /etc/3x-ui/sub_templates/my-theme/). Déjala vacía para usar la página predeterminada.»

En la misma sección se encuentran las configuraciones relacionadas cuyos valores están disponibles en la plantilla:

En la descripción del campo «Directorio del tema de suscripción» hay un enlace **«Guía de plantillas ↗»** a la documentación para crear plantillas de diseño propias para la página de suscripción.

- **«Título de suscripción»** (`subTitle`) – nombre visible para el cliente;
- **«URL de soporte»** (`subSupportUrl`) – enlace al soporte técnico.

Parámetro de configuración

Parámetro	Valor predeterminado	Finalidad
<code>subThemeDir</code>	<code>""</code> (vacío)	Ruta absoluta al directorio con su plantilla HTML. Vacío = página integrada predeterminada.

Cómo establecer su propia plantilla

1. Cree en el servidor una carpeta para el tema (en cualquier lugar), por ejemplo `/etc/3x-ui/sub_templates/my-theme/`.
2. Coloque dentro un archivo HTML con el nombre `index.html` o `sub.html`.

Ejemplo: ruta al tema. Estructura final en el servidor y valor del campo en la configuración:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html      (o sub.html – tiene prioridad)
```

Configuración → Suscripción → «Directorio del tema de suscripción»:
`/etc/3x-ui/sub_templates/my-theme/`

La ruta debe ser **absoluta** (comenzar con `/`). Si en la carpeta no hay ni `index.html` ni `sub.html`, el panel entregará la página integrada. 3. En el panel, abra **Configuración** → **Suscripción** e introduzca la ruta **absoluta** a esa carpeta en el campo «Directorio del tema de suscripción». 4. Guarde la configuración.

Comportamiento de selección de archivo y renderizado:

- Si en el directorio hay `sub.html`, se usa exactamente ese; de lo contrario se toma `index.html`. Es decir, `sub.html` tiene prioridad sobre `index.html`.
- La plantilla se renderiza con el motor estándar de Go `html/template`.
- La plantilla analizada **se almacena en caché** y solo se relee del disco cuando cambia la hora de modificación del archivo. Por lo tanto, las ediciones de la plantilla se recogen sin reiniciar el panel, pero sin la sobrecarga de leer/analizar en cada solicitud.
- La respuesta se forma en un búfer completo y solo entonces se entrega al cliente: si la plantilla falla durante la ejecución, la página parcialmente generada (corrupta) no llegará al usuario.

Comportamiento predeterminado y recuperación (fallback)

- Campo vacío → se entrega la página SPA integrada (los datos se inyectan en `window.__SUB_PAGE_DATA__`).
- La ruta no existe o no es un directorio → se usa la página predeterminada.
- En el directorio no hay ni `index.html` ni `sub.html` → se escribe en el log la advertencia «subThemeDir set but no usable template found» y se entrega la página predeterminada.
- El archivo de plantilla existe pero no se puede analizar → se escribe en el log el error «custom template parse failed» y se entrega la página predeterminada.
- Error al ejecutar la plantilla → se escribe en el log «custom template execution failed» y se entrega la página predeterminada.

Es decir, cualquier problema con la plantilla personalizada no «rompe» la suscripción – el panel siempre degrada a la página integrada. Todas las páginas de suscripción (tanto la personalizada como la estándar) se entregan con encabezados de prohibición de caché (`Cache-Control: no-cache, no-store, must-revalidate`) para que los clientes reciban siempre los datos actualizados sobre tráfico y fecha de vencimiento.

Variables disponibles en la plantilla

Al contexto de la plantilla se le pasa un conjunto de datos del cliente de la suscripción. Acceso – mediante `{{ .nombre }}`:

Variable	Tipo	Descripción
<code>{{ .sId }}</code>	string	ID de suscripción (UUID).
<code>{{ .enabled }}</code>	bool	Si el cliente/suscripción está habilitado.
<code>{{ .download }}</code>	string	Volumen de descarga formateado (p. ej. «2.5 GB»).
<code>{{ .upload }}</code>	string	Volumen de envío formateado.
<code>{{ .total }}</code>	string	Límite total de tráfico formateado.
<code>{{ .used }}</code>	string	Tráfico utilizado formateado (download + upload).
<code>{{ .remained }}</code>	string	Tráfico restante formateado.
<code>{{ .expire }}</code>	int64	Fecha de vencimiento – tiempo Unix en segundos (<code>0</code> = sin vencimiento). Para <code>Date</code> en JS multiplique por 1000.
<code>{{ .lastOnline }}</code>	int64	Hora de la última conexión – tiempo Unix en milisegundos (<code>0</code> = nunca).
<code>{{ .downloadByte }}</code>	int64	Descarga en bytes exactos.
<code>{{ .uploadByte }}</code>	int64	Envío en bytes exactos.
<code>{{ .totalByte }}</code>	int64	Límite total en bytes exactos.
<code>{{ .subUrl }}</code>	string	URL de la página de suscripción.
<code>{{ .subJsonUrl }}</code>	string	URL de la configuración JSON de suscripción.
<code>{{ .subClashUrl }}</code>	string	URL de la configuración Clash/Mihomo.
<code>{{ .subTitle }}</code>	string	Título de suscripción de la configuración (puede estar vacío).
<code>{{ .subSupportUrl }}</code>	string	URL de soporte de la configuración (puede estar vacío).
<code>{{ .links }}</code>	[]string	Lista de cadenas de configuración (VMess, VLESS, etc.). Iteración: <code>{{ range .links }} ... {{ end }}</code> .
<code>{{ .emails }}</code>	[]string	Lista de emails relacionados con la suscripción.
<code>{{ .datepicker }}</code>	string	Formato de calendario actual del panel: <code>gregorian</code> o <code>jalali</code> (tomado de la configuración «Tipo de calendario»; si está vacío – <code>gregorian</code>).

Ejemplo mínimo del cuerpo de la plantilla que usa algunas de las variables:

```
<h1>{{ .subTitle }}</h1>
<p>Utilizado: {{ .used }} de {{ .total }} (restante {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

****Ejemplo: fecha de vencimiento desde `expire`.**** El campo `{{ .expire }}` es tiempo Unix en ****segundos****, por lo que para JavaScript se multiplica por 1000; el valor `0` significa «sin vencimiento»:

```
```html
<script>
 var exp = {{ .expire }};
 document.write(exp === 0
 ? 'Без срока'
 : 'До ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Tenga en cuenta: `{{ .lastOnline }}` ya está en **milisegundos** — no es necesario multiplicarlo por 1000.



---

## ## 11. Xray: enrutamiento, outbounds, DNS y extensiones

La sección **«Configuración de Xray»** es un editor de la plantilla de configuración de Xray-core, a partir de la cual el panel genera el `config.json` definitivo para ejecutar el núcleo. Sugerencia de la sección de plantilla: **«A partir de la plantilla se crea el archivo de configuración de Xray.»** A diferencia de los inbounds (que se almacenan por separado en la base de datos y se insertan en la plantilla al ensamblar la configuración), todo lo demás –registros, enrutamiento, outbounds, DNS, política, estadísticas– se define aquí.

> Importante: el valor de la plantilla se almacena en la base de datos bajo la clave `xrayTemplateConfig`. Al guardar, el panel lo somete a una serie de transformaciones automáticas (ver [11.10](#1110-guardado-reinicio-y-transformaciones-automáticas)). Cualquier JSON sintácticamente incorrecto será rechazado con el error **«xray template config invalid»**.

### #### Ubicación en el menú: «Salientes» y «Enrutamiento»

**«Salientes» (Outbounds)** y **«Enrutamiento» (Routing)** son elementos independientes del menú lateral (justo debajo de «Hosts», encima de «Configuración del panel»), cada uno con su propia dirección: `/outbound` y `/routing`. Los enlaces directos a estas páginas y la recarga de la página funcionan como se espera. En el submenú **«Configuraciones de Xray»** quedan únicamente: Principal, Balanceador, DNS y Plantilla avanzada. En la descripción a continuación, las secciones [11.3](#113-reglas-de-enrutamiento-routing) y [11.4](#114-outbounds-conexiones-salientes) corresponden a las páginas «Enrutamiento» y «Salientes».

### ### 11.1. Estructura del editor: pestañas/modos

El editor ofrece varios modos de visualización de la plantilla (filtros por secciones JSON):

Modo	Qué muestra
---	---
<b>Principal</b>	Secciones básicas (Registro, enrutamiento básico, configuración principal)
<b>Plantilla avanzada</b>	Plantilla JSON completa de Xray
<b>Todo</b>	Todas las secciones simultáneamente

Grupos lógicos de configuración dentro del editor:

- **Configuración principal** (sugerencia: **«Estos parámetros describen la configuración general»**).
- **Registro** (ver [11.9](#119-registros-y-estadísticas-stats-metrics)).
- **Conexiones básicas**: bloqueos y rutas directas.
- **Entrantes** (sugerencia: **«Modificar la plantilla de configuración para conectar determinados clientes»**).
- **Salientes** (ver [11.4](#114-outbounds-conexiones-salientes)).
- **Balanceador** (ver [11.5](#115-balanceadores-balancers)).
- **Enrutamiento** (sugerencia: **«¡El orden de cada regla es importante!»**, ver [11.3](#113-reglas-de-enrutamiento-routing)).
- **DNS / Fake DNS** (ver [11.6](#116-dns)).

### ### 11.2. Configuración principal (General)

## #### Freedom Protocol Strategy

Campo	Etiqueta	Descripción	Por defecto
`FreedomStrategy`		**Configuración de la estrategia del protocolo Freedom**   Estrategia de salida de red para el outbound directo (freedom). Sugerencia: *«Establecer la estrategia de salida de red en el protocolo Freedom»*. Controla el campo `domainStrategy` dentro de `settings` del outbound con protocolo `freedom`.   En la plantilla de referencia, `domainStrategy` para el outbound freedom `direct` es **`AsIs`** (la dirección no se resuelve, se pasa tal cual).	
		`domainStrategy` para freedom (valores de Xray-core): `AsIs` (no resolver el dominio en el servidor), así como la familia `UseIP` / `UseIPv4` / `UseIPv6` y sus variantes «forzadas» `ForceIP*`, que obligan al servidor de salida a resolver el dominio y conectarse por la IP obtenida. Cambie a `UseIPv4` si el servidor de salida no tiene IPv6 o necesita forzar el uso exclusivo de IPv4.	

## #### Freedom Happy Eyeballs (IPv4/IPv6)

Campo	Etiqueta	Descripción
`FreedomHappyEyeballs`		**Freedom Happy Eyeballs (IPv4/IPv6)**   Sugerencia: *«Marcación de doble pila para el outbound directo (freedom) – útil en servidores de salida con IPv4 e IPv6.»* Activa el algoritmo Happy Eyeballs (intento simultáneo en ambas familias de direcciones) para el outbound freedom.
`try delay`	(sugerencia)	*«Milisegundos antes de intentar con la otra familia de direcciones. 150–250 ms es un buen punto de partida.»* Retardo antes de cambiar a la familia de direcciones alternativa. El rango recomendado es 150–250 ms.

## #### Overall Routing Strategy

Campo	Etiqueta	Descripción	Por defecto
`RoutingStrategy`		**Configuración del enrutamiento de dominios**   Estrategia general de resolución DNS para el enrutamiento. Sugerencia: *«Establecer la estrategia general de enrutamiento de resolución DNS»*. Controla el campo `routing.domainStrategy`.   En la plantilla de referencia, `routing.domainStrategy` = **`AsIs`**.	
		`routing.domainStrategy` define cómo se hacen coincidir las reglas IP de enrutamiento con las solicitudes de dominio: `AsIs` (solo reglas de dominio, sin resolución), `IPIfNonMatch` (si el dominio no coincide con las reglas – resolver y verificar reglas IP), `IPOnDemand` (resolver inmediatamente al encontrar una regla IP). Para que funcionen las reglas IP (por ejemplo, `geoip:*`) con solicitudes de dominio, normalmente se requiere `IPIfNonMatch`.	

## #### Outbound Test URL

Campo	Etiqueta	Descripción	Por defecto
`outboundTestUrl`		**URL para probar el saliente**   URL para verificar la conectividad al probar el outbound. Sugerencia: *«URL para verificar la conexión del saliente»*. Se almacena separado de la plantilla, bajo la clave `xrayOutboundTestUrl`.   **`https://www.google.com/generate_204`**	

El valor pasa por saneamiento. Al realizar la prueba, se verifica adicionalmente como una URL pública – es una protección contra SSRF: el usuario no puede inyectar una URL arbitraria (incluidas las internas) a través del cliente; la URL de prueba siempre se

toma de la configuración del servidor. Un valor vacío al guardar/probar se reemplaza por el `generate\_204` predeterminado.

#### #### Block BitTorrent

Campo	Etiqueta	Descripción
`Torrent`	**Bloquear BitTorrent**	Agrega en `routing.rules` una regla que envía el tráfico con `protocol: ["bittorrent"]` al outbound `blocked`. En la plantilla de referencia esta regla está presente por defecto.

#### #### Límites de conexión (Connection Limits)

Sugerencia: *«Políticas de nivel de conexión para usuarios de nivel 0. Deje el campo vacío para usar el valor predeterminado de Xray.»* Estos parámetros se escriben en `policy.levels.0`.

Campo	Etiqueta	Descripción	Por defecto
`connIdle`	**Tiempo de espera de inactividad** (segundos)	**Cierra la conexión tras el número de segundos de inactividad indicado. Reducir el valor libera memoria y descriptores de archivo más rápido en servidores con alta carga (valor predeterminado en Xray: 300).»	vacío → predeterminado Xray <b>300</b>
`bufferSize`	**Tamaño del búfer** (KB)	**Tamaño del búfer interno por conexión en KB. Establezca 0 para minimizar el uso de memoria en servidores con poca RAM (el valor predeterminado de Xray depende de la plataforma).»	Marcador de posición: <b>«auto»</b> . vacío → depende de la plataforma; `0` – minimizar

*«Ejemplo (policy.levels.0).»* Los campos de este grupo se escriben en la política de nivel 0. En un servidor con carga alta y poca RAM se puede acelerar la liberación de recursos así:

```
```json
"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}
```

Aquí la conexión se cierra tras 120 s de inactividad (en lugar de los 300 por defecto), y `bufferSize: 0` minimiza el consumo de memoria en búferes. Un campo dejado vacío en el formulario simplemente no se incluirá en el JSON – y Xray aplicará su valor predeterminado.

11.3. Reglas de enrutamiento (routing)

Lista de reglas `routing.rules`. **El orden es crítico** (*«¡El orden de cada regla es importante!»*): las reglas se evalúan de arriba abajo, se aplica la primera que coincida. Sugerencia: *«Arrastre para cambiar el orden»*. Botones de control de orden: **Primero, Último, Subir, Bajar**.

Cada regla tiene `type: "field"`. Botones: **Crear regla, Editar regla**. Sugerencia para campos de lista: *«Elementos separados por comas»*.

En la página «Enrutamiento», los botones «**Importar reglas**» y «**Exportar reglas**» están agrupados en el menú desplegable «**más**» (more) – igual que en la página «Salientes». El botón «**Exportar reglas**» no descarga el archivo de inmediato, sino que abre una ventana modal con vista previa del JSON y botones «**Copiar**» y «**Descargar**»: el contenido se puede revisar antes de guardarlo. La exportación de salientes en la página «Salientes» funciona de forma análoga.

Route Tester (probador de ruta)

En la pestaña Routing hay una subpestaña **Route Tester** – pregunta al Xray en ejecución qué outbound procesaría una conexión concreta, sin enviar tráfico real. Especifique un dominio o IP, el puerto, la red (TCP/UDP) y, si es necesario, el inbound y el protocolo interceptado (`http` / `tls` / `quic` / `bittorrent`), luego pulse **Test Route**. La decisión se toma directamente del motor de enrutamiento activo.

La respuesta muestra el outbound seleccionado y, si se usa un balanceador, también la etiqueta del balanceador. Si ninguna regla coincidió, el probador informa que el tráfico va al outbound por defecto (el primero de la lista `outbounds`). Esto es útil para verificar el orden de las reglas antes de confiar en ellas.

Activar y desactivar una regla individual

Una regla de enrutamiento individual puede **desactivarse** temporalmente con un interruptor, sin eliminarla. En la tabla de reglas hay una columna «**Activar**» con un interruptor (Switch), y en el formulario de la regla hay un campo «**Activar**» – también un interruptor. Una regla desactivada no se incluye en la configuración final de Xray, pero se conserva en la plantilla y puede reactivarse en cualquier momento.

La regla de servicio de estadísticas (`inboundTag: ["api"]` → `outboundTag: "api"`) no se puede desactivar – su interruptor está bloqueado para no romper el conteo de tráfico del panel (ver [11.10](#)).

Campos del formulario de regla

Campo del formulario	Etiqueta	Campo JSON	Descripción
Origen	Origen	<code>source</code>	Direcciones IP/subredes de origen. Lista separada por comas.
Puerto de origen	Puerto de origen	<code>sourcePort</code>	Puerto(s) de origen.
Destino	Destino	<code>domain + ip + port</code>	Dominios, IPs y puertos de destino. Los dominios admiten los prefijos <code>domain:</code> , <code>full:</code> , <code>regex:</code> , <code>keyword:</code> , así como <code>geosite:*</code> ; las IPs admiten <code>geoip:*</code> y CIDR.
Red	–	<code>network</code>	<code>tcp</code> , <code>udp</code> o <code>tcp,udp</code> .
Protocolo	–	<code>protocol</code>	<code>http</code> , <code>tls</code> , <code>bittorrent</code> (determinado por sniffing).
Usuario	Usuario	<code>user</code>	Filtro por e-mail/identificador de usuario.
Atributos / Valor	Atributos / Valor	<code>attrs</code>	Atributos de cabeceras HTTP para hacer coincidir.
VLESS route	VLESS route	–	Enrutamiento por el campo route para VLESS.

Campo del formulario	Etiqueta	Campo JSON	Descripción
Etiquetas de entrantes	Etiquetas de entrantes	<code>inboundTag</code>	Una o más etiquetas de inbound a las que se aplica la regla (incluidas las integradas <code>api</code> y la etiqueta DNS de la configuración de DNS). En las listas de inbound se muestra como «tag (remark)» si el inbound tiene una nota separada, de lo contrario solo la etiqueta; en las reglas guardadas siguen almacenándose únicamente las etiquetas.
Etiqueta del saliente	Etiqueta del saliente / Conexión saliente	<code>outboundTag</code>	Hacia dónde dirigir el tráfico que coincida.
Etiqueta del balanceador	Etiqueta del balanceador / Balanceador	<code>balancerTag</code>	Sugerencia: «Dirige el tráfico a través de uno de los balanceadores de carga configurados».

Exclusión mutua de `outboundTag` y `balancerTag` : «No es posible usar `balancerTag` y `outboundTag` simultáneamente. Si se usan a la vez, solo funcionará `outboundTag`.» En una regla, especifique la etiqueta del saliente o la etiqueta del balanceador, nunca ambas.

Reglas predefinidas de la plantilla de referencia

En el `config.json` estándar, la sección `routing` contiene tres reglas (en este orden):

1. `inboundTag: ["api"] → outboundTag: "api"` – regla de servicio para la API gRPC de estadísticas del panel.
2. `ip: ["geoip:private"] → outboundTag: "blocked"` – bloqueo de rangos privados.
3. `protocol: ["bittorrent"] → outboundTag: "blocked"` – bloqueo de BitTorrent.

La regla `api → api` siempre se sube automáticamente a la posición 0 al guardar (ver 11.10), para que ninguna regla catch-all superior intercepte la solicitud de estadísticas.

Ejemplo de regla. Enviar todo el tráfico hacia sitios rusos y redes privadas directamente (sin pasar por el proxy), y el resto al balanceador. El orden es importante: la regla «enrutar directamente» debe estar por encima de la catch-all. En `routing.rules` :

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

Para que las reglas IP (`geoip:ru`) se activen también para solicitudes de dominio, normalmente se requiere `routing.domainStrategy: "IPIfNonMatch"` en el nivel superior del enrutamiento (ver 11.2).

Grupos de enrutamiento preconfigurados (Conexiones básicas)

En el modo «Conexiones básicas», el panel ayuda a construir reglas típicas a partir de listas listas para usar:

Grupo	Campos	Sugerencia
Bloqueo por protocolos/sitios	—	«Configure para que los clientes no tengan acceso a determinados protocolos»
Bloqueo por países	IPs bloqueadas, Dominios bloqueados	«Estos parámetros bloquearán el tráfico según el país de destino.»
Conexiones directas	IPs directas, Dominios directos	«Una conexión directa significa que cierto tráfico no se redirigirá a través de otro servidor.»
Reglas IPv4	—	«Estos parámetros permitirán a los clientes enrutar hacia dominios de destino solo a través de IPv4»
Reglas WARP	—	«Estas opciones dirigirán el tráfico según el destino específico a través de WARP.»
Enrutamiento NordVPN	—	«Estas opciones dirigirán el tráfico según el destino específico a través de NordVPN.»

MTProto inbound: enrutamiento del tráfico de Telegram a través de Xray

El inbound MTProto tiene un interruptor **«Route through Xray»** (desactivado por defecto) y una selección opcional de **Outbound**. Al activarlo, el panel agrega al config de Xray un puente SOCKS de loopback con la etiqueta del propio inbound, y mtg dirige el tráfico de Telegram a través de él. A partir de ese momento, el tráfico saliente de Telegram lo gestiona el enrutador: se puede hacer coincidir con reglas normales en la pestaña Routing por la etiqueta del inbound o forzarlo hacia un outbound o balanceador seleccionado mediante el campo **Outbound**. Deje **Outbound** vacío para que sean las reglas de enrutamiento las que tomen la decisión.

11.4. Outbounds (conexiones salientes)

Lista de `outbounds`. Botones: **Crear conexión saliente**, **Modificar conexión saliente**. Sugerencia: «Modificar la plantilla de configuración para definir las conexiones salientes de este servidor».

En la plantilla de referencia hay dos outbounds obligatorios:

- `protocol: "freedom", tag: "direct"` — salida directa a internet (con `domainStrategy: "AsIs"` y `finalRules: [{action: "allow"}]`);
- `protocol: "blackhole", tag: "blocked"` — «agujero negro» para el tráfico bloqueado.

Campos generales del formulario de outbound

Campo	Etiqueta	Descripción
Etiqueta	Etiqueta (sugerencia: «Etiqueta única»)	Identificador único del outbound. Marcador de posición: « <i>etiqueta-única</i> ». Validación: « <i>La etiqueta es obligatoria</i> », « <i>La etiqueta ya está siendo usada por otro saliente</i> ».
Protocolo	—	Tipo del saliente (ver más abajo).
Dirección / Puerto	Dirección / Puerto	Destino de la conexión. La dirección y el puerto son obligatorios.
Enviar a través de	Enviar a través de	Dirección IP local del interfaz saliente (<code>sendThrough</code>). Marcador de posición: « <i>IP local</i> ».
Dialer proxy (cadena)	—	Sugerencia: « <i>Conecte este saliente a través de otro saliente (por etiqueta) para construir una cadena de proxy. Deje vacío para conexión directa.</i> » Marcador de posición: « <i>Seleccione el saliente para la cadena</i> ». Se implementa mediante <code>streamSettings.sockopt.dialerProxy</code> .

Protocolos de outbound compatibles

Protocolos compatibles con el formulario:

- **freedom** — salida directa. Campos `settings.domainStrategy`, `finalRules` (ver más abajo), Happy Eyeballs. No se puede probar («*Outbound has no testable endpoint*»).
- **blackhole** — descarta el tráfico. Campo **Tipo de respuesta**. No se puede probar.
- **socks**, **http** — lista `settings.servers[]` con `address / port`; campo **Contraseña de autenticación**.
- **vmess** — `settings.vnext[]` (`address / port`).
- **vless** — `settings.address / settings.port`.
- **trojan**, **shadowsocks** — `settings.servers[]`.
- **wireguard** — `settings.peers[]` con `endpoint`, más claves (ver 11.7).
- **hysteria** — `settings.address / settings.port` (transporte UDP).

Para el outbound de tipo **loopback** está disponible el bloque **Sniffing** con los mismos parámetros que en los inbounds: activación, **destOverride**, **Metadata Only**, **Route Only** y lista de **dominios excluidos**.

En la máscara **UDP** (FinalMask) para **Hysteria2** hay modos adicionales disponibles. La máscara **Salamander** tiene un selector **Mode** con los valores **Salamander** y **Gecko**: el modo Gecko agrega relleno aleatorio de paquetes con campos **Min/Max** de tamaño (`packetSize`, rango 1–2048, por defecto 512–1200) — esto protege contra fingerprinting por longitud de paquetes. La máscara **Realm** (UDP hole-punching) incorpora un bloque opcional **TLS Config** con campos **Server Name** (SNI), **ALPN** (`h3 / h2 / http/1.1`), **Fingerprint** (uTLS) y el interruptor **Allow Insecure**.

Ejemplo: cadena a través de un SOCKS upstream. El outbound `upstream` se conecta a un proxy SOCKS5 externo, y `chained` envía su tráfico a través de él (`dialerProxy`), formando una cadena. En `outbounds`:

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Ahora una regla de enrutamiento con `outboundTag: "chained"` sacará el tráfico a internet a través de `upstream`.

Importar outbound desde un enlace de compartir

Un outbound se puede importar desde un enlace de compartir (`vless://`, `vmess://`, etc.). Al importar, también se conservan los ajustes del multiplexor **xmux** (XHTTP) pasados en el bloque `extra=` del enlace: tras la importación, sus valores se rellenan en el subformulario **XMUX** del outbound creado.

Campos Mux (multiplexado)

Máx. paralelismo, **Máx. conexiones**, **Máx. reutilizaciones**, **Máx. solicitudes**, **Máx. segundos de reutilización**, **Período keep alive**. Estos parámetros configuran el comportamiento mux/XUDP del saliente.

Socketopts (configuración del socket)

Grupo **Socketopts**: **Intervalo keep alive**, **Mark (fwmark)**, **Interfaz**, **Solo IPv6**, **Aceptar proxy protocol**, **Proxy protocol**, **TCP user timeout (ms)**, **TCP keep-alive idle (s)**. Aquí también se configura el dialer-proxy de la cadena.

Freedom finalRules (anulación del bloqueo de IPs privadas)

Para el outbound freedom está disponible el grupo **Reglas finales**:

Campo	Etiqueta	Descripción
<code>overrideXrayPrivateIp</code>	Anular el bloqueo predeterminado de IP privadas en Xray	Elimina la restricción integrada de Xray para conexiones salientes hacia IPs privadas.
<code>action</code>	Acción	<code>allow</code> (como en la plantilla de referencia: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Redirect) u otras.
<code>blockDelay</code>	Retardo del bloqueo (ms)	Retardo antes de descartar la conexión.

Campo	Etiqueta	Descripción
redirect / fragment	Redirect / Fragment	Acciones de redirección y fragmentación del tráfico.

Máscara fragment: Lengths y Delays por fragmento

En la máscara **fragment** (tipo fragment en FinalMask, para TCP), los campos individuales Length y Delay se reemplazan por listas **Lengths** y **Delays**: para cada segmento se puede definir un rango de longitud independiente (por ejemplo `100-200`) y retardos en milisegundos (por ejemplo `10-20` o `0`). Las líneas de la lista se pueden agregar y eliminar; los valores individuales guardados anteriormente se trasladan automáticamente a un array de un elemento.

Otros campos del formulario

- **UDP over TCP** y **Versión UoT** — para protocolos tipo shadowsocks.
- **Sin cabecera gRPC**, **Tamaño de chunk Uplink** — parámetros del transporte gRPC.
- Campos TLS/uTLS: **Verificar nombre del peer**, **Pinned SHA256**, **Short ID**, **Vision testpre**, marcador de posición «nombre del servidor».

Prueba de salientes

Botones: **Probar**, **Probar todos**. Estados: **Probando conexión...**, **Prueba exitosa**, **Prueba fallida**, **No se pudo probar la conexión saliente**. Resultado: **Resultado de la prueba**, latencia en milisegundos.

Dos modos (sugerencia: «TCP: sonda de solo dial rápida. HTTP: solicitud completa a través de xray.»):

- **TCP** (mode=tcp) — conexión simple hasta `host:port`, ejecutada en paralelo por todos los endpoints, con un tiempo de espera de ~5 s. Solo verifica la accesibilidad TCP, no valida el protocolo proxy. Para `freedom / blackhole` /la etiqueta `blocked` devuelve «Outbound has no testable endpoint».
- **HTTP** (mode=http o vacío) — levanta una instancia temporal de Xray, realiza una solicitud HTTP real (URL de sonda = `outboundTestUrl` del servidor), mide la latencia real. Modo autoritativo pero costoso: se serializa con un bloqueo global («Another outbound test is already running, please wait»). El tiempo de espera de un intento es 10 s, la ventana de espera del resultado es 15 s (aumentados para que los outbounds saludables en canales lentos o tunelizados no se marquen como «Failed»). Ante un fallo, la causa real (error DNS, connection refused, expiración del deadline, error TLS, etc.) se escribe en el registro del panel/Xray, al que apuntan los mensajes generales de tiempo de espera.

Los protocolos UDP (`wireguard`, `hysteria`) y los transportes UDP (`kcp`, `quic`, `hysteria`) **siempre** se prueban en modo HTTP, incluso si se solicitó TCP — un dial UDP puro no distingue un endpoint «vivo» de uno «muerto». Para wireguard en la configuración de prueba se fuerza `noKernelTun: true`.

Verificación por lotes y desglose por etapas

Probar y **Probar todos** en modo HTTP levantan una única instancia temporal de Xray para el conjunto de outbounds, crean para cada uno un inbound SOCKS de loopback con su regla y envían en paralelo una solicitud

HTTP real a través de él; **Probar todos** verifica los outbounds en lotes. **Probar todos** también verifica los outbounds obtenidos de suscripciones (tabla «de suscripciones», solo lectura) – sus filas también se resaltan con el resultado de la prueba. Los outbounds `freedom` («direct») y `dns` no se prueban en ningún modo (no son proxies): el botón de prueba no está disponible para ellos, **Probar todos** los omite, y la protección del servidor prohíbe su prueba HTTP incluso con llamada directa a la API. Además del éxito/error, el resultado emergente muestra el estado HTTP de la respuesta y el desglose del tiempo por etapas: **Proxy connect** (conexión al proxy), **TLS via outbound** (TLS a través del outbound) y **First byte** (tiempo hasta el primer byte) – esto ayuda a identificar en qué paso ocurrió el retardo o el fallo.

Estadísticas de tráfico de outbounds

El panel lleva contadores de tráfico por etiquetas (`up / down / total`). El botón de reinicio restablece los contadores de una etiqueta concreta o de todas (`tag = "-alltags-"`). Los campos **Información de la cuenta** y **Estado de la conexión saliente** muestran un resumen.

11.5. Balanceadores (Balancers)

Lista de `routing.balancers`. Botones: **Crear balanceador**, **Editar balanceador**.

En la pestaña Balancers hay columnas de estado en vivo: **Live Target** muestra el destino activo actual del balanceador en el Xray en ejecución, y **Override** permite anular manualmente la selección del destino (el valor **Auto (strategy)** devuelve la selección a la estrategia). El estado se actualiza con un botón independiente. Si el balanceador todavía no está activo en el Xray en ejecución, el panel sugerirá guardar primero los cambios o iniciar Xray.

Campo	Etiqueta	Descripción
Etiqueta	Etiqueta (sugerencia: «Etiqueta única»)	Identificador único. Marcador de posición: « <i>etiqueta única del balanceador</i> ». Validación: « <i>La etiqueta es obligatoria</i> », « <i>La etiqueta ya está siendo usada por otro balanceador</i> ».
Selectores	Selectores	Lista de etiquetas de outbound (por subcadena) entre las que el balanceador elige la salida. Debe seleccionarse al menos uno: « <i>Seleccione al menos un saliente</i> ».
Fallback	Fallback	Etiqueta de outbound de reserva si ningún selector es adecuado.
Estrategia	Estrategia	Algoritmo de selección (ver más abajo).

Estrategia y parámetros de observación

La estrategia (`strategy.type`) determina cómo el balanceador elige el outbound entre los selectores. Valores de Xray-core: `random` (aleatorio), `roundRobin` (por turno), `leastPing` (latencia mínima según los resultados del observatory), `leastLoad` (carga mínima). Para `leastLoad / leastPing` se usan los parámetros de `strategy.settings`:

Campo	Etiqueta	Descripción
<code>expected</code>	Esperado	Marcador de posición: « <i>número óptimo de nodos</i> » – número objetivo de nodos activos.
<code>maxRtt</code>	RTT máx.	Límite superior del RTT admisible al seleccionar candidatos.

Campo	Etiqueta	Descripción
tolerance	Tolerancia	Tolerancia al comparar latencias/cargas.
baselines	Baselines	Umbrales de latencia para agrupar nodos.
costs	Costs	Coefficientes de peso (cost) para etiquetas individuales.

Ejemplos de estrategias. El bloque `strategy` vive dentro del balanceador (en JSON, junto a `tag` y `selector`):

```
"strategy": { "type": "random" }      // selección aleatoria entre los selectores
"strategy": { "type": "roundRobin" }  // por turno, uno a uno
"strategy": { "type": "leastPing" }   // latencia mínima (requiere observador)
```

Para `leastLoad`, los parámetros se especifican en `settings`:

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.+$", "value": 5 }
    ]
  }
}
```

Cómo funciona (ejemplo). Supongamos que el observador midió las latencias de las salidas: `A = 250 ms`, `B = 280 ms`, `C = 700 ms`, `D = 1500 ms`. Con la configuración anterior, la selección funciona así:

1. **maxRTT: "1s"** — las salidas con latencia superior a 1 s se descartan: `D` (1500 ms) queda eliminada. Quedan `A`, `B`, `C`.
2. **baselines + expected** — las salidas se agrupan por umbrales de latencia, y se toma el **menor** umbral en el que haya al menos `expected` salidas. El umbral `500ms` ya contiene `A` y `B` — son 2 (= `expected`), por lo que se elige el grupo { `A`, `B` }. `C` (700 ms) no entra en la selección mientras haya suficientes rápidas (es la «reserva caliente»).
3. **tolerance: 0.05** — dentro del grupo seleccionado, las salidas cuyas latencias difieren en no más de un 5% se consideran equivalentes y la carga se distribuye entre ellas. `A` (250) y `B` (280) difieren en ~12% (> 5%), por lo que en igualdad de condiciones se prefiere la más rápida, `A`; si la diferencia estuviera dentro del 5%, el tráfico iría tanto por `A` como por `B`.
4. **costs** — ajustan el «coste» de salidas individuales antes de la comparación: un `value` menor hace la salida más atractiva, uno mayor la hace menos preferida. En el ejemplo, `proxy-premium` recibe `0.1` (se vuelve «más barato» y se elige con preferencia), mientras que todos los `proxy-cheap-*` (por expresión regular, `regex: true`) reciben `5` (se vuelven «más caros» y se usan en último lugar). Así se pueden priorizar salidas de forma suave, sin excluirlas rígidamente.

Resultado: el tráfico irá principalmente por **A** (con latencias similares, a partes iguales con **B**), **C** permanecerá como reserva, **D** queda excluida hasta que su RTT baje de `maxRTT`.

Observador: `observatory` y `burstObservatory` (mediciones para `leastPing` / `leastLoad`)

Las estrategias `leastPing` y `leastLoad` no miden nada por sí solas – necesitan datos de latencia y disponibilidad de cada outbound. Los recopila el **observador** (observatory): sondea periódicamente cada outbound rastreado y registra el tiempo de respuesta y la disponibilidad. Esos mismos datos se muestran en la pestaña «**Observatory**» (estados **Activo** / **No disponible**, «Última actividad», «Último intento»).

No hay un formulario separado para el observador en el panel – el bloque se agrega **manualmente** en el editor de configuración de Xray, en el nivel superior de la configuración (junto a `routing` y `outbounds`), tras lo cual hay que **reiniciar Xray**.

Hay dos variantes disponibles:

- **`observatory`** – sencillo: `subjectSelector` + `probeURL` + `probeInterval`.
- **`burstObservatory`** – avanzado, con configuración fina del ping mediante `pingConfig`; conveniente para varias salidas.

Ejemplo del bloque `burstObservatory`:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Descripción de los campos:

Campo	Qué define
<code>subjectSelector</code>	Lista de prefijos de etiquetas de outbound para observar. Xray toma todos los outbounds cuyas etiquetas comienzan con las cadenas indicadas. En el ejemplo se observan las salidas <code>WS-SE...</code> , <code>WS-FR...</code> , <code>WS-PL...</code> . Estas etiquetas deben coincidir con las seleccionadas en los Selectores del balanceador.
<code>pingConfig.destination</code>	URL solicitada a través de cada outbound para medir la latencia. Se usa una página «ligera» con respuesta <code>204</code> sin cuerpo – por ejemplo <code>https://www.google.com/generate_204</code> . El tiempo hasta la respuesta es la latencia medida.
<code>pingConfig.interval</code>	Con qué frecuencia sondear cada outbound. Cadena de duración: <code>"1m"</code> – una vez por minuto, también <code>"30s"</code> , <code>"5m"</code> , etc. Más frecuente – datos más frescos, pero más tráfico de fondo.

Campo	Qué define
<code>pingConfig.connectivity</code>	(opcional) URL para verificar la conectividad básica del propio servidor. Si no está accesible – el problema está en la red del servidor, y el observador no marca el outbound como no disponible (protección contra falsos positivos ante un fallo local). Normalmente también es un endpoint con respuesta <code>204</code> .
<code>pingConfig.timeout</code>	Cuánto esperar la respuesta de un ping antes de considerar el intento fallido (por ejemplo <code>"5s"</code>).
<code>pingConfig.sampling</code>	Cuántas mediciones recientes guardar y promediar por cada outbound. <code>2</code> – considerar los dos últimos pings (suaviza los picos aleatorios).

Cómo conectar todo:

1. En el editor de Xray, agregue el bloque `burstObservatory` con los `subjectSelector` necesarios.
2. Cree el balanceador: **Estrategia** = `leastPing`, en **Selectores** indique las mismas etiquetas de outbound (`WS-SE`, `WS-FR`, `WS-PL`).
3. Diríjale tráfico con una regla de enrutamiento (campo **Etiqueta del balanceador**, ver [11.3](#)).
4. Reinicie Xray. En la pestaña **«Observatory»** aparecerán los estados de las salidas, y el balanceador comenzará a elegir la más rápida entre las activas.

En una sola regla no se pueden especificar `balancerTag` y `outboundTag` simultáneamente – solo funcionará `outboundTag`.

11.6. DNS

Sección `dns`. Activación: **Activar DNS** (sugerencia: «*Activar el servidor DNS integrado*»).

Parámetros generales de DNS

Campo	Etiqueta	JSON	Descripción / sugerencia
<code>tag</code>	Nombre de etiqueta DNS	<code>dns.tag</code>	«Esta etiqueta estará disponible como etiqueta entrante en las reglas de enrutamiento.» Permite enrutar las propias solicitudes DNS mediante <code>inboundTag</code> .
<code>clientIp</code>	IP del cliente	<code>dns.clientIp</code>	«Se utiliza para notificar al servidor la ubicación de IP especificada durante las consultas DNS» (EDNS Client Subnet).
<code>strategy</code>	Estrategia de consulta	<code>dns.queryStrategy</code>	«Estrategia general de resolución de nombres de dominio». Valores: <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> .

Campo	Etiqueta	JSON	Descripción / sugerencia
<code>disableCache</code>	Deshabilitar caché	<code>dns.disableCache</code>	«Deshabilita el almacenamiento en caché de DNS».
<code>disableFallback</code>	Deshabilitar DNS de reserva	<code>dns.disableFallback</code>	«Deshabilita las consultas DNS de reserva».
<code>disableFallbackIfMatch</code>	Deshabilitar DNS de reserva si hay coincidencia	<code>dns.disableFallbackIfMatch</code>	«Deshabilita las consultas DNS de reserva cuando hay coincidencia con la lista de dominios del servidor DNS».
<code>enableParallelQuery</code>	Habilitar consultas paralelas	—	«Habilitar consultas DNS paralelas a varios servidores para una resolución más rápida».
<code>useSystemHosts</code>	Usar Hosts del sistema	<code>dns.useSystemHosts</code>	«Usar el archivo hosts del sistema instalado».

Ejemplo del bloque `dns` . Las consultas a dominios de Google se resuelven a través del servidor DoH de Cloudflare; todo lo demás, a través de `1.1.1.1` ; para las respuestas de Google solo se esperan IPs no privadas. En el nivel superior de la configuración:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

La cadena de servidor (`"1.1.1.1"`) sin campos es el servidor por defecto para todos los demás dominios. La etiqueta `dns-inbound` puede usarse luego como `inboundTag` en las reglas de enrutamiento para dirigir las propias consultas DNS a través del outbound adecuado.

Caché de registros obsoletos

Campo	Etiqueta	Descripción
<code>serveStale</code>	Usar obsoletos	«Devolver resultados obsoletos de la caché mientras se actualiza en segundo plano».
<code>serveExpiredTTL</code>	TTL de obsoletos	«Tiempo de vida (segundos) de los registros de caché obsoletos; 0 = sin límite».

Servidores DNS (lista `dns.servers`)

Botones: **Crear DNS**, **Editar DNS**, **Eliminar todos** (confirmación: «*Todos los servidores DNS serán eliminados de la lista. Esta acción no se puede deshacer.*»). Plantillas: **Usar plantilla**, ventana **Plantillas DNS**, incluyendo el preset **Familiar**.

Al pulsar **Editar DNS** en un registro de servidor DNS (igual que en un registro de Fake DNS), la ventana de edición carga los valores guardados del servidor, no los valores predeterminados.

Campos del servidor DNS:

Campo	Etiqueta	Descripción
address	—	Dirección DNS (IP, URL DoH, <code>localhost</code> , <code>fakedns</code> , etc.).
domains	Dominios	Lista de dominios para los que se usa este servidor.
expectIPs	IPs esperadas	Aceptar la respuesta solo si la IP está en la lista.
unexpectedIPs	IPs no esperadas	Descartar respuestas con las IPs indicadas.
skipFallback	Omitir Fallback	No usar este servidor como fallback.
finalQuery	Consulta final	Marca el servidor como final en la cadena.
timeoutMs	Tiempo de espera (ms)	Tiempo de espera de la solicitud al servidor.

Hosts (registros estáticos)

Grupo **Hosts** (`dns.hosts`). Botón **Agregar Host**; estado vacío **No se han definido hosts**. Campos: dominio (marcador de posición: «*Dominio (p.ej. domain:example.com)*») y valores (marcador de posición: «*IP o dominio – introduzca y pulse Enter*»).

Registros DNS

Ver 11.9: el indicador **Registros DNS** (`dnsLog`) en la sección de registro.

11.7. Fake DNS

Sección `fakedns`. Botones: **Crear Fake DNS**, **Editar Fake DNS**.

Campo	Etiqueta	Descripción
ipPool	Subred del pool de IP	Rango CIDR del que se asignan IPs ficticias (por ejemplo <code>198.18.0.0/15</code>).
poolSize	Tamaño del pool	Cuántas direcciones mantener en el pool circular.

Fake DNS se usa junto con sniffing en el inbound: el núcleo entrega al cliente una IP ficticia, recuerda la correspondencia dominio↔IP y restaura el dominio al enrutar. Para que Fake DNS funcione, el servidor DNS con la dirección `fakedns` debe agregarse a la lista de servidores DNS.

Ejemplo: combinación Fake DNS + servidor DNS. Primero definimos el pool de direcciones ficticias, luego agregamos el servidor DNS `fakedns` para que las consultas de dominio reciban IPs de ese pool:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Además, en el inbound es necesario activar sniffing con `destOverride: ["fakedns"]`; de lo contrario, el núcleo no tiene de dónde obtener el dominio real para la restauración.

11.8. WireGuard / WARP / NordVPN

Campos de WireGuard (`wireguard`)

Campo	Etiqueta	Descripción
<code>secretKey</code>	Clave secreta	Clave privada del interfaz local.
<code>publicKey</code>	Clave pública	Clave pública del peer.
<code>psk</code>	Clave compartida	PreShared Key (opcional).
<code>allowedIPs</code>	Direcciones IP permitidas	Rangos enrutados al túnel.
<code>endpoint</code>	Punto de conexión	<code>host:port</code> del peer.
<code>domainStrategy</code>	Estrategia de dominio	Estrategia de resolución para el outbound WireGuard.

Cloudflare WARP (`warp`)

La integración usa la API <https://api.cloudflareclient.com/v0a4005> (client-version `a-6.30-3596`). Acciones del controlador (`/xray/warp/:action`): `config`, `reg`, `license`, `data`, `del`.

Paso a paso:

1. **Crear cuenta WARP** → `reg`: el panel genera/acepta claves privada (`privateKey`) y pública (`publicKey`), registra el dispositivo en Cloudflare y guarda `access_token`, `device_id`, `license_key`, `private_key` (así como `client_id`) en la configuración `warp`.
2. **Clave de licencia WARP / WARP+** → `license`: establece la clave WARP+ de 26 caracteres (marcador de posición: «Clave WARP+ de 26 caracteres»). En caso de error: «No se pudo establecer la licencia WARP.» Si la configuración aún no se ha obtenido: «Primero obtenga la configuración de WARP.»
3. **Información de la cuenta: Nombre del dispositivo, Modelo del dispositivo, Dispositivo habilitado, Tipo de cuenta, Rol, Datos WARP+, Cuota, Uso.**
4. **Agregar saliente** – crea un outbound WireGuard con las claves y el endpoint de Cloudflare obtenidos.
5. **Eliminar cuenta** → `del`: borra los datos WARP guardados.

NordVPN (nord / nordvpn)

La integración usa NordLynx (= WireGuard). Acciones del controlador (/xray/nord/:action): countries , servers , reg , setKey , data , del .

Paso a paso:

1. **Token de acceso** → reg : el panel solicita las credenciales NordLynx a api.nordvpn.com y extrae nordlynx_private_key . Guarda private_key y token en la configuración nord . Alternativa – setKey : introducir la **Clave privada** directamente (no puede estar vacía).
2. **País** → countries carga la lista de países; **Ciudad** (o **Todas las ciudades**).
3. **Servidor** → servers carga los servidores del país seleccionado (countryId se valida como número – protección contra inyecciones). Filtro: solo se muestran servidores con **Carga** > 7%. Si no hay servidores: «No se encontraron servidores para el país seleccionado». Si el servidor no tiene clave pública NordLynx: «El servidor seleccionado no informa una clave pública NordLynx.»
4. Creación/actualización del saliente: notificaciones «Saliente NordVPN agregado» / «Saliente NordVPN actualizado».

Prioridad IPv4 y userspace TUN

Los outbounds WireGuard generados por los asistentes de WARP y NordVPN utilizan domainStrategy: "ForceIPv4v6" (prioridad IPv4 con vuelta a IPv6 en hosts solo-v6) en lugar de ForceIP – esto elimina el «bloqueo» del handshake en hosts con IPv6 configurado a medias, cuando se selecciona el registro AAAA del endpoint de Cloudflare. Además, en ellos se activa userspace TUN (noKernelTun: true) en lugar del kernel TUN: este último requiere permisos y enrutamiento fwmark, y falla silenciosamente en muchos VPS, mientras que la verificación de conectividad integrada del panel siempre prueba a través de userspace TUN – ahora el tráfico real y la verificación siguen el mismo camino. El cambio solo afecta a los outbounds recién agregados o restablecidos; las plantillas ya guardadas conservan su configuración.

11.9. Reverse proxy y TUN

Reverse (proxy inverso)

Sección reverse de la configuración de Xray. En el formulario de outbound hay un interruptor al tipo **Proxy inverso**. Botones: **Crear proxy inverso**, **Editar proxy inverso**.

Campo	Etiqueta	Descripción
Tipo	Tipo	Bridge o Portal – dos roles del proxy inverso de Xray.
Dominio	Dominio	Dominio de etiqueta de servicio para el par bridge↔portal.
Etiqueta / Conexión	Etiqueta / Conexión	Etiquetas para la vinculación de bridge y portal.
Reverse Tag	Etiqueta de proxy inverso	Sugerencia: «Etiqueta del saliente para un proxy inverso VLESS simple. Deje vacío para desactivar.» Marcador de posición: «etiqueta del saliente (vacío = desactivado)». Implementa un VLESS reverse simplificado.

En el formulario de outbound también están presentes los campos del flujo inverso: **Sniffing inverso**, **Workers**, **Reservado**, **Intervalo mínimo de carga (ms)**, **Tamaño máximo de carga (bytes)**.

TUN (tun)

Campo	Etiqueta	Descripción	Por defecto
name	—	«Nombre del interfaz TUN.»	xray0
mtu	—	«Unidad máxima de transmisión. Tamaño máximo de los paquetes de datos.»	1500
userLevel	Nivel de usuario	«Todas las conexiones establecidas a través de este flujo entrante usarán este nivel de usuario.»	0

11.10. Registros y estadísticas (Stats, metrics)

Registro (log)

Sugerencia: «Los registros pueden ralentizar el servidor. ¡Habilite solo los tipos de registros que necesite!» Sección log de la plantilla de referencia: access: "none", error: "", loglevel: "warning", dnsLog: false, maskAddress: "" .

Campo	Etiqueta	JSON	Descripción	Por defecto
logLevel	Nivel de registros	loglevel	«Nivel de registro para los registros de errores...» Valores: debug , info , warning , error , none .	warning
accessLog	Registros de acceso	access	«Ruta al archivo de registro de acceso. El valor especial «none» deshabilita los registros de acceso.»	none
errorLog	Registros de errores	error	«Ruta al archivo de registros de errores. El valor especial «none» deshabilita los registros de errores.»	"" (por defecto)
dnsLog	Registros DNS	dnsLog	«Habilitar registros de consultas DNS»	false
maskAddress	Enmascaramiento de dirección	maskAddress	«Al activarlo, la dirección IP real se reemplaza por una de enmascaramiento en los registros.»	"" (desact.)

Estadísticas (stats / policy)

Grupo **Estadísticas**. Activa los contadores en policy.system y policy.levels . En la plantilla de referencia: statsInboundUplink: true , statsInboundDownlink: true , statsOutboundUplink: false , statsOutboundDownlink: false ; para el nivel 0 — statsUserUplink: true , statsUserDownlink: true .

Campo	Etiqueta	Descripción	Por defecto
<code>statsInboundUplink</code>	Estadísticas del uplink entrante	«Activa la recopilación de estadísticas para el tráfico saliente de todos los proxies entrantes.»	true
<code>statsInboundDownlink</code>	Estadísticas del downlink entrante	«Activa la recopilación de estadísticas para el tráfico entrante de todos los proxies entrantes.»	true
<code>statsOutboundUplink</code>	Estadísticas del uplink saliente	«Activa la recopilación de estadísticas para el tráfico saliente de todos los proxies salientes.»	false
<code>statsOutboundDownlink</code>	Estadísticas del downlink saliente	«Activa la recopilación de estadísticas para el tráfico entrante de todos los proxies salientes.»	false

Las estadísticas por clientes e inbounds (uplink/downlink) son la base para mostrar el tráfico en el panel y en los clientes; no se recomienda desactivarlas. Las estadísticas de outbound están desactivadas por defecto y solo son necesarias si se hace un seguimiento del tráfico por etiquetas de salientes.

Metrics

En la plantilla de referencia hay una sección `metrics` (`listen: "127.0.0.1:1111"`, `tag: "metrics_out"`) y la API `metrics_out` correspondiente. El panel usa este listener para recopilar métricas e instantáneas del observatory: analiza `metrics.listen` de la plantilla, consulta `/debug/vars` y agrega el historial de latencias por etiquetas. Si cambia la dirección/puerto de `metrics.listen`, el panel se dirigirá a la nueva dirección; eliminar la sección `metrics` desactivará la recopilación de gráficos del observatory.

La prueba de outbound en modo HTTP levanta una instancia temporal **separada** de Xray con su propio listener `metrics` en un puerto aleatorio — no es el mismo listener que el de la configuración principal.

11.11. Guardado, reinicio y transformaciones automáticas

Botones

Botón	Acción
Guardar	POST <code>/xray/update</code> : valida y guarda la plantilla + <code>outboundTestUrl</code> .
Reiniciar Xray	Recarga el servicio con la configuración guardada. Confirmación: «¿Reiniciar xray?» / «Recarga el servicio xray con la configuración guardada.»

Notificaciones: éxito — «Xray reiniciado correctamente», «Xray detenido correctamente»; errores — «Se produjo un error al reiniciar Xray.», «Se produjo un error al detener Xray.» La ventana **Salida del reinicio de Xray** muestra la salida de diagnóstico del núcleo.

Aplicación de cambios en caliente (sin reinicio completo)

Los cambios en inbounds, outbounds y reglas de enrutamiento se aplican «en vivo»: al pulsar **Guardar**, el panel calcula la diferencia entre la configuración antigua y la nueva y aplica solo las partes modificadas a través de la API gRPC de Xray (HandlerService/RoutingService), sin reiniciar el proceso. El reinicio completo se realiza automáticamente solo cuando cambian secciones sin API de recarga en caliente (`log` , `dns` , `policy` , `observatory` , etc.). Por eso en la página Xray no hay un botón «Reiniciar» separado — **Guardar** ya aplica los cambios. El reinicio del núcleo cuando sea necesario sigue realizándose automáticamente (ver también la recarga automática al actualizar suscripciones y la rotación de WARP).

Restaurar la plantilla por defecto

El endpoint `GET /xray/getDefaultJsonConfig` devuelve la plantilla de referencia (`config.json` , integrada en el binario). Con ella se puede restablecer la configuración a los valores de fábrica.

Transformaciones automáticas al guardar

Al guardar la configuración de Xray, el panel realiza (en este orden):

1. **Eliminación de envoltorios** — elimina envoltorios del tipo `{ "xraySetting": <config>, "inboundTags": ..., "outboundTestUrl": ... }` si por error entraron en el valor (de lo contrario, las capas se acumularían en cada guardado). Se eliminan hasta 8 capas.
2. **Verificación de la configuración** — el JSON se analiza en la estructura de configuración de Xray; en caso de error, se rechaza con «*xray template config invalid*».
3. **Garantía de la regla de estadísticas** — la regla `inboundTag: ["api"]` → `outboundTag: "api"` se sube forzosamente a la posición 0 en `routing.rules` (o se agrega si no existe). Esto garantiza que la solicitud de estadísticas gRPC del panel no sea interceptada por una regla catch-all superior (de lo contrario, los clientes pueden aparecer offline con tráfico cero aunque el proxy esté funcionando).

Por el punto 3, no intente eliminar ni mover la regla `api` → `api` — el panel la devolverá a su lugar en el próximo guardado. Es la infraestructura de servicio de estadísticas, no una ruta de usuario.

11.12. Outbound de suscripción (con actualización automática)

A partir de la versión 3.3.0, el panel puede importar `outbound` s directamente desde una URL de suscripción — el mismo formato que los proveedores VPN entregan a las aplicaciones cliente. Las suscripciones se releen periódicamente en segundo plano, por lo que el conjunto de `outbound` s en el servidor se mantiene actualizado sin editar manualmente la plantilla de configuración.

En el interfaz, la sección se llama «**Suscripciones de salientes**», descripción: «Importar salientes desde URL de suscripciones remotas (vmess/vless/trojan/ss/...). Las etiquetas permanecen inalteradas para su uso en balanceadores y reglas de enrutamiento. La actualización se realiza automáticamente.» La sección está en la página Xray, encima del panel de configuración de `outbound` s.

Cómo funciona

Las suscripciones se almacenan separadas de la plantilla de configuración de Xray. La plantilla **nunca se sobrescribe**: los `outbound` s obtenidos de las suscripciones se agregan a la configuración final al vuelo en cada generación del config de Xray.

Agregar una suscripción

En el formulario «Agregar suscripción» están disponibles los siguientes campos:

Campo	Clave	Por defecto	Uso
URL de suscripción	<code>url</code>	– (obligatorio)	Dirección de la suscripción. Marcador de posición: «https://... (lista de enlaces en base64)». Solo se acepta HTTP(S); la dirección se verifica por seguridad.
Nota	<code>remark</code>	vacío	Etiqueta arbitraria (marcador de posición «p.ej. nodos HK»).
Prefijo de etiqueta	<code>tagPrefix</code>	<code>subN-</code>	Prefijo con el que comienzan las etiquetas de los <code>outbound</code> s importados. Si se deja vacío, el panel asignará el menor número libre disponible de la forma <code>sub1-</code> , <code>sub2-</code> , etc.
Intervalo de actualización	<code>updateInterval</code>	600 segundos (10 minutos)	Con qué frecuencia se relee la suscripción. En la interfaz se especifica en horas/minutos.
Habilitado	<code>enabled</code>	sí (<code>true</code>)	Solo las suscripciones habilitadas se incluyen en la configuración y se actualizan automáticamente.
Permitir direcciones privadas	<code>allowPrivate</code>	no (<code>false</code>)	Permite URL en localhost, LAN e IPs privadas. Desactivado por defecto como protección contra SSRF – actívelo solo para fuentes locales de confianza.
Antes de los salientes manuales	<code>prepend</code>	no (<code>false</code>)	Si está habilitado, los <code>outbound</code> s de esta suscripción se colocan antes de los <code>outbound</code> s manuales de la plantilla, y uno de ellos puede convertirse en el <code>outbound</code> por defecto. De lo contrario, se agregan después .

El botón «**Vista previa**» (`POST /outbound-subs/parse`) permite descargar y analizar la URL antes de guardar y ver qué `outbound` s y etiquetas se obtendrán; no escribe nada en la base de datos. Si no se reconoce nada en la URL, se muestra «No se encontraron salientes para esta URL.»

El orden de varias suscripciones en la lista general de `outbound` s se establece por prioridad (`priority`) y se modifica con las flechas arriba/abajo (`POST /outbound-subs/:id/move`).

Formatos de suscripción aceptados

El cuerpo de la respuesta de la URL se procesa así:

- El contenido se intenta primero como **base64** (variantes estándar y URL-safe, con autocompletar el padding y eliminar espacios/saltos de línea). Si es base64 – se decodifica; de lo contrario, se toma tal cual.
- Luego el cuerpo se divide en líneas. Cada línea no vacía que no comience con `#` se analiza como enlace. Las líneas no reconocidas (comentarios, protocolos no compatibles) se omiten silenciosamente.

- Esquemas de enlace compatibles: `vmess://` , `vless://` , `trojan://` , `ss://` (Shadowsocks), `hysteria2://` / `hy2://` , `wireguard://` / `wg://` .

Es decir, es compatible la suscripción estándar del tipo «lista de enlaces codificados en base64», como la de la mayoría de los proveedores.

Etiquetas estables

A cada enlace se le calcula una «identidad» estable (URI base sin el fragmento de nota; para vmess – el JSON interno sin el campo `ps`). La correspondencia «identidad → etiqueta» se conserva, y en la siguiente actualización el mismo servidor recibe la misma etiqueta, aunque hayan cambiado la nota o los parámetros secundarios. Esto se hizo específicamente para que los balanceadores y las reglas de enrutamiento sigan funcionando tras las actualizaciones:

- Una etiqueta exacta en un balanceador/regla seguirá apuntando al mismo servidor.
- Un selector de prefijo/wildcard (por ejemplo, `hk-*`) recogerá automáticamente los nuevos servidores que la suscripción devuelva más adelante – es la forma recomendada de «suscribirse a un pool».
- Si un servidor desaparece de la suscripción, su etiqueta simplemente desaparece del array final de `outbounds`; si el balanceador tiene un `fallbackTag` , Xray lo usará.
- Si el proveedor cambió el UUID/host/credenciales del servidor, la identidad cambia – se considera un nuevo `outbound` con una nueva etiqueta.

Dentro de una misma descarga, las etiquetas se deduplicarán con el sufijo `-N` . Las etiquetas de suscripciones conservan caracteres no ASCII (por ejemplo, cirílico) y permanecen legibles: las letras y dígitos Unicode se conservan en el slug, y la puntuación se reemplaza por un guion – las etiquetas de nombres en cirílico ya no se reducen solo a números.

Cómo funciona la actualización automática

- La tarea en segundo plano de actualización de suscripciones se ejecuta según un programa **cada 5 minutos**.
- En cada ejecución recorre todas las suscripciones habilitadas y actualiza solo aquellas cuyo intervalo propio ha expirado: una suscripción se actualiza si nunca se ha actualizado o si desde la última actualización ha pasado al menos su `updateInterval` . Así la tarea verifica las suscripciones con frecuencia, pero cada suscripción concreta no se relee más de una vez por su `updateInterval` (por defecto 10 minutos). En la interfaz esto se refleja con la sugerencia correspondiente.
- Actualización: la URL se verifica de nuevo como pública (las direcciones privadas se bloquean si la suscripción no tiene `allowPrivate`), la solicitud se realiza a través del cliente proxy del panel con el encabezado `User-Agent: 3x-ui-outbound-sub/1.0` . La cadena de redirecciones está limitada a 10 saltos, y cada salto también se verifica por privacidad (protección contra SSRF). Se espera HTTP 200; de lo contrario, se registra el error.
- Tras un análisis exitoso, el resultado se guarda, se registra la hora de la última actualización y se borra el error. En caso de error, su texto es visible en la interfaz como «Último error», y los `outbounds` obtenidos anteriormente permanecen en vigor.
- Si al menos una suscripción se actualizó realmente, la tarea marca Xray para reinicio y envía una invalidación a la interfaz para que actualice los nuevos `outbounds` . El reinicio efectivo de Xray ocurre en el próximo ciclo de 30 segundos del gestor.

La actualización manual de una suscripción se realiza con el botón **«Actualizar ahora»** (`POST /outbound-subs/:id/refresh`); también marca Xray para reinicio. Agregar, modificar o eliminar una suscripción también activa el indicador de reinicio de Xray (al eliminar, sus `outbound` s desaparecen de la configuración en el próximo reinicio). La interfaz sugiere: «Después de agregar o actualizar, reinicie Xray (o espere al próximo reinicio automático) para que los salientes queden activos.»

Cómo se incorpora al config de Xray

En cada generación de la configuración de Xray, los `outbound` s de suscripciones activas se dividen en dos grupos — `prepend` (indicador «Antes de los salientes manuales») y los demás — y se fusionan con la plantilla: `[prepend de suscripciones] + [outbounds de la plantilla] + [resto de suscripciones]` . Dentro de cada grupo, las suscripciones se ordenan por prioridad. Los `outbound` s manuales de la plantilla no se modifican; si por algún motivo el array de `outbound` s de la plantilla no se puede analizar, los `outbound` s de suscripción no se mezclan en él (para no perder los manuales).

Los `outbound` s importados también se muestran en el propio panel de `outbound` s en un bloque separado **«De suscripciones de salientes (solo lectura)»** — no se pueden editar allí; la gestión es exclusivamente a través de la sección «Suscripciones de salientes».

11.13. Rotación de IP en WARP

En 3X-UI se puede levantar un `outbound` WARP — una conexión WireGuard saliente hacia Cloudflare WARP (etiqueta `warp` en el config de Xray). El panel registra por sí mismo en los servidores de Cloudflare una cuenta de dispositivo, obtiene las claves WireGuard y las direcciones, y las inserta en el `outbound` con la etiqueta `warp` . A través de este `outbound`, el tráfico sale a internet bajo la IP de Cloudflare WARP. La novedad de la versión 3.3.0 es la posibilidad de cambiar esa IP saliente manualmente o según un programa, sin recrear manualmente la cuenta WARP.

La gestión se encuentra en la sección **Xray** en la tarjeta WARP (tras pulsar «Crear cuenta WARP» y obtener la configuración; hasta ese momento las acciones no están disponibles — el panel avisará «Primero obtenga la configuración de WARP»).

Qué ocurre al cambiar la IP

El botón **«Cambiar IP»** inicia el cambio de IP. La lógica:

1. Se genera un nuevo par de claves WireGuard.
2. Con la nueva clave se vuelve a registrar el dispositivo WARP en los servidores de Cloudflare (nuevo `device_id` , `access_token` , direcciones y datos del peer).
3. Los nuevos datos se escriben en el `outbound` WARP del config de Xray: se actualizan `secretKey` , `address` (v4 /32 y v6 /128), `reserved` (de `client_id`), así como `publicKey` y `endpoint` del peer.
4. Si anteriormente se había configurado una clave de licencia WARP+ (de al menos 26 caracteres), se reinstala automáticamente en la nueva cuenta. Si falla, es solo una advertencia en los registros — el cambio de IP no se cancela.
5. Tras el cambio exitoso, Xray se marca como que requiere reinicio para que el nuevo `outbound` entre en vigor.

En caso de éxito, la interfaz muestra «¡La dirección IP de WARP se cambió correctamente!».

Rotación automática según programa

En la tarjeta WARP hay un interruptor «**Actualización automática de la dirección IP**» y un campo «**Intervalo (días)**». Sugerencia: «0 – desactivar. Cambia automáticamente la dirección IP.»

Parámetro	Valor
Configuración en la base de datos	<code>warpUpdateInterval</code> (entero, ≥ 0)
Valor por defecto	0 (rotación automática desactivada)
Unidad de medida	días
0	desactiva el cambio automático
> 0	cambiar la IP cada N días

Al guardar el intervalo se almacena `warpUpdateInterval`, y si el valor es mayor que 0, se restablece la «hora de la última actualización» al momento actual – de lo contrario, el planificador cambiaría la IP ya en el próximo tick.

El programa lo ejecuta una tarea en segundo plano que se lanza una vez por hora – es decir, el panel verifica cada hora si es momento de rotar. Algoritmo de verificación:

- si el intervalo es ≤ 0 – no hace nada;
- si la «hora de la última actualización» es 0 (por ejemplo, el intervalo se estableció editando directamente la base de datos) – es la primera ejecución: la tarea solo registra la marca de tiempo base y NO cambia la IP de inmediato;
- si desde la última actualización ha pasado al menos `intervalo × 24 × 3600` segundos – se ejecuta el mismo cambio de IP, se actualiza la marca de tiempo y se programa el reinicio de Xray.

Detalle importante: el cambio manual con el botón «Cambiar IP» también restablece la marca de tiempo de la última actualización. Por tanto, tras una rotación manual, el conteo del intervalo automático comienza de nuevo y el cambio programado no se ejecutará inmediatamente después.

«A través del proxy del panel»

Cambiado en 3.3.1. El ajuste independiente «Proxy de red del panel» (`panelProxy`) ha sido eliminado. El tráfico saliente del propio panel (incluidas las solicitudes a la API de WARP) ahora se dirige a través del **outbound de tráfico del panel** seleccionado – un outbound de Xray o un balanceador (ver sección 13). La descripción a continuación corresponde a versiones anteriores a 3.3.1.

Todas las solicitudes a la API de Cloudflare WARP (registro, obtención de configuración, establecimiento de licencia, cambio de IP) no se realizan directamente, sino a través del cliente HTTP del panel con un tiempo de espera de 15 segundos. Este cliente respeta el ajuste «**Proxy de red del panel**» (`panelProxy`) de la configuración del panel.

De la descripción del ajuste: el proxy enruta las solicitudes salientes propias del panel (actualizaciones de bases geo, verificaciones de versiones de Xray/panel, Telegram, y ahora también las llamadas a WARP) — para sortear el filtrado del servidor. Se aceptan direcciones del tipo `socks5://` o `http(s)://`, por ejemplo el inbound SOCKS local del propio Xray. Si el campo está vacío o el proxy está mal configurado — se usa conexión directa (el comportamiento no se rompe).

Utilidad para WARP: si el servidor no puede conectarse directamente a `api.cloudflareclient.com`, el registro y la rotación fallaban antes. Ahora, especificando en `panelProxy` un proxy funcional (incluido el propio inbound de Xray), se puede garantizar la accesibilidad de la API de WARP y el funcionamiento tanto del botón manual como de la rotación programada.

Cuándo resulta útil

- Cambiar periódicamente la IP saliente del outbound que pasa por WARP — reduce el riesgo de bloqueos y rastreo por una misma dirección.
 - «Refrescar» la IP manualmente si la dirección actual de Cloudflare ha sido incluida en listas negras o funciona lentamente.
 - Servidores sin acceso directo a la API de Cloudflare WARP: enrutar las solicitudes a través de `panelProxy` hace que el registro y la rotación sean funcionales.
-

12. Nodos (multipanel, master/slave)

La sección **Nodos** convierte una instalación ordinaria de 3X-UI en un **panel central (maestro)** que monitoriza y gestiona de forma remota otros paneles 3X-UI (secundarios). Cada nodo es una instalación independiente de 3X-UI en su propio servidor; el maestro se comunica con ella a través de su API HTTP propia, consulta su estado y le sincroniza los inbounds y clientes que le corresponden. Esta es la capacidad de **multipanel**: en lugar de acceder a cada panel por separado, se ven todos los servidores en una sola lista y se gestionan de forma centralizada.

Principio fundamental: **un nodo no es un agente, sino un panel 3X-UI completo**. El maestro no «instala» nada en él – simplemente se conecta a su API mediante un token. Eliminar un nodo de la lista solo detiene la monitorización; el panel remoto en sí no se ve afectado (sugerencia: «Esto detendrá la monitorización del nodo. El panel remoto en sí no se verá afectado»).

12.1. Resumen en la parte superior de la lista

Encima de la tabla de nodos se muestran contadores agregados:

Campo	Descripción
Total de nodos	Número total de nodos en la lista.
En línea	Cuántos nodos tienen el estado <code>online</code> .
Fuera de línea	Cuántos nodos tienen el estado <code>offline</code> .
Latencia media	Latencia promedio (ping) hasta los nodos, en milisegundos.

12.2. Añadir y editar un nodo

Los botones **Añadir nodo** y **Editar nodo** abren un formulario con los campos del nodo.

Son obligatorios (sugerencia: «El nombre, la dirección, el puerto y el token API son obligatorios») los campos **Nombre**, **Dirección**, **Puerto** y **Token API**.

Al pulsar «Guardar» (tanto al añadir como al editar), el panel **primero verifica la accesibilidad del nodo** con un tiempo de espera de 6 segundos. Si el nodo no responde, el registro no se guarda y se muestra un error. Es decir, no es posible añadir un nodo que sea inaccesible de antemano.

Campos del formulario

Campo	Por defecto	Valores permitidos	Descripción
Nombre	– (obligatorio)	cadena no vacía, única	Nombre interno del nodo. La columna de nombre tiene restricción de unicidad – no se pueden crear dos nodos con el mismo nombre. Marcador de posición de la sugerencia: <code>ej. de-frankfurt-1</code> . Al guardar, los espacios en los extremos se eliminan.

Campo	Por defecto	Valores permitidos	Descripción
Nota	vacío	cualquier cadena	Nota/descripción opcional del nodo. No afecta al funcionamiento.
Esquema	<code>https</code>	<code>http / https</code>	Protocolo de conexión al panel remoto. Si se deja vacío o se especifica un valor no válido, la normalización establecerá <code>https</code> . Si el nodo responde por HTTP ordinario pero el esquema está en <code>https</code> , el panel devolverá una sugerencia clara: «the server speaks HTTP, not HTTPS; set the node scheme to http».
Dirección	– (obligatorio)	host o IP	Dirección del panel remoto. Marcador de posición: <code>panel.example.com</code> o <code>1.2.3.4</code> . La dirección se normaliza; por defecto, las direcciones privadas/locales están prohibidas para protección contra SSRF – véase «Permitir dirección privada».
Puerto	– (obligatorio)	entero 1–65535	Puerto del panel web del nodo remoto. Los valores fuera del rango son rechazados («node port must be 1-65535»).
Ruta base	<code>/</code>	cadena de ruta	Ruta base (web base path) del panel remoto, si está definida. Se normaliza: se garantiza que empieza y termina con <code>/</code> (valor vacío → <code>/</code>). El panel añade <code>panel/api/server/status</code> a continuación al consultar.
Token API	– (obligatorio)	token del panel remoto	Token Bearer para acceder a la API del nodo. Se transmite en la cabecera <code>Authorization: Bearer <token></code> . Marcador de posición: «Token de la página de Configuración del panel remoto». Sugerencia: «El panel remoto muestra su token API en la sección Configuración → Token API». Es decir, el token debe crearse en el propio nodo (Configuración → Token API) y pegarse aquí.
Habilitado	<code>true</code>	sí/no	Activa la monitorización y sincronización del nodo. Los nodos deshabilitados no son consultados por las tareas en segundo plano (heartbeat y traffic-sync los omiten) y no participan en la actualización masiva del panel.
Permitir dirección privada	<code>false</code>	sí/no	Elimina la protección SSRF y permite conectarse al nodo mediante una dirección privada/local. Sugerencia: «Habilitar solo para nodos en red privada o VPN». Habilitar solo cuando el nodo esté realmente en una red privada o sea accesible mediante VPN.

Obtención y regeneración del token en el lado del nodo

El token se obtiene en el panel remoto en la sección **Configuración → Token API**. Allí también se puede reemitir: botón **Regenerar token** con la advertencia: «La regeneración anulará el token actual. Cualquier panel central que lo use perderá el acceso hasta que se actualice. ¿Continuar?». Tras la regeneración, el token antiguo en el panel maestro dejará de funcionar – hay que actualizarlo en el formulario del nodo.

Conexión saliente (Connection outbound)

El campo **Connection outbound** (Conexión saliente, `outboundTag`) define cómo el tráfico de las solicitudes del maestro a la API de este nodo sale del servidor. Si se selecciona en él la etiqueta de un Xray-outbound, las solicitudes del panel al nodo irán no directamente, sino a través del outbound especificado; el panel añadirá automáticamente a la configuración activa un inbound de puente en loopback y aplicará el cambio en caliente,

sin reinicio. Sugerencia: «Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection».

El selector funciona como el selector de outbound del panel: las etiquetas se agrupan en **Outbounds** (salientes ordinarios) y **Balancers** (balanceadores); los outbounds blackhole se ocultan de la lista. El valor vacío (marcador de posición «Direct connection») = conexión directa al nodo.

Import inbound (selección de inbounds a sincronizar)

En el formulario del nodo existe la configuración **Import inbound** (`inboundSyncMode`) con dos modos: **Todos los inbounds** (`all` , por defecto) y **Seleccionados** (`selected`). Por defecto, el maestro sincroniza al nodo todos los inbounds que tienen seleccionado ese nodo; los nodos existentes continúan funcionando en el modo «Todos los inbounds».

En el modo **Seleccionados**, aparece debajo del campo una selección múltiple de etiquetas de inbound. Pulse **Cargar inbounds** — el maestro, con los parámetros de conexión introducidos (aún no guardados), solicitará al nodo la lista de sus inbounds (endpoint `POST /panel/api/nodes/inbounds`) y mostrará sus etiquetas; marque los deseados. El panel solo sincronizará y desplegará en el nodo las etiquetas marcadas, mientras que el resto de los inbounds existentes directamente en el nodo se mantendrán intactos — el maestro no los elimina ni los gestiona.

Ejemplo: solicitar la lista de inbounds del nodo para importación selectiva. En el cuerpo se transmiten los parámetros de conexión aún no guardados; en la respuesta — las etiquetas de los inbounds disponibles en el nodo:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. Verificación TLS (para nodos https)

El grupo de campos define cómo el maestro verifica el certificado HTTPS del nodo. Estos ajustes **solo son relevantes para el esquema https** ; para los nodos `http` se ignoran.

Verificación TLS — lista desplegable, sugerencia: «Cómo el panel verifica el certificado HTTPS del nodo. Anclaje o Omisión — para certificados autofirmados (solo nodos https)».

Modo	Valor	Por defecto	Descripción
Verificar (CA estándar)	<code>verify</code>	sí (por defecto)	Verificación ordinaria de la cadena de certificados por una CA de confianza. Adecuado para nodos con certificado público/Let's Encrypt. Se usa también para todos los nodos <code>http</code> .

Modo	Valor	Por defecto	Descripción
Anclar certificado (SHA-256)	<code>pin</code>	—	La cadena de CA no se verifica, pero el SHA-256 del certificado hoja del nodo se compara con la huella almacenada (comparación en tiempo constante). Mantiene la protección contra MITM para certificados autofirmados . Requiere rellenar el campo de huella.
Omitir verificación	<code>skip</code>	—	La verificación del certificado se deshabilita completamente. Advertencia: «Omitir la verificación elimina la protección contra ataques de intermediario — el token API puede ser interceptado. Es mejor anclar el certificado».

A los tres modos anteriores se añade en 3.4.0 un cuarto — **Mutual TLS (certificado de cliente)** (`mtls`), disponible, como los demás, solo para el esquema `https`.

Modo	Valor	Por defecto	Descripción
Mutual TLS (certificado de cliente)	<code>mtls</code>	—	Además de verificar el certificado del nodo, el maestro se identifica adicionalmente ante el nodo con un certificado de cliente emitido por su propia CA. Para el nodo en este modo el token API se vuelve opcional — el nodo reconoce al maestro por el certificado. Al seleccionar el modo se muestra la sugerencia: «This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it».

Para habilitar TLS mutuo para el nodo: en el lado del nodo, establezca el modo **Mutual TLS**, copie la CA del panel de gestión desde la sección **Node mTLS** (véase a continuación), configúrela en el nodo como **CA raíz de confianza** y reinicie el nodo.

Si se selecciona cualquier valor distinto de `skip`, `pin` o `mtls`, la normalización forzará `verify`.

Anclaje de certificado

Al seleccionar **Anclar certificado** aparecen:

- **SHA-256 del certificado anclado** — campo de entrada. Se acepta la huella en **base64** (formato `pinnedPeerCertSha256` de Xray) o en **hex** con o sin dos puntos (estilo `openssl -fingerprint`). Sugerencia: «SHA-256 del certificado del nodo en base64 o hex. Pulse "Obtener" para leerlo del nodo ahora». Marcador de posición: «SHA-256 en base64 o hex». Al seleccionar `pin`, una huella vacía o incorrecta genera un error de validación al guardar.

Ejemplo: la misma huella en dos formatos. El campo acepta cualquiera de las variantes — ambas representan el mismo certificado:

```
# base64 (formato pinnedPeerCertSha256 de Xray)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex con dos puntos (estilo openssl x509 -fingerprint -sha256)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Si la huella aún no se conoce, pulse **Obtener** — el maestro la leerá del nodo vía HTTPS y la insertará en el campo.

- Botón **Obtener** — se conecta al nodo vía HTTPS sin verificar el certificado y lee el SHA-256 del certificado hoja actual (endpoint `POST /certFingerprint`), insertándolo en el campo. Tras el éxito — «Certificado actual del nodo obtenido»; en caso de fallo — «No se pudo obtener el certificado». Disponible solo para nodos https.

Node mTLS (autenticación TLS mutua entre paneles)

En la página **Nodos** hay una sección separada **Node mTLS** — configuración de la autenticación TLS mutua, que añade al token API un segundo factor (certificado de cliente) para las llamadas «panel → nodo». El TLS mutuo es opcional; si los campos de la sección están vacíos, los nodos funcionan según el esquema anterior — **solo con el token API** (sugerencia: «Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth»). La sección tiene dos operaciones:

- **Copiar la CA de este panel** (`POST /panel/api/nodes/mtls/ca`) — copia el certificado raíz (CA) de este panel al portapapeles. Esta CA debe entregarse a los nodos gestionados para que confíen en el certificado de cliente del panel; en los propios nodos se establece después el modo de verificación TLS **Mutual TLS** (sugerencia: «Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS»). Tras copiar — «CA certificate copied to clipboard».
- **CA raíz de confianza** (Trusted parent CA , `POST /panel/api/nodes/mtls/trustCA`) — campo que se usa cuando este panel actúa como nodo para un panel superior (de gestión). Pegue aquí la CA del panel de gestión para exigirle el certificado de cliente y pulse **Save trust CA**. El cambio requiere **reiniciar el panel** (sugerencia: «When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply»).

12.4. Qué se muestra por cada nodo

Columnas de la tabla y campos de la tarjeta del nodo (estado observado, se rellena con cada consulta heartbeat):

Campo	Descripción
Estado	<code>online</code> / <code>offline</code> / <code>unknown</code> — véase a continuación.
CPU	Carga del procesador del servidor remoto en porcentaje.
Memoria	Uso de RAM en porcentaje (se calcula como <code>current/total*100</code>).
Tiempo activo	Tiempo de funcionamiento continuo del servidor (en segundos).
Latencia	Tiempo de respuesta del nodo en la última consulta (ms).
Último ping	Momento del último heartbeat exitoso (segundos unix; <code>0</code> = «nunca»; un valor reciente se muestra como «ahora mismo»).
Versión de Xray	Versión del Xray-core en ejecución en el nodo.
Versión del panel	Versión de 3X-UI en el nodo — se compara con la actual para el indicador de actualización.

Campo	Descripción
(inbounds)	Cuántos inbounds están físicamente alojados en este nodo.
(clientes)	Número de clientes en los inbounds del nodo.
(en línea)	Cuántos clientes del nodo están actualmente conectados.
(agotados)	Cuántos clientes del nodo han caducado o han agotado el límite de tráfico . Los clientes deshabilitados manualmente no entran en este contador.
(velocidad)	Velocidad de transferencia actual (en vivo) en los inbounds alojados en el nodo.

Los contadores de inbounds/clientes/en línea se asocian al nodo por su GUID estable (`panelGuid`), no por el id local – para que un cliente en un subnodo se contabilice exactamente bajo ese subnodo, y no bajo el nodo intermedio a través del cual se sincronizó.

Para los inbounds alojados en el nodo, la página muestra los clientes en línea, los contadores y la **velocidad de transferencia actual**. La asociación por GUID estable separa correctamente incluso los nodos «clonados» con el mismo `panelGuid`.

Estados del nodo

Estado	Cuándo se establece
online	El nodo respondió <code>success=true</code> a la consulta <code>panel/api/server/status</code> .
offline	El nodo no respondió, devolvió un error HTTP, <code>success=false</code> o una respuesta no reconocida.
unknown	Valor inicial, mientras el nodo aún no ha sido consultado ni una vez.

En caso de una consulta fallida, el texto del error se guarda y se muestra con una formulación clara, lo que ayuda a diagnosticar la causa del «offline».

12.5. Acciones sobre el nodo

- **Probar conexión** (`POST /test`) – en el formulario del nodo, verifica la conexión con los parámetros introducidos (aún no guardados) con un tiempo de espera de 6 s. Resultado: «Conexión correcta ({ms} ms)» o «No se pudo conectar». Útil para depurar la dirección/puerto/token/TLS antes de guardar.
- **Comprobar ahora** (botón «Comprobar ahora», `POST /probe/:id`) – consulta no planificada de un nodo ya guardado; actualiza inmediatamente el estado y las métricas (CPU/memoria/tiempo activo/latencia/versiones) y registra el heartbeat. En caso de fallo – «La comprobación falló».

Ejemplo: verificar y consultar un nodo a través de la API del maestro. «Probar conexión» prueba los parámetros aún no guardados del formulario:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

Consulta no planificada de un nodo ya guardado con id 7:

```
POST /panel/api/nodes/probe/7
```

- **Actualizar panel** (`POST /updatePanel` con cuerpo `{ids:[...]}`) – inicia en el nodo su actualizador automático estándar: el nodo descarga la última versión de 3X-UI y se reinicia con ella. El botón **Actualizar seleccionados ({count})** realiza esto para varios nodos marcados a la vez. Junto al nodo se muestra un indicador: **Actualización disponible** o **Al día**, según la comparación de la versión del panel del nodo con la última disponible.

Ejemplo: actualizar varios nodos con una sola solicitud. En el cuerpo se transmiten los id de los nodos marcados; solo se actualizarán los habilitados y `online`, el resto se devolverá como omitidos.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

Respuesta del tipo «Actualización iniciada en 2 nodos, 1 falló»: el nodo 12, por ejemplo, podía estar offline y por eso fue omitido.

- Confirmación: «¿Actualizar {count} nodos a la última versión? Cada nodo seleccionado descargará la última versión y se reiniciará. Solo se actualizan los nodos habilitados en línea».
- **Solo se actualizan los nodos habilitados con estado `online`**. Un nodo deshabilitado aparece en los resultados marcado como «node is disabled», uno offline – como «node is offline». Resultado: «Actualización iniciada en {ok} nodos, {failed} fallaron». Si no se selecciona ningún nodo adecuado – «Seleccione al menos un nodo habilitado en línea».
- **Set Cert from Panel** (auxiliar, `GET /webCert/:id`) – al crear un inbound en el nodo, permite insertar las rutas al certificado TLS **propio** del nodo (no del panel central), para que los archivos existan exactamente en el nodo. Requiere que el nodo esté habilitado y accesible.
- **Eliminar nodo** (`POST /del/:id`) – confirmación: «¿Eliminar el nodo "{name}"? Esto detendrá la monitorización del nodo. El panel remoto en sí no se verá afectado». Elimina el registro del nodo y sus estadísticas de tráfico acumuladas; el panel remoto continúa funcionando con normalidad. **Un nodo solo se puede eliminar después de que se hayan desvinculado todos sus inbounds**. Si al nodo todavía hay al menos un inbound asociado (a través de `node_id`), el panel rechazará la eliminación con un error del tipo «cannot delete node: N inbound(s) still attached to it; detach or delete them first» – primero desconecte o elimine esos inbounds, y después elimine el nodo. Esto evita inbounds «huérfanos» con una referencia pendiente al nodo eliminado.

12.6. Historial de métricas

El botón/gráfico de historial accede a `GET /history/:id/:metric/:bucket`. Las métricas disponibles son: `cpu` y `mem` – se acumulan con cada heartbeat exitoso. El tamaño del intervalo de agregación (`bucket`, en segundos) está restringido por una lista blanca:

Ejemplo: solicitud de historial. Gráfico de carga de CPU del nodo 7 con agregación por intervalos de 60 segundos (se devuelven hasta 60 puntos):


```
GET /panel/api/nodes/history/7/cpu/60
```

Para memoria y modo «tiempo real» (2 s) – respectivamente `.../7/mem/60` y `.../7/cpu/2`. Los valores fuera de la lista blanca son rechazados («invalid metric» / «invalid bucket»).

Bucket (s)	Propósito
2	Modo de tiempo real
30	Intervalos de 30 s
60	Intervalos de 1 min
120	Intervalos de 2 min
180	Intervalos de 3 min
300	Intervalos de 5 min

Se devuelven hasta 60 puntos. Una métrica o bucket no válidos son rechazados («invalid metric» / «invalid bucket»).

12.7. Cómo se sincronizan inbounds y clientes

Un inbound «pertenece» a un nodo a través del campo `node_id` (en el editor de inbound se selecciona el nodo):

Ejemplo: token en el formulario del nodo. El token se obtiene en el panel secundario (Configuración → Token API) y se pega en el campo **Token API** del maestro. En cada consulta el maestro lo envía en la cabecera:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Si en el panel secundario se ha definido una **ruta base** (web base path), por ejemplo `/secret/`, el maestro la insertará automáticamente antes de `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

1. **Despliegue de configuración (reconcile).** Con cualquier cambio en un inbound/cliente vinculado a un nodo, el nodo se marca como «sucio». Una tarea en segundo plano, para cada nodo habilitado **con estado online** y con cambios pendientes, despliega en el nodo sus inbounds (por `node_id`) y luego restablece el indicador de estado «sucio». Un nodo que está deshabilitado, offline o «sucio» se considera «en espera» – el despliegue en él se pospone hasta que se restaure la conexión.
2. **Recopilación de tráfico.** La misma tarea solicita al nodo una instantánea del tráfico y la fusiona en las estadísticas locales. Basándose en el tráfico fusionado, se realiza la verificación del agotamiento de límites/plazos y, si es necesario, la desactivación de clientes; el contador «agotados» por nodo refleja exactamente esto. Si el nodo no está disponible, sus clientes en línea se borran.

Para un cliente vinculado simultáneamente a varios paneles, el maestro, en la misma tarea, envía adicionalmente a los nodos el consumo de tráfico **total de todos los paneles** de ese cliente (en una tabla separada en el nodo, clave – GUID del maestro; se sobrescribe en cada envío, por lo que el

restablecimiento en el lado del maestro también se propaga). En el nodo, en el tráfico del cliente, se muestra el mayor de los dos valores – el local o el recibido – y al superar la cuota total, el cliente se desactiva **localmente en el propio nodo** (mediante el mismo mecanismo de reinicio de Xray en la desactivación automática, que corta las conexiones ya establecidas). Esto elimina la situación en que el nodo solo veía su parte del tráfico, subestimaba el consumo y seguía atendiendo a un cliente que ya había agotado el límite total. Al restablecer el tráfico, en la renovación automática o al eliminar el cliente, los contadores enviados se borran.

3. **Heartbeat.** Una tarea en segundo plano independiente consulta periódicamente todos los nodos **habilitados** (con límite de paralelismo) a través de `panel/api/server/status`, actualiza el estado/métricas/versiones y, si hay clientes web, distribuye el árbol de nodos actualizado por WebSocket.

12.8. Cadenas de nodos (subnodos / nodos transitivos)

La topología puede no ser plana: un nodo puede ser a su vez maestro de sus propios nodos. Dichos paneles subordinados se muestran como **Subnodos** – son **proyecciones de solo lectura** obtenidas del nodo directo.

- Sugerencia: «Solo lectura: nodo subordinado accesible a través de {padre}. Gestiónelo desde el panel propio de {padre}». Es decir, el subnodo no se puede editar, eliminar ni actualizar aquí – todas las operaciones sobre él se realizan desde el panel de su padre directo.
- La identidad del subnodo se determina por su GUID; gracias a esto, los clientes en línea y los inbounds se contabilizan exactamente bajo el nodo físico que los aloja, incluso en la cadena `Nodo1 → Nodo2 → Nodo3` (el maestro «desciende» un nivel a través de cada nodo directo).
- Si el nodo directo deja de estar disponible, su caché de subnodos se borra y los subnodos desaparecen del árbol hasta que se restaure la conexión.

12.9. Nodos: novedades en 3.3.0

En la versión 3.3.0 la sección **Nodos** recibió tres mejoras notables: atribución correcta del tráfico y de los clientes en línea en topologías multinivel (multi-hop), sincronización de client-IP entre nodos e indicador de estado separado para el caso en que el panel del nodo esté activo pero el núcleo Xray en él haya caído. Las palabras inbound/outbound no se traducen a continuación.

1. Multi-hop: atribución correcta del tráfico por la cadena de subnodos

Antes, los contadores (número de inbounds, clientes en línea, agotados) se calculaban a nivel del nodo «directo». Si tenía una cadena del tipo `Maestro → Nodo1 → Nodo2 → Nodo3`, todo lo que físicamente residía en `Nodo2 / Nodo3` se atribuía erróneamente a `Nodo1`, a través del cual llegaba al maestro. En 3.3.0 la atribución se hace por la fuente real.

Cómo funciona:

- **Los subnodos se muestran como filas independientes.** Cada panel publica la lista de sus nodos directos; solo se incluyen los nodos con `Guid` conocido – la identidad estable es necesaria para atribuir el nodo a un «salto» hacia arriba. El maestro extrae periódicamente (desde la tarea heartbeat) estas listas y las almacena en caché, añadiendo luego a los nodos directos los subnodos «transitivos».

- **Los nodos transitivos son de solo lectura.** En la interfaz se etiquetan como **«Subnodo»** con la sugerencia: *«Solo lectura: nodo subordinado accesible a través de {padre}. Gestiónelo desde el panel propio de {padre}.»* Esa fila no tiene botones de gestión — el nodo se gestiona desde el panel de su padre inmediato.
- **Jerarquía mediante GUID.** El `ParentGuid` del nodo directo es el GUID del propio maestro; el del transitivo es el GUID de su nodo padre. Así se construye el árbol.
- **Fuente de verdad para los contadores — `origin_node_guid` en el inbound.** Es el `panelGuid` del nodo que aloja físicamente ese inbound. Se establece al sincronizar el inbound desde el nodo y **se conserva tal cual en los saltos siguientes**, por lo que un inbound profundamente anidado se atribuye al nodo real, no al intermediario. Con este GUID se recalculan los contadores de número de inbounds, clientes en línea y clientes agotados. Lógica de selección de clave:

Estado del inbound	Se atribuye a
<code>origin_node_guid</code> definido	este GUID (nodo fuente real)
vacío, pero <code>node_id</code> definido	GUID sintético del nodo (build antiguo que aún no informó su <code>panelGuid</code>)
vacío y <code>node_id</code> vacío	GUID propio del maestro (inbound en Xray local)

Los clientes en línea también se agrupan por GUID, por lo que cada fila de nodo muestra solo los que están realmente conectados a él.

Lo que ve el usuario: en una topología plana (nodos directamente bajo el maestro) nada cambia — los contadores por GUID y por `id` coinciden. Pero en cuanto aparece una cadena de nodos, surgen filas de «Subnodos» en la lista, y los números de inbounds/en línea/agotados de cada nodo reflejan ahora exactamente su propia carga, no la suma de todo lo que transitó por él.

2. Sincronización de client-IP desde access.log entre nodos

El límite por IP (`limitIp` en el cliente) se basa en las direcciones que Xray escribe en su `access.log`. Antes, cada nodo solo veía las conexiones hacia sí mismo, por lo que la restricción «no más de N IP por cliente» no funcionaba en el clúster: un cliente podía conectarse a diferentes nodos y eludir el límite. En 3.3.0 las IP observadas se sincronizan por todo el clúster.

Cómo funciona:

- En cada nodo, una tarea en segundo plano analiza el `access.log`, extrayendo de cada línea la IP, el email del cliente y la marca de tiempo, y los almacena en una tabla local (un registro por email, las IP se guardan como matriz JSON `{ip, timestamp}`). Las direcciones locales `127.0.0.1` y `::1` se descartan.
- La sincronización **cada 10 segundos** realiza un intercambio bidireccional por cada nodo habilitado en línea: extrae las IP del nodo y las fusiona en la tabla local, y luego entrega al nodo la imagen consolidada del maestro.
- La fusión combina las observaciones antiguas y las entrantes **sin duplicar** una IP vista en varios nodos, y **sin resucitar** registros obsoletos: se aplica el mismo umbral de antigüedad que en la tarea local — **30 minutos**. Por cada IP se conserva la marca de tiempo más reciente. Los registros de otros nodos reciben un nuevo id local (los espacios de id de los nodos son independientes); la inserción concurrente del mismo email está protegida contra duplicados.

- Al calcular el límite, se considera «activa» una IP que haya sido observada en el escaneo local actual o que tenga una marca de tiempo muy reciente de la base sincronizada (**dentro de los 2 minutos**). Esto es lo que hace que el límite funcione a escala de todo el clúster, incluso si la dirección fue observada en otro nodo. Al superar el límite, las IP «activas» más antiguas se envían al registro fail2ban y las conexiones se interrumpen a la fuerza (remove/re-add del cliente a través de la API de Xray).

Lo que ve el usuario: la restricción por número de IP ahora se aplica a todo el clúster, no a cada nodo por separado; en el panel, por cliente, se ven las IP observadas en cualquier nodo (dentro de una ventana de 30 minutos). No hay ningún botón/configuración separado para esto – la sincronización se realiza automáticamente en segundo plano, siempre que el nodo tenga el access.log habilitado y accesible (para el propio límite también se necesita Fail2Ban en el nodo).

3. Indicador de estado separado: el panel del nodo está en línea, pero Xray ha caído

Antes, el estado del nodo era, en esencia, «en línea / fuera de línea». Si el panel del nodo respondía, el nodo se consideraba en línea – incluso cuando el núcleo Xray en él no funcionaba y los clientes de hecho no podían conectarse. En 3.3.0 la salud del panel y la salud del núcleo Xray están separadas.

Cómo funciona:

- Al consultar el nodo, el maestro toma de la respuesta de `/panel/api/server/status` remoto los campos `xray.state` y `xray.errorMsg` y los guarda en el nodo. Estos campos se rellenan incluso cuando el ping al panel es exitoso pero el núcleo no está sano – precisamente para distinguir la disponibilidad del panel del estado de Xray.
- Valores de `xray.state`: "running" (en ejecución), "stop" (detenido), "error" (error).
- Estos valores se traducen a estados del nodo. A los habituales se añaden nuevos:

Clave de estado	Etiqueta	Cuándo se muestra
online	«En línea»	el panel responde, Xray está en ejecución (running)
offline	«Fuera de línea»	el panel no está disponible / el ping falló
unknown	«Desconocido»	el estado aún no está determinado
xrayError	«Error de Xray»	el panel está en línea, pero el núcleo Xray está en estado error (hay errorMsg)
xrayStopped	«Detenido»	el panel está en línea, pero Xray está detenido (stop)

- Para este tipo de estado en la interfaz se utiliza **un indicador violeta separado** (color diferente al verde de «en línea» y al rojo de «fuera de línea»). El violeta señala directamente: se puede alcanzar el nodo, el problema está en el núcleo Xray en sí, no en la red ni en el propio panel.

Lo que ve el usuario: en lugar del engañoso «verde» cuando el núcleo ha caído, el nodo se resalta en **violeta** con el estado **«Error de Xray»** o **«Detenido»**. Esto indica de inmediato que hay que reparar Xray en el nodo (reiniciar el núcleo, revisar errorMsg), y no investigar la accesibilidad del nodo en sí. El mismo xrayState /

`xrayError` se propaga también a los subnodos transitivos (véase el punto 1), por lo que el estado incorrecto del núcleo es visible en toda la cadena.

13. Configuración del panel

La sección «Configuración» (título de la página – **Configuración**, en inglés *Panel Settings*) controla el comportamiento del propio panel web 3X-UI: en qué dirección y puerto escucha, cómo está protegido, cómo interactúa con el bot de Telegram y los servicios externos, y en qué zona horaria ejecuta las tareas programadas. Cada parámetro se almacena en la tabla `settings` de la base de datos como un par «clave – valor»; si el valor no está en la BD, se aplica el valor por defecto.

Importante – aplicación de cambios. Cualquier cambio en esta página debe guardarse con el botón **Guardar** (*Save*) y luego reiniciar el panel para que los cambios surtan efecto. El mensaje literal dice: «Guarda los cambios y reinicia el panel para aplicarlos.» Al guardar aparece la notificación «Configuración modificada».

13.1. Guardar y reiniciar el panel

Elemento	Función
Guardar (<i>Save</i>)	Escribe todos los campos del formulario en la BD (<code>POST /panel/setting/update</code>). Antes de escribir, los valores pasan por validación – las direcciones, puertos o rutas incorrectas serán rechazados y el panel devolverá un error.
Reiniciar panel (<i>Restart Panel</i>)	Reinicia el servidor web del panel (<code>POST /panel/setting/restartPanel</code>). El reinicio ocurre con un retraso de 3 segundos. Mensaje literal: «¿Está seguro de que desea reiniciar el panel? Confirme y el reinicio ocurrirá en 3 segundos. Si el panel no está disponible, revise el log del servidor». Si tiene éxito: «El panel se reinició correctamente».
Restaurar configuración predeterminada (<i>Reset to Default</i>)	Elimina todos los ajustes guardados en la BD, tras lo cual el panel usa los valores por defecto. Las credenciales del administrador no se restablecen con esta operación.

El reinicio se realiza enviando la señal `SIGHUP` al proceso del panel (o mediante el gancho de reinicio registrado). En Windows no se admite el reinicio automático mediante señal. **Los cambios en los parámetros de escucha (IP, puerto, ruta, dominio, certificados, zona horaria) solo se aplican tras reiniciar el panel.**

13.2. Configuración general (pestaña «Panel» / *General*)

Idioma de la interfaz (*Language*)

Idioma de la interfaz web del panel. Idiomas disponibles: `en-US` (inglés), `ru-RU` (ruso), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. Es una configuración de visualización y no afecta al funcionamiento de Xray.

Tipo de calendario (*Calendar Type*)

- **Clave:** `datepicker`
- **Valor por defecto:** `gregorian` (gregoriano).

- **Función:** tipo de calendario utilizado en los selectores de fecha (por ejemplo, al definir la fecha de vencimiento de los clientes). Mensaje literal: «Las tareas programadas se ejecutarán de acuerdo con este calendario.» El valor alternativo es el calendario persa (jalali), muy demandado entre el público iraní del panel.

Tamaño de paginación (*Pagination Size*)

- **Clave:** `pageSize`
- **Valor por defecto:** `25`
- **Valores admitidos:** entero de `0` a `1000`.
- **Función:** número de filas por página en las tablas (listas de conexiones/inbound). Mensaje literal: «Define el tamaño de página para la tabla de conexiones. Establece 0 para desactivar» — con `0` la paginación se desactiva y todos los registros se muestran en una sola lista.
- **No requiere reiniciar el panel** (ajuste de visualización).

Reiniciar Xray tras desactivación automática (*Restart Xray After Auto Disable*)

- **Clave:** `restartXrayOnClientDisable`
- **Valor por defecto:** `true`
- **Función:** cuando un cliente se desactiva automáticamente (por vencimiento del plazo o por alcanzar el límite de tráfico), Xray se reinicia para interrumpir las conexiones ya establecidas de ese cliente. Mensaje literal: «Cuando un cliente se desactiva automáticamente por vencimiento del plazo o límite de tráfico, reiniciar Xray.» La función en sí no ha cambiado — el interruptor simplemente se encuentra en la pestaña «Panel» (*General*) junto al resto de los ajustes generales.

Modelo de comentario y carácter separador (*Remark Model & Separation Character*)

- **Clave:** `remarkModel`
- **Valor por defecto:** `-ieo`
- **Función:** define cómo se forma el nombre (remark) de la configuración en la suscripción. La cadena consta del **primer carácter** — separador — seguido de la **secuencia de letras de orden**:
 - `i` — comentario del inbound (*inbound remark*);
 - `e` — email del cliente;
 - `o` — etiqueta adicional (*extra*).

Con el valor por defecto `-ieo` el separador es `-` y el orden de las partes es: inbound → email → extra (por ejemplo, `MyInbound-user@mail-extra`). Las partes vacías se omiten. El campo «Ejemplo de comentario» (*Sample Remark*) en la interfaz muestra una vista previa del nombre generado. La inclusión del email en el nombre depende adicionalmente del parámetro «Incluir Email en el nombre» en la configuración de suscripción (sección sobre suscripciones).

Ejemplo: cómo el valor de `remarkModel` afecta al nombre de la configuración. Supongamos que el inbound se llama `VLESS-Reality`, el email del cliente es `alex@vpn` y la etiqueta adicional es `RU`. Entonces:

Valor del campo	Nombre resultante (remark)
-ieo (por defecto)	VLESS-Reality-alex@vpn-RU
_ie	VLESS-Reality_alex@vpn
-ei	alex@vpn-VLESS-Reality
o (espacio como separador, solo etiqueta)	RU

El primer carácter de la cadena siempre es el separador; las demás letras determinan qué partes y en qué orden formarán el nombre.

13.3. Acceso al panel: IP, puerto, ruta, dominio, certificado

Este grupo define el punto de entrada de red del panel. **Todos los cambios aquí solo se aplican tras reiniciar el panel.**

Campo	Clave	Valor por defecto	Descripción
Dirección IP de escucha del panel (<i>Listen IP</i>)	webListen	"" (vacío)	IP en la que escucha el panel web. Vacío = escuchar en todas las IP. Mensaje literal: «Déjelo vacío para conectarse desde cualquier IP». Si se especifica, debe ser una dirección IP válida (de lo contrario el guardado se rechaza).
Dominio del panel (<i>Listen Domain</i>)	webDomain	"" (vacío)	Nombre de dominio del panel para verificar la solicitud por dominio. Vacío = aceptar conexiones desde cualquier dominio e IP. Mensaje literal: «Déjelo vacío para conectarse desde cualquier dominio e IP.»
Puerto del panel (<i>Listen Port</i>)	webPort	2053	Puerto en el que funciona el panel. Mensaje literal: «Puerto en el que funciona el panel». Admite 1-65535. El puerto debe estar libre; el panel y el servicio de suscripción no pueden usar simultáneamente el mismo par IP:puerto.
Ruta URI (<i>URI Path</i>)	webBasePath	/	Ruta base de la URL del panel (basePath). Mensaje literal: «Debe comenzar con '/' y terminar con '/'». Al guardar, el panel agrega automáticamente la barra inicial y final / si están ausentes. Se rechazan los caracteres no permitidos en la ruta.

Certificado del panel (TLS / HTTPS)

Campo	Clave	Valor por defecto	Descripción
Ruta al archivo de clave pública del certificado del panel (<i>Public Key Path</i>)	webCertFile	""	Ruta completa al archivo de certificado (cadena). Mensaje literal: «Introduzca la ruta completa comenzando con '/'».
Ruta al archivo de clave privada del certificado del panel (<i>Private Key Path</i>)	webKeyFile	""	Ruta completa al archivo de clave privada. Mensaje literal: «Introduzca la ruta completa comenzando con '/'».

Si se especifica **al menos una** de las rutas de certificado/clave, al guardar el panel intenta cargar el par «certificado + clave»; si hay un error (archivo inexistente, clave y certificado no coinciden) el guardado se rechaza. Cuando ambas rutas correctas están definidas, el panel pasa a HTTPS. Ambos campos vacíos = el panel funciona con HTTP simple.

Advertencias de seguridad (*Security warnings*). El panel muestra el bloque «Su panel puede estar expuesto:» con advertencias si detecta una configuración insegura:

- funcionamiento con HTTP simple – «configure TLS para producción»;
- puerto estándar 2053 – «cámbielo por uno aleatorio»;
- ruta base por defecto / – «cámbiela por una aleatoria»;
- ruta de suscripción estándar /sub/ y de suscripción JSON /json/ – «cámbielas». Son recomendaciones, no bloqueos.

13.4. Sesión, proxy del panel y proxies de confianza (pestaña «Proxy y servidor» / *Proxy and Server*)

Duración de sesión (*Session Duration*)

- **Clave:** `sessionMaxAge`
- **Valor por defecto:** 360 (minutos, es decir, 6 horas).
- **Valores admitidos:** de 1 a 525600 minutos (1 año).
- **Función:** cuánto tiempo permanece el administrador autenticado sin necesidad de volver a iniciar sesión. La unidad es el **minuto**. Mensaje literal: «Duración de la sesión en el sistema (valor: minuto)».

Outbound para el tráfico del panel (*Panel Traffic Outbound*)

- **Clave:** `panelOutbound`
- **Valor por defecto:** "" (vacío = conexión directa).
- **Función:** define el **outbound** de Xray a través del cual el panel envía sus **propias solicitudes** – verificaciones de versiones y descarga del panel/Xray, comunicaciones con Telegram, actualización habitual de archivos geo – para eludir el filtrado del servidor de GitHub/Telegram. El campo es una **lista desplegable**: en ella se enumeran los outbounds de la plantilla de configuración de Xray, los outbounds de las suscripciones a outbound, así como los **balanceadores** de enrutamiento (en un grupo separado). Los outbounds de tipo `blackhole` están excluidos de la lista – no tiene sentido enrutar las descargas hacia un «agujero negro». Mensaje literal: «Enruta las solicitudes propias del panel – verificaciones de versiones y descargas del panel/Xray, Telegram y la actualización habitual de archivos geo – a través de este outbound de Xray para eludir el filtrado del servidor de GitHub/Telegram. Un inbound puente local se agrega automáticamente a la configuración en ejecución y se aplica al instante. La actualización automática de Geodata integrada en Xray no se ve afectada; tiene su propio outbound de descarga. Déjelo vacío para conexión directa.»

Cómo funciona. Al seleccionar un outbound, el panel agrega por sí solo a la configuración activa un inbound loopback de servicio (puente SOCKS con la etiqueta `panel-egress`) y una regla de

enrutamiento que redirige el propio tráfico HTTP del panel hacia el outbound seleccionado. Si se selecciona un balanceador, se inserta `balancerTag` en la regla y el tráfico del panel se distribuye entre sus miembros. El puente y la regla se aplican **al instante**, sin reiniciar completamente el panel. Deje el campo vacío para conexión directa. La actualización automática de datos geo integrada en Xray **no se ve afectada** por este ajuste — tiene su propio outbound dentro del enrutamiento de Xray.

- **Formato:** `socks5://` (o `socks5h://`) o `http(s)://`, con autenticación si es necesario, en la forma `socks5://user:pass@host:port`. Los esquemas admitidos son estrictamente: `socks5`, `socks5h`, `http`, `https` — otros esquemas se consideran no válidos y el panel revierte a conexión directa. Un ejemplo típico es el inbound SOCKS local del propio Xray.
- Mensaje literal: «Enruta las solicitudes salientes propias del panel (actualizaciones geo, verificaciones de versiones de Xray/panel, Telegram) a través de este proxy para eludir el filtrado del servidor de GitHub/Telegram. Acepta `socks5://` o `http(s)://`, por ejemplo el inbound SOCKS local de Xray. Déjelo vacío para conexión directa.»
- Un proxy no válido no provoca un error al guardar — el panel simplemente utiliza la conexión directa y escribe una advertencia en el log.

Ejemplo de valores del campo. Si en el servidor ya hay un inbound SOCKS local de Xray en el puerto `10808`, dirija a través de él las solicitudes propias del panel:

```
socks5://127.0.0.1:10808
```

Para un proxy HTTP externo con autenticación:

```
http://user:pass@proxy.example.com:8080
```

Tras guardar y reiniciar, el panel obtendrá las actualizaciones de bases geo, verificará versiones y se comunicará con Telegram a través del proxy especificado.

CIDR de proxies de confianza (*Trusted proxy CIDRs*)

- **Clave:** `trustedProxyCIDRs`
- **Valor por defecto:** `127.0.0.1/32, ::1/128` (solo el host local).
- **Formato:** lista de direcciones IP o subredes CIDR separadas por comas (por ejemplo `10.0.0.0/8, 192.168.1.5`). Cada elemento se verifica como IP o CIDR — un valor incorrecto se rechaza al guardar.
- **Función:** enumera las fuentes a las que se permite establecer los encabezados `X-Forwarded-Host`, `X-Forwarded-Proto` y el encabezado de IP real del cliente. Mensaje literal: «IP/CIDR separadas por comas a las que se permite establecer los encabezados forwarded host, proto e IP del cliente.» Es necesario configurarlo si el panel funciona detrás de un proxy inverso (nginx, Caddy, etc.) para determinar correctamente las IP de los clientes y el esquema.

Ejemplo: panel detrás de un proxy inverso. Si nginx está en el mismo host y reenvía solicitudes al panel, deje la confianza solo al host local (valor por defecto):

```
127.0.0.1/32, ::1/128
```

Si el proxy se encuentra en un servidor separado dentro de la red interna `10.0.0.0/8`, agregue su subred; de lo contrario el panel ignorará los encabezados que este envíe y verá la IP del proxy en lugar de la del cliente real:

```
127.0.0.1/32, ::1/128, 10.0.0.0/8
```

Ejemplo del bloque nginx correspondiente que transmite la IP real y el esquema:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Bot de Telegram (pestaña «Bot de Telegram» / *Telegram Bot*)

Habilitar bot de Telegram (*Enable Telegram Bot*)

- **Clave:** `tgBotEnable`
- **Tipo/por defecto:** booleano, `false`.
- **Función:** habilita el funcionamiento del bot de Telegram. Mensaje literal: «Acceso a las funciones del panel a través del bot de Telegram».

Token de Telegram (*Telegram Token*)

- **Clave:** `tgBotToken`
- **Por defecto:** `""`.
- **Función:** token del bot. Mensaje literal: «Debe obtener el token del gestor de bots de Telegram @botfather».
- **Particularidad de seguridad:** el token es un valor secreto. En la respuesta del panel a la lectura de configuración no se devuelve (el campo se vacía, solo se devuelve el indicador «configurado/no configurado»). Si al guardar se deja el campo vacío, el token guardado anteriormente **se conserva** (no se sobrescribe).

Idioma del bot de Telegram (*Telegram Bot Language*)

- **Clave:** `tgLang`
- **Por defecto:** `en-US`.
- **Función:** idioma de los mensajes del bot (independiente del idioma de la interfaz web). La lista de idiomas disponibles coincide con los idiomas del panel.

ID de usuario del administrador del bot (*Admin Chat ID*)

- **Clave:** `tgBotChatId`
- **Por defecto:** `""`.
- **Formato:** uno o varios User ID numéricos de Telegram **separados por comas**.

- **Función:** destinatarios de notificaciones y administradores a quienes se permite gestionar el panel a través del bot. Mensaje literal: «Uno o varios User ID del administrador/es del bot de Telegram. Para obtener el User ID use @userinfobot o el comando '/id' en el bot.»

Frecuencia de notificaciones (*Notification Time*)

- **Clave:** tgRunTime
- **Por defecto:** @daily (una vez al día).
- **Formato:** cadena en formato **Crontab** (se admiten tanto expresiones cron estándar como abreviaciones del tipo @daily, @hourly, @every 1h). Mensaje literal: «Especifique el intervalo de notificaciones en formato Crontab». Controla los informes periódicos del bot.

Ejemplos de valores del campo.

Valor	Cuándo envía el bot el informe
@daily	una vez al día a medianoche (por defecto)
@hourly	cada hora
@every 6h	cada 6 horas
0 9 * * *	todos los días a las 09:00
30 8 * * 1	cada lunes a las 08:30

La hora se calcula en la zona horaria del ajuste «Zona horaria» (punto 13.6).

Proxy SOCKS (*SOCKS Proxy*)

- **Clave:** tgBotProxy
- **Por defecto:** "" .
- **Función:** proxy SOCKS5 específico para la conexión del bot a Telegram. Mensaje literal: «Si necesita un proxy Socks5 para conectarse a Telegram, configure sus parámetros según el manual.» Se aplica únicamente al tráfico del bot (distinto del «Proxy de red del panel» general del punto 13.4).

Servidor API de Telegram (*Telegram API Server*)

- **Clave:** tgBotAPIServer
- **Por defecto:** "" (usar el servidor estándar api.telegram.org).
- **Formato:** URL http(s)://...; al guardar se verifica la validez de la URL — una dirección no válida se rechaza. Mensaje literal: «Servidor API de Telegram utilizado. Déjelo vacío para usar el servidor por defecto.» Es necesario para un servidor de la API del bot de Telegram desplegado de forma independiente.

Notificaciones del bot (grupo «Notificaciones» / *Notifications*)

Campo	Clave	Por defecto	Descripción
Copia de seguridad de la base de datos (<i>Database Backup</i>)	<code>tgBotBackup</code>	<code>false</code>	Enviar a Telegram el archivo de copia de seguridad de la BD junto con el informe. Mensaje literal: «Enviar notificación con el archivo de copia de seguridad de la base de datos».
Notificación de inicio de sesión (<i>Login Notification</i>)	<code>tgBotLoginNotify</code>	<code>true</code>	Notificar ante intentos de inicio de sesión en el panel. Mensaje literal: «Muestra el nombre de usuario, la dirección IP y la hora cuando alguien intenta acceder a su panel.»
Antelación para notificación de vencimiento (<i>Expiration Date Notification</i>)	<code>expireDiff</code>	<code>0</code>	Cuántos días antes del vencimiento del cliente enviar la notificación. <code>0</code> – desactivado. Admite <code>>= 0</code> . Mensaje literal: «Recibir notificación de vencimiento de sesión antes de alcanzar el valor umbral (valor: día)».
Umbral de tráfico para notificación (<i>Traffic Cap Notification</i>)	<code>trafficDiff</code>	<code>0</code>	Umbral de tráfico restante para la notificación. Mensaje literal: «Recibir notificación de agotamiento de tráfico antes de alcanzar el umbral (valor: GB)». Admite <code>0–100</code> .
Umbral de carga de CPU (<i>CPU Load Notification</i>)	<code>tgCpu</code>	<code>80</code>	Notificar a los administradores si la carga de CPU supera el umbral (en %). Admite <code>0–100</code> . Mensaje literal: «Notificar a los administradores en Telegram si la carga de CPU supera este umbral (valor: %)».

13.6. Fecha y hora (pestaña «Fecha y hora» / *Date and Time*)

Zona horaria (*Time Zone*)

- **Clave:** `timeLocation`
- **Valor por defecto:** `Local` (zona horaria del sistema del servidor).
- **Formato:** nombre de zona de la base de datos IANA tz (por ejemplo, `Europe/Moscow`, `UTC`, `Asia/Tehran`).
- **Función:** zona horaria en la que el panel ejecuta las tareas programadas (informes del bot, restablecimiento/verificaciones de tráfico, vencimientos de plazos). Mensaje literal: «Las tareas programadas se ejecutan de acuerdo con la hora en esta zona horaria».
- **Validación:** al guardar se verifica la zona – una zona inexistente se rechaza. Si posteriormente la BD contiene un valor incorrecto, el panel en tiempo de ejecución revierte a `Local` y, si este tampoco está disponible, a `UTC`.

13.7. Tráfico externo y comportamiento de Xray (pestaña «Tráfico externo» / *External Traffic*)

Campo	Clave	Por defecto	Descripción
Información de tráfico externo (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	Notificar a la API externa en cada actualización de tráfico. Mensaje literal: «Notificar a la API externa en cada actualización de tráfico.»
URI de información de tráfico externo (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	URL a la que el panel envía las actualizaciones de tráfico. Se verifica la validez de la URL al guardar. Mensaje literal: «Las actualizaciones de tráfico se envían a este URI».
Reiniciar Xray tras desactivación automática (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	Reiniciar Xray cuando un cliente se desactiva automáticamente por vencimiento del plazo o superación del límite de tráfico. Mensaje literal: «Cuando un cliente se desactiva automáticamente por vencimiento del plazo o límite de tráfico, reiniciar Xray.» El interruptor se encuentra en la pestaña «Panel» (General) – véase el punto 13.2; aquí se incluye a modo de referencia completa.

13.8. Otros: plantilla de configuración de Xray y URL de verificación

Plantilla de configuración de Xray (*xrayTemplateConfig*)

- **Clave:** `xrayTemplateConfig`
- **Por defecto:** plantilla JSON integrada (embedded) incluida en la compilación.
- **Función:** plantilla JSON base de configuración de Xray-core sobre la cual el panel construye inbounds/outbounds. Este valor **no se devuelve** en la salida habitual de todos los ajustes y se edita en la página de configuración de Xray por separado, no en la lista general de campos de configuración del panel. La plantilla estándar por defecto está disponible mediante `GET /panel/setting/getDefaultJsonConfig`.

URL de verificación de outbounds (*xrayOutboundTestUrl*)

- **Clave:** `xrayOutboundTestUrl`
- **Por defecto:** `https://www.google.com/generate_204`
- **Función:** URL utilizada al verificar la operatividad de las conexiones salientes (outbound). Al establecerse pasa por saneamiento como URL HTTP(S).

13.9. Cuenta de administrador y tokens de API

Estos parámetros se encuentran en la pestaña adyacente («Cuenta» / *Authentication*) y se tratan en detalle en la sección de seguridad; aquí se incluye un resumen breve de las claves.

- **Cambio de credenciales** (campos «Nombre de usuario actual», «Contraseña actual», «Nuevo nombre de usuario», «Nueva contraseña») se guarda mediante una solicitud separada `POST /panel/setting/updateUser`. Se requieren el nombre de usuario y contraseña actuales correctos; el nuevo nombre de usuario y contraseña no deben estar vacíos. Mensajes: «Ha cambiado correctamente las credenciales del administrador.» / «Nombre de usuario o contraseña incorrectos».
- **Autenticación de dos factores (2FA)** – claves `twoFactorEnable` (por defecto `false`) y el secreto `twoFactorToken`. El token es secreto: con 2FA habilitado, dejar el campo vacío al guardar no sobrescribe el token existente. Al **habilitar** 2FA por primera vez, el panel invalida las sesiones actuales (se incrementa la «época de inicio de sesión»).
- **Los tokens de API** se gestionan mediante endpoints separados (`/panel/setting/apiTokens...`): lista, creación (`apiTokens/create`), eliminación, habilitación/deshabilitación. El propio token se muestra **solo una vez al crearlo** y no se almacena en formato legible: «Copie este token ahora. Por razones de seguridad no se almacena en formato legible y no volverá a mostrarse.»

Los detalles sobre 2FA, contraseñas, sincronización LDAP y formatos de suscripción (JSON/Clash, fragmentation, noises, mux) se tratan en las secciones correspondientes del manual.

13.10. Cambios de API en 3.3.0 (importante para integraciones)

En la versión 3.3.0 cambió la estructura de las rutas de la API del servidor. Si tiene integraciones externas (scripts, bots, paneles centrales, tareas CI) que acceden al panel por HTTP, **es necesario actualizarlas**; de lo contrario dejarán de funcionar.

 **BREAKING CHANGE:** los endpoints `/panel/setting/*` y `/panel/xray/*` se han trasladado bajo `/panel/api`

Antes, la gestión de la configuración del panel y la configuración de Xray vivía por separado, bajo las rutas `/panel/setting/*` y `/panel/xray/*`. Ahora ambos conjuntos están registrados dentro del grupo de API común `/panel/api`. Las rutas antiguas **se han eliminado por completo** – una solicitud a ellas devolverá 404.

Por qué se hizo: todo el grupo `/panel/api` pasa por una verificación de acceso unificada, es decir, estos endpoints ahora aceptan el mismo encabezado `Authorization: Bearer <token>` que el resto de la API. El token de API es un acceso de administrador completo, y de esta forma toda la superficie de la API se ha vuelto uniforme.

Lo que NO ha cambiado: las páginas de la interfaz web (rutas SPA) `/panel/settings` y `/panel/xray` permanecen en su lugar – se habla únicamente de los endpoints de la API del servidor.

Tabla de correspondencia de rutas (antigua → nueva)

El prefijo de todas las rutas a continuación es – simplemente se añadió `api/` después de `/panel/`.

Antes (≤ 3.2.x)	Ahora (3.3.0)	Método
/panel/setting/all	/panel/api/setting/all	POST
/panel/setting/defaultSettings	/panel/api/setting/defaultSettings	POST
/panel/setting/update	/panel/api/setting/update	POST
/panel/setting/updateUser	/panel/api/setting/updateUser	POST
/panel/setting/restartPanel	/panel/api/setting/restartPanel	POST
/panel/setting/getDefaultJsonConfig	/panel/api/setting/ getDefaultJsonConfig	GET
/panel/setting/apiTokens	/panel/api/setting/apiTokens	GET
/panel/setting/apiTokens/create	/panel/api/setting/apiTokens/create	POST
/panel/setting/apiTokens/delete/:id	/panel/api/setting/apiTokens/ delete/:id	POST
/panel/setting/apiTokens/ setEnabled/:id	/panel/api/setting/apiTokens/ setEnabled/:id	POST
/panel/xray/	/panel/api/xray/	POST
/panel/xray/update	/panel/api/xray/update	POST
/panel/xray/getDefaultJsonConfig	/panel/api/xray/getDefaultJsonConfig	GET
/panel/xray/getXrayResult	/panel/api/xray/getXrayResult	GET
/panel/xray/getOutboundsTraffic	/panel/api/xray/getOutboundsTraffic	GET
/panel/xray/resetOutboundsTraffic	/panel/api/xray/resetOutboundsTraffic	POST
/panel/xray/testOutbound	/panel/api/xray/testOutbound	POST
/panel/xray/warp/:action	/panel/api/xray/warp/:action	POST
/panel/xray/nord/:action	/panel/api/xray/nord/:action	POST
/panel/xray/outbound-subs (y / outbound-subs/*)	/panel/api/xray/outbound-subs (y / outbound-subs/*)	GET/POST/ DELETE

Los propios nombres de sub-rutas, los cuerpos de solicitudes y los formatos de respuesta no han cambiado – solo cambió el **prefijo**.

Cómo corregir las integraciones existentes

1. Busque en sus scripts/configuraciones todas las ocurrencias de `/panel/setting/` y `/panel/xray/`.
2. Reemplace el prefijo: añada `api/` inmediatamente después de `/panel/` (por ejemplo, `/panel/setting/all` → `/panel/api/setting/all`).
3. No es necesario modificar los cuerpos de solicitudes, los parámetros ni el formato de las respuestas – solo cambia la URL.

4. Como los ajustes y la configuración de Xray están ahora bajo `/panel/api`, se puede (y debe) acceder a ellos con el mismo token de API `Authorization: Bearer <token>` que a `/panel/api/inbounds/*` y demás endpoints. No olvide el middleware CSRF, que está habilitado para todo el grupo `/panel/api`.

Ejemplo: lectura de todos los ajustes mediante la API. Antes ($\leq 3.2.x$):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Ahora (3.3.0) — se añadió `api/` después de `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Del mismo modo para el reinicio del panel: `POST /panel/api/setting/restartPanel`. La ruta antigua `/panel/setting/restartPanel` ahora devolverá 404.

API tipada: esquemas y documentación (Swagger / OpenAPI)

En 3.3.0 la especificación OpenAPI pasó a ser totalmente tipada. Antes, las respuestas tipadas se describían con un objeto vacío `{}`; ahora los componentes y esquemas (`components.schemas`) se generan directamente a partir de los modelos de datos. Gracias a esto:

- La interfaz de Swagger UI muestra los modelos de datos reales en lugar de marcadores de posición genéricos.
- Los generadores externos (`openapi-generator` y similares) pueden construir clientes listos en el lenguaje deseado a partir de la especificación.
- Cada respuesta tipada lleva un `$ref` a un modelo concreto y ejemplos de respuestas adjuntos.

Dónde consultar la documentación de la API:

- **Página de Swagger integrada.** En el menú del panel — el elemento «**Documentación de API**» (ruta SPA `/panel/api-docs`). Aquí se enumeran de forma interactiva todos los endpoints con descripciones, cuerpos de solicitudes y ejemplos de respuestas.
- **La especificación OpenAPI 3.0 en bruto** se sirve en la dirección `/panel/api/openapi.json`. Esta URL puede pasarse directamente a Postman, Insomnia o `openapi-generator`. La especificación está integrada en el binario durante la compilación; cuando el panel se ejecuta bajo un `webBasePath` no estándar, el campo `servers` de la especificación se reescribe automáticamente según la ruta base actual, para que el botón «Try it out» y los generadores externos apunten al prefijo correcto.

14. Bot de Telegram

El panel 3X-UI incluye un bot de Telegram integrado que permite recibir notificaciones sobre el estado del servidor y los clientes, así como gestionar clientes individuales directamente desde el mensajero. El bot funciona mediante long polling (consulta continua a Telegram), por lo que no necesita un dominio externo ni un puerto abierto — basta con acceso saliente a los servidores de Telegram.

El bot distingue dos tipos de interlocutores:

- **Administrador** — usuario cuyo Telegram User ID está especificado en la configuración del bot (campo «User ID администратора бота»). Tiene acceso a todas las funciones: estadísticas del servidor, copia de seguridad, gestión de clientes, reinicio de Xray.
- **Cliente** — cualquier otro usuario cuyo Telegram User ID está vinculado a un cliente de inbound concreto (campo `tgId` del cliente). Solo puede ver la información de sus propias suscripciones.

Ejemplo: vincular un cliente a Telegram. Para que un usuario reciba estadísticas de su suscripción, su Telegram User ID numérico se introduce en el campo `tgId` del cliente. En la configuración JSON del cliente tiene este aspecto:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

Tras esto, el usuario con User ID `123456789` podrá consultar al bot `/usage ivan` y ver sus estadísticas. El mismo ID puede configurarlo el administrador mediante el botón « Установить пользователя Telegram» en la tarjeta del cliente — no es necesario editar el JSON manualmente.

14.1. Activación y configuración del bot

Todos los parámetros del bot se configuran en el panel en la sección **Настройки** → **Telegram-бот**. Tras modificar la configuración es necesario guardarla y reiniciar el panel — el bot se inicializa al arrancar el servidor web.

Campo (UI)	Clave de configuración	Valor por defecto	Descripción
Включить Telegram бота	<code>tgBotEnable</code>	<code>false</code>	Interruptor principal. Sugerencia: «Доступ к функциям панели через Telegram-бота». Mientras esté desactivado, el bot no se inicia y las tareas de notificación no se planifican.
Telegram-токен	<code>tgBotToken</code>	(vacío)	Token del bot. Sugerencia: «Необходимо получить токен у менеджера ботов Telegram @botfather». Sin un token válido el bot no arranca.

Campo (UI)	Clave de configuración	Valor por defecto	Descripción
SOCKS-прокси	tgBotProxy	(vacío)	Proxym para conectarse a Telegram. Sugerencia: «Если для подключения к Telegram вам нужен прокси Socks5, настройте его параметры согласно руководству».
Telegram API Server	tgBotAPIServer	(vacío)	Servidor de API de Telegram alternativo. Sugerencia: «Используемый API-сервер Telegram. Оставьте пустым, чтобы использовать сервер по умолчанию».
User ID администратора бота	tgBotChatId	(vacío)	Uno o varios Telegram User ID de administradores separados por coma. Sugerencia: «Для получения User ID используйте @userinfobot или команду /id в боте».
Частота уведомлений для администраторов от бота	tgRunTime	@daily	Programación del informe periódico en formato crontab. Sugerencia: «Укажите интервал уведомлений в формате Crontab».
Резервное копирование базы данных	tgBotBackup	false	Sugerencia: «Отправлять уведомление с файлом резервной копии базы данных». Adjunta la copia de seguridad al informe periódico.
Уведомление о входе	tgBotLoginNotify	true	Sugerencia: «Отображает имя пользователя, IP-адрес и время, когда кто-то пытается войти в вашу панель».
Порог нагрузки на ЦП для уведомления	tgCpu	80	Umbral de carga de CPU en porcentaje (validación 0–100). Sugerencia: «Уведомление администраторов в Telegram, если нагрузка на ЦП превышает этот порог (значение: %)». Con valor 0 la comprobación de CPU queda desactivada.
Язык Telegram-бота	—	—	Idioma en el que el bot redacta todos sus mensajes.

Obtener el token mediante @BotFather

1. Abre en Telegram el chat con **@BotFather**.
2. Envía el comando `/newbot` y sigue las instrucciones (nombre del bot y `username` único que termine en `bot`).
3. BotFather proporcionará un token del tipo `123456789:AA...` . Cópialo en el campo **Telegram-токен**.

Obtener el User ID del administrador

El User ID es el identificador numérico de la cuenta (no el username). Puedes averiguarlo de dos maneras:

- Escribiendo al bot **@userinfobot**.
- Iniciando el bot ya configurado y enviándole el comando `/id` — responderá con tu ID.

Introduce el número obtenido en el campo **User ID администратора бота**. Para designar varios administradores, enumera sus ID separados por coma (por ejemplo, `11111111,22222222`). Cada ID se valida como número entero; un valor incorrecto producirá un error de inicio del bot.

Ejemplo: valor del campo «User ID администратора бота». Un solo administrador — simplemente un número:

123456789

Dos administradores separados por coma (los espacios no son necesarios):

123456789,987654321

Cada valor debe ser un número entero. Entradas como `@username` o `123 456` (con espacio dentro del número) no son válidas – el bot no arrancará.

Proxy

Se admiten los esquemas `socks5://`, `http://` y `https://`. Si el campo de proxy se deja vacío, el bot intenta utilizar el proxy general del panel (si está configurado y su esquema es compatible). Una URL con un esquema no compatible o con sintaxis incorrecta se ignora – el bot se conecta directamente. El proxy es útil cuando el acceso directo a la API de Telegram desde el servidor está bloqueado.

Notificaciones por correo electrónico (SMTP)

Además de Telegram, los mismos eventos pueden recibirse por correo. El canal se configura en **Настройки** → **Email** en la pestaña **SMTP Settings**:

Campo (UI)	Clave de configuración	Valor por defecto	Descripción
Enable Email Notifications	<code>smtpEnable</code>	<code>false</code>	Interruptor principal de las notificaciones por correo vía SMTP.
SMTP Host	<code>smtpHost</code>	(vacío)	Host del servidor SMTP (por ejemplo, <code>smtp.gmail.com</code>).
SMTP Port	<code>smtpPort</code>	<code>587</code>	Puerto del servidor SMTP.
SMTP Username	<code>smtpUsername</code>	(vacío)	Nombre de usuario para la autenticación SMTP. Se usa también como dirección del remitente (From).
SMTP Password	<code>smtpPassword</code>	(vacío)	Contraseña para la autenticación SMTP. Se almacena de forma oculta; si ya está establecida, el campo muestra que está «configurada» y puede dejarse vacío para conservar la actual.
Recipients	<code>smtpTo</code>	(vacío)	Lista de destinatarios separados por coma (por ejemplo, <code>admin@example.com, ops@example.com</code>).
Encryption	<code>smtpEncryptionType</code>	<code>starttls</code>	Tipo de cifrado de la conexión: <code>none</code> (sin cifrado), <code>starttls</code> (STARTTLS) o <code>tls</code> (TLS implícito).

El botón **Send Test Email** envía un correo de prueba y muestra el resultado por fases: **Connection** (conexión), **Authentication** (autenticación) y **Send** (envío). Si algo falla, el diagnóstico indica en qué fase se produjo el error (por ejemplo, «Authentication failed – check username and password» o «Server requires STARTTLS – change encryption type»), lo que facilita ajustar los parámetros.

En la segunda pestaña (**Notifications**) se seleccionan los eventos sobre los que se enviarán correos — con los mismos grupos de tarjetas que para Telegram (véase «Шина событий и выбор уведомлений» en la sección 14.5).

Servidor de API de Telegram

Por defecto el bot utiliza la API oficial de Telegram. En el campo **Telegram API Server** se puede especificar la dirección de un servidor Bot API propio (`telegram-bot-api`). La URL se valida por seguridad; una dirección bloqueada o incorrecta se descarta y se usa el servidor predeterminado.

14.2. Menú principal y botones

El menú se invoca con el comando `/start` . Los botones son un teclado inline adjunto al mensaje; el conjunto de botones depende de si eres administrador o cliente.

Menú del administrador

Botón	Acción
 Отсортированный отчёт об использовании трафика	Enumera todos los clientes ordenados por tráfico con el consumo de cada uno; los emails «extra» sin datos se marcan con «  Нет результатов».
 Состояние сервера	Resumen del servidor (véase la sección 14.5). El botón «  Обновить» refresca los datos.
Сбросить весь трафик	Reinicia los contadores de tráfico de todos los clientes. Solicita confirmación («Вы уверены?  »), luego muestra «  Успешно» o «  Неудача» para cada cliente y finalmente «  Сброс трафика завершён для всех клиентов».
 Бэкап БД	Envía el archivo de base de datos y <code>config.json</code> (véase la sección 14.6).
 Лог банов	Envía los archivos de registro de IPs baneadas por exceder el límite de IP.
 Входящие подключения	Resumen de todos los inbound: Remark, puerto, tráfico, número de clientes, fecha de expiración.
 Скоро конец	Lista de inbound y clientes que pronto agotarán su tráfico o plazo (véase la sección 14.5).
 Команды	Muestra la ayuda de comandos del administrador.
 Онлайн	Número y lista de clientes conectados; tocar un email abre la tarjeta del cliente. Botón «  Обновить».
 Все клиенты	Abre la selección de inbound y luego la lista de sus clientes para ver/gestionar.
 Новый клиент	Inicia el asistente para añadir un cliente (selección de inbound → borrador → confirmación).
Настройки подписки / индивидуальные ссылки / QR-код	Selección de inbound y cliente para obtener el enlace de suscripción, enlaces individuales o códigos QR.

Menú del cliente

El cliente dispone de un conjunto limitado de botones:

Botón	Acción
Статистика клиента	Muestra los datos de todas las suscripciones vinculadas al Telegram User ID del cliente.
Команды	Muestra la ayuda de comandos del cliente.
Настройки подписки	Selección del propio cliente → enlace de suscripción.
Индивидуальные ссылки	Selección del propio cliente → enlaces individuales.
QR-код	Selección del propio cliente → códigos QR.

Si el usuario no tiene ningún cliente con su Telegram User ID, el bot responde: « Ваша конфигурация не найдена!  Пожалуйста, попросите администратора использовать ваш Telegram User ID в конфигурации. Ваш User ID: ...». Este ID debe facilitarse al administrador para que lo introduzca en el campo del cliente.

14.3. Comandos del bot

El bot tiene cuatro comandos registrados visibles en el menú «/» de Telegram:

Comando	Descripción (del menú)	Acceso	Qué hace
/start	Показать главное меню	todos	Saludo; al administrador le muestra adicionalmente «  Добро пожаловать в бота управления !» y el menú principal.
/help	Справка по боту	todos	Muestra el saludo general y la invitación a elegir una opción del menú.
/status	Проверить статус бота	todos	Responde «  Бот функционирует нормально».
/id	Показать ваш Telegram ID	todos	Devuelve «  Ваш User ID: ... ». Útil para conocer el propio User ID.

Además de los registrados, se procesan otros tres comandos con argumento (no aparecen en el menú «/» pero funcionan):

- **/usage [Email]** — búsqueda de un cliente por email.
 - Para el **administrador** muestra la tarjeta completa del cliente (con botones de gestión).
 - Para el **cliente** muestra únicamente su propia suscripción con el email indicado (según la vinculación del Telegram User ID). Sin argumento el bot solicita el email: « Пожалуйста, укажите email для поиска».
- **/inbound [nombre de conexión]** — solo para el administrador. Busca un inbound por Remark y muestra sus parámetros con las estadísticas de todos los clientes. Sin argumento (o para un cliente) — « Неизвестная команда».
- **/restart** — solo para el administrador. Reinicia Xray Core. Posibles respuestas: « Ядро Xray успешно перезапущено», « Xray Core не запущен» (si el núcleo no está en ejecución), « Ошибка при перезапуске Xray-core. <Ошибка>». Cualquier argumento tras **/restart** produce un mensaje de comando desconocido con la sugerencia **/restart**.

En chats de grupo, un comando del tipo **/comando@botusername** solo se procesa si el username coincide con el nombre del bot actual.

Ayuda del administrador (botón «Команды»):

 Для перезапуска Xray Core: `/restart`
 Для поиска клиента по email: `/usage [Email]`
 Для поиска входящих подключений (со статистикой клиентов): `/inbound [имя подключения]`
 Ваш Telegram User ID: `/id`

Ayuda del cliente:

 \$
 Для просмотра информации о вашей подписке: `/usage [Email]`
 Ваш Telegram User ID: `/id`

14.4. Gestión de clientes (solo administrador)

Al abrir la tarjeta de un cliente (mediante «Все клиенты», «Онлайн», «Скоро конец» o `/usage`), el administrador ve la información del cliente (email, inbound vinculados, estado «Activo», estado de conexión, fecha de expiración, consumo de tráfico) y los botones inline de gestión:

Botón	Función
 Обновить	Recargar la tarjeta del cliente.
 Сбросить трафик	Reiniciar el contador de tráfico del cliente. Requiere confirmación «  Подтвердить сброс трафика? ».
 Лимит трафика	Establecer el límite de tráfico. Valores predefinidos: ☹ Sin límite (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB o «  Своё » – introducción de un número con el teclado numérico integrado (botones 0–9, «  » – reiniciar a 0, «  » – borrar el último dígito, «  Подтвердить: N »). El valor se expresa en gigabytes.
 Изменить дату окончания	Opciones predefinidas: ☹ Sin límite, «  Своё », añadir 7/10/14/20 días, 1/3/6/12 meses. Un número positivo prorroga el plazo (suma días a la fecha de expiración actual o a «ahora» si ya ha vencido); 0 elimina la restricción de plazo.
 Лог IP	Muestra las IPs registradas del cliente (con marcas de tiempo si las hay). Desde el registro están disponibles «  Обновить » y «  Очистить IP » (con confirmación «  Подтвердить очистку IP? »).
 Лимит IP	Límite de IPs simultáneas. Opciones: ☹ Sin límite (0), 1–10 o «  Своё » (teclado numérico).
 Установить пользователя Telegram	Muestra el Telegram User ID actualmente vinculado al cliente; permite eliminar la vinculación («  Удалить пользователя Telegram » con confirmación). La vinculación de un nuevo usuario se realiza mediante el selector de contactos de Telegram del sistema.
 Вкл./Выкл.	Activa o desactiva el cliente. Requiere confirmación «  Подтвердить вкл/выкл пользователя? ».

Todas las operaciones que modifican la configuración (límite de tráfico/IP, fecha de expiración, vinculación/desvinculación del usuario de Telegram, activar/desactivar) marcan Xray para reinicio si es necesario, de modo que los cambios surtan efecto. Tras una operación exitosa, el bot muestra una confirmación del tipo «  :... » y vuelve a mostrar la tarjeta.

Cualquier entrada numérica en los asistentes está limitada a valores < 999999.

14.5. Notificaciones e informes

Las notificaciones se envían a todos los administradores (todos los User ID de `tgBotChatId`).

Bus de eventos y selección de notificaciones

Las notificaciones están construidas sobre un bus de eventos unificado con dos canales de entrega: **Telegram** y **correo electrónico (SMTP)**. Para cada canal se selecciona de forma independiente sobre qué eventos notificar. En **Настройки** → **Telegram** se hace en la pestaña **Notifications**; en **Настройки** → **Email**, en la pestaña homónima.

Los eventos se agrupan en tarjetas; cada grupo tiene un interruptor principal con un contador de eventos activados (n/total) y un estado intermedio cuando solo una parte está seleccionada. Los grupos disponibles son:

- **Outbound** – «Down» (`outbound.down`) y «Up» (`outbound.up`): caída y recuperación de un outbound.
- **Xray Core** – «Crash» (`xray.crash`): terminación anormal del núcleo Xray.
- **Nodes** – «Down» (`node.down`) y «Up» (`node.up`): un nodo se vuelve inaccesible o se recupera.
- **System** – «CPU high (%)» (`cpu.high`) y «Memory high (%)» (`memory.high`): alta carga del procesador y de la memoria RAM. Junto a ambos eventos hay un campo inline para el umbral en porcentaje.
- **Security** – «Login attempt» (`login.attempt`): intento de inicio de sesión en el panel.

El conjunto de eventos activados se almacena de forma independiente: para Telegram en `tgEnabledEvents`, para Email en `smtpEnabledEvents`. Por defecto en ambos canales están activados «Login attempt» y «CPU high» (valor `login.attempt,cpu.high`).

Notificación de inicio de sesión en el panel

Se gestiona con la casilla **Уведомление о входе** (`tgBotLoginNotify`, activada por defecto). Con cada intento de acceso al panel web, los administradores reciben un mensaje:

- En caso de éxito: « Успешный вход в панель.» + host, nombre de usuario, IP, hora.
- En caso de fallo: « Ошибка входа в панель.» + host, **motivo** (por ejemplo, «Ошибка 2FA» si el segundo factor es incorrecto), nombre de usuario, IP, hora.

Exceso de carga en CPU y memoria

Cada minuto el panel comprueba la carga del procesador y de la memoria RAM. Si el umbral `tgCpu` > 0 y la carga media de CPU durante ese minuto lo supera, los administradores reciben: « Загрузка процессора составляет N%, что превышает пороговое значение M%». Análogamente se comprueba la carga de RAM frente al umbral `tgMemory` (80% por defecto) – evento «Memory high (%)».

Ambos umbrales se configuran mediante los campos inline junto a los eventos «CPU high (%)» y «Memory high (%)» en el grupo **System** de la pestaña Notifications (véase «Bus de eventos y selección de notificaciones» más arriba). Para el canal Email se usan las claves independientes `smtpCpu` y `smtpMemory`. Con un umbral de 0 la comprobación correspondiente queda desactivada.

Informe periódico (programado)

Se planifica según la expresión cron del campo **Частота уведомлений** (`tgRunTime` , por defecto `@daily`). Si el valor está vacío o es incorrecto, se usa `@daily` . El informe incluye:

Constructor de programación

El campo **Частота уведомлений для администраторов от бота** no se rellena escribiendo una cadena manualmente, sino mediante un constructor de programación. Primero se selecciona el modo en la lista desplegable:

- **@every – repetir con intervalo** – aparecen un campo numérico y un selector de unidad (**Секунды / Минуты / Часы**); el resultado se compone como una expresión del tipo `@every 6h` .
- **@hourly – cada hora, @daily – cada día a las 00:00, @weekly – cada semana, @monthly – cada mes** – presets listos que se guardan como el macro correspondiente (`@hourly` , `@daily` , `@weekly` , `@monthly`).
- **Произвольный (crontab)** – campo para una expresión crontab propia. El planificador del panel trabaja con segundos habilitados, por lo que la expresión arbitraria consta de **6 campos**: segundo, minuto, hora, día del mes, mes, día de la semana (por ejemplo, `0 30 8 * * *` – cada día a las 08:30:00). Al cambiar a este modo, el campo se rellena con el equivalente crontab de la selección actual, para tener un punto de partida.

Ejemplo: valores del campo «Частота уведомлений» (`tgRunTime`). Se admiten tanto abreviaciones predefinidas como el formato crontab completo:

Valor	Cuándo se activa
<code>@daily</code>	una vez al día a medianoche (valor por defecto)
<code>@hourly</code>	cada hora
<code>@every 6h</code>	cada 6 horas
<code>0 9 * * *</code>	cada día a las 09:00
<code>0 9 * * 1</code>	cada lunes a las 09:00
<code>0 */12 * * *</code>	cada 12 horas (a las 00:00 y las 12:00)

Orden de los campos en crontab: minuto, hora, día del mes, mes, día de la semana.

1. La línea «  Запланированные отчёты: <расписание>» y la fecha/hora actuales.
2. **Estado del servidor** (véase más abajo).
3. Bloque «Скоро конец» por inbound y clientes.
4. Notificaciones personales a los clientes con Telegram User ID vinculado – a cada cliente no administrador se le envía la lista de sus suscripciones que pronto agotarán tráfico o plazo (teniendo en cuenta las desactivadas).
5. Si está activado **Резервное копирование базы данных** (`tgBotBackup`) – copia de seguridad de la BD a los administradores.

Estado del servidor contiene: host, versión de 3X-UI y Xray, IPv4/IPv6, tiempo de actividad (en días), carga media (Load1/2/3), RAM (actual/total), número de clientes online, contadores de conexiones TCP/UDP, tráfico de red total (↑/↓) y estado de Xray.

«Скоро конец» muestra:

- por inbound: número de desactivados y número de «próximo a agotarse», seguido de la enumeración de dichos inbound (Remark, puerto, tráfico, fecha de expiración);
- por clientes: lo mismo, más tarjetas de clientes y botones con sus emails (tocar uno abre la tarjeta del cliente).

Los umbrales de «próximo a agotarse» se toman de la configuración general del panel: margen de tráfico (en GB) y margen de plazo (en días). Se considera que un inbound/cliente está «próximo a agotarse» cuando le queda menos del umbral de tráfico hasta el límite 0 menos del umbral de días hasta la fecha de expiración.

14.6. Copia de seguridad y registros

- **Copia de seguridad de la BD** (botón « Бэкап БД» o la casilla en el informe periódico): el bot envía la hora de la copia, el archivo de base de datos (`x-ui.db`, o `x-ui.dump` para PostgreSQL) y el archivo de configuración de Xray `config.json`.
- **Registro de baneos** (botón « Лог банов»): envía el archivo de registro actual y el anterior con las IPs baneadas por exceder el límite de IP. Los archivos vacíos no se envían.

14.7. Particularidades de funcionamiento

- **Los mensajes largos** se dividen en partes (umbral ~2000 caracteres); el teclado inline se adjunta a la última parte.
- **Concurrencia**: los comandos y las pulsaciones de botones se procesan de forma concurrente (pool de hasta 10 manejadores simultáneos).
- **Fiabilidad de envío**: ante errores de conexión, los mensajes se reenvían con retardo exponencial (1s/2s/4s, hasta 3 intentos).
- **Caché**: los datos de «Estado del servidor» se almacenan en caché para que las pulsaciones frecuentes de «Обновить» no sobrecarguen el sistema.
- **Reinicio del bot**: al guardar la configuración y reiniciar el panel, el ciclo de polling anterior se detiene correctamente y se inicia uno nuevo — en todo momento solo funciona una instancia de recepción de actualizaciones.

15. Bases geográficas (geoip / geosite y personalizadas)

Las bases geográficas son archivos binarios `.dat` que Xray-core utiliza para el enrutamiento y el filtrado de tráfico según la pertenencia a un país (rangos de IP) o según la categoría de dominios. El panel es capaz de descargar y actualizar tanto el conjunto estándar de archivos geográficos como fuentes personalizadas arbitrarias indicadas por URL. Todos los archivos se almacenan en el directorio `bin`, junto al binario de Xray (la ruta predeterminada es `bin`, y puede cambiarse mediante la variable de entorno `XUI_BIN_FOLDER`).

15.1. Qué son geoip.dat y geosite.dat

- **geoip.dat** — base de correspondencias «dirección IP → código de país/región». Se usa en reglas de enrutamiento con la forma `geoip:<código>`, por ejemplo `geoip:ru`, `geoip:cn`, así como para etiquetas especiales como `geoip:private` (redes privadas/locales). Conceptualmente, responde a la pregunta «¿en qué país se encuentra esta IP?».
- **geosite.dat** — base de correspondencias «dominio → categoría/lista». Se usa con la forma `geosite:<categoría>`, por ejemplo `geosite:category-ads-all` (dominios publicitarios), `geosite:google`, `geosite:ru`. Conceptualmente, son listas de dominios agrupadas por categoría.

Estos archivos son necesarios para construir reglas del tipo «todo el tráfico hacia IPs/dominios rusos va directo, el resto pasa por el outbound», y similares. Las propias reglas se configuran en la sección de enrutamiento de Xray; las bases geográficas únicamente les suministran los datos. Sin archivos geográficos actualizados, las reglas que hagan referencia a `geoip:` / `geosite:` no funcionarán o se basarán en listas desactualizadas.

Ejemplo: regla «dominios e IPs rusos de forma directa». Esta regla en la sección de enrutamiento dirige todo el tráfico hacia recursos rusos al outbound con la etiqueta `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Archivos geográficos estándar y su actualización

El panel contiene una lista de permisos (allowlist) fija de seis archivos estándar con fuentes de descarga predefinidas. La actualización se realiza a través de `POST /panel/api/server/updateGeofile/:fileName` (o sin nombre de archivo para actualizar todos a la vez).

Ejemplo: actualización de un archivo y de todos a la vez a través de la API. Actualizar solo `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Actualizar los seis archivos estándar con una sola solicitud (sin especificar nombre de archivo):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Respuesta exitosa:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Nombre de archivo	Fuente (repositorio de releases)
geoip.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoip.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoip_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoip.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoip_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoip.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Particularidades de la actualización de los archivos estándar:

- **Botón de actualización de un archivo.** Antes de la descarga se muestra una confirmación: «Do you really want to update the geofile? This will update the #filename# file.». Si la operación es exitosa, aparece la notificación «Geofile updated successfully».
- **Botón «Update all»** descarga los seis archivos. Confirmación: «This will update all geofiles.».
- **Descarga condicional.** Si el archivo ya existe localmente, se incluye en la solicitud el encabezado `If-Modified-Since` con la hora de modificación del archivo. Una respuesta `304 Not Modified` del servidor indica que el archivo no ha cambiado — no se vuelve a descargar, solo se actualiza la marca de tiempo del archivo.
- **Seguridad del nombre de archivo.** Solo se aceptan nombres que estén en la allowlist; el nombre se verifica para asegurarse de que no contiene `.`, separadores de ruta `/` ni `\`, rutas absolutas, y debe coincidir con el patrón `^[a-zA-Z0-9._-]+\..dat$`. Cualquier nombre fuera de la lista es rechazado con el error «Invalid geofile name».
- **Reinicio de Xray.** Tras la descarga de los archivos geográficos, Xray-core se reinicia para que relea las bases actualizadas. Si el reinicio falla, se añade la cadena de error correspondiente al mensaje de error.

Actualización de bases geográficas desde la línea de comandos (x-ui)

Las bases geográficas también pueden actualizarse sin el panel, a través del menú interactivo `x-ui` (opción de actualización de archivos geográficos) o mediante el comando no interactivo `x-ui update-all-geofiles`. Para cada archivo del conjunto (geoip/geosite, incluidos los conjuntos IR y RU) se muestra un estado individual: «actualizado», «ya está al día» o «error de descarga». En caso de descarga fallida, no se imprime ningún mensaje de éxito falso. El reinicio de Xray (y por tanto la interrupción de las conexiones activas) ocurre solo si al menos un archivo fue realmente actualizado; si ningún archivo cambió (todos devolvieron `304 Not Modified`), el panel y Xray no se reinician.

15.3. Actualización automática de geodatos mediante Xray (Geodata Auto-Update)

Las fuentes `.dat` adicionales con URL arbitrarias no se añaden a través del panel, sino mediante la sección nativa `geodata` de Xray-core. La sección correspondiente se encuentra en la ventana modal de actualizaciones de Xray (Dashboard → actualizaciones de Xray, `xrayUpdates`) — es la pestaña «Geodata Auto-Update». El panel aquí solo edita la clave `geodata` en la plantilla de configuración de Xray; la descarga, verificación y recarga en caliente de los archivos las gestiona el propio núcleo de Xray.

En la parte superior de la sección se muestra una sugerencia: «Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.».

Campos de la sección

- **Schedule (cron)** — cadena cron de 5 campos; el valor predeterminado es `0 4 * * *` (diariamente a las 04:00). Al guardar, se verifica que la cadena contenga exactamente 5 campos; de lo contrario, se muestra el error «Cron debe contener 5 campos, p. ej. `0 4 * * *`».
- **Download through outbound (optional)** — lista desplegable con las etiquetas de los outbound disponibles (más los outbound de suscripciones) a través del cual Xray descargará los archivos; los outbound con protocolo `blackhole` no aparecen en la lista. El campo puede dejarse vacío — en ese caso se usará una conexión directa. Esta elección es independiente del outbound para las propias solicitudes del panel (véase §11): la actualización automática de geodata tiene su propio outbound de descarga separado.
- **Lista de archivos** — cada fila define un par «URL + Nombre de archivo» (*File name*). La URL debe comenzar con `https://` (de lo contrario: «Se requiere una URL HTTPS para cada archivo.»). El nombre de archivo se indica de forma simple, sin rutas ni separadores — solo los caracteres `^[A-Za-z0-9._-]+$` (de lo contrario: «El nombre de archivo debe ser simple, por ejemplo `geosite_custom.dat` (sin rutas).»). Al introducir la URL, el panel intenta completar automáticamente el nombre de archivo a partir del último segmento de la ruta. El botón «Add file» añade una fila; el botón de papelera la elimina.

Si la lista está vacía, se muestra la sugerencia: «No files configured. Reference files in routing rules as `ext:geosite_custom.dat:category`.».

Guardado

El botón «Save & Restart Xray» muestra la confirmación «Save geodata settings? This updates the Xray config template and restarts Xray.». Tras guardar, la clave `geodata` se escribe en la plantilla de configuración (`POST /panel/api/xray/update`) y Xray se reinicia (`POST /panel/api/server/restartXrayService`). Si la lista de archivos está vacía, la clave `geodata` se elimina de la plantilla.

Aspectos importantes:

- **El archivo debe existir previamente en `bin`**. Xray solo actualiza los archivos `.dat` que ya están presentes en la carpeta `bin` en el momento del arranque. Por eso, un nuevo archivo personalizado primero debe colocarse manualmente en `bin` (o al menos crear allí una versión vacía/desactualizada con el nombre requerido), y solo entonces Xray empezará a mantenerlo actualizado según el calendario.
- **Recarga en caliente**. Tras la descarga programada, Xray relee las bases actualizadas sin reiniciar el proceso por completo.

- **Compatibilidad.** Los archivos geográficos descargados anteriormente (tanto los estándar como los personalizados) siguen funcionando en las reglas de enrutamiento con la sintaxis `ext:` sin cambios.

Si la lista está vacía, se muestra la sugerencia: «No custom geo sources yet — click Add to create one».

Columnas de la tabla y campos de la fuente

Campo (UI)	JSON	Valor predeterminado	Descripción
Tipo (<i>Type</i>)	<code>type</code>	— (obligatorio)	Tipo de recurso: solo <code>geosite</code> o <code>geoip</code> . Determina el nombre del archivo resultante.
Alias (<i>Alias</i>)	<code>alias</code>	— (obligatorio)	Identificador corto de la fuente. A partir de él y del tipo se construye el nombre del archivo.
URL (<i>URL</i>)	<code>url</code>	— (obligatorio)	Enlace directo al archivo <code>.dat</code> (http/https).
Habilitado (<i>Enabled</i>)	—	—	Indicador de actividad de la fuente en la lista.
Actualizado (<i>Last updated</i>)	<code>lastUpdatedAt</code>	<code>0</code>	Hora de la última actualización exitosa (tiempo Unix; <code>0</code> — todavía no se ha actualizado).
Enrutamiento (ext:...) (<i>Routing (ext:...)</i>)	—	—	Cadena lista para usar en reglas de enrutamiento: <code>ext:<archivo.dat>:tag</code> .
Acciones (<i>Actions</i>)	—	—	Botones «Editar», «Eliminar», «Actualizar ahora».

Adicionalmente, en la base de datos se almacenan campos internos: `localPath` (ruta real al archivo en el directorio `bin`), `lastModified` (valor del encabezado `Last-Modified` del servidor, usado para la descarga condicional), `createdAt` y `updatedAt`.

Nomenclatura de archivos

El nombre del archivo resultante se genera automáticamente a partir del tipo y el alias:

- tipo `geoip` → `geoip_<alias>.dat`;
- tipo `geosite` → `geosite_<alias>.dat`.

Por ejemplo, una fuente con tipo `geosite` y alias `myads` creará el archivo `geosite_myads.dat`.

Ejemplo: añadir una fuente a través de la API. Añadir una lista propia de dominios publicitarios como recurso `geosite` con el alias `myads`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

El panel descargará el archivo en el directorio `bin` como `geosite_myads.dat`, guardará el registro y reiniciará Xray.

Botones y acciones

- **Añadir** (*Add*) – abre el formulario «Add custom geo». El botón de guardado es «Save». API: `POST /add`.
- **Editar** (*Edit*) – formulario «Edit custom geo». API: `POST /update/:id`. Al cambiar el tipo o el alias, el archivo antiguo se elimina y el nuevo se descarga de nuevo.
- **Eliminar** (*Delete*) – confirmación «Delete this custom geo source?». Elimina el registro de la base de datos y el propio archivo `.dat`. API: `POST /delete/:id`. Si la operación es exitosa: «Пользовательский гео-файл «» eliminado».
- **Actualizar ahora** (*Update now*) – vuelve a descargar la fuente concreta y actualiza la marca de tiempo. API: `POST /download/:id`. Si la operación es exitosa: «Geofile «» actualizado».
- **Actualizar todo** – actualiza todas las fuentes personalizadas a la vez. API: `POST /update-all`. Si todo es exitoso: «All custom geo sources updated». Si al menos una fuente no se actualizó, la operación se considera parcialmente fallida con el mensaje «One or more custom geo sources failed to update», y en la respuesta se enumeran las fuentes exitosas y las fallidas.

Tras cualquiera de las acciones (añadir, editar, eliminar, actualizar, actualizar todo cuando hay éxitos), Xray-core se reinicia.

Paso a paso: añadir una fuente

1. Pulse «Añadir».
2. En el campo «Tipo» seleccione `geosite` o `geoip`.
3. En el campo «Alias» introduzca el identificador (solo letras latinas minúsculas, dígitos, `-` y `_`; texto de marcador de posición: `a-z 0-9 _ -`).
4. En el campo «URL» indique el enlace directo al archivo `.dat` (debe comenzar con `http://` o `https://`).
5. Pulse «Guardar». El panel descargará inmediatamente el archivo en el directorio `bin`, guardará el registro y reiniciará Xray.

15.4. Validación y restricciones

Al crear y modificar una fuente se realizan verificaciones estrictas. Mensajes de error:

Condición	Mensaje (RU)	Mensaje (EN)
Tipo distinto de <code>geosite / geoip</code>	Тип должен быть geosite или geoip	<i>Type must be geosite or geoip</i>
Alias vacío	Укажите псевдоним	<i>Alias is required</i>
Caracteres no permitidos en el alias (no coincide con <code>^[a-z0-9_-]+\$</code>)	Псевдоним содержит недопустимые символы	<i>Alias must match allowed characters</i>
Alias reservado	Этот псевдоним зарезервирован	<i>This alias is reserved</i>
URL vacía	Укажите URL	<i>URL is required</i>

Condición	Mensaje (RU)	Mensaje (EN)
URL no analizable	Некорректный URL	<i>URL is invalid</i>
Esquema distinto de http/https	URL должен использовать http или https	<i>URL must use http or https</i>
Host vacío/incorrecto o bloqueado por la protección SSRF	Некорректный хост URL	<i>URL host is invalid</i>
Duplicado «tipo + alias»	Такой псевдоним уже используется для этого типа	<i>This alias is already used for this type</i>
Fuente no encontrada	Источник не найден	<i>Custom geo source not found</i>
Error de descarga	Ошибка загрузки	<i>Download failed</i>

Sugerencias en el formulario (validación en el cliente): «Alias may only contain lowercase letters, digits, - and _» y «URL must start with http:// or https://».

Restricciones técnicas adicionales:

- **Alias reservados.** No se pueden usar alias que entren en conflicto con los archivos estándar. Están reservados (la comparación es insensible a mayúsculas y minúsculas, y el guion se equipara al guion bajo): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. Por ejemplo, `geosite-ru` será rechazado como `geosite_ru`.
- **Protección SSRF.** El host de la URL se resuelve a IP, y si apunta a una dirección privada/interna, la descarga queda bloqueada (el usuario ve «URL host is invalid»). Esto impide usar el panel para acceder a servicios internos.
- **Protección contra path traversal.** La ruta final del archivo debe encontrarse dentro del directorio `bin` (con resolución de enlaces simbólicos); cualquier intento de salir de ese directorio es rechazado.
- **Tamaño mínimo del archivo.** Un archivo descargado se considera válido solo si tiene al menos 64 bytes; un archivo demasiado pequeño se rechaza con un error de descarga.
- **Proxy y descarga condicional.** Si en la configuración del panel se ha definido un proxy, la descarga se realiza a través de él; en caso contrario, se utiliza una conexión directa con transporte seguro frente a SSRF. Al igual que con los archivos estándar, se aplica `If-Modified-Since / 304 Not Modified` (un archivo sin cambios no se vuelve a descargar). El tiempo de espera de descarga es de 10 minutos; la comprobación de accesibilidad de la URL (HEAD y, si es necesario, GET parcial) es de 12 segundos.

15.5. Verificación automática al arrancar el panel

Al iniciarse, el panel recorre todas las fuentes personalizadas y verifica la existencia e integridad de cada archivo local (el archivo no existe, es un directorio o tiene menos de 64 bytes). Si el archivo falta o está dañado, se realiza una comprobación de la fuente y un intento de nueva descarga. Esto garantiza que, tras una reinstalación o pérdida del directorio `bin`, los archivos geográficos personalizados se restauren automáticamente.

15.6. Uso de las bases geográficas en las reglas de enrutamiento

En las reglas de enrutamiento de Xray, las bases geográficas se usan en campos como `domain / ip` mediante prefijos:

- **geoip**: para bases de IPs — `geoip:<código>`. Ejemplos: `geoip:ru`, `geoip:cn`, `geoip:private`. Se obtiene de `geoip.dat` (o de `geoip_RU.dat`, etc., si la regla apunta a un archivo específico).
- **geosite**: para bases de dominios — `geosite:<categoría>`. Ejemplos: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Se obtiene de `geosite.dat`.

Ejemplo: bloqueo de publicidad con geosite. Regla que envía todos los dominios publicitarios a un «agujero negro» (se presupone un outbound con la etiqueta `blocked` y el protocolo `blackhole`):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Para archivos **personalizados** se usa la sintaxis de archivo externo `ext:`. La sugerencia en la interfaz dice: «In routing rules use the value column as `ext:file.dat:tag` (replace tag)». Formato:

```
ext:<nombre_archivo.dat>:<etiqueta>
```

donde `<nombre_archivo.dat>` es `geoip_<alias>.dat` o `geosite_<alias>.dat`, y `<etiqueta>` es la lista/categoría concreta dentro del archivo. El panel, en la columna «Enrutamiento (ext:...)», sugiere una plantilla lista del tipo `ext:geosite_myads.dat:tag` — solo hay que reemplazar `tag` por la etiqueta deseada. El nombre de dicho archivo se define en la sección «Geodata Auto-Update» (véase §15.3) en el campo «Nombre de archivo» — por ejemplo `geosite_custom.dat`; en las reglas se hace referencia a él como `ext:geosite_custom.dat:category`.

Ejemplo: regla basada en un archivo personalizado. Si se ha añadido una fuente de tipo `geosite` con el alias `myads`, y dentro del archivo `.dat` la lista está marcada con la etiqueta `ads`, la regla de enrutamiento es la siguiente:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

Para una fuente de IPs (tipo `geoip`, alias `mycorp`, etiqueta `office`), el campo sería `"ip": ["ext:geoip_mycorp.dat:office"]`.

16. Operación: copias de seguridad, registros, actualización, CLI

Esta sección cubre el mantenimiento cotidiano del panel: creación y restauración de copias de seguridad de la base de datos, visualización de registros (logs) del panel y de Xray, reinicio y detención de servicios, actualización de Xray y del propio panel, tareas periódicas (cron) y desinstalación del panel. Algunas operaciones se realizan desde la interfaz web (pestañas en la página «Dashboard» y «Configuración del panel»), otras, desde el menú de consola `x-ui` en el servidor.

16.1. Copia de seguridad y restauración de la base de datos

Todos los datos del panel (inbound, clientes, grupos, nodos, configuraciones) se almacenan en una única base de datos. La gestión de copias de seguridad está disponible en la página «**Dashboard**», en la pestaña «**Backup**», cuyo bloque se denomina «**Backup y Restauración**».

El panel admite dos motores de base de datos, y el comportamiento de la copia de seguridad depende de cuál esté en uso:

- **SQLite** (opción predeterminada): los datos se almacenan en el archivo `x-ui.db`.
- **PostgreSQL**: si el panel está configurado con PostgreSQL, el bloque muestra el siguiente aviso:

«Este panel funciona con PostgreSQL. "Backup" descarga un archivo `pg_dump` (.dump), y "Restauración" lo carga de nuevo mediante `pg_restore`. En el servidor deben estar instaladas las herramientas cliente de PostgreSQL (`pg_dump` y `pg_restore`).»

Exportación (creación de copia)

El botón «**Exportar base de datos**» (en inglés `Back Up`) descarga el archivo de copia de seguridad en su dispositivo.

Motor de BD	Nombre del archivo	Qué ocurre en el servidor
SQLite	<code>x-ui.db</code>	Primero se realiza un checkpoint WAL para que el archivo contenga las últimas entradas; luego el archivo se lee completo y se envía para descarga
PostgreSQL	<code>x-ui.dump</code>	Se ejecuta <code>pg_dump</code> y el archivo se envía para descarga

Mensajes de ayuda en la interfaz:

- **SQLite**: «Haga clic para descargar el archivo .db que contiene una copia de seguridad de su base de datos actual en su dispositivo.»
- **PostgreSQL**: «Haga clic para descargar un volcado de PostgreSQL (.dump) de la base de datos actual en su dispositivo.»

Técnicamente, la exportación es una solicitud `GET /panel/api/server/getDb`. El nombre del archivo adjunto lo establece el servidor (`Content-Disposition`) según el motor en uso.

El nombre del archivo de copia de seguridad se forma a partir de la dirección del servidor, no como un valor fijo `x-ui.db` / `x-ui.dump`. Al descargarlo desde el navegador, se toma del dominio del panel visible en la barra de direcciones (host de la solicitud); en caso contrario, se utiliza el dominio web configurado o, si no existe, la IP pública del servidor (primero IPv4, luego IPv6), con `x-ui` como valor de respaldo. De este modo es fácil distinguir las copias de seguridad de distintos servidores. La extensión se mantiene como `.db` para SQLite y `.dump` para PostgreSQL; las copias enviadas por Telegram también se nombran según el dominio/IP.

Ejemplo: descarga de copia de seguridad mediante API. La misma exportación puede obtenerse desde la consola, por ejemplo, en un script de copia de seguridad automática. Se necesita una sesión autenticada (cookie de inicio de sesión):

```
# 1) Iniciamos sesión y guardamos la cookie de sesión
curl -s -c cookies.txt \
  -d 'username=admin&password=admin' \
  https://panel.example.com:2053/panel/login

# 2) Descargamos el archivo de base de datos (el nombre lo establece el servidor: x-
ui.db o x-ui.dump)
curl -s -b cookies.txt -OJ \
  https://panel.example.com:2053/panel/api/server/getDb
```

Si el panel está accesible bajo una ruta base (Web Base Path), hay que añadirla a la URL: `...:2053/<base_path>/panel/api/server/getDb`.

Importación (restauración)

El botón «**Importar base de datos**» (en inglés `Restore`) abre el selector de archivo y lo sube al servidor para su restauración (`POST /panel/api/server/importDB`, campo del formulario `db`).

Mensajes de ayuda en la interfaz:

- SQLite: «Haga clic para seleccionar y cargar un archivo .db desde su dispositivo y restaurar la base de datos desde la copia de seguridad.»
- PostgreSQL: «Haga clic para seleccionar y cargar un archivo .dump para restaurar la base de datos PostgreSQL. Esto reemplazará todos los datos actuales.»

Proceso de importación para SQLite (es importante entender que es atómico y con reversión):

1. El archivo cargado se verifica como formato válido; debe ser una base SQLite válida. En caso contrario, se devuelve el error «Invalid db file format».
2. El archivo se guarda como `x-ui.db.temp` temporal y se comprueba su integridad.
3. **Xray se detiene** antes de reemplazar la base de datos.
4. La base de datos actual se renombra a `x-ui.db.backup` (copia de respaldo).
5. El archivo temporal se mueve a la posición de la base de datos activa, se realiza la inicialización y las migraciones de esquema y, a continuación, la migración de inbound.
6. **Si algún paso falla**, se realiza una reversión: se restaura la base de datos anterior desde `x-ui.db.backup` y Xray se reinicia con los datos antiguos.
7. Si todo tiene éxito, el archivo de respaldo se elimina y **Xray se reinicia automáticamente** ya con los datos restaurados.

Mensajes de la interfaz según el resultado:

Resultado	Texto
Éxito	«Base de datos importada correctamente»
Error de importación	«Se ha producido un error al importar la base de datos»
Error de lectura del archivo	«Se ha producido un error al leer la base de datos»

La restauración reemplaza completamente los datos actuales. Dado que Xray se detiene brevemente durante el proceso, las conexiones existentes de los clientes se interrumpen durante la importación.

Archivo de migración entre motores (SQLite ⇌ PostgreSQL)

Además de la copia de seguridad habitual, existe la función «**Descargar archivo de migración**» (Download Migration, solicitud GET /panel/api/server/getMigration). Genera un archivo portable para cambiar a otro motor de base de datos:

Motor actual	Qué se descarga	Nombre del archivo	Propósito
SQLite	Volcado SQL portable (texto)	x-ui.dump	Inicializar PostgreSQL con sus datos
PostgreSQL	Base SQLite construida a partir de los datos de PostgreSQL	x-ui.db	Volver el panel a SQLite

Mensajes de ayuda:

- En SQLite: «Haga clic para descargar una exportación portable .dump (texto SQL) de su base de datos SQLite.»
- En PostgreSQL: «Haga clic para descargar una base de datos SQLite (.db) construida a partir de sus datos de PostgreSQL y lista para ejecutar el panel en SQLite.»

La conversión .db ⇌ .dump para SQLite también puede realizarse desde la CLI con el comando x-ui migrateDB [file] (véase la sección 16.7).

Copia de seguridad a través del bot de Telegram

Si el bot de Telegram está configurado (véase la sección sobre notificaciones), puede enviar la copia de seguridad directamente al chat del administrador. La copia de seguridad a través de Telegram incluye **dos archivos**: la propia base de datos (x-ui.db, o x-ui.dump en PostgreSQL) y la configuración de Xray config.json. El mensaje va precedido de la línea «🕒 Hora de la copia de seguridad: ...».

Hay dos formas de obtener una copia de seguridad en Telegram:

1. **Bajo demanda.** El botón « Backup BD» en el menú del bot: el bot envía los archivos inmediatamente al chat actual.

2. **Automáticamente con el informe.** En la configuración del bot hay un interruptor **«Database Backup»** con la descripción «Enviar notificación con el archivo de copia de seguridad de la base de datos». Cuando está activado, en cada envío periódico del informe, el bot adjunta la copia de seguridad a todos los administradores. La frecuencia del informe se define mediante la programación cron del bot (véase la sección 16.6). Entre archivos y entre administradores, el bot hace pausas para no superar los límites de Telegram.

La copia de seguridad a través del bot se envía solo si el bot está activo; en PostgreSQL también requiere que `pg_dump` esté disponible en el servidor.

16.2. Visualización de registros

El panel dispone de dos visores de registros independientes, ambos accesibles desde la pestaña **«Logs»** del «Dashboard». Cada ventana puede actualizarse (icono «actualizar» en el encabezado) y descargar el contenido mostrado como archivo `x-ui.log` (botón con icono de descarga).

Registros del panel (aplicación / syslog)

Ventana de registros del panel (`POST /panel/api/server/logs/{count}`). Controles:

Elemento	Valor predeterminado	Descripción
Número de líneas	20	Lista desplegable: 10 / 20 / 50 / 100 / 500
Nivel	Info	Nivel mínimo: Debug / Info / Notice / Warning / Error
SysLog (casilla)	desactivado	Origen de los registros: del búfer de la aplicación o del diario del sistema

El comportamiento depende de la casilla **SysLog**:

- **Desactivado (predeterminado):** los registros se toman del búfer circular interno del panel, filtrados según el nivel seleccionado. Las entradas se muestran con nivel (DEBUG / INFO / NOTICE / WARNING / ERROR) y origen: X-UI: para mensajes del propio panel, XRAY: para mensajes redirigidos de Xray.
- **Activado:** el panel ejecuta en el servidor `journalctl -u x-ui --no-pager -n <count> -p <level>`, es decir, muestra el diario del sistema del servicio `x-ui`. El número de líneas permitido es de 1 a 10000; el nivel acepta valores de syslog (`emerg/0`, `alert/1`, `crit/2`, `err/3`, `warning/4`, `notice/5`, `info/6`, `debug/7`). En Windows el modo SysLog no está disponible: se mostrará un aviso indicando que hay que desmarcar la casilla y usar los registros de la aplicación. Si `systemd` /el servicio no está disponible, aparecerá un mensaje de error al intentar ejecutar `journalctl`.

Ejemplo: el mismo diario desde la consola del servidor. Cuando el panel no está accesible (por ejemplo, no arranca), el diario del sistema puede leerse directamente; ese es exactamente el comando que el panel ejecuta en modo SysLog:

```
# últimas 100 líneas de nivel warning y superior
journalctl -u x-ui --no-pager -n 100 -p warning

# seguir el diario en tiempo real
journalctl -u x-ui -f
```

El nivel en esta ventana filtra la **salida**. El nivel mínimo que se escribe realmente en consola/syslog lo determina el nivel de registro del panel (variable de entorno; predeterminado `Info` ; en archivo el panel siempre escribe en nivel `DEBUG`).

Registros de Xray (diario de acceso)

Ventana separada para el access-log de Xray (`POST /panel/api/server/xraylogs/{count}`). Analiza las líneas del diario de acceso de Xray y las muestra en una tabla: **Date, From, To, Inbound, Outbound, Email**.

Elemento	Valor predeterminado	Descripción
Número de líneas	20	10 / 20 / 50 / 100 / 500
Filtro	vacío	Búsqueda de texto por subcadena (se aplica al presionar Enter)
Direct (casilla)	activado	Mostrar conexiones directas (tráfico a través de freedom-outbound)
Blocked (casilla)	activado	Mostrar conexiones bloqueadas (tráfico a blackhole-outbound)
Proxy (casilla)	activado	Mostrar tráfico proxificado

El tipo de evento se determina automáticamente por la etiqueta del outbound en la línea del registro: coincidencia con etiquetas freedom → «DIRECT» (verde), blackhole → «BLOCKED» (rojo), todo lo demás → «PROXY» (azul). Las líneas `api -> api` y las líneas vacías se omiten.

Para que esta ventana muestre entradas, Xray debe tener habilitado el **diario de acceso** con una ruta de archivo (distinta de `none`); véase más abajo. Si el access-log está desactivado o el archivo no está disponible, la ventana estará vacía («No Record...»).

16.3. Nivel y configuración del registro de Xray

Los parámetros de registro del propio Xray se configuran en la página «**Configuraciones de Xray**», en el bloque «**Log**» con la advertencia:

«Los registros pueden ralentizar el servidor. ¡Habilite solo los tipos de registros que necesite!»

Campo	Nombre	Valor predeterminado	Descripción
Nivel de logs (<code>logLevel</code>)	Log Level	<code>warning</code>	Nivel de detalle del registro de errores de Xray. Valores permitidos: <code>debug</code> , <code>info</code> , <code>notice</code> , <code>warning</code> , <code>error</code> . Ayuda: «Nivel de registro para los registros de errores, que indica la información que se debe registrar.»
Logs de acceso (<code>accessLog</code>)	Access Log	<code>none</code>	Ruta al archivo de registro de acceso. El valor especial <code>none</code> desactiva los logs de acceso. Ayuda: «Ruta al archivo de registro de acceso. El valor especial "none" desactiva los logs de acceso.»
Logs de errores (<code>errorLog</code>)	Error Log	vacío (ruta predeterminada)	Ruta al archivo de logs de errores; <code>none</code> los desactiva. Ayuda: «Ruta al archivo de logs de errores. El valor especial "none" desactiva los logs de errores.»
Logs de DNS (<code>dnsLog</code>)	DNS Log	<code>false</code> (desact.)	Habilitar el registro de solicitudes DNS. Ayuda: «Habilitar logs de solicitudes DNS».
Enmascaramiento de dirección (<code>maskAddress</code>)	Mask Address	vacío (desact.)	Cuando se activa, la dirección IP real se sustituye automáticamente por una enmascarada en los registros. Ayuda: «Cuando se activa, la dirección IP real se reemplaza por una enmascarada en los registros.»

Dado que el valor predeterminado de «**Logs de acceso**» es `none` , la ventana «Registros de Xray» (sección 16.2) está vacía de forma inicial. Para que funcione, configure aquí la ruta al access-log y reinicie Xray.

Tenga en cuenta que el access-log vacío solo afecta a esta ventana. La lista de clientes en línea en el «Dashboard» y el límite de número de IP en el formulario del cliente **no dependen** del access-log; el panel determina los clientes en línea y cuenta sus direcciones IP a través de la API de estadísticas en línea del núcleo Xray (estadísticas de conexiones). En versiones antiguas del núcleo donde esta API no está disponible, el panel vuelve automáticamente al método anterior (lectura del access-log), y en ese caso la ruta al access-log aquí sigue siendo necesaria para el límite de IP.

Límite por número de IP y fail2ban. El propio límite de número de IP por cliente (campo «IP Limit» en el formulario del cliente y en la adición masiva) solo se aplica en el servidor si **fail2ban** está instalado: es fail2ban quien bloquea las direcciones que superan el límite. El panel verifica la presencia de fail2ban (`GET /panel/api/server/fail2banStatus`); si no está instalado, el campo «IP Limit» queda deshabilitado con un mensaje explicativo (en Windows, con un mensaje separado), y los límites previamente definidos en esos servidores se anulan automáticamente, ya que de todas formas no tenían efecto. El bloqueo de fail2ban se aplica tanto a TCP como a UDP. En servidores normales, fail2ban ahora se instala y configura automáticamente al instalar y actualizar el panel (véase la sección 16.5).

Ejemplo: bloque `log` con el que la ventana «Registros de Xray» comenzará a mostrar entradas. En la configuración JSON de Xray queda así:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

Lo fundamental es reemplazar `"access": "none"` por una ruta de archivo (por ejemplo, `"./access.log"`). Tras guardar, reinicie Xray y la tabla en la ventana «Registros de Xray» comenzará a mostrar filas.

16.4. Gestión de Xray: detención y reinicio

El estado de Xray se controla desde la tarjeta de Xray en el «Dashboard». El estado actual se muestra como uno de estos valores: **Iniciado** (Running), **Detenido** (Stopped), **Desconocido** (Unknown), **Error** (Error). En caso de error, aparece el mensaje emergente «Error al iniciar Xray».

Botón	Nombre	Endpoint	Acción
Detener	Stop	POST /panel/api/server/stopXrayService	Detiene el proceso Xray. Si tiene éxito, se muestra la notificación de advertencia «Xray service has been stopped».
Reiniciar	Restart	POST /panel/api/server/restartXrayService	Reinicia (o inicia) Xray con la configuración actual. Si tiene éxito, se muestra la notificación «Xray service has been restarted successfully».

Tras cualquiera de las operaciones, el panel difunde el nuevo estado por WebSocket, por lo que el estado en el «Dashboard» se actualiza sin recargar la página. Si la operación falla, el estado de Xray pasa a «Error» y el texto del error se incluye en la notificación.

Además del reinicio manual, el panel comprueba periódicamente si es necesario reiniciar Xray (tarea en segundo plano cada 30 s) y si el proceso ha caído (comprobación cada segundo); véase la sección 16.6.

16.5. Reinicio y actualización del panel

Reinicio del panel

En la página «**Configuración del panel**» existe la acción «**Reiniciar panel**» (Restart Panel , POST /panel/api/setting/restartPanel). Al confirmarlo, el panel se reinicia **en 3 segundos**.

Mensajes:

- Confirmación: «¿Está seguro de que desea reiniciar el panel? Confirme y el reinicio se producirá en 3 segundos. Si el panel no está disponible, compruebe el log del servidor.»
- Éxito: «El panel se ha reiniciado correctamente».

Técnicamente, en Linux el reinicio se realiza enviando la señal `SIGHUP` al proceso del panel (o a través de un hook registrado). En Windows el envío de `SIGHUP` no está disponible.

Actualización automática del panel (Update Panel)

En el «Dashboard» está disponible la función **«Actualizar panel»** (`Update Panel`): actualiza 3X-UI a la última versión directamente desde la interfaz web.

Antes de actualizar, el panel compara versiones (`GET /panel/api/server/getPanelUpdateInfo`), consultando la última versión de 3x-ui en GitHub:

Campo	Nombre
Versión actual del panel	Current panel version
Última versión del panel	Latest panel version
El panel está actualizado / «Actualizado»	Panel is up to date / Up to date: se muestra cuando no hay nueva versión

Para iniciar la actualización: `POST /panel/api/server/updatePanel` . Diálogo de confirmación:

- «¿Realmente desea actualizar el panel?»
- «Esto actualizará 3X-UI a la versión #version# y reiniciará el servicio del panel.»

Tras iniciarla, aparece el mensaje emergente «Actualización del panel iniciada» (`Panel update started`); si la verificación de versión falla: «Comprobación de actualización del panel fallida» (`Panel update check failed`).

Qué ocurre en el servidor: la actualización automática solo está disponible **en Linux** (en otros sistemas operativos se devuelve el error «panel web update is supported only on Linux installations»). El panel descarga el script oficial `update.sh` de GitHub (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh`) y lo ejecuta en un proceso separado: preferentemente a través de `systemd-run` en una unidad separada (`x-ui-web-update-<timestamp>`), o como proceso independiente desacoplado si `systemd` no está disponible. Al finalizar, el script actualiza los componentes y reinicia el servicio del panel. Para su ejecución se requiere `bash` .

Si durante la actualización el script genera una nueva ruta base aleatoria para la interfaz web (Web Base Path), el servicio `x-ui` se reinicia automáticamente para que la nueva ruta entre en vigor de inmediato. (Sin el reinicio, el servidor seguiría sirviendo la ruta antigua mientras la interfaz mostraría la nueva, haciendo inaccesible la nueva dirección hasta un reinicio manual.)

En los nodos, el mismo 3x-ui se actualiza de forma centralizada a través de `POST /panel/api/nodes/updatePanel` ; véase la sección sobre nodos.

Instalación automática de fail2ban

Para que el límite por número de IP de los clientes (sección 16.3) funcione de forma inmediata, al instalar y actualizar el panel en un servidor normal, `fail2ban` ahora se instala y configura automáticamente (antes esto

solo ocurría en la imagen Docker). El comportamiento lo controla la variable de entorno

`XUI_ENABLE_FAIL2BAN` : la configuración se realiza si la variable no está definida o es igual a `true` . La ejecución manual está disponible con el comando `x-ui setup-fail2ban` . Un fallo en la configuración de fail2ban no interrumpe la instalación o actualización del panel.

Instalación y actualización en hosts solo con IPv6

Los scripts `install.sh` y `update.sh` ahora funcionan correctamente en servidores solo con IPv6: la descarga de la versión, del script `x-ui.sh` y de los archivos de servicio ya no fuerza IPv4 (`curl -4`), sino que usa el protocolo disponible. Por tanto, el panel puede instalarse y actualizarse incluso en un host sin dirección IPv4.

Anulación del puerto del panel con la variable `XUI_PORT`

El puerto de escucha de la interfaz web puede anularse con la variable de entorno `XUI_PORT` : solo tiene efecto durante la ejecución del proceso actual y **no modifica** el valor guardado `webPort` en la base de datos. Los valores admitidos son de `1` a `65535` ; un valor vacío, incorrecto o fuera de rango se ignora (se usa `webPort`) y se registra una advertencia en el log. Resulta útil en implementaciones, especialmente en Docker: al usar red bridge, el puerto publicado del contenedor debe coincidir con `XUI_PORT` ; por ejemplo, `XUI_PORT=8080` y `ports: "8080:8080"` .

Actualización y cambio de versión de Xray-core

También en el «Dashboard» se puede gestionar la versión de Xray-core de forma independiente al panel.

- **Actualizaciones de Xray** (`Xray Updates`) / **Versión** (`Version`): lista desplegable de versiones disponibles. Mensajes de ayuda: «Seleccione la versión deseada» y la advertencia «Importante: las versiones antiguas pueden no ser compatibles con la configuración actual».
- Instalación/cambio de versión: `POST /panel/api/server/installXray/{version}` . Diálogo: «Cambiar versión de Xray» / «¿Está seguro de que desea cambiar la versión de Xray?». Si tiene éxito: «Xray actualizado correctamente».

Ejemplo: cambio de versión de Xray-core mediante API. La versión se especifica con la etiqueta de versión de XTLS/Xray-core (con el prefijo `v`). Por ejemplo, cambiar a `v1.8.24` :

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` es el archivo con la cookie del ejemplo de la sección 16.1.) Tras la instalación, Xray se reiniciará automáticamente con la versión seleccionada.

En el servidor, al cambiar la versión, Xray primero se detiene, se descarga el archivo de la versión deseada desde GitHub (XTLS/Xray-core), el binario se extrae y sustituye, y a continuación Xray se reinicia con verificación del tamaño del archivo/binario.

16.6. Tareas periódicas (cron)

El panel registra una serie de tareas en segundo plano al iniciarse. Sus programaciones son fijas (no se configuran en la UI, excepto la programación del informe de Telegram y la sincronización LDAP). A continuación se muestran las tareas relacionadas con la operación.

Tarea	Programación	Propósito
Comprobación de estado de Xray	cada 1 s	Control de que el proceso Xray está en ejecución
Comprobación de necesidad de reinicio de Xray	cada 30 s	Reinicio si la configuración está marcada como modificada
Recopilación de tráfico de Xray	cada 5 s (inicio 5 s después del arranque)	Contabilidad de tráfico de inbound/clientes
Comprobación de IP de clientes	cada 10 s	Control del límite de IP por registro
Heartbeat y sincronización de tráfico de nodos	cada 5 s	Intercambio con los nodos
Limpieza de logs	diariamente (@daily)	Limpia los logs de límite de IP y el access-log persistente, rotando el log actual a <code>*.prev.log</code>
Reinicio de tráfico por período	<code>@hourly</code> , <code>@daily</code> , <code>@weekly</code> , <code>@monthly</code>	Reinicia los contadores de tráfico de los inbound (y sus clientes) que tienen configurado el período de reinicio automático correspondiente
Informe de Telegram	configurado en los ajustes del bot (predeterminado <code>@daily</code>)	Envío de informe a los administradores; si la opción está habilitada, con la copia de seguridad adjunta (sección 16.1)
Reinicio del almacén hash de Telegram	cada 2 m	Solo cuando el bot está activo
Control de carga de CPU para Telegram	cada 10 s	Solo si se ha definido un umbral de CPU > 0

Información adicional:

- **Reinicio periódico de tráfico** solo se activa para los inbound que tienen seleccionado el modo de reinicio automático correspondiente (por hora/día/semana/mes). La tarea reinicia el tráfico del propio inbound y de todos sus clientes.
- **Comprobación de vencimiento y agotamiento.** La desactivación de clientes por vencimiento y por agotamiento del límite de tráfico se realiza en el marco de la contabilidad de tráfico: los clientes con `expiry_time` vencido o volumen agotado se marcan y desactivan; si es necesario, se calcula el siguiente período (para límites cíclicos y el modo «conteo desde el primer uso»). En el «Dashboard» y en las listas esto se refleja con los estados «Vencido»/«Agotado»/«Por vencer».
- **Copia de seguridad automática en Telegram:** es un efecto secundario de la tarea del informe; no existe una programación cron exclusiva para la copia de seguridad. Por tanto, la frecuencia del backup automático es igual a la frecuencia del informe del bot.

16.7. Menú de consola y CLI (`x-ui`)

En el servidor, el panel se gestiona con el comando `x-ui`. Sin argumentos abre el menú interactivo «3X-UI Panel Management Script»; con un argumento, ejecuta un subcomando específico. Elementos del menú relacionados con la operación:

N.º en menú	Elemento	Acción
1	Install	Instalación del panel (descarga y ejecuta <code>install.sh</code>)
2	Update	Actualización de todos los componentes de <code>x-ui</code> a la última versión sin pérdida de datos; a continuación, reinicio automático
3	Update Menu	Actualización solo del script de menú <code>x-ui</code>
4	Legacy Version	Instalación de una versión específica (antigua) del panel introduciendo el número (p. ej., <code>2.4.0</code>)
5	Uninstall	Desinstalación completa del panel y Xray (véase 16.8)
6	Reset Username & Password	Restablecimiento del usuario/contraseña del administrador
7	Reset Web Base Path	Restablecimiento de la ruta base de la interfaz web
8	Reset Settings	Restablecimiento de la configuración a los valores predeterminados
9	Change Port	Cambio del puerto del panel
10	View Current Settings	Visualización de la configuración actual
11–13	Start / Stop / Restart	Inicio, detención, reinicio del servicio del panel
14	Restart Xray	Reinicio solo de Xray
15	Check Status	Estado actual del servicio
16	Logs Management	Visualización y limpieza de registros (véase más abajo)
17–18	Enable / Disable Autostart	Habilitar/deshabilitar el inicio automático del servicio al arrancar el sistema operativo
25	Update Geo Files	Actualización de archivos geográficos (GeoIP/GeoSite)
27	PostgreSQL Management	Gestión de PostgreSQL

Gestión de registros en CLI (elemento 16)

En el submenú «Logs Management»:

- **Debug Log:** visualización en flujo del diario del servicio: `journalctl -u x-ui -e --no-pager -f -p debug` (en Alpine: `grep` sobre `/var/log/messages`).
- **Clear All logs:** limpieza del diario del sistema: `journalctl --rotate + journalctl --vacuum-time=1s`, tras lo cual el servicio se reinicia. (No disponible en Alpine.)

Subcomandos directos de `x-ui`

Todos los subcomandos disponibles:

Comando	Descripción
<code>x-ui</code>	Abrir el menú administrativo
<code>x-ui start</code>	Iniciar el panel
<code>x-ui stop</code>	Detener el panel
<code>x-ui restart</code>	Reiniciar el panel
<code>x-ui restart-xray</code>	Reiniciar Xray
<code>x-ui status</code>	Estado actual
<code>x-ui settings</code>	Mostrar la configuración actual
<code>x-ui enable</code>	Habilitar el inicio automático al arrancar el sistema operativo
<code>x-ui disable</code>	Deshabilitar el inicio automático
<code>x-ui log</code>	Visualización de registros
<code>x-ui banlog</code>	Visualización de registros de bloqueos de Fail2ban
<code>x-ui setup-fail2ban</code>	Instalar y configurar fail2ban para el límite de IP (véase 16.5)
<code>x-ui update</code>	Actualizar el panel
<code>x-ui update-all-geofiles</code>	Actualizar todos los archivos geográficos (con reinicio posterior)
<code>x-ui migrateDB [file]</code>	Conversión de base de datos <code>.db</code> a <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	Instalar una versión antigua
<code>x-ui install</code>	Instalar el panel
<code>x-ui uninstall</code>	Desinstalar el panel

El comando `x-ui update` descarga y ejecuta el `update.sh` oficial (el mismo que la actualización web de la sección 16.5), solicitando confirmación: «This function will update all x-ui components to the latest version, and the data will not be lost.» Al finalizar, el panel se reinicia automáticamente.

Flags `-webCert` / `-webCertKey` en el subcomando `setting` . Las rutas al certificado y a la clave privada de la interfaz web pueden definirse directamente en el subcomando `x-ui setting -webCert <ruta> -webCertKey <ruta>` ; especificar cualquiera de estos flags guarda la ruta correspondiente (igual que el subcomando `cert` por separado) y el panel pasa inmediatamente a HTTPS.

Obtención del token de API mediante CLI

El comando para obtener el token de API mediante CLI (elemento de menú/comando `x-ui`) no muestra un token emitido previamente. Los tokens de API se almacenan solo como hashes, por lo que no es posible recuperar un token existente en texto plano. Si ya hay tokens configurados, el comando informa de su cantidad, recomienda gestionarlos en el panel (**Settings** → **API Tokens**, véase la sección sobre tokens de API) y genera inmediatamente un **nuevo token de respaldo** con un nombre del tipo `cli-fallback-<timestamp>`, mostrándolo para que la CLI siga siendo útil sin necesidad de acceder a la interfaz.

16.8. Desinstalación del panel

La desinstalación se realiza desde la CLI: elemento de menú **5 (Uninstall)** o comando `x-ui uninstall`. Antes de desinstalar se solicita confirmación (predeterminado «no»): «Are you sure you want to uninstall the panel? xray will also uninstalled!».

Al confirmar, el script:

1. Detiene el servicio y deshabilita su inicio automático (`systemctl stop/disable x-ui`, o en Alpine: `rc-service / rc-update`), elimina el archivo de unidad del servicio y recarga la configuración de systemd.
2. Elimina los directorios de datos y aplicación (`/etc/x-ui/`, directorio de instalación) y el archivo de entorno del servicio (`/etc/default/x-ui`, `/etc/conf.d/x-ui` o `/etc/sysconfig/x-ui`, según la distribución).
3. Elimina el propio script `x-ui` y muestra el mensaje «Uninstalled Successfully», así como el comando para una nueva instalación.

La desinstalación es irreversible: junto con el panel se elimina Xray y todos los datos (incluida la base de datos). Si los datos pueden ser necesarios, exporte la base de datos previamente (sección 16.1).

16.9. Comando `x-ui migrateDB`

A partir de la versión 3.3.0, el script de gestión `x-ui.sh` recibió el subcomando `migrateDB`, un envoltorio alrededor del binario integrado `x-ui` (`x-ui migrate-db`) para la conversión de la base de datos del panel SQLite entre dos formatos: el binario `.db` y el volcado de texto portable `.dump` (texto SQL estándar).

Qué hace el comando

El comando funciona en dos direcciones, y la dirección se determina **automáticamente** según el archivo de entrada:

Dirección	Cómo se llama	Qué ocurre
<code>.db → .dump</code>	dump (exportación)	la base SQLite binaria se exporta a un archivo SQL de texto
<code>.dump → .db</code>	restore (restauración)	a partir del archivo SQL de texto se reconstruye la base SQLite binaria

Por debajo, el script llama al binario del panel:

- exportación: `x-ui migrate-db --src <entrada> --dump <salida>`
- restauración: `x-ui migrate-db --restore <entrada> --out <salida>`

Sintaxis de uso

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** : archivo de entrada (primer argumento). Si no se especifica, se usa la base instalada del panel por defecto: `/etc/x-ui/x-ui.db`.
- **[output]** : ruta al archivo de salida (segundo argumento). Es opcional; si se omite, el nombre se elige automáticamente junto al archivo de entrada (véase más abajo).

Ejemplos:

```
x-ui migrateDB                               # exportar /etc/x-ui/x-ui.db -> /etc/x-ui/x-
ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db      # reconstruir .db a partir del volcado
```

Cómo se determina la dirección

El script examina la extensión del archivo de entrada:

- `*.db`, `*.sqlite`, `*.sqlite3` → modo **dump** (exportación a texto);
- `*.dump`, `*.sql` → modo **restore** (reconstrucción de base de datos).

Si la extensión no se reconoce, el script lee los primeros 16 bytes del archivo: la firma `SQLite format 3` indica una base binaria (modo dump); en caso contrario, el archivo se considera un volcado (modo restore).

Nombre del archivo de salida cuando no se proporciona el segundo argumento:

- en exportación: mismo nombre que la entrada, con extensión `.dump`;
- en restauración: mismo nombre con extensión `.db`.

Comprobaciones de seguridad y comportamiento

- **Presencia del binario.** Si el binario `x-ui` no se encuentra o no es ejecutable, se muestra el error «x-ui binary not found ... Is the panel installed?».
- **Compatibilidad de la función con la versión.** El script comprueba que el binario admite `migrate-db --dump/--restore` (mediante `x-ui migrate-db -h`). Si no es así, se recomienda actualizar primero el panel con `x-ui update`.
- **Existencia del archivo de entrada.** Si el archivo de entrada no existe, se imprime un error junto con la línea de sintaxis.
- **Sobreescritura del archivo de salida.** Si el archivo de salida ya existe, se solicita confirmación (predeterminado «no»); sin confirmación, la operación se cancela. En la restauración, el archivo de salida existente se elimina previamente.

- **Protección de la base activa.** Al restaurar en la base predeterminada `/etc/x-ui/x-ui.db` mientras el panel está en ejecución, la operación se rechaza con la indicación de detener primero el panel (`x-ui stop`) o elegir otra ruta de salida. Esto evita la sobrescritura de la base de datos activa de un servicio en funcionamiento.
- Si la construcción de la base falla, el archivo de salida incompleto se elimina.

Para qué sirve

- **Copia de seguridad.** El `.dump` de texto es legible por humanos, adecuado para almacenamiento en sistemas de control de versiones y para la inspección diferencial del contenido de la base de datos.
- **Transferencia.** El volcado es portable entre máquinas y resistente a diferencias de versión del formato de archivo SQLite: en un nuevo servidor se puede reconstruir desde él una `.db` funcional.
- **Diagnóstico.** Desde un `.dump` se puede examinar visualmente la estructura y los datos del panel sin necesidad de herramientas SQLite a mano.

Modo interactivo

Además de la invocación directa, la conversión también está disponible desde el menú interactivo. En el submenú de PostgreSQL (`x-ui` → sección de gestión de PostgreSQL) hay el elemento **9. Convert SQLite `.db` <--> `.dump`**: solicita la ruta al archivo de entrada (predeterminado `/etc/x-ui/x-ui.db`) y al archivo de salida (puede dejarse vacío para nombre automático), y la dirección, como en el modo CLI, se determina automáticamente.

Documento elaborado a partir del código fuente de 3X-UI. Si algún elemento de la interfaz difiere en su versión, prevalece el comportamiento del panel y los mensajes de ayuda del propio UI.